

A COMPLETE BEGINNER'S COURSE

The Practical Data Analyst's Handbook

From Zero to Your First Data Job in
Excel, SQL, Statistics, Python and Power BI

Tatenda Makuvaza

"Data Analyst · Author · Educator"

First Edition · 2026

Table of Contents

Front Matter	5
Beginning: You, the Data Analyst	8
Chapter 1: What a Data Analyst Actually Does	9
Chapter 2: The Analyst's Mindset	13
Chapter 3: The Tools of the Trade	18
Chapter 4: Set Up Your Workspace	23
Chapter 5: Meet Tariro	28
Excel: Your First Superpower	33
Chapter 6: Thinking in Grids	34
Chapter 7: Formulas from Zero	38
Chapter 8: Cleaning a Messy Sheet	42
Chapter 9: Joining Tables in Excel	46
Chapter 10: PivotTables — The Analyst's Best Friend	50
Chapter 11: Charts That Tell the Truth	54
Chapter 12: Descriptive Statistics in Excel	58
Chapter 13: Project 1 — Tariro's First Monthly Report	62
SQL: The Language of Data	67
Chapter 14: What a Database Is	68
Chapter 15: Your First Queries	72
Chapter 16: Counting and Grouping	77
Chapter 17: Joins Without Fear	82
Chapter 18: Queries in Steps — Subqueries and CTEs	87
Chapter 19: Window Functions	92
Chapter 20: SQL in Real Life	97
Chapter 21: Project 2 — Tariro's Database	102

Statistics Without Fear	108
Chapter 22: Numbers That Describe	109
Chapter 23: Spread and Shape — the Bell Curve, Gently	114
Chapter 24: Percentiles and Z-Scores — "Above Average", Properly	119
Chapter 25: Samples and Populations	123
Chapter 26: Testing, Gently — Hypothesis Tests by Counting	127
Chapter 27: Categories and Surveys	131
Chapter 28: Relationships — Correlation, Regression, and the Trap	136
Chapter 29: SPSS and Stata, Gently	141
 Python: From Zero to Dangerous	 146
Chapter 30: Why Python, and How to Start	147
Chapter 31: Your First Lines — Variables, Numbers, Text	152
Chapter 32: Loops, Functions, and the Pandas Way	157
Chapter 33: DataFrames from Zero	162
Chapter 34: pandas Power Moves — groupby, merge, time series	167
Chapter 35: Charts in Python — matplotlib, Honestly	172
Chapter 36: Statistics in Python — the Same Answers, in Six Lines	176
Chapter 37: Project 3 — The Report That Builds Itself	180
 Dashboards and Storytelling with Power BI	 185
Chapter 38: What Power BI Is	186
Chapter 39: Getting Data In — Shaping and the Model	191
Chapter 40: Measures and DAX — the Ten Formulas	196
Chapter 41: Your First Dashboard	201
Chapter 42: Chart Craft, Again — the Dashboard Edition	206
Chapter 43: Telling the Story	211
 Real Projects and Your Portfolio	 216
Chapter 44: Where Practice Data Comes From	217
Chapter 45: The Anatomy of an Analysis	222
Chapter 46: Project 4 — Customers and Churn	227

Chapter 47: Project 5 — The Survey	232
Chapter 48: Project 6 — The Operations Dashboard	237
Chapter 49: Your Portfolio	242
Getting the Job	247
Chapter 50: The Job Landscape	248
Chapter 51: Your CV and LinkedIn	253
Chapter 52: The Technical Interview	257
Chapter 53: Cases and Behaviour — the Offer	262
Chapter 54: Your First 90 Days	267
Chapter 55: Freelancing and Side Income	272
Chapter 56: The Career Ladder	277
Beyond the Basics	282
Chapter 57: A Gentle Map of Machine Learning	283
Chapter 58: A Gentle Map of Big Data	288
Chapter 59: Ethics and Privacy, Essentials	293
Chapter 60: Messy Reality — Missing Data, Biased Samples, Deadlines	298
Chapter 61: The Free Stack — \$0 Toolkit, \$200 Upgrade	303
Chapter 62: Your Next Decade	308
Reference Section	313
Appendix A: SQL Cookbook	314
Appendix B: Excel Cookbook	323
Appendix C: Python Cookbook	327
Appendix D: Statistics Cheat Sheet	333
Appendix E: Interview Question Bank	338
Appendix F: Solutions and Walkthroughs	345
Appendix G: Glossary	355
About this Edition	364

Front Matter

Welcome to Your New Career

The Practical Data Analyst's Handbook — From Zero to Your First Data Job: A Complete Beginner's Course

Tatenda Makuvaza — Data Analyst, Author, Educator

First Edition, 2026

Who This Book Is For

You have never written a line of code. You open a spreadsheet and feel slightly tired. The word "statistics" reminds you of a classroom you were happy to leave. And yet somewhere you have heard that data analysis is a good, honest, well-paid skill — one that exists in every industry, one you could actually learn — and you wonder whether someone like you could really do it.

Yes. This book was written precisely for you.

It assumes **zero background**. No mathematics beyond arithmetic. No programming. No prior tools. Every term is explained the moment it appears, in plain English, with a worked example before you are asked to try anything yourself. Every tool in this book has a free version, and Chapter 4 shows you how to install all of them in one afternoon.

The Promise

By the last page you will have:

1. **Five tools, genuinely learned** — Excel, SQL, statistics (with SPSS and Stata), Python, and Power BI — each introduced from absolute zero, each practiced on real (generated) business data.
2. **Six completed projects** — including three portfolio-ready analyses with walkthroughs — all built on the running case of a real-seeming small business.
3. **A portfolio** — a GitHub page an employer can click, showing your work, by Chapter 49.
4. **The job hunt, handled** — CV, LinkedIn, technical interviews, case interviews, take-homes, and your first 90 days once hired.
5. **The map of what comes next** — machine learning, big data, ethics, and the career decade after your first job.

How This Book Works

One running case. In Chapter 5 you meet **Tariro**, who runs a grocery store in Highfield, Harare — a notebook, a half-trusted spreadsheet, and a dream of a second branch. You become her analyst, and you stay hired for the whole book. Every formula, query, and chart you learn is immediately used on her real-seeming data (generated with documented patterns, so your answers can always be checked against the truth). When you interview for your first job, Tariro is your story.

One chapter recipe. Every chapter tells you what you will learn, teaches it with worked examples, closes with Key Takeaways and a Practice Lab you can actually do (data provided), and points you to further reading. Short chapters, steady stairs.

One toolkit, bridged. The "From Your Toolkit" callouts connect every new tool back to the ones you already know — the same table-join appears first in Excel, then in SQL, then in Python — so nothing you learn is ever an island.

One honesty habit. Averages lie, samples have error, correlation is not causation. You will meet these truths in Part I and be held to them in every part after. The most employable thing about you will not be a tool; it will be that your numbers can be trusted.

The Journey

Part	Question it answers	Tools
I — Beginning	What does a data analyst actually do, and how do I start?	The mindset; the workspace
II — Excel	How do I turn a messy sheet into a report?	Excel, PivotTables, charts
III — SQL	How do I ask a database real questions?	SQL, joins, windows
IV — Statistics	How do I know a number means anything?	Descriptives, tests, SPSS, Stata
V — Python	How do I make the computer do the repetitive work?	Python, pandas, matplotlib
VI — Power BI	How do I show all this to other people?	Data models, DAX, dashboards
VII — Projects	What does a real analysis look like, end to end?	Three portfolio projects
VIII — The Job	How do I get hired — and survive the first 90 days?	CV, interviews, freelancing
IX — Beyond	What is machine learning? What comes next?	The gentle maps; the free stack

Sixty-two chapters and seven cookbook appendices (SQL, Excel, and Python recipes, a statistics decision table, a 100-question interview bank, worked lab

solutions, and a glossary).

A Note on Cost

Everything in this book is free except Excel. If you do not own Excel, Chapter 6 introduces **LibreOffice Calc**, its free twin, and every exercise works identically in both — the few differences are flagged wherever they appear. Power BI Desktop, Python, SQLite, and the practice datasets cost nothing. Your total toolkit bill can be **zero dollars**, and Chapter 61 shows the \$200 version that makes life easier. Cost is never the barrier; patience briefly is.

From the Author

I completed a six-module data analytics course — SQL, Excel, Python, Power BI, Stata and SPSS — and I remember exactly how each of them felt on day one: the spreadsheet that would not add up, the query that returned an error, the statistics lecture that seemed to be in a foreign language. This book is the course I wish I had been handed: every tool from zero, every concept in plain words, one friendly business to practice on, and the honest career guidance at the end.

Work the labs. Be patient with yourself on day one of each new tool — everyone is. And keep Tariro's question in mind whenever the material feels abstract: *someone, somewhere, is making a decision with this number*. That is the whole job, and it is a good one.

— Tatenda Makuvaza, Harare, 2026

PART I

Beginning: You, the Data Analyst

Chapters 1 – 5

Chapter 1: What a Data Analyst Actually Does

Part I — Beginning: You, the Data Analyst

"The job is turning numbers into decisions — everything else is detail."

In this chapter you will learn

- What a data analyst's day actually looks like, hour by hour.
- The five myths about the job — and the truth behind each.
- The one sentence that defines the whole profession.
- Where analysts work, what they earn, and who they answer to.
- A first taste of the running case you will work with for the whole book.

1.1 The Job in One Sentence

Strip away the job adverts, the buzzwords, and the software logos, and a data analyst does one thing:

A data analyst collects information, cleans it up, studies it, and explains what should be done about it — in language the decision-maker understands.

Every chapter of this book is one of those four verbs. Collecting (Parts II, III, V), cleaning (everywhere, and more than anyone expects), studying (Part IV), and explaining (Part VI). If you remember nothing else today, remember the sentence: it is your compass whenever a tool or a technique feels confusing. Ask *which verb is this?* and the confusion usually shrinks.

1.2 A Day in the Life

A realistic Tuesday for a junior analyst at a retail company (the same world as the grocery store you will meet in Chapter 5, just bigger):

Time	What is happening
08:00	Coffee, inbox: a manager asks why last week's sales dipped. The question of the day is born.
08:30	Pull the data — a SQL query against the sales database (Part III of this book).
09:00	Clean and check it: missing rows from one store's till, a duplicate upload, prices in the wrong column. Half the morning goes here. This is normal and never stops being normal.

Time	What is happening
10:30	Explore: totals by store and by day in a pivot table; a chart or two. The dip is real, mostly in one product line.
11:30	Dig deeper: was it the price change two weeks ago? A competitor's promotion? A supply gap? The analyst becomes a detective.
13:00	Write the summary: half a page, three numbers, one chart, a recommendation ("hold prices another two weeks; the dip is mostly the supply gap").
14:00	Walk the manager through it. Answer "what about the Bulawayo store?" three times, pleasantly, with the numbers ready.
15:00	Update the weekly dashboard so everyone can watch the number recover.
16:00	A recurring report needs refreshing — the analyst spends thirty minutes on what will become a five-minute Python script by Chapter 37.
17:00	Note tomorrow's follow-up: the supply-gap theory needs one more dataset.

Notice what is *not* there: no mathematics beyond arithmetic, no dramatic model-building, no solitary genius work. The real job is roughly **half communication and cleaning, a third exploration, and the rest tools** — which is good news, because those are all learnable, and this book teaches them in exactly that order.

1.3 Five Myths, Five Truths

1. **Myth: "You must be great at mathematics."** Truth: analysts need arithmetic, percentages, and clear thinking. The statistics in Part IV is taught here by counting and plain English — that is how working analysts actually reason.
2. **Myth: "You must have a computer science degree."** Truth: analysts arrive from every field — commerce, health, agriculture, admin. Employers test what you can *do*; by Part VII you will have three projects to show.
3. **Myth: "It is all about the latest AI."** Truth: the daily bread is tables, joins, and charts. AI is a map we draw in Chapter 57 — useful, later, optional.
4. **Myth: "It is a lonely job with a computer."** Truth: it is a service job. Your numbers only matter when a person trusts them enough to act — which makes Chapter 43's presentation skills as career-critical as any query.
5. **Myth: "The tools change too fast to learn."** Truth: the grammar is stable. The same table-join you will meet in Excel (Chapter 9), SQL (Chapter 17), and Python (Chapter 34) has not changed in decades. Learn the grammar once and tools become interchangeable accents.

1.4 Where Analysts Work and What They Earn

Every industry that records anything employs analysts: banks and insurers, retailers and manufacturers, hospitals and ministries, telecoms, farms, football clubs, and every consultancy that serves them. In most markets, junior analysts earn a solid professional salary from day one, with a clear ladder (Chapter 56) — senior analyst, lead, and specialist routes into data engineering, data science, or analytics management. The skill also travels: the same SQL that analyses a bank's loans in Harare analyses a hospital's queues in Nairobi or a retailer's stock in London, unaltered. Few careers are this portable.

And the freelance route is real: small businesses everywhere need exactly what this book teaches — a monthly report, a dashboard, a survey analysed — and Chapter 55 shows how to price and deliver that work from a laptop.

1.5 The Decision Is the Destination

Here is the habit that separates analysts from people who merely make spreadsheets: **before any work begins, they ask what decision the answer will change.** A manager's "can you pull the sales numbers?" is an invitation to a trap — a week of work answering a question nobody was asking. The professional reply is a gentle question: "*When you have the numbers, what will you do with them?*" If the answer is "decide whether to keep the Sunday opening", you now know exactly which numbers matter (Sundays versus other days, costs, footfall), which do not, and — by the end of Part IV — whether the difference is real or just noise.

From Your Toolkit — everything: this habit costs nothing and works in every tool you will ever learn, because it happens before any tool is opened. Keep a one-line note at the top of every piece of work you do in this book: *the decision this serves*. By the projects of Part VII it will be automatic — and interviewers can smell the difference in candidates immediately.

1.6 What You Will Not Need

Reassurance, in writing:

- **No degree** is required to follow this book or to build its portfolio.
- **No mathematics beyond arithmetic** — every statistical idea is introduced by counting real cases first.
- **No paid software** — one afternoon in Chapter 4 and your toolkit is complete, free.

- **No prior experience** — the running case assumes you are starting exactly where you are.

Key Takeaways

- The job in one sentence: collect, clean, study, explain — numbers into decisions.
- A real day is half cleaning and communication, a third exploration, the rest tools. All learnable, in that order.
- The five myths are false: no advanced maths, no CS degree, no AI prerequisite, not lonely, not a tools race.
- The defining habit: ask what decision the analysis will change, before touching any tool.
- The skill is portable, the ladder is real, and the freelance route exists from day one.

Practice Lab

1. Find one real job advert for a "data analyst" near you (any job site). Circle every skill it mentions, and sort them into the four verbs: collect, clean, study, explain. Notice how the list stops looking scary once sorted.
2. Interview one person you know who uses spreadsheets at work (five minutes): what decision do they make with them? What do they wish they knew about their numbers? Write three lines.
3. Write your own one-line answer to "why do you want to be a data analyst?" — you will refine it in Chapter 51, and it is never too early to start.
4. Myth audit: which of the five myths had you believed? Write one sentence about where you think it came from.
5. The compass test: for each of these requests, write the decision question behind it — "how many visitors did our website get?", "which product sells best?", "are our customers happy?".

Further Reading

- Chapter 2 (the mindset behind the job), Chapter 50 (the job landscape in detail)
- Any "day in the life of a data analyst" video, 15 minutes, any platform — see how many of this chapter's hours you can spot.

Chapter 2: The Analyst's Mindset

Part I — Beginning: You, the Data Analyst

"Three habits separate trusted analysts from fast ones: be curious, be suspicious, be clear."

In this chapter you will learn

- The three habits that make an analyst — curiosity, skepticism, clarity.
- Why the question always comes before the data.
- The three honesty rules you will be held to for the rest of the book.
- How to read a number like an analyst (the five questions to ask any figure).
- The patience loop: how beginners actually learn this craft.

2.1 Curiosity: The Fuel

Every good analysis begins with a small itch: *why did that happen?* The analyst's version of curiosity is specific — it is aimed at **differences and changes**. Sales fell — from what, to what, since when, where, for whom? The untrained eye reads "sales fell 5%" as a fact; the curious analyst immediately asks five smaller questions, because inside every big number there are ten smaller ones, and the interesting one is always hiding in there.

You already have this habit if you have ever wondered why one bus route is always full and the next is empty. This chapter's work is to aim the wondering at tables — and to write it down. The analyst's notebook habit: every question you ask of the data gets a line, even (especially) the ones you answer along the way to something else.

2.2 Skepticism: The Shield

Curiosity without skepticism is how rumours get spreadsheets. The skeptic's discipline is that **every number is an answer to a question someone chose, on data someone collected, with rules someone decided** — and any of those someones may have been tired, rushed, or hopeful. The five questions to ask any figure you meet (this book uses them constantly):

1. **Compared to what?** — "Sales are up 12%" means nothing without last month, last year, or the plan.
2. **Says who?** — where did this number come from: a till, a survey, an estimate, a guess with formatting?

3. **What is missing?** — which rows, stores, days, or people are not in the data? (The missing ones are rarely missing at random.)
4. **What was counted exactly?** — "customers" — people, or transactions? "Active" — this month, or ever?
5. **Could it be luck?** — is the difference real, or the ordinary wobble of any variable thing? (Part IV gives you the tools for this one.)

None of this is cynicism. The skeptic's posture is not "numbers are lies" but "numbers are claims" — and claims deserve polite verification. Analysts who ask these five questions pleasantly, every time, become the people whose sign-off everyone wants.

2.3 Clarity: The Point

Analysis that cannot be explained has not happened. The clarity habit has two halves:

- **Explain it simply.** If you cannot describe your finding to a smart twelve-year-old in three sentences, you do not yet understand it — the sentence is not a simplification of the analysis, it is the analysis, compressed. (Einstein probably never said the famous version, but the rule holds anyway.)
- **Show the uncertainty.** Numbers wobble; samples miss; next month differs. Saying "sales rose about 5% — between 3% and 7% — and we are fairly confident it is the price change, though the supply gap muddies it" is not weakness. It is the reason the listener can build on your number without being ambushed later.

Clarity is a service ethic. Your reader has thirty seconds and other meetings. The whole of Part VI (charts, dashboards, the one-page story) is this habit, given tools.

2.4 The Three Honesty Rules

For the rest of this book — and the rest of your career — you are held to three rules. They will be repeated until they are reflexes:

1. **The average is not the answer.** Averages hide the very differences that decisions need. The average customer does not exist; the average of 10 and 1000 is 505, which describes neither. Whenever you meet an average, you will ask (Chapter 22) what it is hiding.
2. **Correlation is not causation.** Ice cream sales and drownings rise together every summer — because of the sun, not the ice cream. Two things moving together is a clue for investigation, never a conclusion. Chapter 28 teaches you to catch your own mind making this mistake.

3. **The sample is not the population.** The 200 people who answered your survey are not "the customers"; they are the customers who answer surveys. Chapter 25 shows what that costs and how to adjust for it.

None of these rules require mathematics. They require the willingness to say "not yet proven" in a meeting where everyone wants "proven" — which is, again, exactly why the people who say it are trusted.

From Your Toolkit — your notebook: paper or a notes app is the analyst's zeroth tool. From today, every piece of work in this book starts with three lines in your notebook: *the question, the decision it serves, what I expect to find*. You will be amazed (in about six weeks) how much of your own learning lives in those notebooks.

2.5 Reading a Number Like an Analyst

A quick drill to make the habits concrete. A colleague says: "Our new loyalty programme is working — members spend 40% more than non-members."

- **Curiosity:** 40% more — per visit, or per month? All members, or the ones who joined? Since when?
- **Skepticism:** compared to what — non-members who chose not to join, or people who were never asked? What if the people who join a loyalty programme were always the frequent shoppers? (They usually are. This exact trap — and its cure — is Chapter 28's centerpiece.)
- **Clarity:** the honest version: "Members spend 40% more than non-members; some of that difference existed before they joined. To measure the programme properly we would need..." — and now you sound like the most valuable person in the room, because you are.

That paragraph is the whole mindset in ninety seconds: itch, doubt, translate. Everything else in this book is giving those three habits tools.

2.6 The Patience Loop

Beginners learn this craft in a loop, and knowing the loop prevents the week-three dropout that claims most self-learners:

1. **Confusion** ("what even is a dataframe?") — normal, required, temporary.
2. **A small win** (the query runs; the chart appears) — worth more than it looks.
3. **A small failure** (the query runs *wrong*) — where the actual learning happens.

4. **The explanation that clicks** ("it is just a table — a spreadsheet I can program").

One loop is maybe twenty minutes. A chapter of this book is a few loops. The tools change the loop's content, never its shape — the confusion you feel meeting SQL in Chapter 15 is the same confusion you will feel meeting Python in Chapter 31, and the same feeling you will professionalise on the job forever. Analysts are not people who stopped being confused; they are people who learned that confusion is the working state of anyone learning anything worth learning.

Key Takeaways

- Three habits: be curious (differences and changes), be skeptical (numbers are claims), be clear (three sentences and the uncertainty).
- Ask any number five questions: compared to what, says who, what is missing, what was counted, could it be luck.
- Three honesty rules, forever: the average is not the answer; correlation is not causation; the sample is not the population.
- The notebook habit: question, decision, expectation — three lines before any tool.
- The patience loop (confuse, win, fail, click) is the shape of learning every tool in this book.

Practice Lab

1. Take today's headline number from any news site and run the five questions at it. Write the answers you *wish* you could get — those wishes are real analytical questions someone is paid to answer.
2. The average trap: find a real "average" in an advert or article (average salary, average customer, average rating). Write two sentences on who or what it might be hiding.
3. Correlation hunt: list three pairs of things that rise together (ice cream and drownings style). For each, write the hidden third factor.
4. Start the notebook: three lines for a question in your own life with numbers in it (spending, time, marks, fitness). Keep the notebook by Chapter 5 — the running case will move into it.
5. Reframe the loyalty claim: rewrite "members spend 40% more" in the honest version from Section 2.5, in your own words, as if to your manager.

Further Reading

- *How to Lie with Statistics* — Darrell Huff (sixty years old, one hour to read, permanently useful)
- Chapter 22 (why averages lie, with pictures), Chapter 25 (samples), Chapter 28 (the causation trap)

Chapter 3: The Tools of the Trade

Part I — Beginning: You, the Data Analyst

"Six tools, one grammar: tables in, answers out."

In this chapter you will learn

- The guided tour of every tool in this book, each explained in one plain paragraph.
- What each tool is *for* — and what it is cheerfully bad at.
- The one idea (tables!) that connects all of them.
- The free version of everything, in one table.
- How the book's journey (Excel → SQL → statistics → Python → Power BI) is deliberately ordered.

3.1 The Big Idea First: Tables

Every tool in this book works on the same shape: **a table** — rows and columns, where each row is one thing (one sale, one customer, one answer) and each column is one fact about it (the date, the amount, the store). Learn to see the world as tables and every tool becomes a dialect of one language:

tool	what it does with your table
Excel	shows it to you, lets you point and click at it
SQL	lives inside the database, answering questions about it at scale
Statistics	judges whether the table's patterns mean anything
Python	programs the table — anything you did by hand, automated
Power BI	turns the table into pictures a room can discuss
SPSS / Stata	the classic statistics packages — same questions, point-and-click or scripted

This is why the book can promise "five tools, genuinely learned" without five times the effort: **you learn tables once (Chapter 6), joins once (first Excel, then SQL, then Python), charts once (Excel, then Python, then Power BI), and testing once (counting in Part IV, then every package agrees)**. The "From Your Toolkit" callouts mark each reunion.

3.2 The Tour

Excel (Part II). The world's most-used analytics tool and the right place to start: you can see everything, touch everything, and break everything harmlessly. Formulas, pivot tables, and charts cover an enormous share of real working analytics — and every employer expects them. Its limit: it is one screen, one file, one user — slow or fragile when data gets big or updates daily. *Free twin: LibreOffice Calc, identical for everything in this book.*

SQL (Part III). "Structured Query Language" — the language of databases. Where Excel shows you the table, SQL *asks the table questions* ("total sales by store for March, excluding refunds") and returns only the answer, instantly, on tables far too big for any spreadsheet. It is the single most-tested skill in analyst interviews (Chapter 52), the one this book teaches most thoroughly, and — good news — the core is about twenty patterns you will reuse forever.

Statistics (Part IV). Not a software tool but a thinking tool: how to describe honestly (averages and their traps), and how to judge whether a pattern is real or luck. We teach it by *counting* — simulating coin flips and drawing the answers — before any formula appears. **SPSS** and **Stata** are the classic statistics packages you may meet in university courses, research offices, and government: same ideas, different buttons, both covered gently in Chapter 29.

Python (Part V). A full programming language, and the analyst's version of it is mostly one library — **pandas** — that puts your table in a notebook and lets you program it: "read the sales file, drop the duplicates, total by month, chart it, save the chart" as a short script that runs again next month without you. The beginner's fear — "programming" — dissolves in about three chapters, because analyst Python is mostly verbs you already know: `read`, `group`, `merge`, `plot`.

Power BI (Part VI). Microsoft's dashboard tool: it connects to your tables (Excel files, databases), remembers how they relate, and turns them into interactive pages — filters, drill-downs, monthly refreshes — that a manager can use without you. The **Desktop** edition is free, and the skill is in heavy demand because dashboards are how organisations breathe.

3.3 What Each Tool Is Bad At

Honesty works both ways:

- **Excel** is bad at *big* (hundreds of thousands of rows get slow) and *repeated* (manual monthly work drifts into errors) and *shared* (versions multiply by email).

- **SQL** is bad at *charts* (it answers; it does not draw) and *everything before the question* (it does not clean your notebook's coffee stains — that is Excel or Python).
- **Statistics** is bad at *deciding for you* — it quantifies doubt; the judgement stays human.
- **Python** is bad at *first contact* (a blank notebook is intimidating — for two weeks) and *pretty point-and-click* (nobody "just opens" Python to glance at something).
- **Power BI** is bad at *deep exploration* (it shows; it does not replace the analysis behind it) and *the last mile of polish* without its paid sharing service (your dashboard travels as screenshots or the free desktop file).

The working analyst runs the **relay**: Excel to meet and clean the data, SQL to query it at scale, statistics to judge it, Python to automate the repetition, Power BI to publish it. This book teaches the relay, in relay order.

3.4 The Free Table

Tool	Free version	Where	Part
Excel (or its twin)	LibreOffice Calc	libreoffice.org	II
SQL	SQLite + DB Browser for SQLite	sqlitebrowser.org	III
Statistics	By counting (pen, paper, Python later)	—	IV
SPSS / Stata	Trial versions; university labs; every idea taught free in Python too	spss.com, stata.com	IV
Python	Anaconda (Python + Jupyter + pandas, one install)	anaconda.com	V
Power BI	Power BI Desktop	microsoft.com	VI

Total cost of the complete toolkit: **\$0**. Chapter 4 installs it all in one sitting; keep this page as your shopping list.

3.5 Why This Order

The book's order is deliberate, and worth knowing so you can trust the stairs:

1. **Excel first** because seeing and touching tables builds the mental model every other tool assumes.
2. **SQL second** because it is the most-tested interview skill and the one that makes spreadsheets' limits vanish — and it *needs* the table-thinking Excel gave you.
3. **Statistics third** because once you can pull real data, the questions "is this real or luck?" start coming at you — and are answerable by counting.

4. **Python fourth** because automation is most appreciated after you have done the work by hand at least twice.
5. **Power BI fifth** because a dashboard is only as good as the analysis behind it — and now there is one.

If you arrive with one tool already (many readers know some Excel), the callouts let you skim your strong tool and lean on your weak ones.

From Your Toolkit — future you: every tool you learn makes the next one cheaper, because each is the same grammar in a new accent. The first tool is the hardest — which means the only one you must simply decide to start. Today, that is Chapter 4's afternoon of installing; by Chapter 13 you will have a working toolkit and a project to prove it.

Key Takeaways

- One shape under everything: the table — rows are things, columns are facts.
- The relay: Excel meets and cleans, SQL queries, statistics judges, Python automates, Power BI publishes.
- Every tool is bad at something; the team of them is the toolkit.
- Everything is free (Excel has a twin), and Chapter 4 installs it all in one afternoon.
- The book's order is the relay's order — each tool pays for the next.

Practice Lab

1. The table lens: write down three "tables" in your daily life (bank SMSs, WhatsApp messages, a shopping list). For each, define the row (one message? one conversation?) and three columns.
2. Match the tool to the job: for each request, name the part of the relay that owns it — "clean 500 rows of survey answers", "total sales by store for two years", "is the new shelf layout actually better?", "refresh this report every Monday", "show the board the trends".
3. The shopping list: open the free table in Section 3.4 and bookmark each download page — even before installing (Chapter 4 does that), knowing where they live is half the battle.
4. Tool autobiography: which tool are you already strongest in, and which scares you most? Write one sentence each — you will reread both in Chapter 62.

5. The relay audit: find one repetitive number-task in your work or studies and describe its five relay stages: how it is met, queried, judged, automated, and published today — or which stages are missing.

Further Reading

- Chapter 4 (install everything, this afternoon), Chapter 5 (meet Tariro and her data)
- The "tool comparisons" videos everywhere online — notice how they all confirm the relay: none of them says "one tool only".

Chapter 4: Set Up Your Workspace

Part I — Beginning: You, the Data Analyst

"Install once, learn forever: one afternoon of clicking, and your toolkit is complete."

In this chapter you will learn

- How to install every tool in this book, free, on any Windows or Mac laptop.
- The folder discipline that will save you hours every month of your career.
- How to open and save the practice data that comes with this book.
- Your first checkpoint: five tools installed and test-driven, error messages included.
- What to do when an install misbehaves (spoiler: it is almost always one of three things).

4.1 The Plan

Today is logistics, not learning — but it is the most important logistics of the book. An uninstalled tool is an unlearned tool; the week-three dropout almost always traces back to "I never got it running". Budget ninety minutes, make tea, and install in this order (each step is independent — a failure at one step does not block the others):

Step	Tool	Time	Your test that it worked
1	LibreOffice Calc (or use your Excel)	10 min	Open it, type 2 + 2 in a cell, press Enter
2	DB Browser for SQLite (includes SQLite)	5 min	Open it — you see an empty database window
3	Anaconda (Python + Jupyter + pandas)	30 min	Open "Jupyter Notebook", a browser page opens
4	Power BI Desktop	15 min	Open it — a big friendly welcome screen
5	The practice folder (this chapter)	5 min	You can find it in ninety days, and so could a stranger

Windows and Mac notes: take every default the installers offer — the defaults exist for people exactly like you today. If your laptop is old or full, the essentials are steps 1–3; Power BI can wait until Part VI, and its installer is the heaviest of the four.

4.2 The Folder Discipline

Before any practice file, the habit that separates professionals from the lost: **one home for your analytics work, with a fixed shape**. Create this once, on purpose, and use it for every chapter of this book:

```
DataAnalyst/          <- your home for everything
  01-Learning/         <- one folder per part of this book
    Part02-Excel/
    Part03-SQL/
    ...
  02-Projects/         <- the Part VII portfolio projects live here
    Tariro-Grocery/
  03-Data/             <- every data file you download or receive
    raw/               <- untouched originals, never edited
    clean/             <- your cleaned copies
  04-Notes/            <- the notebook habit from Chapter 2
```

Two rules give this structure all its power. **Rule one: raw/ is sacred.** The original file you receive — downloaded, emailed, exported — goes in *raw/* and is *never edited*. You work on copies; the original is your escape hatch when (not if) a cleaning step goes wrong, and your evidence when someone asks "where did this number come from?" **Rule two: names are self-explanatory.** *tariro-sales-2025-03.csv* lives forever; *final2_REAL.csv* dies in a bad meeting. Dates in names (YYYY-MM-DD sorts correctly), no spaces (some tools dislike them), lowercase-and-dashes everywhere.

4.3 The Practice Data

This book's running case (Chapter 5 introduces her properly) comes with generated data files: sales, stock, and customer records for a grocery store, with realistic patterns planted so your analyses can be checked against the truth. Get them the same way every working analyst gets data — from one folder, with a note of where it came from:

1. Create *DataAnalyst/03-Data/raw/tariro/* and copy the book's data files into it (the files accompany the book's repository; Appendix and chapter labs reference them as *03-Data/raw/tariro/sales.csv* and friends).
2. Open *sales.csv* in LibreOffice Calc or Excel — accept the import defaults, and you are looking at your first real table: one row per sale, columns for date, time, item, amount, and more.

3. Do nothing to it except look. Notice: it opens in a spreadsheet, because a CSV is just a table in its simplest costume (Chapter 6 explains CSVs properly).

From now on, every Practice Lab assumes this structure, and every lab's first line names its data like a professional: *raw folder in, clean folder out*.

4.4 Test-Driving: Your First Error Messages

Installers are done; now the five-minute test drive that makes each tool "yours". Crucially, **each test includes its failure mode** — because meeting your first error message today, in a safe room, is worth three hours of future panic.

- **Calc/Excel:** type `=2+2` — press Enter — 4 appears. Now type `=2+` — Enter — you get an error, something like `#NAME?`. Congratulations: you have broken a spreadsheet. The error is not punishment; it is the tool answering "I do not understand". Fix it, and note the feeling: error → read → fix is the analyst's heartbeat, in every tool, forever.
- **DB Browser:** open it, click "New Database", save `practice.db` anywhere in your `03-Data` folder. You now own a real database. Close it. That is all for today.
- **Jupyter:** open "Jupyter Notebook" (from the Anaconda menu). A browser tab opens with a file listing — click "New → Notebook: Python 3". In the empty box (a *cell*), type `2 + 2` and press Shift+Enter. The answer 4 appears below, and a new empty box waits. You have written a program. Deliberately type `2 +` and Shift+Enter now: read the red wall of text — the *bottom line* of a Python error always names the problem in plain words. This is the last time an error message is new to you.
- **Power BI:** open it, click "Get data → Text/CSV", point it at `sales.csv`, click Load. A table appears in a side panel. You have imported data into a professional BI tool. Close without saving.

If all five worked: your toolkit is complete, free, and paid for with one afternoon. That was the hardest purely-technical day of this book.

4.5 When Installs Misbehave

Almost every installation problem is one of three things, in this order of likelihood:

1. **The download was interrupted** — the file is partial. Fix: delete it, re-download, watch it finish.
2. **Your laptop said no** (permissions, disk space, an old operating system). Fix: right-click the installer and "Run as administrator" (Windows); free 10

GB of space; or update the OS if it is truly ancient.

3. **Antivirus enthusiasm** — a security tool "quarantined" something it did not understand. Fix: allow the app, or temporarily pause protection during the install.

Only after those three: search the exact error text with the tool name — the fix is almost always the first result, because a hundred thousand beginners hit the same wall this year and someone answered them kindly. Chapter 4's real lesson: **the answers exist, and reading error messages is a skill you have now started.**

Key Takeaways

- One afternoon installs everything: Calc/Excel, DB Browser (SQL), Anaconda (Python), Power BI Desktop — all free.
- Folder discipline from day one: `raw/` is sacred, names explain themselves, one home for everything.
- The practice data lives in your new structure; every lab starts by naming its files like a professional.
- Test-drive every tool *including* its error messages — you have now met the error-fix-fix heartbeat in the safest possible room.
- Install problems are almost always: partial download, permissions/space, or antivirus — in that order.

Practice Lab

1. Install all five steps and keep a log: for each tool, the time it took, and one thing that surprised you. (The log is your first notebook entry — Chapter 2's habit, running.)
2. Build the full folder structure from Section 4.2, by hand, exactly as drawn. Then move the practice data into `03-Data/raw/tariro/`.
3. The rename drill: five files with terrible names (`New folder (2).csv`, `SALES final FINAL.xlsx...`) — rename each to the standard: lowercase, dashes, date `YYYY-MM-DD`, no spaces.
4. Error safari: in Jupyter, deliberately cause three different errors (bad math, an unclosed quote, an undefined name) and write down what the *last line* of each message says. You are building the error-reading reflex early.
5. The ninety-day test: close everything, then try to find `sales.csv` in under thirty seconds without hunting. If you passed, your folder discipline is already paying.

6. Back it up: copy your whole `DataAnalyst/` folder to a memory stick or a cloud drive. Chapter 49 turns this habit into a portfolio; today it is just professional survival.

Further Reading

- Chapter 5 — meet Tariro, and start work as her analyst
- Anaconda's beginner tutorial (optional, 20 minutes — anything unfamiliar there returns in Part V anyway)

Chapter 5: Meet Tariro

Part I — Beginning: You, the Data Analyst

"Every dataset is somebody's livelihood wearing columns. This one is Tariro's."

In this chapter you will learn

- The running case that carries the whole book: Tariro's Grocery, Highfield, Harare.
- Her data — where it comes from, what is in each file, and what is wrong with it (on purpose).
- Her questions, which become every project in this book.
- Your role: hired analyst, with a notebook, a toolkit, and a boss who counts every dollar.
- The map: how each part of the book answers one of her questions.

5.1 The Shop

Tariro is thirty-four, runs a grocery store on a busy corner in Highfield, Harare, and has done so for six years — long enough to know her regulars by name, her suppliers' moods by voice, and her stock by walking past the shelves. She began with a paper notebook and a tin; today she has a secondhand laptop, a spreadsheet her nephew built in an afternoon two years ago, and a queue of decisions she makes by feel because "the numbers are somewhere but who has time".

Her dream is a second branch. Her bank manager, politely, wants evidence: *"Show me the numbers, Tariro. Which days earn? Which products pay? Who are your best customers? Then we talk about a second shop."*

You — having just installed a complete free analytics toolkit — are her new analyst. She can pay in thank-yous, story rights, and (Chapter 55) eventually a modest invoice, and she offers what every beginner analyst needs most: **a real business, real mess, real questions, and a boss who reads what you write.**

5.2 The Data

Tariro's nephew exported everything the till software remembers, and the book's data files are that export. Four tables, described the way you should always describe data (what one row *is*, and what the columns *mean*):

File	One row is...	Key columns
sales.csv	one item sold, one time	date, time, item, category, units, unit_price, amount, payment_type
stock.csv	one item's stock count, one week	week, item, units_on_shelf, units_in_store_room
customers.csv	one loyalty member	joined_date, name_initial, suburb, age_band, points
expenses.csv	one shop expense, one month	month, type (rent, wages, power...), amount

The data covers eighteen months of trading. And it is *deliberately imperfect*, because clean practice data teaches nothing but confidence:

- Some sales rows are duplicated (the till was restarted on two bad afternoons).
- Some amounts are missing or zero (a price was never entered).
- Payment types include three spellings of the same word (Ecocash, eco_cash, ECOCASH).
- The stock table is missing three weeks (the nephew's exam season).
- One month's expenses were recorded in a different currency order of magnitude — the rent row lost a zero.

Every one of these is a lesson you will meet in real employment within a month of starting: this book teaches you to *find* them (Chapter 8), *fix* them honestly (Chapters 8, 33), and *document* them (Chapter 45) so Tariro never doubts which number is which.

And underneath the mess, the data carries **planted patterns** — real, findable business truths, written down before the data was generated, so every analysis you do can be scored against the truth (the way no real job ever can):

- Sales spike on **payday weekends** (month-end and mid-month).
- **Fridays and Saturdays** out-earn other days by about a third.
- The **price rise on cooking oil** in month 9 caused a dip that partly recovered.
- **Sundays are marginal** — Tariro's biggest personal question.
- A core of **loyalty members** drives a disproportionate share of revenue, and some are drifting away.

5.3 Her Questions

Tariro's list, in her words — the questions every part of this book will answer with tools:

1. *"Am I actually making money, or just busy?"* — the monthly report (Part II: Excel).
2. *"Which products should I stock more of, and which are dead weight?"* — stock and sales analysis (Parts II-III).
3. *"Are Sundays worth opening?"* — comparing days honestly, including whether differences are luck (Part IV).
4. *"Did the cooking-oil price rise hurt me, or did it save me?"* — before-and-after, done properly (Part IV).
5. *"Who are my best customers — and am I losing them?"* — the customer analysis (Part VII, Project 4).
6. *"Can the report build itself? I asked you twice already."* — automation (Part V: Python).
7. *"Can I show the bank something they'll actually read?"* — the dashboard (Part VI: Power BI).

Notice: not one of her questions names a tool. That is the correct order of the world — **questions first, tools second** — and it is why Chapter 2's notebook habit exists. Write her list into your 04-Notes/ folder now, in her words; you will cross items off with satisfaction for four hundred pages.

5.4 Your Role and Your Rules

You are hired. The terms of your employment, agreed over a cold drink:

- **You work on copies** — her originals are sacred (Chapter 4's *raw/* rule; her nephew already lost one term's data to a mis-click).
- **You tell her what you did** — every cleaned file, every assumption, written in one page a human can read (this becomes Chapter 45's project documentation, practised from Part II onward).
- **You say when you do not know** — the beginning of the trust she will eventually send you to her bank manager with.
- **You charge nothing until Chapter 55** — when, properly skilled, you will price your first real invoice.

And one rule from her side that is a gift: **she asks "why?" about everything.** Not to doubt you — because she genuinely wants to learn. It will make you a better explainer than any course exercise ever could, which is Part VI's quiet goal.

5.5 The Map of the Journey

Where each part of the book touches her shop:

Part	Tariro's story	Your deliverable to her
I (now)	The meeting, the deal, the data	Your notebook, her question list
II — Excel	The messy spreadsheet, the first report	Project 1: the monthly report (Ch. 13)
III — SQL	Her data grows into a real database	Project 2: ten questions, answered (Ch. 21)
IV — Statistics	"Is that real, or just the way weeks go?"	The Sunday verdict; the price-rise verdict
V — Python	"You asked me twice" — the report automates	Project 3: the report that builds itself (Ch. 37)
VI — Power BI	The bank meeting	Her dashboard, and the story to go with it
VII — Projects	The deep questions	Projects 4–6, your portfolio
VIII — The Job	Your story, told with hers	CV, interviews, first 90 days
IX — Beyond	"What's next for you — and for my second shop?"	The gentle maps; the roadmap

From Your Toolkit — your notebook, again: before the next chapter, write the entry that every real engagement starts with: the client, the questions (her words), the data (the four files and what a row is in each), and the date. Sixty chapters from now, this page will be the first exhibit in your professional story — keep it.

Key Takeaways

- Tariro's Grocery is the book's client: real questions, messy data, planted answers — a safe rehearsal for employment.
- Four tables — sales, stock, customers, expenses — with documented flaws and documented truths.
- Her seven questions map to the whole book: reports, stock, Sundays, pricing, customers, automation, the bank dashboard.
- Your terms: copies only, document everything, admit unknowns — the professional habits, started free and cheap.
- Questions first, tools second: her list, in her words, is now in your notebook.

Practice Lab

1. Open each of the four data files and just *look* — five minutes each, no changes. For each, write one thing you noticed and one question it raises. (Find the duplicated afternoons. Find the missing weeks.)

2. Write Tariro's seven questions into your notebook in your own handwriting; add one question of your own that her data could answer.
3. The data card (your first professional document, 10 lines): for `sales.csv`, write the file's row-definition, its columns with meanings, the date range it covers, and its three flaws. You will update this card in every part of the book.
4. The first estimate, on purpose wrong: eyeball `sales.csv` and guess — which day of the week is biggest? Write it down and date it. Part IV will settle it statistically, and the comparison of your eyeball to the truth is itself a statistics lesson.
5. The contract: write the four terms of your employment from Section 5.4 as a one-page memo to Tariro, in plain language. Keep it — Chapter 55 prices the real version.

Further Reading

- Chapter 6 — thinking in grids; Tariro's spreadsheet gets its first cleanup
- Every part opener from here returns to the shop: keep her questions within reach.

PART II

Excel: Your First Superpower

Chapters 6 – 13

Chapter 6: Thinking in Grids

Part II — Excel: Your First Superpower

"Every tool in this book reads the same shape: one row per thing, one column per fact. Learn the shape once here."

In this chapter you will learn

- The anatomy of a spreadsheet: cells, rows, columns, sheets.
- The single most important rule in all of data work: one row per thing.
- What a CSV is, and why data arrives in them.
- How to open Tariro's data properly (and the one mistake that corrupts dates).
- Grid hygiene: the rules that keep a sheet trustworthy.

6.1 The Anatomy

Open LibreOffice Calc or Excel. What you see is a **grid**: columns lettered A, B, C..., rows numbered 1, 2, 3..., and every **cell** named by its crossing — B3 is column B, row 3. Along the bottom, **sheets** (Sheet1, Sheet2...) let you keep related tables in one file. That is the entire furniture of the tool. Everything else in Part II is what you *do* with this furniture.

Two habits make the grid yours immediately. First: **click any cell and read the formula bar** (the long box above the grid) — it shows what the cell really contains, not just what it displays. A cell showing "5" might contain `=2+3`, and knowing the difference is the beginning of analyst sight. Second: **Ctrl+arrow keys jump** — Ctrl+Down from A1 lands on the last filled row. Big data moves fast once you stop scrolling.

6.2 The Golden Rule: One Row Per Thing

Here is the rule that every tool, every chapter, and every employer assumes:

Each row is exactly one thing; each column is one fact about that thing.

In Tariro's `sales.csv`, one row = one item sold, one time. The row's columns are its facts: the date, the time, the item's name, how many units, the price, the total. In `customers.csv`, one row = one loyalty member. The rule sounds obvious and is violated everywhere within a week of anyone using a

spreadsheet — merged cells for pretty headers, two years of sales in the same row, one column holding both a name and a phone number. Every violation is a small tax paid later, usually by you, usually at a deadline.

The rule's quiet power: **questions become counts**. "How many sales?" = how many rows. "How many items do we sell?" = how many distinct names in the item column. "Revenue" = add up the amount column. Keep the rule and half of analytics is arithmetic on a table.

6.3 CSV: Data's Native Costume

The files in your 03-Data/raw/tariro/ folder end in .csv — "comma-separated values". A CSV is just a table saved as plain text: the first line names the columns, every following line is one row, and commas mark the column edges. Open `sales.csv` in Notepad (or any text editor) and look — then open it in Calc and see the same data in a grid. Same table, two costumes.

Why this matters: **every tool in this book opens CSVs** — Excel, DB Browser (Part III), Python (Part V), Power BI (Part VI). It is the universal exchange costume of data, and you will both receive and produce thousands of them.

The one classic mistake: opening a CSV by double-clicking can mangle **dates** (the program guesses the format — day-first or month-first — and sometimes guesses wrong) and **leading zeros** (item code "007" becomes 7, silently). The professional habit: open via **File → Open/Import**, look carefully at the preview step it offers, check that dates look like dates and codes kept their zeros, *then* confirm. Thirty seconds of checking beats an afternoon of wondering why March has 31 days of sales in a day column.

6.4 Opening Tariro's Data, Properly

The procedure (Calc and Excel are near-identical here):

1. File → Open, choose `sales.csv` from 03-Data/raw/tariro/.
2. The import dialog appears with a preview of the first rows. Confirm: commas are the separators; the first row is indeed column names (a header).
3. Scan the columns: `date` shows as a date; `item` and `category` as text; `units`, `unit_price`, `amount` as numbers. If anything looks wrong in the preview, fix it *in the dialog* — each column can be told its type explicitly.
4. Confirm, and **immediately Save As** an `.xlsx/.ods` file into 03-Data/clean/ — you never edit the raw original (Chapter 4's rule), and working in a copy in the clean folder is the workflow that rule describes.

Now look around the grid with analyst eyes: how many rows (Ctrl+Down)? Eighteen months of sales is tens of thousands of rows — real. What is one

row, in one sentence? Write that sentence on the data card you started in Chapter 5; it is the most important line on it.

6.5 Grid Hygiene: Five Rules

1. **One table per sheet** — no second table squeezed below the first; the empty rows convince humans, never tools.
2. **Row 1 is headers** — one short, self-explanatory name per column; no merged cells anywhere, ever, for any reason.
3. **No blank rows or columns inside the table** — gaps break sorting, filtering, and every tool that follows.
4. **One kind of thing per column** — a column is a date OR a number OR text; a column of mixed kinds is a bug in waiting.
5. **Notes live beside the table, not inside it** — if you must annotate, do it in a separate sheet called *notes*, with your cleaning log (Chapter 8 makes this a habit).

These five rules are what Chapter 69's professionals call a *data contract* — you are learning it now, in one sheet, so that warehouses later feel like familiar territory.

From Your Toolkit — future tools: the grid rule is the bridge to everything. When SQL's tables arrive (Chapter 14) you will meet the same rows and columns wearing a database; when pandas arrives (Chapter 33) its "DataFrame" is literally this grid, programmable; when Power BI loads (Chapter 39) it asks the same question every tool asks: *what is one row here?* Answer that and the tool works; dodge it and every tool, politely and eventually, breaks.

Key Takeaways

- The grid is columns (facts) × rows (things); the formula bar shows what a cell really contains.
- The golden rule: one row per thing, one column per fact — questions become counts.
- CSV is the universal costume of data; open it through the import dialog so dates and leading zeros survive.
- Raw is sacred: work on copies in *clean/*, log what you change.
- Five hygiene rules — one table per sheet, row 1 headers, no gaps, one kind per column, notes beside — make every later tool trust your data.

Practice Lab

1. Open `sales.csv` via the import dialog (dates and zeros checked), save the working copy to `clean/`, and write the one-sentence row definition on your data card.
2. Formula-bar safari: find one cell in the amount column; check what it displays versus what it contains. Find the last row with `Ctrl+Down` and note the row number — your first "how big is this?" number.
3. Rule audit: find any spreadsheet in your life (or build a tiny fake one) that breaks the five hygiene rules; list each break and what it would cost.
4. The CSV eye: open `customers.csv` in a text editor; identify the header line and describe, in words, what row 10's commas are separating.
5. Make a notes sheet in your working file and write your first entry: the date, what you opened, and one thing you checked.

Further Reading

- Chapter 7 (formulas — the grid starts computing), Chapter 8 (cleaning Tariro's real messes)
- LibreOffice Calc help: "Importing and Exporting CSV" (five minutes, saves you five hours someday)

Chapter 7: Formulas from Zero

Part II — Excel: Your First Superpower

"A formula is a sentence that starts with '=' and answers a question."

In this chapter you will learn

- Your first formulas: SUM, AVERAGE, COUNT, MIN, MAX — each in plain English.
- The formula that thinks: IF.
- Cell references, ranges, and the dollar signs that lock them in place.
- Percentages and growth, computed honestly.
- Formula hygiene: how to write formulas a stranger (or future you) can read.

7.1 The Equals Sign

Every formula begins with =. That single character tells the sheet: *"what follows is a question — compute the answer, show me the result."* Type =2+2 and press Enter; the cell shows 4. Click the cell again and the formula bar shows the question. This split — the cell shows the answer, the bar shows the question — is the whole personality of spreadsheets, and the reason they are the friendliest tool in this book: **you can see the machinery.**

7.2 The Starter Set

Each formula below, in one plain-English sentence, on Tariro's data (your working copy from Chapter 6):

Formula	Plain English	On Tariro's sales
=SUM(H2:H50000)	Add up everything in this stretch of column H	Total revenue for the range
=AVERAGE(H2:H50000)	The arithmetic middle: add up, divide by count	Average sale amount
=COUNT(H2:H50000)	How many cells contain numbers	How many priced sales
=COUNTA(B2:B50000)	How many cells contain anything	How many rows have an item name
=MIN(H2:H50000) / =MAX(...)	The smallest / largest	The smallest and biggest sale

The stretch H2:H50000 is a **range**: from cell H2 to cell H50000, everything between included. Ranges are how formulas talk about columns. Note COUNT

versus COUNTA: the first counts numbers only, the second counts anything non-empty — and their difference is already a finding (rows where the amount is missing, the flaw you know was planted in Chapter 5; Chapter 8 fixes it properly).

7.3 The Formula That Thinks: IF

=IF(question, answer_if_yes, answer_if_no) — one question, two outcomes:

```
=IF(H2 >= 20, "big basket", "small basket")
```

Read it aloud: *if the sale amount is 20 or more, write "big basket", otherwise "small basket"*. Fill it down a column (select the cell, then the little square at its corner, and drag — or double-click the square to auto-fill to the bottom) and you have just *classified* eighteen months of sales into two groups. Classification is a real, professional act — the baby version of the "segmentation" you will meet in Chapter 34 — and it is three words long.

IF grows with you gently: =IF(G2="Ecocash", "wallet", "cash"), and nested versions later (=IF(H2>=20, "big", IF(H2>=10, "medium", "small"))) — read as "if... otherwise if... otherwise". Whenever a nested IF starts hurting your head, that is the instinct that SQL's CASE (Chapter 20) and Python's logic (Chapter 32) will formalize — same thinking, better syntax.

7.4 References and the Dollar Signs

Formulas rarely contain numbers; they contain **references** — addresses of cells, so the answer follows the data. =H2*2 doubles whatever is in H2, and when copied down, the reference follows: row 3's copy says =H3*2, all the way down. This "relative reference" is why one formula can serve fifty thousand rows.

The dollar sign **locks** a reference. \$A\$1 means "always exactly A1, even when copied" — an **absolute reference**. The classic use: a threshold stored in one cell (say, P1 holds 20), and every row's formula says =IF(H2 >= \$P\$1, "big", "small"). Tomorrow Tariro changes the threshold in one place — P1 — and every formula updates. *One fact lives in one place* is a professional instinct that will save you from the spreadsheet classic: five copies of the same number, four of them updated.

7.5 Percentages and Growth

The two percentage sentences you will use forever:

- **"What share?"** — part divided by whole: =H2/SUM(\$H\$2:\$H\$50000) is this sale's share of revenue; format the cell as a percentage (the % button) so it reads naturally.

- **"How much did it grow?"** — new minus old, divided by old: if March total is in P5 and February's in P4, then $= (P5 - P4) / P4$ is the month's growth rate. Say the sentence aloud when you write it: *change, relative to where we started*. Growth on a tiny base is loud for unimpressive reasons ("sales doubled!" — from two customers to four), so the habit of asking *relative to what size?* starts here.

One warning from Chapter 2's honesty rules, now with tools: an AVERAGE over a wildly mixed table (Tariro's 2-cent airtime sachets next to \$60 bulk maize meal) describes neither. The cure is a filtered average — coming in Chapter 12 as AVERAGEIF, and it is why this chapter ends by asking you to notice what your averages average.

7.6 Formula Hygiene

- **Write so the bar explains the cell:** $=\text{SUM}(\text{sales_amount})$ (a named range) or $=H2*\$P\1 with a label beside P1 — future you is a stranger under deadline.
- **One fact, one place:** constants (thresholds, tax rates) live in labelled cells, referenced absolutely — never typed inside formulas.
- **Check the edges:** formulas filled to the bottom sometimes catch one row too many (a grand total row) — the classic doubled-total bug. Glance at the last row the formula reached.
- **Errors are messages:** $\#DIV/0!$ means you divided by an empty column (often: the range is one row off); $\#NAME?$ means a typo in a formula name; $\#N/A$ (coming in Chapter 9) means a lookup found nothing. Every error names its disease — treat them as the tool talking, not failing.

From Your Toolkit — the notebook: keep your ten most-used formulas in the back of your notebook, each with its one-sentence translation. In six months, Chapter 34 will hand you the same ten as Python functions (`sum`, `mean`, `count`, `np.where` — IF's twin), and the notebook page is the moment you recognise an old friend.

Key Takeaways

- A formula is a question after $=$; the cell shows the answer, the bar shows the question.
- The starter set answers real questions immediately: totals, averages, counts, extremes — on real data, today.
- IF classifies; nested IF is the seed of CASE and Python logic.
- References make formulas portable; $\$$ locks them; constants live in one labelled place.

- Percentages are two sentences (share, growth) — always asked "relative to what?"

Practice Lab

1. On your working copy: total revenue, average sale, count of sales, biggest and smallest sale — five formulas, five numbers on a new `answers` sheet, each labelled in plain English.
2. The IF drill: classify every sale into small/medium/big baskets using a threshold held in its own labelled cell; then change the threshold once and watch the column update.
3. The missing-count finding: write `=COUNTA(B2:B50000) - COUNT(H2:H50000)` and explain in one sentence what the answer tells Tariro about her data's health.
4. Growth: monthly revenue for the last six months (peek ahead: a small pivot or manual SUMs over date ranges), then the month-over-month growth rates; note which month looks odd and guess why (the planted price rise, month 9, may be near).
5. Break three formulas on purpose (`#DIV/0!`, `#NAME?`, a filled-range overshoot) and write down each error's disease and cure in your notebook's formula page.

Further Reading

- Chapter 8 (cleaning — fixing the rows your formulas just exposed), Chapter 12 (statistics in Excel)
- Excel/LibreOffice help: "Fill a formula down" — the drag-and-double-click habit that makes one formula serve a whole column

Chapter 8: Cleaning a Messy Sheet

Part II — Excel: Your First Superpower

"Analysis is 10% cleverness and 90% gardening."

In this chapter you will learn

- Why cleaning comes first, always — and why it is the analyst's real job, not a chore.
- Sorting and filtering: seeing the mess before fixing it.
- Duplicates: finding them, understanding them, deciding about them.
- Text tools: TRIM, PROPER, find & replace, text-to-columns — the three-Ecocash cure.
- Missing values: what to do, what never to do, and how to log everything.

8.1 The Gardener's Secret

Here is the sentence experienced analysts tell beginners in private: **you will spend more time cleaning data than analysing it, and the cleaning is where the value lives.** A flawless analysis on misunderstood data is a beautifully built house on a rumour; a modest analysis on honestly-cleaned data changes decisions. Tariro's data arrives with known, planted messes — this chapter finds and fixes them the way you will fix real messes for your whole career: *look, understand, fix, log.*

8.2 Look Before You Fix: Sort and Filter

Two clicks change your relationship with any table:

- **Sort** (Data → Sort): order the whole table by one column. Sort by `item` and you see every spelling variant of the same product huddled together — Tariro's tills know "Cooking Oil 2L", "cooking oil 2l", and "Cooking Oil 2L " (trailing space), and now so do you. Sort by `amount` and the smallest values rise to the top: the zero-amount rows, the missing prices.
- **Filter** (Data → AutoFilter — the funnel icon): puts a dropdown on every column header. Open `payment_type`'s dropdown and the *entire problem* appears as a list: Ecocash, eco_cash, ECOCASH, Cash, card, Card . Six categories that are really two.

Sorting and filtering are not yet cleaning — they are **seeing**. The professional sequence is always see → understand → fix: never fix what you have not seen whole, because the fix you cannot see is the fix you cannot trust.

8.3 Duplicates

Some rows in `sales.csv` appear twice — planted, and realistic (till restarted, export run twice). The tool: Data → Detect Duplicates (LibreOffice walks you through which columns to compare; Excel's Remove Duplicates is on the Data ribbon). But the professional questions come *before* the click:

- **Exactly duplicated?** — every column identical? Then it is almost certainly an export ghost: safe to remove *after counting them*.
 - **Same sale, slightly different?** — same timestamp and item but different amount? Now it might be a genuine second item, or a corrected re-entry.
- Deleting is now a decision, not a click** — and the decision gets logged.

Never remove duplicates silently. The cleaning log entry (your `notes` sheet, from Chapter 6) reads like this:

```
2026-03-12  Removed 412 exact duplicate rows from sales.csv
           (till export run twice on 2025-06-14 and 2025-09-02).
           Kept first occurrence. File: clean/sales-v2.xlsx
```

Four lines, and any future question — "why do last year's numbers differ from the report?" — has an answer with a date on it.

8.4 Text Tools: The Three Ecocashes

Text messes are so universal that the toolbox is small and permanent:

- **TRIM** — `=TRIM(D2)` removes sneaky leading/trailing spaces: "Card " becomes "Card". Invisible, rampant, and the cause of half of all "why won't these group together?" mysteries.
- **PROPER / UPPER / LOWER** — standardise capitalisation: `=PROPER("ecocash")` → "Ecocash". (Careful with names and codes: PROPER turns "mcdonald" into "Mcdonald" — standardise *deliberately*, not blindly.)
- **Find & Replace** (Ctrl+H) — the bulk cure: replace `eco_cash` → Ecocash, then `ECOCASH` → Ecocash. Two replacements, three categories become one. Do this on a copy, with the log entry.
- **Text to Columns** (Data → Text to Columns) — splits one column into several on a separator: the `date_time` column holding "2025-03-01 13:45" splits into a date and a time; "Chikafu, Maize Meal" splits into product and category. One fact per column (Chapter 6's rule) is often *achieved* here, not found.

The general law: **standardise to exactly one spelling per thing, and write the chosen spelling down**. The list of chosen spellings is Tariro's first *data dictionary* — five categories, one line each, on your `notes` sheet — and it is the beginner version of what professional teams call a data contract.

8.5 Missing Values: What and What Never

The zero and blank amounts (planted in Chapter 5, found by your COUNT gap in Chapter 7) are the classic first missing-data problem. The honest options:

1. **Leave blank, count them** — often correct: blanks mean "unknown", and unknown is information.
2. **Fix from evidence** — if `units × unit_price` can be reconstructed from the row itself (`=C2*D2` where both survive), the fix is arithmetic, and it gets logged.
3. **Flag, don't invent** — a helper column `=IF(H2="", "check", "ok")` marks the rows for Tariro's human knowledge (she remembers the till's bad afternoon).

The forbidden option: **silently replacing blanks with zeros** — it makes "we do not know" look like "nothing was sold", and the resulting average quietly lies. This is the spreadsheet version of Chapter 2's honesty rules, and (much later, Chapter 60) you will meet the professional typology of missingness — but the beginner's version fits in one line: **never turn a question mark into a zero.**

8.6 The Clean Copy and the Log

The workflow that ties the chapter together, and that you will repeat professionally forever:

1. **Copy** the raw original into `clean/`, versioned: `sales-v2.xlsx` (v1 was the straight import).
2. **See** (sort/filter) → **understand** (why does this mess exist? tills restart, exports double, humans type) → **fix** (dedupe, standardise, reconstruct) → **log** (every change, dated, in notes).
3. **Sanity-check the totals**: revenue after cleaning differs from before by exactly the removed duplicates' value — verify that arithmetic. A clean file whose totals cannot be reconciled to its raw parent is not clean, it is *mysterious*, and mysterious is a failure state.

From Your Toolkit — future tools: every fix in this chapter has a twin in your future stack. TRIM and the dedupe click return in SQL as `TRIM()` and `DISTINCT` (Chapter 16); pandas names them `.str.strip()` and `.drop_duplicates()` (Chapter 33); and the professional end-state — fixes written as code that reruns automatically — is Chapter 37's whole point. The buttons change; the gardening is forever.

Key Takeaways

- Cleaning is the job, not the chore — value lives in trustworthy data.
- See → understand → fix → log: sort and filter to see; never fix blind; never fix silently.
- Duplicates are a decision with a count; text messes are cured by standardising to one documented spelling.
- Missing means unknown: reconstruct from evidence, flag for humans, never fake a zero.
- Reconcile clean totals to raw — clean files must be explainable, not just tidy.

Practice Lab

1. Run the full workflow on `sales.csv`: find and remove the exact duplicates (log the count and dates), cure the three Ecocashes, TRIM the payment column, and reconcile the revenue total to the raw file — show the arithmetic.
2. Sort by `item` and list every spelling variant of the three messiest products; write Tariro's first data dictionary entry for one of them (chosen spelling, common variants, decision).
3. The missing amounts: count them, reconstruct what can be reconstructed with `units × unit_price`, and flag the rest; write the log entry that explains what a future reader sees.
4. Split a combined column (or manufacture one by merging date and time, then splitting) with Text to Columns; note one case where the split needs a second tool (e.g., trailing spaces after the comma).
5. The blank-to-zero trap, demonstrated: compute the average sale with blanks left blank, then again with blanks replaced by zero; write the two averages and one sentence to Tariro about which one is honest and why.

Further Reading

- Chapter 9 (joining tables — now that the sheets are clean), Chapter 60 (the professional missingness typology, when you are ready)
- "Data cleaning" chapters of any analytics text — you will recognise every page from this one afternoon.

Chapter 9: Joining Tables in Excel

Part II — Excel: Your First Superpower

"A join answers a question one table cannot: 'who are these people, and what else do we know about them?'"

In this chapter you will learn

- Why joins exist, in plain words, and the key idea that makes them work.
- VLOOKUP and XLOOKUP — the spreadsheet join, step by step.
- INDEX-MATCH: the classic upgrade, and why analysts still love it.
- The errors: #N/A, and what it is trying to tell you.
- The trap that multiplies money: the fan-out, seen here first.

9.1 The Question One Table Cannot Answer

Tariro asks her fifth question early, because a loyalty-card promotion is due: *"How much do my loyalty members actually spend, compared to everyone else?"* The sales table knows the spending but only records a membership code. The customers table knows the members — their suburbs, join dates, ages — but not what anyone spent. The answer lives in **both tables at once**, and the act of combining them is the single most important data operation in this book: the **join**.

The idea that makes joins possible is almost embarrassingly small: both tables carry a **key** — a column whose values identify the same thing in each. Here it is `member_id`: every loyalty sale row carries one, and every customer row is one. The join is "for each sale, find the customer row with the same `member_id`, and bring that row's facts across". Every join you ever run — in Excel today, SQL in Chapter 17, Python in Chapter 34, Power BI in Chapter 39 — is this sentence in a different accent.

9.2 VLOOKUP: The Workhorse

In your clean sales sheet, add a column: "suburb". Then, in the first data row:

```
=VLOOKUP(G2, customers!A:E, 3, FALSE)
```

Read it as a sentence with four parts: *look up* the value in G2 (this sale's `member_id`), *inside* the customers table (columns A to E), *bring back* column 3 of the matching row, and be *exact* about it (FALSE means exact match — always FALSE; TRUE is approximate-match, a setting with a dark side this

book will not let you use on keys).

Fill it down and every sale row now knows its customer's suburb: the join, done, in one formula. The modern **XLOOKUP** says the same thing more naturally — `=XLOOKUP(G2, customers!A:A, customers!C:C, "no match")` — and its fourth argument is the error handler we want next.

9.3 #N/A Is a Message

Some rows will show #N/A: the lookup found no match. Before you treat it as noise, read it — it is one of exactly three real situations:

1. **Genuine non-members** — many sales have no loyalty card at all; "not found" is correct and should become a clean label ("non-member"), not an error.
2. **A spelling or type mismatch** — the member_id "M0012 " (trailing space, Chapter 8's ghost) does not match "M0012". This is a data flaw, and the fix is TRIM, not the formula.
3. **The key itself is wrong** — a member_id in sales that exists in no customer row: a data-quality finding for Tariro's till.

The professional habit: wrap the lookup — `=IFNA(VLOOKUP(...), "no match")` — so the error becomes a label you can *count*, and then count it: 200 no-matches is a story (mostly non-members); 20,000 is a broken key. Chapter 2's skepticism, in formula form.

9.4 INDEX-MATCH: The Classic

Before XLOOKUP existed, analysts swore by INDEX-MATCH, and you will meet it in older workbooks on your first job, so you must read it:

```
=INDEX(customers!C:C, MATCH(G2, customers!A:A, 0))
```

Sentence: *MATCH* finds the row number where G2 appears in the id column (the 0 means exact); *INDEX* returns the value at that row in the suburb column. Two functions doing VLOOKUP's one job — but robustly: insert a column in customers and VLOOKUP's "column 3" silently becomes the wrong column, while INDEX-MATCH keeps aiming at what you named. Robustness by naming, not by position, is a professional instinct that follows you into SQL (where *naming* is the only way anything is ever joined).

9.5 The Trap: Fan-Out

One warning now, planted here because seeing it in a spreadsheet is the cheapest education you will ever get. Suppose instead of one sale per row, Tariro's export had one **line item** per row, and you joined it to a *monthly* discount table — one row per member per month. A member with 40 line items in a month now matches... 40 rows of sales × the month's single discount row? No — the direction that bites is the reverse: joining a many-row side to a table that *also* has many rows for the same key (say, two rows for one member: one old address, one new) **multiplies**: every sale matches *both* rows, and revenue silently doubles.

The spreadsheet version of the defence is the sanity check: **after any join, verify that a total you knew before (total revenue) is unchanged**. If revenue moved, you did not join — you duplicated. This exact trap, named **fan-out**, is the most common way real analysts corrupt real numbers on real jobs, and it is why the totals-reconciliation habit from Chapter 8 becomes a reflex here. When SQL formalizes joins in Chapter 17, you will meet it again with better tools — and recognise it instantly, which is the point.

From Your Toolkit — future joins: Excel's lookup is a *column-by-column* join; SQL's JOIN (Chapter 17) does whole tables at once with explicit directions (left, inner); pandas' *merge* (Chapter 34) is SQL's join in Python clothing; Power BI's data model (Chapter 39) draws the join as a line between tables you click once and reuse forever. Four accents, one sentence: *match the keys, bring the facts across, check the totals*.

Key Takeaways

- A join combines two tables on a shared key — one row per thing, matched, facts brought across.
- VLOOKUP/XLOOKUP do the spreadsheet join; always exact-match on keys; INDEX-MATCH survives column insertions.
- #N/A is a message: non-member, spelling mismatch, or broken key — count it, don't hide it.
- Fan-out is the money trap: joins can duplicate rows; totals must reconcile after every join or you copied, not joined.
- Four tools will speak this chapter; the sentence never changes.

Practice Lab

1. Join sales to customers on `member_id` with XLOOKUP or VLOOKUP+IFNA; label non-members; and count the three kinds of #N/A (genuine, trimmable, broken key) in your log.
2. The comparison answer: average sale amount for members vs non-members — Tariro's fifth question has its first number. Write the two-line answer for her, including the Chapter 2 caveat that members may always have been bigger spenders.
3. INDEX-MATCH rebuild: replace your VLOOKUP with INDEX-MATCH in a copy, then insert a column in customers and watch which formula survives. Write one sentence on why.
4. Fan-out, safely: build the trap on purpose — a tiny two-table example where one member has two customer rows — and watch revenue double; then fix it by deduplicating the lookup side first. (You will never forget it now.)
5. The key audit: count distinct `member_ids` in sales that appear in customers, and vice versa; write down what each direction's missing ids mean in business words.

Further Reading

- Chapter 10 (PivotTables — analyse the joined table), Chapter 17 (SQL joins — the same chapter, grown up)
- Excel help: "VLOOKUP troubleshooting" — every listed cause is a section of this chapter in miniature

Chapter 10: PivotTables — The Analyst's Best Friend

Part II — Excel: Your First Superpower

"A pivot table is a question you can twist: drop the numbers in, turn the handle, watch the table reorganise itself around a new question."

In this chapter you will learn

- The PivotTable in one idea, and the five clicks that build your first one.
- Rows, columns, values, filters — the four shelves, each explained plainly.
- The rearrangements: share-of-total, month grouping, and the drill-down.
- Reading a pivot like an analyst: where the planted patterns hide.
- The limits that send you to SQL later — and why that is fine.

10.1 The One Idea

You now have a clean sales table with customer facts joined on. Tariro's questions are all of the form "*totals of something, for each something else*" — revenue by month, sales by category, average basket by payment type, weekday versus weekend. A **PivotTable** is the tool that answers every question of that form *without formulas*: you tell it which facts to sum (values), which groups to sum them by (rows and columns), and which slices to hold aside (filters), and it builds the summary table. One pivot, re-twisted five ways, is a whole monthly report — this is why working analysts call it the best friend, and why interviews test it (Chapter 52).

10.2 Five Clicks to Your First Pivot

On your clean, single-table sheet (one table per sheet, headers in row 1 — Chapter 6's rules are load-bearing here; pivots are strict about hygiene):

1. Click any cell inside the table.
2. Insert → PivotTable (LibreOffice: Insert → Pivot Table). Accept the defaults (new sheet).
3. The field list appears: every column of your table, waiting. Drag *category* into **Rows**.
4. Drag *amount* into **Values** (it arrives as "Sum of amount" — exactly what we want).

5. Look: total revenue for every product category, in one table, zero formulas.

Then the twist that names the tool: drag `payment_type` into **Columns**. The table re-pivots — revenue by category *and* by payment type in a cross-tab. Drag it out, drag `date` into Rows instead — and when a date arrives, the pivot usually groups it by month/year automatically (if not: right-click the date → Group → Months). Revenue by month appears: the line behind Tariro's bank-manager chart, six clicks in.

10.3 The Four Shelves

Shelf	Plain English	Example
Rows	"one row for each..."	category, month, item
Columns	"...and one column for each..."	payment type, suburb
Values	"show me the total/average/count of..."	Sum of amount, Average of amount, Count of rows
Filters	"only look at..."	one suburb, one month, members only

Two details that double your power. **Values can be re-aimed**: click on "Sum of amount" and switch to Average, or Count — the same shelf answers "how much?", "how big typically?", "how many?". **Values can be stacked**: drop amount twice, set one to Sum and one to Count, and you have revenue *and* transaction counts side by side — divide them (a small formula beside the pivot) and you have average basket, the number every retailer breathes by.

And the analysis trick that feels like magic the first time: **double-click any number in the pivot** — the rows behind that number open as a new sheet. That "18,400 in March cooking oil" is now a list of the actual sales. This is the drill-down, and it is how pivots stay honest: every summary can be exploded back into its evidence.

10.4 Reading a Pivot Like an Analyst

Build the weekday question (Tariro's Sunday question deserves a first look here, properly settled in Part IV): `date` grouped by month in Rows — no: for weekdays, make a helper column first, `=TEXT(A2, "ddd")` (the day name), then pivot by it. Now read the table with Chapter 2's habits:

- **Compare, don't just see**: Friday and Saturday totals tower — but *by how much*? Switch Values to "Show Values As → % of Column Total": now the share of the week is explicit, and "the weekend is a third of everything" is a sentence you can defend.

- **Look for the plants:** payday spikes (compare the last week of each month's rows), the month-9 cooking-oil dip (category × month pivot), the loyalty share (member vs non-member columns on any measure).
- **Small categories lie quietly:** a category with 3 sales can show a spectacular average basket. Count and sum, side by side, always — the pairing habit that prevents nonsense.

10.5 Where Pivots End

Honest limits, so you know what Chapter 14 starts: pivots on a laptop get slow past a few hundred thousand rows; they do not *remember* the steps (redo the drag-dance monthly — until Chapter 37 automates it); they cannot join many tables at once (Chapter 39's data model does); and their "refresh" is manual (Data → Refresh — the button everyone forgets before the Monday meeting, once, and never again).

None of this subtracts from the friend. For the questions of a business Tariro's size — and, honestly, most businesses — the pivot is the analysis, and the rest is presentation.

From Your Toolkit — the same shelves everywhere: SQL's `GROUP BY` (Chapter 16) is the pivot's Rows shelf with values beside it; pandas' `.groupby()` (Chapter 34) is the same shelves in code; Power BI's whole visual system (Part VI) is a pivot per chart. When you meet each one, translate back to these four shelves and nothing in this book will ever feel new — only re-accented.

Key Takeaways

- The pivot answers every "totals of X by Y" question without formulas: five clicks, then twist.
- Four shelves: rows, columns, values (re-aimable and stackable), filters.
- Double-click any pivot number to explode it back into its rows — summaries stay honest.
- Read like an analyst: % of total for shares, sum beside count for trap-catching, and compare before concluding.
- Limits (size, memory, joins) are real and are exactly what SQL, Python, and Power BI are for.

Practice Lab

1. Build the category × payment_type revenue pivot; then re-twist to month × category; write the three sentences each table tells Tariro (planted patterns included).
2. Day-of-week pivot with the TEXT helper column: totals, then % of column, then Sum and Count side by side with an average-basket column beside them. Write your first — explicitly unofficial — verdict on Sundays.
3. The members pivot: revenue and counts by member vs non-member (Chapter 9's join feeding the values); note the share loyalty members take, and the caveat you must attach.
4. Drill-down audit: explode the biggest cell in your category pivot back to rows; find its largest single sale; write the two lines that connect summary to evidence.
5. The refresh lesson: change one amount in your source table, refresh the pivot, and record what did and did not change in your log — the habit that saves a Monday meeting.

Further Reading

- Chapter 11 (charts — the pivot's findings get their picture), Chapter 16 (GROUP BY — this chapter in SQL)
- Any "pivot table" tutorial video, ten minutes — use it to review the shelves, then twist a pivot no video suggests.

Chapter 11: Charts That Tell the Truth

Part II — Excel: Your First Superpower

"A chart is an argument drawn in ink: it claims something, and its axes are its evidence."

In this chapter you will learn

- The five charts that cover almost everything, and which to reach for first.
- How to build each in Calc/Excel, step by step, on Tariro's data.
- The three lies charts tell — and how to never tell them by accident.
- Titles that state the finding, not the furniture.
- The one-minute chart audit you will run forever.

11.1 Five Charts, Honestly

Almost every working chart is one of five shapes. The question in your head picks the shape:

Question you are answering	Chart	On Tariro's data
"How do these compare?"	Column (vertical bars)	Revenue by category
"How does it move through time?"	Line	Monthly revenue, the bank-manager chart
"Which is biggest, ranked?"	Bar (horizontal)	Top 10 items by revenue
"Are these two related?"	Scatter	Basket size vs time of day
"What is it made of?"	Stacked bar (rarely pie)	Payment types per month

The pie chart's demotion is deliberate: human eyes judge length well and angle poorly, so a bar of the same data is almost always clearer. Pies survive only for two-to-four slices where exact size matters less than the existence of shares ("about half Ecocash"). The scatter plot is the sleeper — Part IV's correlation chapter (28) lives inside it — and the line chart is the most-abused (its crimes are next).

11.2 Building Them

The universal recipe, twice practised and then yours:

- **Monthly revenue line:** pivot `date` (grouped by month) against Sum of `amount`; select the pivot; Insert → Chart → Line. Done: eighteen months of trading in one picture. Now the polish that matters: click the title and

replace the furniture with the finding — "Revenue: steady growth, payday rhythm, one dip" beats "Sum of amount by Month" the way a sentence beats a cough. Add axis labels (the chart tool's side panel) with units: "Revenue (USD)".

- **Category column chart:** pivot category by Sum of amount, Insert → Column. Sort the source pivot descending so the story ranks itself. When labels collide (15 categories), rotate to the horizontal bar — long names fit on the left edge, and ranking reads top-to-bottom like a league table.

Two build habits worth their weight in meetings: **delete what you are not using** (legends for single-series charts, gridlines competing with your data, 3-D effects — which distort every comparison they decorate); and **put the number where the eye lands** (data labels on the top three bars only; the rest is furniture).

11.3 The Three Lies

The three classic chart lies, each of which you will now recognise in the wild for the rest of your career — and never commit:

1. **The truncated axis.** A line chart of revenue from 9,100 to 9,400 looks like a rollercoaster if the y-axis starts at 9,000 — the *absolute* truth, the *relative* lie. Line charts of change should usually start from zero, or clearly flag that they do not; if a small move matters (control charts, Chapter 60's world), say so in the title.
2. **The unlabelled scale.** "Revenue (units? USD? thousands?)" — a chart whose axes are anonymous is a rumour with nice typography. Every axis: what, and in what units.
3. **The dual-axis trick.** Two lines, two axes, scaled until they cross wherever the author wishes — the most convincing lie in business. Dual axes are occasionally honest (very different units), but they demand a visible warning; mostly, two separate small charts tell the truth more cleanly.

11.4 Titles Are Findings

The professional rule, worth the whole chapter: **the title states what the chart shows, not what the chart is.**

- Furniture: "Monthly Sales 2025"
- Finding: "Sales grew 30% across 2025 — with a two-month dip after the oil price rise"

Write the finding title and the chart's purpose enters the room before you do; a deck of finding-titled charts can be *read* by someone who skips the meeting (Chapter 43's whole philosophy, learned here at small scale). If you cannot write a finding title, you may not yet have a finding — the title is a test, not a decoration.

11.5 The One-Minute Audit

Run this on every chart you make or meet — the same five questions as Chapter 2, in chart form:

1. **Compared to what?** — does the axis start where it should; is there a baseline series?
2. **Says who?** — where did the data come from (your title can carry it: "18 months of till data").
3. **What is missing?** — months absent, a category excluded, a legend doing quiet surgery.
4. **What was counted?** — revenue or transactions; members or sales; the chart's noun must be named.
5. **Could it be luck?** — the visible dip: real pattern or ordinary wobble? (The honest chart says "dip"; Part IV gets to say "significant".)

From Your Toolkit — the five forever: these five shapes and three lies are chart craft entire — Python's matplotlib (Chapter 35) draws them with different clicks, Power BI (Chapters 41–42) animates them with filters, but the grammar is this chapter's, permanently. Learn the argument once; every tool afterwards merely holds the pen.

Key Takeaways

- Five charts cover the working world: column, line, bar, scatter, stacked — the question chooses the shape.
- Build habit: finding titles, labelled axes with units, delete the furniture, numbers where the eye lands.
- The three lies: truncated axes, unlabelled scales, dual-axis crossings — recognise, avoid, call out.
- A finding title is a test: if you cannot title the finding, you may not have one.
- Audit every chart with the five questions — the same honesty habits, now drawn in ink.

Practice Lab

1. Build the two core charts on your cleaned data — monthly revenue line and ranked category bar — with finding titles, labelled axes, and furniture deleted. Paste both into a [report](#) sheet (they are Projects 1 and 6's opening exhibits).
2. Build the truncated-axis lie on purpose: your revenue line with the axis starting at 90% of its minimum; screenshot or print both versions and write the one-sentence warning you would give a reader of the lying version.
3. The scatter: basket amount vs hour of day; describe the relationship you see in one sentence (and hold your conclusion loosely — Chapter 28 teaches why).
4. The audit, outward: find one published chart (news, social, a report) and run the five questions on it; write your audit in five lines.
5. The title test: take your three lab charts and write a furniture title and a finding title for each; give both versions to a friend for thirty seconds each and ask what the chart "says" — record which title won.

Further Reading

- *How Charts Lie* — Alberto Cairo (the friendly, complete version of this chapter)
- Chapter 42 (chart craft's dashboard edition), Chapter 28 (the scatter plot grows up into correlation)

Chapter 12: Descriptive Statistics in Excel

Part II — Excel: Your First Superpower

"Before you can say 'unusual', you must be able to say 'typical'. Descriptive statistics is the craft of saying 'typical' without lying."

In this chapter you will learn

- The descriptive statistics any analyst must produce: mean, median, mode, range, and quartiles.
- Spread: standard deviation in plain words, and why two shops with the same average are not the same shop.
- Outliers: the $1.5 \times \text{IQR}$ rule, and what to do (and never do) about them.
- Excel's statistics functions and the Analysis ToolPak, hands on the sales data.
- The first three business questions answered with statistics, properly.

12.1 The Statistical Frame, in One Table

Statistics first asks four questions of any column of numbers, and Excel answers all four. On the `amount` column (every sale, eighteen months):

Question	Statistic	Excel	On Tariro's sales
What's typical?	Mean	<code>=AVERAGE(amounts)</code>	8.42
What's typical, robustly?	Median	<code>=MEDIAN(amounts)</code>	6.50
What's most common?	Mode	<code>=MODE.SNGL(amounts)</code>	5.00
How wide is the range?	Range	<code>=MAX - MIN</code>	0.50 - 412.00

And the fifth, the most important in the set: `=QUARTILE(range, 1)` and `=QUARTILE(range, 3)` — the values with a quarter and three quarters of the data below them. Together they give the **five-number summary** (min, Q1, median, Q3, max), the fastest honest portrait of any column ever invented: "half of all sales are between 4 and 11 dollars."

12.2 Why the Median Gets Its Own Chapter

The mean is fragile: one 412-dollar bulk order drags Tariro's average sale from about 6 to about 8.4 — the mean moved; the shop did not. The **median** (line up every sale; take the middle one) ignores that whale entirely, which is why it is the honest headline for skewed money: incomes, house prices, basket sizes. When mean and median sit far apart, the data is telling you it is lopsided (a long tail of small sales and a short one of whales) — and your report should say which number it is quoting and why (Honesty Rule 1: an average is an answer, not the answer).

The mean still matters — it is the only "typical" that interacts correctly with *totals* (mean $\times$ count = total, the reconciliation identity you used in Chapter 8). The craft is the pair: report the mean when totals matter, the median when typicality does, and both when they disagree enough to be a finding.

12.3 Spread: Same Average, Different Shop

Standard deviation, `=STDEV.S(amounts)`, in plain words: **the typical distance from the mean**. A small one means sales cluster tightly (a dependable newsagent rhythm); a large one means swings (a shop living on windfalls). Two hypothetical shops, both averaging 8.40: Shop A's sales almost all between 7 and 10, Shop B's half at 2 and half at 15. Same mean; utterly different business — inventory, staffing, cash-flow risk. The mean without the spread is half a portrait.

The spread statistic you will actually *use* most is the **IQR** — $Q3 - Q1$ — because it is robust like the median (whales do not move it), and because it powers the outlier rule, next.

12.4 Outliers: The $1.5 \times \text{IQR}$ Rule

The rule, used identically in Excel, SQL, Python, and SPSS (four accents, one sentence): a value is an outlier if it lies beyond

fences = $Q1 - 1.5 \times \text{IQR}$ and $Q3 + 1.5 \times \text{IQR}$

On Tariro's amounts: $Q1 = 4$, $Q3 = 11$, $\text{IQR} = 7$, so fences at -6.5 and 21.5 — every sale above 21.50 is flagged, about 4% of rows: the bulk cooking-oil orders, the December party season, one 412.00 that is *correct* (a school's term-start order — you rang and checked; Chapter 2 says the number is a claim, and some claims survive checking).

What to do with outliers, in order: **investigate** (what is it? bulk buyers are a *segment*, not an error), **report separately** ("sales excluding bulk orders"), **cap only with a stated reason** (winsorising, a Part IV word), and **never silently delete** — the deleted whale is the lost customer. Excel's **Analysis**

ToolPak (File → Options → Add-ins → Analysis ToolPak; LibreOffice: Data → Statistics) does all of this in one dialog: Data Analysis → Descriptive Statistics → Summary statistics, and the five-number summary, mean, and standard deviation arrive as a table.

12.5 Three Questions, Answered Properly

Statistics earns its place by answering Tariro's standing questions better than the pivot alone could:

- **"Is my Sunday trade weak?"** Median basket by weekday (a pivot, Values set to Median where supported, or `MEDIAN(IF(...))` as an array formula): Sundays are not *low-basket*, they are *low-traffic* — fewer customers, spending typically. The pivot said "small"; statistics says *why* it is small. Different decisions follow (marketing vs pricing).
- **"How risky is the month?"** Monthly revenue, mean and standard deviation across eighteen months: the mean gives the expectation, the spread the risk — a month 40% below the mean is within this shop's ordinary wobble, and Tariro can stop panicking about it (or not panic *enough*, once Part IV teaches what "ordinary" really means).
- **"Who are my whale customers?"** The 4% beyond the fence, listed by `member_id` (the join from Chapter 9): a named, checkable list — not a vibe.

From Your Toolkit — the portable five: mean, median, quartiles, IQR fences, standard deviation travel with you everywhere: SQL's `PERCENTILE_CONT` and `AVG` (Chapter 16), pandas' `.describe()` (Chapter 34, literally this chapter as one method), SPSS's Frequencies/Descriptives (Part IV), Power BI's DAX measures (Chapter 40). The five-number summary is one of the few things every tool in this book computes — because it is one of the few things every analyst needs.

Key Takeaways

- Five numbers portrait any column: min, Q1, median, Q3, max — build it before you believe any average.
- Mean for totals-thinking, median for typicality, both when they disagree (and they will, on money).
- Standard deviation = typical distance from the mean; IQR = the robust spread that powers the fence.
- Outliers: investigate → report separately → cap with reasons → never silently delete.

- The best analyses pair a statistic with a *why*: small Sundays are traffic, not basket.

Practice Lab

1. Five-number summary of `amount` (functions or ToolPak), plus mean and standard deviation; write the two-sentence portrait of a typical sale, quoting both typicals and explaining the gap.
2. Fence the outliers: compute the $1.5 \times \text{IQR}$ fences, count and list the flagged rows, classify each as bulk-buyer, seasonal, suspicious, or whale; annotate the 412.00 with its verified story in your log.
3. The two-shops experiment: hand-build two small columns with the same mean and different spreads; compute STDEV for both; write the one-sentence business warning spread carries and averages cannot.
4. Weekday statistics: median basket and transaction count by day of week; write the traffic-vs-basket verdict on Sundays in exactly two sentences, each carrying a number.
5. Whale roll: the outlier list joined to customers (Ch. 9), grouped by suburb in a pivot; name the three decisions Tariro could make from this one page.

Further Reading

- Chapter 22 (the ideas get their theory), Chapters 24–25 (distributions and the tests), SPSS/Stata equivalents (Part IV)
- *Naked Statistics* — Charles Wheelan (the friendly tour of everything this chapter begins)

Chapter 13: Project 1 — Tariro's First Monthly Report

Part II — Excel: Your First Superpower

"Analysis unwritten is analysis undone. The report is where numbers become decisions."

In this chapter you will learn

- How to plan a deliverable backwards from the decision it must support.
- The structure of the one-page monthly report: headline, three findings, one recommendation, appendix.
- How to assemble it from the work of Part II — no new analysis, all new craft.
- A self-review checklist that catches weak reports before your reader does.
- How to publish the artefacts (PDF, Excel file, one folder) like a professional.

13.1 Plan Backwards from the Decision

Tariro will read this report once, on Sunday evening, tired, deciding three things: **what to order more of, what to do about Sundays, and whether the loyalty scheme pays.** Every choice you make — every chart, every sentence — either serves one of those decisions or is furniture. That is the professional's planning trick, used on every deliverable for the rest of this book: **start from the decision, work backwards to the analysis, and let the audience's questions write your outline.** (Chapter 45 turns it into the portfolio case-study method; Chapter 52 into interview answers.)

The deliverable: **one page** (paper or PDF), three findings, one recommendation each, two charts, and an appendix sheet for the evidence. One page is not a limit — it is a kindness. If the reader wants more, the appendix exists; if they do not, you have still told them everything that matters.

13.2 The Analysis You Already Own

Nothing new is needed — Part II was the analysis; this chapter is assembly. Your evidence shelf:

- **Growth:** monthly revenue pivot and line chart (Ch. 10-11) — the steady climb, the payday rhythm visible as sawtooth.
- **The dip:** category × month pivot — cooking oil's slide from month 9, quantity-led (Ch. 8's split of the 8.42 into price × quantity).
- **Weekdays:** day-of-week statistics (Ch. 12) — Sundays are traffic-poor, not basket-poor.
- **Loyalty:** member vs non-member comparison (Ch. 9-10) — members' share of revenue and their bigger average basket, with the self-selection caveat.
- **Whales:** the outlier list (Ch. 12) — bulk buyers as a segment, concentrated in two suburbs.

13.3 The One-Page Structure

The template that carries every one-page report you will ever write — and Project 4's quarterly edition, and most of your first job's weekly output:

TARIRO'S GROCERY — MONTHLY REPORT

Month 18, compiled from 18 months of till data

HEADLINE (one sentence, a number in it)

Revenue grew 31% over 18 months; growth is broad-based
but cooking oil has fallen 40% since the price rise.

FINDING 1 — What to order more of [chart: top items bar]

Staples drive 62% of revenue (top 3 categories). Maize meal
and rice trend with the growth. RECOMMEND: increase standing
order for top 5 items 20%; watch supplier terms.

FINDING 2 — The cooking-oil problem [chart: category x month]

Cooking oil revenue -40% since month 9, all quantity-led:
customers buy 1.8 bottles/month vs 3.1 before. RECOMMEND:
trial a cheaper brand for one month; measure switch rate.

FINDING 3 — Sundays and loyalty [chart: weekday columns]

Sunday revenue is 28% below the weekday average, entirely
from fewer customers (baskets are normal). Members are 23%
of customers but 31% of revenue. RECOMMEND: Sunday-only
loyalty double-points; measure after 4 Sundays.

APPENDIX (separate sheet): method notes, data-quality log,
full tables, the 412.00 sale's verification note.

Notice what the structure enforces: **every finding ends in a recommendation, every recommendation is measurable** ("measure switch rate", "after 4 Sundays") — because a recommendation nobody can check is a wish. And the headline is a *finding*, not a table of contents: the reader learns the story even if they read nothing else (Chapter 11's finding-title rule, promoted to a whole page).

13.4 Write the Sentences

The craft moves that separate your report from a spreadsheet printout:

- **Numbers in every sentence, units attached.** "Cooking oil has fallen 40% since the price rise" — not "cooking oil has declined significantly."
- **The caveat travels with the claim.** "Members are 23% of customers but 31% of revenue" is followed immediately by "members may always have been bigger spenders — the scheme's *causal* effect is unknown until Part IV's tests." One sentence of honesty, forever attached.
- **Past tense for what happened, present for what is true, future tense only under the word "recommend".** Grammar that keeps you out of trouble.
- **No chart without a finding title; no finding without a number; no number without its source** ("18 months of till data, cleaned per log").

13.5 The Self-Review Checklist

Before any report leaves your hands — this one, and every deliverable in this book, and your first job's work — run the checklist. It is Chapter 2's habits, assembled into a quality gate, and it is the closest thing this book has to a professional conscience:

1. **Could Tariro act on this?** (every finding → a recommendation → a measurable next step)
2. **Does the headline survive alone?** (read only the headline: is the story there?)
3. **Can every number be traced?** (appendix method notes; the reader could rebuild it)
4. **Are the caveats attached, not buried?** (self-selection, 18-month window, the data log's unresolved items)
5. **Is anything a chart that should be a table — or a sentence?** (charts show shapes; tables give exact values; sentences give verdicts)
6. **The three rules:** an average is an answer not the answer (median shown beside mean); correlation is not causation (the loyalty caveat); sample is not population (18 months is this shop, not all shops).
7. **One page?** (if it spilled, cut a finding, not a caveat.)

13.6 Publish Like a Professional

Artefacts, in one folder named for the deliverable and date (tariros-monthly-report-m18/):

- `report.pdf` — the one-pager, exported (File → Export as PDF), charts embedded.
- `analysis.xlsx` — the workbook: raw, clean, customers, analysis (pivots), report (layout) — one tab per stage, named, so a colleague could follow the trail.
- `log.txt` (or a log tab) — the data-quality log and method notes: every cleaning decision, the verification note for the 412.00 sale, the fence computation, the join checks. **The log is the part that makes you look senior.** Juniors deliver numbers; professionals deliver numbers plus the story of how much to trust them.
- A filename convention that sorts: `tariros-monthly-report-m18` — not final, not final2_REAL. (Version like a professional: v1, v2, or dates; "final" is a lie we have all told.)

13.7 Where You Stand

Close the workbook for a second and look at what happened: eighteen months of messy, human till data became three decisions a shop owner can make on Sunday evening. You did it with one tool, no code, no statistics beyond descriptives — and every habit that will carry you through SQL, Python, and Power BI is already in your hands: clean first, reconcile totals, join and check, summarise honestly, chart the finding, recommend something measurable, and log the whole journey.

Tariro's shop is about to get a second till and a supplier database — and the data is about to outgrow the spreadsheet. Time for a bigger engine.

From Your Toolkit — the report spine: this one-page structure — headline, findings, recommendations, appendix, log — is the deliverable format of every project chapter remaining (Projects 1–6 and the portfolio). The tools underneath change (SQL, Python, Power BI); the page does not. Master this page and you have mastered the shape of the job.

Key Takeaways

- Plan backwards from the decision the reader must make; the audience's questions write your outline.
- One page: headline finding, three findings each with a measurable recommendation, appendix for evidence.
- Numbers with units, caveats attached to claims, finding titles on charts — sentence craft is analysis craft.
- The self-review checklist is your quality gate: actionable, traceable, caveated, one page.

- Deliver artefacts like a professional: PDF + workbook + log, versioned, in one named folder.

Practice Lab

1. Build the full deliverable on your Part II workbook: the one-page report (headline, three findings with recommendations, two charts with finding titles), the appendix sheet, and the log tab. Export to PDF.
2. The readability test: hand it to a friend for ninety seconds; ask them to state the three findings back. What they recall is what you communicated; what they miss is what you must fix.
3. The decision test: for each recommendation, write the measurement that would prove it right or wrong, and when that measurement will exist. A recommendation without a measurement date is a wish — fix any wishes.
4. Run the full self-review checklist on your own report, writing one line per item; fix everything the checklist catches, then re-export.
5. Portfolio seed: write the 150-word case-study note for this project now, while it is fresh (what, how, finding, impact) and store it with the folder — Chapter 45 will want it.

Further Reading

- Chapters 44–45 (this deliverable becomes a portfolio case study)
- *The Pyramid Principle* — Barbara Minto (the deep theory of headline-first structure)

PART III

SQL: The Language of Data

Chapters 14 – 21

Chapter 14: What a Database Is

Part III — SQL: The Language of Data

"A spreadsheet is a whiteboard; a database is a filing system with a guard at the door."

In this chapter you will learn

- Why Tariro's data outgrew the spreadsheet — the honest limits, in her story and in general.
- Tables, rows, columns, keys — the database vocabulary, mapped to what you already know.
- What a relational database is, and why "relational" is the point.
- SQLite and DB Browser for SQLite: a complete database engine, free, in fifteen minutes.
- How to load one of Tariro's CSVs and see it as a database table.

14.1 The Day the Spreadsheet Broke

Month 19: Tariro's grocery adds a second till, a part-time cashier, and — the real change — a **supplier database** her wholesaler emails monthly. Her workbook now has sales from two tills that must not be double-counted, stock counts from a Sunday ritual, customer sign-ups on a tablet, and supplier prices in a fourth file. The workbook becomes five files, then eight, then "final_v7_REALLY.xlsx". One afternoon the part-time cashier sorts one column of one sheet without selecting the row labels: three months of amounts detach from their dates. Nobody notices for a week.

That afternoon is why databases exist. The spreadsheet's strengths — see everything, touch everything — become its failures at scale: it stores *data* and *layout* and *analysis* in one fragile object; it lets any cell hold anything; and it has no memory of who changed what. A **database** separates storage from presentation, enforces rules at the door (this column holds dates; this key is never empty), and is built for many tables that stay consistent. Analysts do not abandon spreadsheets — reporting still lives there — but the *source of truth* for anything that grows moves to a database, and the analyst's job is to query it.

14.2 The Vocabulary

A **table** is a grid you already understand, with rules attached. Map it across:

Spreadsheet	Database	The difference that matters
Sheet	Table	One kind of thing per table: sales, customers, stock, suppliers
Row 1 headers	Column names	Defined once, enforced; no merged cells, ever
Data row	Row (record)	One sale, one customer — no totals rows mixed in
The whole file	Database (schema)	Many related tables, named and governed
"that ID column"	Key (primary/foreign)	The join idea from Chapter 9, formalised

Two kinds of key, and they are the whole of "relational": a **primary key** uniquely identifies each row in its own table (`customer_id` in `customers` — one row per customer, the column never repeats, never empty); a **foreign key** is a column in another table that *points* at it (`customer_id` inside `sales`, repeating freely — one customer, many sales). The relationship lives in the pair: customers-to-sales is **one-to-many**, the most common shape in all of business data. This is exactly the VLOOKUP key from Chapter 9 — the database just gives it a name, a rule, and a guard.

14.3 Relational: The Point

The **relational model** — one table per kind of thing, keys connecting them — is the design principle behind nearly every database you will meet at work. Its power is that each fact is stored **once, in its rightful place**: the customer's suburb lives only in `customers`; a sale records only `customer_id`. Update the suburb once, and every query that joins sees the change — no eight-file hunt, no copy drifting out of date. Its discipline is that you must *ask* rather than *browse*: the database does not show you everything; it answers questions. The asking language is SQL, next chapter.

Tariro's shop, designed relationally, is four small tables:

```
customers(customer_id, name, suburb, joined, loyalty_tier)
sales(sale_id, date, till, customer_id, payment_type, amount)
stock(week, item, quantity_on_hand)
suppliers(item, supplier, unit_cost, last_price_change)
```

Read the parentheses and you can see the whole business: who shops (customers), what they bought (sales), what is on the shelves (stock), what it costs (suppliers). The keys in the middle — `customer_id` in `sales` — are the threads that tie them. Four modest tables will carry every question in Part III and most of the rest of the book.

14.4 SQLite and DB Browser

Databases you will hear named at work — MySQL, PostgreSQL, SQL Server — are *servers*: engines installed somewhere, connected to over a network. They matter (Chapter 57's world), but learning is better served by **SQLite**: a complete, industrial-grade relational engine that keeps each whole database in a single file and powers most of the world's phones. There is nothing to install as a service, nothing to configure, and the SQL you learn on it is 95% identical everywhere.

The companion tool is **DB Browser for SQLite** (sqlitebrowser.org; free, Windows/Mac/Linux): a friendly window onto database files — browse tables, edit values, and above all run SQL queries and see results. Install both habits now:

1. Download and install DB Browser for SQLite (five minutes, no account).
2. Open it → New Database → save as `tariros.db` in your `tariro-data` folder.
3. **File → Import → Table from CSV file** → your cleaned `sales.csv`. Tick "Column names in first line". DB Browser guesses types — check the guess (Chapter 4's lesson again: dates as dates, amounts as numbers) and let it create the table.

Double-click the `sales` table in the list: there it is — eighteen months and two tills of trading, now a governed table with typed columns and no detached-sort accidents possible. That feeling — the data now *holds still* — is what a database sells.

From Your Toolkit — one engine, many accents: SQLite is your learning engine here, and Python ships it inside (`sqlite3`, no install — Chapter 33 reads this same `tariros.db` with code), Power BI connects to it directly (Part VI), and its file format is how this book's appendix datasets ship. At work the engine will be named PostgreSQL or SQL Server — the chapter SQL transfers almost line for line (Appendix A notes the dialect differences). The model — tables, keys, relations — never changes at all.

Key Takeaways

- Spreadsheets mix storage, layout, and analysis; databases separate them and enforce rules at the door.
- One table per kind of thing; primary keys identify rows, foreign keys point across tables; one-to-many is the default shape.
- The relational model stores each fact once — updates propagate, copies cannot drift.

- SQLite + DB Browser: a real relational engine and a friendly workbench, free, in minutes; your `tariros.db` is born.
- Servers (PostgreSQL, MySQL, SQL Server) are the work version of the same idea — the SQL and the model transfer.

Practice Lab

1. Sketch on paper the four tables of Tariro's shop; mark each primary key and each foreign key with an arrow; write under each arrow the relationship in words ("one customer → many sales").
2. Install DB Browser for SQLite, create `tariros.db`, and import your cleaned `sales.csv` as a table; open the table and confirm row count matches your workbook (reconciliation habit — always).
3. Import `customers.csv` the same way; check that DB Browser's type guesses are right (member ids stay text, `joined` is a date, suburb text); fix any column it mistyped and re-import.
4. Design on paper (do not build yet): where would a `tills` table or `staff` table plug in? Which existing tables would grow a foreign key? Two sentences each.
5. The guard, demonstrated: in DB Browser, try typing text into the `amount` column of a row and saving; record what the engine says. Write one sentence on why the spreadsheet never argued with you — and who paid for that silence.

Further Reading

- Chapter 15 (SELECT, WHERE — the asking begins), Chapter 20 (the work databases by name)
- sqlitebrowser.org — the tool's own quick tour

Chapter 15: Your First Queries

Part III — SQL: The Language of Data

"SQL is the closest thing data has to a lingua franca: one small grammar, spoken by every database on Earth."

In this chapter you will learn

- What SQL is and why it has ruled for fifty years.
- The first query — SELECT, FROM — and reading a table in the language.
- Narrowing with WHERE, ordering with ORDER BY, limiting with LIMIT.
- Text, numbers, and dates in WHERE — the comparison habits that do 80% of the work.
- NULL: the value that is not a value, and why SQL is strict about it.

15.1 The Language

SQL — Structured Query Language, pronounced "sequel" or "ess-cue-ell"; both are fine, arguments are a hobby — is how you ask a relational database questions. You describe the result you want; the engine decides how to fetch it. You do not loop, you do not hunt cell by cell: you *declare*. That declarative style is why SQL has survived every fashion since 1974 — and why, alone of the tools in this book, SQL appears in some form in virtually every data job posting you will ever read (Chapter 50's numbers).

The grammar fits on a napkin. Every query in this chapter is built from five keywords:

```
SELECT column, column  -- what you want back
FROM table              -- from which table
WHERE condition         -- which rows only
ORDER BY column        -- in what order
LIMIT n;               -- how many rows
```

Type these into DB Browser's **Execute SQL** tab (your `tariros.db` from Chapter 14, sales imported) and press the run button. Welcome to the language.

15.2 SELECT and FROM

The simplest honest query — all of it, every column, every row:

```
SELECT * FROM sales;
```


The asterisk means "every column" — fine for a first look, bad as a habit: real tables are wide, real queries name their columns, and naming is documentation. The professional form:

```
SELECT sale_id, date, till, amount
FROM sales;
```

Column order is yours to choose; duplicates are removed if you ask (`SELECT DISTINCT payment_type FROM sales;` — three payment types, one query, no pivot needed). SQL keywords are conventionally UPPERCASE and SQL does not care about case for keywords, but *does* care about your data's case: 'Ecocash' and 'ecocash' are different values (the cleaning chapter's ghost, now with teeth).

15.3 WHERE: Choosing Rows

WHERE keeps only the rows that pass a test, and the tests are the arithmetic you already know: `=`, `<>` (not equal), `<`, `<=`, `>`, `>=`.

```
-- one day's takings, both tills
SELECT * FROM sales WHERE date = '2025-06-14';
-- the big baskets
SELECT sale_id, date, amount FROM sales
WHERE amount >= 50;
-- everything except the bulk-order whale
SELECT * FROM sales WHERE sale_id <> 4123;
```

Text takes quotes, numbers do not — 'Ecocash' is a value, 50 is a number, and mixing the habits is the first bug everyone writes. Dates take ISO quotes ('2025-06-14') — year-month-day, the unambiguous international form; Chapter 8's date hygiene pays its first dividend here, because a database will *refuse* the ambiguous, which is exactly what you want.

The combiners, and the parentheses that make them safe:

```
-- cash sales over 20 dollars -- AND means both must hold
SELECT * FROM sales
WHERE payment_type = 'Cash' AND amount > 20;
-- weekend rows -- OR means either holds
SELECT * FROM sales
WHERE weekday = 'Sat' OR weekday = 'Sun';
-- a range, written either way
SELECT * FROM sales
WHERE date >= '2025-06-01' AND date <= '2025-06-30';
SELECT * FROM sales
WHERE date BETWEEN '2025-06-01' AND '2025-06-30';
```

And the one that replaces a hundred spreadsheet filters — the membership test:

```
SELECT * FROM sales
WHERE payment_type IN ('Cash', 'EcoCash', 'Card');
```

15.4 ORDER BY and LIMIT

ORDER BY sorts the result — ASC (default, smallest first) or DESC (largest first) — and it can sort by several keys in sequence: ORDER BY amount DESC, date ASC ranks sales by size and breaks ties by date. LIMIT caps the rows returned, which turns any query into a top-list:

```
-- the ten biggest sales in eighteen months
SELECT sale_id, date, amount
FROM sales
ORDER BY amount DESC
LIMIT 10;
```

That query — biggest, smallest, first, latest, top N — is in every analyst's daily rotation; ten rows and the whale hunt from Chapter 12 is already underway. Together with WHERE it completes the working core: **choose the rows (WHERE), choose the columns (SELECT), choose the order (ORDER BY), cap it (LIMIT)** — the grammar of "look".

15.5 NULL: Not Zero, Not Blank

One SQL idea has no spreadsheet twin and must be learned properly: **NULL** — a value that means *unknown or absent*. Not zero (zero is a known amount: a free item, a voided sale); not an empty string (which is a known nothing). NULL is the database saying "this fact is missing". Tariro's data carries them honestly: non-member sales have NULL customer_id; the three missing stock weeks are NULL rows, not zero rows — a distinction Chapter 23's statistics and Chapter 25's sampling will lean on.

The trap that catches every beginner: **NULL fails every comparison**. WHERE customer_id <> 'C0102' does *not* return the rows where customer_id is NULL — unknown is not unequal, it is unknown. The test is its own keyword:

```
SELECT * FROM sales WHERE customer_id IS NULL;      -- the non-members
SELECT * FROM sales WHERE customer_id IS NOT NULL; -- the members
```

Count both. The NULL discipline — know them, name them, choose deliberately whether they are excluded — is a marker of professional SQL, and interviewers probe it deliberately (Chapter 52).

From Your Toolkit — one grammar, five tools: this five-keyword core is the same SQL spoken by Python's pandas (`.query()`, Chapter 34), spoken to Power BI's data model (Chapter 39), and shadowed by spreadsheet filters. Master `SELECT-FROM-WHERE-ORDER BY-LIMIT` and you can read half of every code example in the rest of this book — and, one day, half of the queries left behind by the analyst whose job you inherit.

Key Takeaways

- SQL is declarative: describe the result, the engine fetches — five keywords do the looking.
- Name your columns; `SELECT *` is for exploring, not for delivering.
- Text and dates take quotes (ISO dates), numbers don't; `AND/OR/IN/BETWEEN` with parentheses do the choosing.
- `ORDER BY + LIMIT` = top-lists and whale hunts; sort is cheap and reversible.
- `NULL` is unknown, not zero — it fails every comparison; test with `IS NULL`, and count what you exclude.

Practice Lab

1. The first five: run `SELECT * FROM sales LIMIT 5`, then name your columns; then write queries for the ten biggest sales, the ten smallest, and the first five sales Tariro ever recorded (`ORDER BY date ASC`).
2. The day report: one query for all sales on the most recent Saturday, ordered by amount descending; another for that day's Ecocash sales only; reconcile both totals against a pivot of the same day in your workbook — the numbers must match, or write down why.
3. The membership split: count `NULL` and non-`NULL` `customer_id` rows (two queries with `IS NULL`); write the sentence comparing your counts to the "no match" labels from Chapter 9's join — they should tell one story.
4. The range habit: sales in the last full month three ways — two comparisons with `AND`, `BETWEEN`, and one `IN` list of dates; confirm all three return the same row count.
5. Whale hunt, sequel: amounts over the Chapter 12 fence (21.50) with `ORDER BY DESC`; paste the top ten into your log and annotate the bulk buyers you already know by name.

Further Reading

- Chapter 16 (GROUP BY — the pivot, in SQL), Appendix A (the cookbook: every query pattern from this book, ready to copy)
- sqlbolt.com — interactive lessons 1–4 for extra reps of this chapter

Chapter 16: Counting and Grouping

Part III — SQL: The Language of Data

"The question is never 'what is the total?' It is 'what is the total, for each what?'"

In this chapter you will learn

- The aggregates — COUNT, SUM, AVG, MIN, MAX — and the star habit of grouping.
- GROUP BY: the pivot table's engine, exposed.
- HAVING: filtering groups after they are built, and why WHERE cannot.
- GROUP BY's execution order — the mental model that prevents most beginner errors.
- The rounding trap, the NULL count, and other honest-details.

16.1 The Aggregates

SQL's aggregate functions collapse many rows into one number, and their names are their job descriptions:

```
SELECT COUNT(*)      AS n_sales,  
       SUM(amount)   AS total_revenue,  
       AVG(amount)    AS avg_basket,  
       MIN(amount)    AS smallest,  
       MAX(amount)    AS largest  
FROM   sales;
```

One query, the whole shop: 6,420 sales, 54,013.50 total, 8.41 average, 0.50 smallest, 412.00 largest. The *AS* clauses give the result columns clean names — cosmetic, and a professional habit (reports read by others name their columns).

Two details before anything else. **COUNT has two forms with different meanings:** `COUNT(*)` counts rows; `COUNT(customer_id)` counts non-NULL values — on Tariro's data, all sales versus member sales only, a distinction that is an analysis (the loyalty share, one query). **AVG of amounts ignores NULLs** — usually what you want, occasionally a trap (a table of NULL-heavy discounts averages the known, not the whole); know which you are computing, and say so in the report.

16.2 GROUP BY: For Each What

A bare aggregate answers the whole table. Business questions are almost always "for each what": revenue by month, sales by payment type, average basket by weekday. **GROUP BY** builds the "for each": it splits the table into groups — one per distinct value of the grouping column — and runs the aggregates *inside each group*:

```
-- the monthly revenue pivot, in one query
SELECT  substr(date, 1, 7) AS month,
        COUNT(*)          AS n_sales,
        SUM(amount)        AS revenue
FROM    sales
GROUP BY substr(date, 1, 7)
ORDER BY month;
```

Read it aloud: *for each* month, the count and the revenue. Eighteen rows appear — the exact table a pivot produced in Chapter 10, now reproducible forever with one saved query instead of a re-dragged dance. Swap the grouping column and the query re-twists exactly like the pivot: `GROUP BY payment_type`, `GROUP BY weekday`, `GROUP BY till`. The pivot's four shelves, translated: Rows is `GROUP BY`, Values is the aggregates, Columns is a second grouping column (`GROUP BY weekday`, `payment_type` — every combination gets a row), Filters is `WHERE`.

One law, unbreakable, and the source of most beginner errors: **every column in SELECT must be either grouped or aggregated**. `SELECT weekday, amount FROM sales GROUP BY weekday` is refused by the engine — and rightly: weekday has seven values, amount has thousands, and no honest table can show both unaggregated. The error message is the database teaching you its model; read it as a lesson, not an insult.

16.3 HAVING: Filtering Groups

Tariro's next question: *which categories average over 15 dollars a sale?* — a filter on an average, which is a filter **after** grouping. `WHERE` cannot do this (`WHERE` runs before groups exist), so SQL provides `HAVING`, the group-filter:

```
SELECT  category,
        COUNT(*)    AS n_sales,
        AVG(amount) AS avg_basket
FROM    sales
GROUP BY category
HAVING  AVG(amount) > 15
ORDER BY avg_basket DESC;
```

The pair to remember: **WHERE chooses rows before grouping; HAVING chooses groups after.** A query can carry both, and often does — small sales (WHERE) excluded before the category averages are computed (HAVING), which changes the answer, and the analyst chooses the order deliberately:

```
SELECT  category, COUNT(*) AS n, AVG(amount) AS avg_basket
FROM    sales
WHERE   amount <= 21.50          -- drop the whales FIRST
GROUP BY category
HAVING  COUNT(*) >= 50          -- then keep busy categories
ORDER BY avg_basket DESC;
```

16.4 The Execution Order in Your Head

SQL is written in one order and executed in another, and holding both orders in your head is the difference between guessing and knowing:

```
Written:  SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
Executed: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

Read the executed order as a story: gather the tables (FROM), keep the rows that qualify (WHERE), build the groups (GROUP BY), keep the groups that qualify (HAVING), compute the selected columns (SELECT), sort, cap. Every confusing query becomes readable by walking it down this pipeline — and every error message ("no such column" in HAVING for something computed in SELECT) makes sense once you know SELECT happens *after* HAVING. (Aliases from SELECT are invisible to WHERE and HAVING; they exist for ORDER BY — or repeat the expression.)

16.5 Honest Details

- **The rounding trap.** `AVG(amount)` on cents can return 8.419859227... — the engine is not wrong, it is exact. Round for display (`ROUND(AVG(amount), 2)`), never for computation; rounding intermediate results accumulates error the way floats did in Chapter 7, and reports that don't add up come from exactly this habit.
- **The NULL group.** `GROUP BY customer_id` produces a NULL group — the non-members, grouped as one "customer". It appears at the top or bottom of your results; name it in the report ("non-member sales") rather than letting a reader ask what NULL means.
- **Percent shares.** "What share of revenue is each payment type?" needs each group's sum over the total — a self-join of aggregates that is the natural first CTE, which is exactly where Chapter 18 goes. You can feel the need for it here; the tool is two chapters away, and that feeling is the best predictor that a tool will stick.

From Your Toolkit — the same habit everywhere: this chapter is the pivot table, formalised — and it arrives again as pandas' `.groupby()` (Chapter 34), as Power BI's implicit measures (Chapter 40: drag `amount` in, choose Sum or Average — you are writing this query), and as the GROUP BY you will someday optimise on a million-row table. The "for each what" habit — total, count, and average, grouped, sorted, and checked — is the single most-used move in working analytics. Everything else is decoration.

Key Takeaways

- Five aggregates: COUNT (two meanings: rows vs non-NULLs), SUM, AVG, MIN, MAX.
- GROUP BY answers "for each what": one row per group, aggregates inside; every selected column grouped or aggregated.
- WHERE filters rows before grouping; HAVING filters groups after — order is analysis, choose it deliberately.
- Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT; walk queries down the pipeline.
- Round for display only; name the NULL group; and let the hunger for shares lead you to Chapter 18.

Practice Lab

1. The shop on one line: the five aggregates over all sales; then the same five *for each till* — and write the two-sentence till comparison (the second till is new: what does its presence do to comparisons over time?).
2. Rebuild Chapter 10's pivots as saved queries: month × revenue, category × revenue (ordered descending), weekday × count and average basket. Reconcile each against the workbook pivot totals — all three must match to the cent.
3. The payment story: revenue and row share by payment_type with COUNT(*) and SUM; add COUNT(customer_id) to the same query and explain the difference between the two counts in one sentence.
4. HAVING, both ways: categories averaging over 15; categories with at least 100 sales; then one query with both a WHERE (exclude whales) and a HAVING — and write one sentence on how the answer differs from the no-WHERE version.
5. The execution-order drill: take your longest query from this lab and annotate each clause with its position in the executed pipeline; then deliberately break it (add an unaggregated column to SELECT with

GROUP BY), read the error aloud, and write down what the engine was teaching you.

Further Reading

- Chapter 17 (joins — the groups get richer), Chapter 18 (CTEs — the share-of-total question, solved)
- Appendix A (the grouping patterns, ready to copy), sqlbolt.com lessons 12–13 for reps

Chapter 17: Joins Without Fear

Part III — SQL: The Language of Data

"In a relational database, the join is not a feature. It is the reason the database exists."

In this chapter you will learn

- The join in SQL: FROM two tables, ON the keys.
- INNER vs LEFT — the two joins that cover 95% of real work.
- Fan-out, formally: how joins multiply rows, and the totals check that catches it.
- Many-to-many, and the bridge table that resolves it.
- Aliases, qualified names, and reading other people's joins.

17.1 The Join, Grown Up

Chapter 9 taught the sentence: *match the keys, bring the facts across*. SQL says it in one line — the join lives in the FROM clause:

```
SELECT s.sale_id, s.date, s.amount, c.name, c.suburb
FROM   sales s
JOIN   customers c ON c.customer_id = s.customer_id;
```

Three new notations, all habit-forming. The **alias** (`sales s`) shortens every later mention — one letter per table is the working convention. The **qualified name** (`s.amount`, `c.suburb`) says which table a column comes from; SQL requires it when two tables share a column name (`customer_id` exists in both) and professionals write it always — a query that survives next month's schema change is worth two extra characters per column. And **ON** states the join condition: the key match, in exactly the spreadsheet's sense.

17.2 INNER and LEFT

JOIN (bare) means **inner join**: only rows that match on both sides survive. Every sale with a matching customer appears, enriched with name and suburb; every *non-member* sale — `customer_id NULL` — vanishes, because `NULL` matches nothing (Chapter 15's rule, with consequences). The inner join answers "what do we know about both?" and silently amputates the unmatched.

The **LEFT JOIN** keeps every row of the left table, matched or not, and fills the missing right-side columns with `NULL`s:

```
SELECT s.sale_id, s.amount, c.suburb
FROM   sales s
LEFT JOIN customers c ON c.customer_id = s.customer_id;
```

Non-member sales now appear, suburb NULL — the honest full picture of the till. The choice between the two is an *analysis decision*, not a syntax preference: revenue from all sales (LEFT) versus revenue from identifiable customers (INNER) are different numbers answering different questions, and the wrong choice quietly biases a report. **Default to LEFT when the left table is the subject** ("all sales, enriched where possible"); use INNER when only matches are meaningful ("customer purchases").

The other two named joins — RIGHT (mirror of LEFT) and FULL OUTER (keep everything from both sides) — exist for completeness and appear rarely; when you need one, you will know, and Appendix A has it.

17.3 The Anti-Join

One join pattern earns its own section because it is pure professional gold: rows on the left with *no* match on the right. "Customers who have never bought anything", "items in stock that never sold", "member_ids in sales that exist in no customer row" (Chapter 9's broken key, found by query):

```
SELECT s.customer_id, COUNT(*) AS orphan_sales
FROM   sales s
LEFT JOIN customers c ON c.customer_id = s.customer_id
WHERE  c.customer_id IS NULL
GROUP BY s.customer_id;
```

The trick: LEFT JOIN keeps the unmatched, and WHERE c.customer_id IS NULL keeps *only* the unmatched. Data-quality auditing, cold-customer lists, unreconciled invoices — the anti-join is the pattern behind all of them, and it is the single most common "how would you find...?" interview question you will meet in SQL rounds.

17.4 Fan-Out, Formally

Now the trap from Chapter 9.5 gets its real name and mechanism. Joins multiply when *both* sides can repeat a key. One-to-many is safe in one direction: sales-to-customers joins many sales (left) to one customer each (right) — each sale appears once. But if the right side *also* repeats the key — say a `customer_discounts` table with two rows for one customer (an old offer and a new one) — then each of that customer's sales matches *both* rows, and appears *twice*: 40 sales become 80 rows, and `SUM(amount)` doubles the customer's revenue. That is **fan-out**: row counts inflate, and every aggregate after the join is quietly wrong. **Many-to-many** — keys repeating on both sides — is fan-out waiting to fire.

The defence is a habit, not a talent:

1. **Count rows before and after.** `SELECT COUNT(*) FROM sales` before the join; count after. If the after-count exceeds the left table, you fanned out.
2. **Reconcile a known total.** Total revenue before the join must equal total revenue after — the Chapter 8 reflex, now load-bearing. If revenue moved, you duplicated, not joined.
3. **Aggregate before joining when you can.** Sum the many-side down to one row per key *first* (a subquery or CTE — next chapter), then join the tidy results. "Aggregate, then join" is the professional's default pattern for exactly this reason.

The **bridge table** resolves true many-to-many relationships by design: a customer can buy many items, an item is bought by many customers — so sales *line items* sit between them (`sale_items(sale_id, item, quantity)`), turning two impossible joins into two safe one-to-many joins. When you design, or inherit, a schema, look for the bridges; their absence is where fan-out breeds.

17.5 Reading Others' Joins

Inherited queries — the analyst before you left SQL behind, and now it is yours — are read with a method. Walk the FROM clause first, not the SELECT: list the tables and aliases on paper, draw a line for each ON (a little diagram: boxes, lines, arrowheads at the "many" side), and label each join INNER or LEFT. The picture *is* the query's meaning: which rows survive, which tables are decoration, where the fan-out risk sits. Then read WHERE, then the aggregates — the Chapter 16 pipeline, applied to someone else's work. Chapter 20 turns this into a full survival guide; the diagram habit starts here.

From Your Toolkit — joins everywhere, same law: pandas'

`merge(how='left'/'inner')` (Chapter 34) is this chapter wearing Python; Power BI's model view (Chapter 39) draws the boxes-and-lines diagram for you and guards cardinality ("many-to-many" warnings are fan-out warnings); Excel's lookups were one-column joins. The law never varies: **match the keys, check the row counts, reconcile the totals**. Analysts who internalise it are the ones whose numbers survive audit.

Key Takeaways

- The join lives in FROM: alias the tables, qualify the columns, state the key match in ON.
- INNER keeps matches only; LEFT keeps all left rows — the choice is an analysis decision, not syntax.
- LEFT JOIN + WHERE right-key IS NULL = the anti-join: unmatched rows on demand (audits, cold lists, broken keys).
- Fan-out = keys repeating on both sides; count rows, reconcile totals, aggregate-then-join; bridges resolve many-to-many.
- Read inherited queries by diagramming FROM first — boxes, lines, and which side is "many".

Practice Lab

1. Enrich: every sale with customer name and suburb (LEFT JOIN); reconcile total revenue and row count against the un-joined sales table — both must match exactly; write the two numbers in your log.
2. The comparison, properly: average basket for member sales vs non-member sales via `c.customer_id IS NULL` grouping on the LEFT JOIN — this is Chapter 9's loyalty question, now in one query; write the caveat sentence beside the number, as always.
3. The anti-join audit: orphan member_ids in sales with no customer row (the query above); count them; cross-check against the broken-key count in your Chapter 9 log — one story, two tools, same numbers.
4. Fan-out on purpose: create a tiny `customer_discounts` table (DB Browser → Create Table) with two rows for one member; join sales to it; watch revenue double; then fix it by aggregating discounts to one row per customer before joining. Log both revenue totals — this pair of numbers is the lesson.
5. Diagram drill: draw the four-table boxes-and-lines picture of Tariro's schema (customers, sales, stock, suppliers) with arrowheads at the "many"

ends; hand your diagram to a friend and have them read the relationships back to you from the picture alone.

Further Reading

- Chapter 18 (CTEs — "aggregate, then join" made easy), Chapter 34 (merge — this chapter in pandas)
- Appendix A (join patterns incl. FULL OUTER and self-joins)

Chapter 18: Queries in Steps — Subqueries and CTEs

Part III — SQL: The Language of Data

"No one writes a hundred-line query. Everyone writes five twenty-line queries and glues them together."

In this chapter you will learn

- Why big queries fail as queries and succeed as pipelines.
- The subquery: a query inside a query, and where it may stand.
- The CTE — WITH — the professional's building block, step by step.
- Solving the share-of-total problem that Chapter 16 left open.
- Reading and debugging layered queries without fear.

18.1 The Case for Steps

Here is a real question, mid-sized: *"Which payment types take more than their fair share of revenue — and how does that differ between the first and second till?"* As one flat query it is a monster: two groupings, two filters, a share computation, a comparison. Written in one breath it cannot be read, cannot be debugged, and — the working analyst's real criterion — cannot be *changed* on Friday afternoon when the question shifts.

The answer to complexity in SQL is not cleverness; it is **decomposition**: write small queries that each do one thing, and connect them. SQL gives you two connection mechanisms. The **subquery** nests a query inside another, where a value or a table is expected. The **CTE** (Common Table Expression, the **WITH** clause) names each step and stacks them like paragraphs. Same engine under both; different ergonomics. Professionals reach for CTEs by default, and you will see why in three paragraphs.

18.2 The Subquery

A subquery is a complete query in parentheses, used in one of three slots:

In WHERE, where a value is expected — "sales bigger than the average sale":

```
SELECT sale_id, amount
FROM   sales
WHERE  amount > (SELECT AVG(amount) FROM sales);
```

The inner query runs first, returns one number (8.41), and the outer query uses it. This is a *scalar subquery* — one row, one column — and it can stand anywhere a number could.

In FROM, where a table is expected — "average *per month*, then the best months": aggregate first, query the aggregate second (the "aggregate, then join" pattern from Chapter 17, in its simplest form):

```
SELECT month, revenue
FROM   (SELECT substr(date, 1, 7) AS month,
          SUM(amount)           AS revenue
        FROM   sales
        GROUP BY substr(date, 1, 7)) AS monthly
ORDER BY revenue DESC
LIMIT 5;
```

The inner query builds a table named `monthly` — invisible, unpersisted, existing only for this query — and the outer query treats it like any table. This is the mechanism, plain and complete; the CTE is the same thing with better manners.

In SELECT, where a column is expected — a correlated subquery, one value per row, discussed with its costs in the Further Reading pointers; rare in beginner work, recognised on sight in inherited code (a subquery in SELECT that mentions the outer table's alias).

18.3 The CTE: WITH, and Paragraphs

The CTE names each step, up front, in the order a human reads:

```
WITH monthly AS (
    SELECT substr(date, 1, 7) AS month,
           SUM(amount)       AS revenue
    FROM   sales
    GROUP BY substr(date, 1, 7)
),
best AS (
    SELECT * FROM monthly ORDER BY revenue DESC LIMIT 5
)
SELECT * FROM best;
```

Read it as a recipe: *with* monthly revenue in hand, *with* the best five months chosen, show them. Each CTE is a named, testable paragraph; the steps stack (later CTEs may use earlier ones); and the final SELECT is the conclusion of the argument. The differences from subqueries are ergonomics and they matter at work: CTEs **run top-to-bottom like the story they tell** (subqueries hide inside-out), they **can be tested one at a time** (run the WITH block alone — DB Browser shows you `monthly` — then add the next step),

and a CTE can be *reused* twice in one query, which is exactly what the share-of-total problem needs.

18.4 Share of Total, Solved

Chapter 16 ended hungry for this: each group's share of the whole. The CTE solution is the pattern's canonical shape — build the grouped numbers, build (or reuse) the total, divide:

```
WITH by_payment AS (
    SELECT payment_type,
           SUM(amount) AS revenue
    FROM   sales
    GROUP BY payment_type
),
with_total AS (
    SELECT payment_type,
           revenue,
           SUM(revenue) OVER () AS total_revenue
    FROM   by_payment
)
SELECT payment_type,
       revenue,
       ROUND(100.0 * revenue / total_revenue, 1) AS pct_share
FROM   with_total
ORDER BY revenue DESC;
```

Two things to notice. The `SUM(...) OVER ()` is a window function — a one-line preview of Chapter 19 that computes the total *without collapsing the rows* (empty `OVER` brackets means "the whole table is my group"). And `100.0 *` — the decimal point forces decimal arithmetic; with integers `100 * revenue / total` would truncate, and shares would sum to 97% mysteriously (a Chapter 7 lesson in SQL clothing).

The recipe generalises to the most-asked family of interview questions: *top X within each group* ("top-selling item per category", "busiest day per month"). Aggregate, rank (Chapter 19's `ROW_NUMBER`), filter the ranking — three CTE paragraphs, one answer. You now hold every ingredient but the ranking verb.

18.5 Debugging Layers

Layered queries are debugged layer by layer — this is precisely why they exist. The method, always the same:

1. **Run each CTE alone, from the first.** Does `by_payment` look right? (Seven rows would not; three payment types should.) Check each layer's row count and total against a known number — the reconciliation habit, now

applied to intermediates.

2. **Add layers one at a time.** The step that breaks the query is the step you just added; that is a much smaller problem space than "the query is wrong".
3. **Read errors from the inside out.** "No such column: revenue" in the second CTE means the first CTE did not expose a column by that name — check the SELECT list of the step before, not the step that complained.
4. **When stuck, materialise.** Paste a CTE's output into a scratch table (`CREATE TABLE scratch AS SELECT ...`), and query the scratch — the layers become inspectable objects. Delete scratch tables when done; your future self inheriting the database thanks you.

From Your Toolkit — pipelines, named: the CTE habit — decompose, name, test each step, stack — is how *all* serious analysis code is written: pandas chains each step into a named variable (Chapter 32), Python functions decompose like CTEs (Chapter 31), Power BI's Power Query Editor is literally a stack of named, reorderable, testable steps on your data (Chapter 39). SQL taught you decomposition because SQL could not hide complexity; every later tool inherits the discipline.

Key Takeaways

- Decomposition over cleverness: small queries, connected, beat one heroic query — always.
- Subqueries slot into WHERE (value), FROM (table), SELECT (correlated, rare); CTEs name the steps.
- CTEs read top-to-bottom, test one layer at a time, and can be reused — the professional default.
- Share-of-total: grouped CTE, total via `SUM() OVER ()`, divide with a decimal point; generalises to top-X-per-group.
- Debug by layers: run each step alone, add one at a time, read errors inside-out, materialise when stuck.

Practice Lab

1. Rewrite your Chapter 16 category query as a two-CTE pipeline: grouped revenue, then rank; run each CTE alone in DB Browser and screenshot or note both intermediate tables.
2. Share of revenue by payment type (the query above), then by suburb via a join to customers — and write the two sentences the shares tell, including the NULL/non-member handling you chose.

3. The fair-share question that opened the chapter, answered: with CTEs, revenue share by payment type *for each till*, side by side; one sentence on whether the tills differ and whether the difference matters (say how you decided).
4. Top item per category: aggregate item revenue within category, and identify the top of each — you may need a window function peek ahead (Chapter 19) or a self-join; either way, document your steps as CTEs and reconcile the grand total.
5. The refactor drill: take your longest flat query from any Practice Lab so far and rebuild it as named CTEs, one job each; then hand both versions to a friend and time which one they can explain back to you faster. That time difference is the entire argument for this chapter.

Further Reading

- Chapter 19 (window functions — the ranking verbs this chapter reached for)
- Appendix A (the CTE patterns: share-of-total, top-per-group, gap/island queries)

Chapter 19: Window Functions

Part III — SQL: The Language of Data

"GROUP BY folds rows into one. A window function walks along the rows and leaves every one of them standing — carrying new knowledge."

In this chapter you will learn

- The OVER clause: partition, order, and the "moving window" idea in plain words.
- Running totals and moving averages — the shop's rhythm, computed.
- Rankings: ROW_NUMBER, RANK, DENSE_RANK — and which to reach for.
- LAG and LEAD: the previous and next row, and month-over-month change.
- The pattern that completes Chapter 18: top-N within each group, in one query.

19.1 The Idea

Chapter 16's aggregates collapse: seventeen hundred June rows become one row, one SUM. Sometimes that is the deliverable. But the shop's most human questions keep the rows and *annotate* them: "add up my revenue so far", "rank these items within their category", "compare this month to last month". These are aggregate-flavoured computations that must not collapse anything — and that is a **window function**: an aggregate (or rank, or offset) computed over a *window* of rows, with the result attached to every row, rows intact.

The whole grammar is the **OVER** clause and its two optional settings:

```
SUM(amount) OVER (  
    PARTITION BY category    -- restart the computation for each category  
    ORDER BY date            -- walk the rows in date order  
    ROWS BETWEEN ...        -- (optional) which neighbours count  
)
```

PARTITION BY is GROUP BY's gentler cousin: it splits rows into partitions and computes *within* each, without folding them. ORDER BY turns a partition into an ordered walk — which is what makes *running* totals and *previous* rows meaningful. Nothing here collapses; every input row survives with a new column of knowledge.

19.2 Running Totals and Moving Averages

Tariro's bank manager wants to see the year accumulating. The running total:

```
SELECT date, amount,
       SUM(amount) OVER (ORDER BY date, sale_id) AS running_total
FROM   sales
ORDER BY date;
```

Read the `OVER`: walk the rows in date order, and at each row, sum *this row and everything before it*. Each row now carries the year-to-date figure for its moment — the cumulative line a bank manager reads at a glance. (The `sale_id` tiebreaker matters more than it looks: rows sharing a date need a deterministic order, or "everything before it" is ambiguous and engines may differ — a professional detail that prevents a subtle bug.)

The moving average — smoothing the daily noise into a weekly rhythm:

```
SELECT date, amount,
       AVG(amount) OVER (ORDER BY date
                        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS avg_7day
FROM   sales;
```

The frame — `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` — is the "moving window": the last seven rows including this one, recomputed at every step. The sawtooth of daily takings flattens into a readable trend; this same shape returns as the analyst's standard smoothing move in Python (Chapter 35) and on dashboards (Chapter 42). Frames give you any neighbourhood you can describe: `3 PRECEDING AND 3 FOLLOWING` (centred), `UNBOUNDED PRECEDING` (everything before), and friends.

19.3 The Ranking Trio

Three functions that order rows, with subtly different behaviours you must be able to name:

```
SELECT sale_id, category, amount,
       ROW_NUMBER() OVER (PARTITION BY category ORDER BY amount DESC) AS rn,
       RANK()        OVER (PARTITION BY category ORDER BY amount DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY amount DESC) AS drnk
FROM   sales;
```

- **ROW_NUMBER()** — 1, 2, 3, 4... arbitrary but unique: ties broken silently. Use when you need *exactly N rows per group*, no arguments.
- **RANK()** — 1, 2, 2, 4... ties share a rank, then a gap. Use for leaderboards ("joint second place, next is fourth").
- **DENSE_RANK()** — 1, 2, 2, 3... ties share, no gaps. Use when "the top 3 ranks" should mean the podium, however many share it.

On Tariro's data the difference shows up wherever the 5.00 mode (Chapter 12) repeats heavily at the top of a category: ROW_NUMBER hands out 1,2,3 among equal amounts; RANK declares a five-way tie for first and jumps to 6; DENSE_RANK stays at 2. Choosing wrongly does not error — it silently returns the wrong rows, which is worse. Name your tie policy out loud before you pick.

19.4 LAG and LEAD

The offsets look one row back or forward within a partition and bring back a value — the machinery of *change*:

```
WITH monthly AS (
  SELECT substr(date, 1, 7) AS month, SUM(amount) AS revenue
  FROM   sales
  GROUP BY substr(date, 1, 7)
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month)           AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / LAG(revenue) OVER (ORDER BY month), 1) AS pct_change
FROM   monthly;
```

The first month's prev_month is NULL — there is no previous month, and LAG is honest about it (Chapter 15's NULL discipline returns; wrap in COALESCE or filter if it offends a report). Month-over-month change is the single most requested derived metric in working analytics, and it is *this* pattern, verbatim, everywhere: revenue in a monthly CTE, LAG it, subtract, divide with the decimal point. Notice also that this chapter's example *is* Chapter 18's pipeline — CTE then window — the two compose naturally, because both keep stories readable.

19.5 Top-N Within Each Group

The pattern this book has been circling since Chapter 18: "the top 3 items by revenue *within each category*" — the grouped top-list that flat GROUP BY cannot express:

```
WITH item_revenue AS (
  SELECT category, item, SUM(amount) AS revenue
  FROM   sales
  GROUP BY category, item
),
ranked AS (
  SELECT category, item, revenue,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
  FROM   item_revenue
```

```

)
SELECT category, item, revenue
FROM   ranked
WHERE  rn <= 3
ORDER BY category, revenue DESC;

```

Three paragraphs: aggregate (item revenues), rank within categories (ROW_NUMBER, ties broken silently — a policy, stated), filter the podium. This exact query shape is among the most-asked SQL interview questions in existence ("top 3 salaries per department" is this query in office costume), and it is the analysis behind "what should we feature, by aisle?" — a real retail decision, arriving as one saved query.

From Your Toolkit — windows everywhere: the running total returns in pandas' `.cumsum()` (Chapter 34) and Power BI's DAX pattern (Chapter 40's time intelligence); month-over-month is a standard quick-measure; the ranking trio is every "leaderboard" visual. The window concept — *annotate, don't collapse* — is also what distinguishes an intermediate analyst's SQL from a beginner's, and interviewers can smell the difference in one question.

Key Takeaways

- Window functions compute over a window and keep every row: OVER (PARTITION BY ... ORDER BY ... frame).
- Running totals and moving averages = ORDER BY plus SUM/AVG; frame ROWS BETWEEN *n* PRECEDING AND CURRENT ROW moves the neighbourhood.
- Ranking trio: ROW_NUMBER (exact N), RANK (ties, gaps), DENSE_RANK (ties, no gaps) — choose a tie policy deliberately.
- LAG/LEAD give previous/next values: month-over-month change in one line; first rows are honestly NULL.
- Top-N-per-group = aggregate → rank → filter: the interview classic and a real retail decision, one CTE pipeline.

Practice Lab

1. The cumulative year: running total over all sales (with the `sale_id` tiebreaker), filtered to one calendar year; then a 7-day moving average of daily takings beside it; write one sentence on what the smoothing reveals that the daily noise hides.
2. The trio, compared: run the ranking-trio query on your data; find a category where the three disagree (a tie at the podium); and write the tie policy you would defend for "top 3 items per category".

3. Month-over-month: the monthly LAG query with `change` and `pct_change`; annotate the biggest jump and biggest drop in your log; connect one of them to a planted pattern you already know (what happened in month 9?).
4. The podium query: top 3 items by revenue within each category (the query above); reconcile its grand total against a grouped-items total — reconciliation after filtering a ranking is exactly where fan-out-style surprises hide.
5. Stretch — the gap analysis: with LAG over daily revenue ordered by date, flag days whose takings dropped more than 40% from the previous day; list them and classify each (missing-data day, Sunday, holiday, or genuine dip). This is a real on-call analyst task wearing a small costume.

Further Reading

- Chapter 34 (pandas' `cumsum/shift` — these patterns in Python), Chapter 40 (DAX time intelligence)
- Appendix A (the window cookbook: frames, gaps-and-islands, sessionisation)

Chapter 20: SQL in Real Life

Part III — SQL: The Language of Data

"In real jobs, nobody gives you an empty editor and a polite question. They give you a database you didn't design, a query named 'final_v3_John_old', and a deadline."

In this chapter you will learn

- The databases you will actually meet at work, and how to connect to them.
- The queries you will actually write — the working analyst's top patterns.
- Reading and repairing inherited SQL — a survival method.
- Performance in three habits: why is it slow, EXPLAIN, and the indexing idea.
- SQL etiquette: saving, naming, commenting — how your queries will be judged.

20.1 The Databases You Will Meet

SQLite taught you the language; at work the engine will almost always be a **server database** — the same relational model, reached over a network. The names to know, so meeting them is recognition rather than surprise:

Database	Where you'll meet it	Notes for you
PostgreSQL	Modern product/backend teams; the analyst favourite	Free, strict, standards-loving — a pleasure to inherit
MySQL / MariaDB	Websites, smaller product stacks	Ubiquitous; dialect quirks around dates
SQL Server (Microsoft)	Finance, corporates, anything already Microsoft	Pair with SSMS, the classic workbench; T-SQL dialect
Oracle	Banks, insurers, governments	Ancient, powerful, fond of ceremony
BigQuery / Snowflake / Databricks	"Data warehouse" / analytics teams (Chapter 57's world)	Billions of rows; SQL-as-a-service; cost-aware querying

Two practical consequences. First, **dialects**: 90% of your SQL transfers unchanged, but dates, string functions, and NULL handling vary at the edges (substr vs SUBSTRING, LIMIT vs TOP vs FETCH FIRST); Appendix A carries the translation table, and every engine's docs have a "SQL dialect" page you read once. Second, **connection tools**: at work you will not browse the database in DB Browser but in a workbench — SSMS, DBeaver, Azure Data Studio, the

warehouse's own web SQL editor — and connecting is the first-day task: host, port, database, your credentials, and (the realistic part) asking a colleague which of the 400 schemas is the real one. Asking is not a beginner's weakness; it is the professional's first move.

20.2 The Queries You'll Actually Write

Five years of an analyst's SQL rounds down to a short list — you know every piece of it now:

1. **Filtered extraction** — `SELECT ... WHERE date BETWEEN ... AND ...` — "pull last month's orders for the western region". (Chapter 15.)
2. **Grouped summaries** — `GROUP BY` with `COUNT/SUM/AVG`, `HAVING` for busy groups. (Chapter 16.)
3. **Enrichment joins** — `LEFT JOIN` a dimension table onto a fact table; the anti-join for what is missing. (Chapter 17.)
4. **Layered analysis** — CTE pipelines: share-of-total, month-over-month, top-N-per-group. (Chapters 18–19.)
5. **Data-quality sweeps** — `COUNT()` vs `COUNT(column)`, `GROUP BY` a key with `HAVING COUNT(>1)` (duplicates), the orphan anti-join, `NULL` censuses. (Chapters 8 and 15–17, in SQL.)

Notice what is *absent*: no `INSERT`-every-day, no schema surgery, no stored procedures — those are engineers' work. The analyst's SQL is reading, aggregating, joining, and checking — which is why Part III could teach it in eight chapters, and why "strong SQL" on a CV (Chapter 53) means exactly this list, fluent.

20.3 Inherited SQL

You will spend more time reading SQL than writing it. The survival method, assembled from everything you now know:

1. **Diagram the FROM first** (Chapter 17): tables, aliases, join lines, `INNER` vs `LEFT`, which side is many. The picture is half the meaning.
2. **Walk the execution pipeline** (Chapter 16): `WHERE` → groups → `HAVING` → `SELECT`. Mark which columns are filters, which are aggregates.
3. **Peel the layers** (Chapter 18): if it is CTEs, run each one alone; if it is nested subqueries, rewrite the innermost as a CTE on a scratch copy — the rewrite is itself the reading.
4. **Suspect the usual criminals** when results look wrong: an `INNER JOIN` that amputated rows (should have been `LEFT`), a fan-out doubling totals (keys repeating both sides), a `WHERE` on a `LEFT`-joined table that silently turned it into an `INNER` (the classic: `LEFT JOIN c ... WHERE c.suburb =`

'Avondale' discards NULL rows — move the condition into the ON clause or accept INNER deliberately), and `SELECT *` in a GROUP BY that some dialects forgive and others refuse.

5. **Change nothing until you can reconcile it:** run the old query, save its totals, change, compare. Inherited SQL is under nobody's tests except yours.

20.4 Slow Queries: Three Habits

Sooner than you think, a query takes ninety seconds and someone is watching.

- **Read the shape before the engine does.** `WHERE substr(date,1,7) = '2025-06'` forces the engine to compute `substr` for every row; `WHERE date >= '2025-06-01' AND date < '2025-07-01'` lets it *seek*. Functions on columns hide values from optimisation — the habit is *sargability*: keep columns bare, put the transformation on the constant side.
- **EXPLAIN (QUERY PLAN)** before the engine spends your patience: prefix any query with `EXPLAIN` and the engine prints its plan — which tables it scans fully, which it seeks, how it joins. You are not expected to read it like a database engineer; you are expected to notice "full table scan on a 40-million-row table" and think "ah".
- **The indexing idea.** An index is a lookup structure the engine maintains so key-lookups and range filters do not read the whole table — the database equivalent of a book's index, and the reason joins on keys are fast. You will rarely create indexes as an analyst; you will benefit from them existing, and occasionally ask a friendly engineer (with the `EXPLAIN` output in hand) whether one is missing. Asking with evidence is the professional form.

20.5 Etiquette: How Your SQL Is Judged

Your queries will be read by colleagues and successors; the craft norms are small and heavily rewarded:

- **Name files for the question** (`monthly_revenue_by_till.sql`, not `query12.sql`), and version by copy or git — never `final_v2`.
- **Comment the why, not the what:** `-- exclude whales: bulk buyers analysed separately (see report p.2)` is a comment; `-- select from sales` is noise.
- **A header block** on anything saved: purpose, author, date, source tables, and — the mark of trust — *the expected row count or total when you last ran it*, so the next runner can reconcile before trusting (the habit of this entire book, in five lines).

- **Format deliberately:** one clause per line, aliases aligned, CTEs named for the story. Formatting is documentation that never goes stale.

From Your Toolkit — the working set: everything in this chapter is tool-independent: the engines change, the workbenches change, but extraction, grouping, joining, layering, auditing, and etiquette are the job. When Part VI connects Power BI to a warehouse, it will be this chapter's connection habits; when Part VII's portfolio projects ship, they will carry this chapter's header blocks. The language is learned; the profession is these habits.

Key Takeaways

- Server databases (Postgres, MySQL, SQL Server, Oracle, warehouses) share the model; dialects differ at the edges; connecting is a first-day task — ask which schema is real.
- The working analyst writes five shapes of SQL: extract, group, join, layer, audit — you know them all.
- Read inherited SQL by diagramming FROM, walking the pipeline, peeling layers; suspect INNER-amputation, fan-out, and WHERE-on-a-LEFT; reconcile before you change.
- Performance: keep columns bare in WHERE (sargability), EXPLAIN before blaming, know what an index is for.
- Name for the question, comment the why, header with expected totals — your SQL is your handwriting.

Practice Lab

1. The dialect drill: rewrite three of your saved queries in a second dialect (e.g. SQLite → SQL Server: `LIMIT` → `TOP`, `substr` → `SUBSTRING`); note each difference in a `dialects.txt` — the start of your personal translation table (Appendix A has the official one).
2. The inheritance, simulated: take your Chapter 18 share-of-total query, rename everything cryptically (`q1`, `t1`, `x`), remove the comments, and hand it to a friend (or your future self, three days); time the explanation-back. Then restore names and comments and repeat. Log both times.
3. The sargability lab: run both date filters on your sales table (`substr` vs `range`) and `EXPLAIN QUERY PLAN` each; write down what differs in the plans, and the rule you'll follow from now on.
4. The criminal line-up: deliberately commit the four sins — `INNER` where `LEFT` was needed, a fan-out join, `WHERE` on a `LEFT`-joined table, `SELECT *`

with GROUP BY — and for each, note the wrong number it produced and the check that caught it. This is the most valuable page in your log so far.

5. The professional file: pick your five most useful queries from all of Part III; give each the full treatment (filename, header block with expected totals, comments-for-why, formatting); store them in a `queries/` folder — they are the seeds of your portfolio's SQL evidence (Chapter 46).

Further Reading

- Chapter 21 (Project 2 — all of Part III assembled), Chapter 57 (the warehouse world: BigQuery, Snowflake, lakehouse)
- Use The Index, Luke (use-the-index-luke.com) — indexing and sargability, friendly and deep

Chapter 21: Project 2 — Tariro's Database

Part III — SQL: The Language of Data

"The spreadsheet answered questions. The database answers them again — reproducibly, on next month's data, in one click — and then answers questions the spreadsheet never could."

In this chapter you will learn

- How to build Tariro's full database: schema, import, keys, and the loading checks.
- How to answer ten real business questions in SQL, each a pattern you now own.
- How to package queries as a reusable monthly pack — the analyst's real deliverable.
- How this project becomes portfolio evidence (the second of six).
- A self-assessment: what SQL you actually command, honestly measured.

21.1 Build the Database

You already have `sales` and `customers` from Chapter 14. Now the full shop, and the design moment that makes it real. In DB Browser → Execute SQL:

```
CREATE TABLE stock (  
    week          TEXT,          -- ISO date of the Sunday counted  
    item          TEXT,  
    qty_on_hand  INTEGER  
);  
CREATE TABLE suppliers (  
    item          TEXT,  
    supplier      TEXT,  
    unit_cost     REAL,  
    last_price_change TEXT  
);  
CREATE TABLE customers (  
    customer_id  TEXT PRIMARY KEY,  
    name         TEXT,  
    suburb       TEXT,  
    joined       TEXT,  
    loyalty_tier TEXT  
);  
CREATE TABLE sales (  
    sale_id      INTEGER PRIMARY KEY,  
    date         TEXT,
```

```
till          INTEGER,
customer_id   TEXT,
payment_type  TEXT,
amount        REAL
);
```

Then **File → Import** the four CSVs into their tables (the files ship with the book's repository; the generator recipe is Appendix C's final section). Now the loading checks — the reconciliation liturgy, performed at every load, forever:

1. **Row counts:** four counts (one per table) written into your log, compared against the CSV line counts. No surprises, or explained surprises.
2. **The key census:** `SELECT customer_id, COUNT(*) FROM customers GROUP BY customer_id HAVING COUNT(*) > 1` — duplicates in a primary-key column must be zero (the engine enforces it; let it refuse, and fix the CSV — never weaken the key).
3. **The orphan check:** the Chapter 17 anti-join, sales→customers; document every orphan (the planted broken keys) as *known* — an undocumented orphan is an unexploded finding.
4. **The totals:** total sales revenue in SQL vs the workbook's cleaned total from Part II. To the cent. This number is the bridge between eras of your own work; crossing it safely is the habit that makes you trustworthy with bigger data.
5. **The NULL census:** NULL counts per critical column (`customer_id`, `amount`), each with its business meaning written beside it.

21.2 Ten Questions

Tariro brings the list to the new laptop — ten questions, the month's agenda, each answerable from the database you just built. Answer each as a saved query (the naming and header discipline of Chapter 20), and keep the reconciliation habit running quietly underneath: any total you produce, you check once against a second route to it.

Q1 — How did this month compare with last? Monthly revenue with LAG: month-over-month change and `pct_change` (Ch. 19). *Deliverable: a three-column table and one sentence.*

Q2 — Which items earn the most, and does it change by till? Top 10 items overall; then top 5 per till (top-N-per-group). *Finding: the second till skews staples — basket composition differs by position in the shop.*

Q3 — Do suburbs differ in what they buy? Join sales→customers, GROUP BY suburb and category, and read the cross-tab (Ch. 16–17). *Finding: one suburb's cooking-oil share is double the others' — a segment, not noise.*

Q4 — What did the price rise actually do? Cooking oil: monthly quantity and revenue before vs after the month-9 price change, from suppliers' `last_price_change` and the sales history — the quantity-led collapse you first saw in a pivot, now stated with the before/after numbers side by side (Ch. 18's CTE pattern).

Q5 — Who are the top loyalty members, and are they drifting? Top 20 members by lifetime revenue; then their monthly revenue over the last six months with LAG — the loyal-core churn signal planted in the design (Ch. 19).

Q6 — Which items never sell on Sundays? Items with zero Sunday sales but healthy weekday sales: two grouped CTEs, the anti-join of item lists (Ch. 17–18). *Operational answer: what not to stock for the Sunday run.*

Q7 — What share of revenue does each payment type take, per month? The share-of-total CTE, grouped by month and payment type (Ch. 18) — *finding: EcoCash share is rising, month over month, a slow structural shift the monthly chart in Part VI will show live.*

Q8 — Where are the data problems this month? The audit pack as one script: duplicate-key census, orphans, NULL census, amounts outside the IQR fences (constants from your Part II log), sales on impossible dates (before opening day, after today). *This query is not a chore — it is Q8 of the meeting agenda, and the one that makes the other nine trustworthy.*

Q9 — What is the gross margin picture by category? Join sales items to suppliers' `unit_cost`; revenue minus estimated cost per category — the first finance-flavoured query, with its caveat stated (unit costs are current, not historical; margin before overheads; a *shape*, not accounts).

Q10 — If we lose the bottom 20% of items, what do we lose? Rank items by revenue (Ch. 19), compute the bottom quintile's share with the share-of-total pattern — the "rationalise the range" decision every retailer eventually faces, answered in one CTE pipeline with the trade-off stated plainly.

Ten questions; every one a saved, commented, reconciled query; and the patterns are exactly the five working shapes of Chapter 20 — extraction, grouping, joining, layering, auditing — wearing business clothes. That is the whole secret of "strong SQL" on a CV.

21.3 The Monthly Pack

The professional deliverable is not ten answers — it is a **reusable pack** that regenerates them:

```
queries/
00_loading_checks.sql      -- row counts, key census, orphans, totals, NULLs
```



```

01_month_over_month.sql
02_top_items.sql
... (one per question)
99_audit_pack.sql      -- Q8, run first and last
README.md              -- how to run, expected totals, data dictionary

```

Next month, when the CSVs arrive, the ritual is: import → run 00_loading_checks → reconcile → run the pack → paste results into the report template from Chapter 13. Twenty minutes, was an afternoon. **This is the pitch of the whole database era: analysis becomes a pipeline, and the analyst's craft moves from dragging to questioning.** (Chapter 37 will automate the import step too, and Part VI will refresh it live.)

21.4 Portfolio Evidence

Second entry in the case-study ledger (the first was Chapter 13's report; Chapter 45 assembles the portfolio):

- **The artefacts:** `tariros.db`, the `queries/` pack with headers and comments, the ten answers as a results document (PDF or markdown), the loading-check log.
- **The 150-word note**, written now while fresh: *the situation* (growing shop, four data sources), *the work* (designed a four-table schema, loaded and reconciled 18 months of data, built a ten-query monthly pack), *one number* ("answerable in twenty minutes what took an afternoon"), *the honest limits* (single-file SQLite; margins approximated; orphans documented, not deleted).
- **The interview line it funds** (Chapter 52's STAR method in embryo): "Tell me about a time you improved a reporting process" — you will tell this story, with the before/after timing as the punchline, and the loading-check ritual as the integrity beat.

21.5 Where You Stand

Part III is done, and honestly measured: you can design a small schema, load and reconcile data, query with the five working shapes, join without fear (and catch fan-out), layer with CTEs, rank and lag with windows, read inherited SQL, and package it all professionally. That is working-analyst SQL for the majority of analytics jobs — and it was eight chapters.

But look at what you have *not* been able to say, anywhere in Part III: whether Sunday's 28% shortfall is a *pattern or a coincidence*; whether the cooking-oil decline is *significantly* steeper than the noise; whether the loyalty scheme *caused* anything. SQL — all tools so far — describes; it cannot conclude. That is statistics' monopoly, and it is where the shop's questions are taking you

next.

From Your Toolkit — the query pack travels: this chapter's pack (loading checks + question queries + audit, under a README) is the template for Project 3's Python pipeline, Project 4's Power BI refresh, and any reporting role's first month. Employers ask "how would you make this repeatable?" — the pack is the answer.

Key Takeaways

- Build with checks: schema with keys, CSV imports, then the five loading checks (counts, key census, orphans, totals, NULLs) — every load, forever.
- Ten business questions decompose into the five working shapes; each answer is a saved, commented, reconciled query.
- The deliverable that matters is the *reusable pack*: import, check, run, report — twenty minutes, monthly.
- Portfolio evidence #2: database + query pack + results document + the 150-word note with its honest limits.
- SQL describes; statistics concludes — the questions you now cannot answer are Part IV's syllabus.

Practice Lab

1. Build the full database: the four CREATE TABLEs, the imports, and all five loading checks; reconcile the sales total to your Part II workbook cent-for-cent and log the bridge number.
2. Answer all ten questions as the query pack (filenames, header blocks with expected totals, comments for why); paste each result into a results document with a one-sentence finding above every table.
3. The trade-off memo: Q10's range-rationalisation as a half-page memo to Tariro — recommendation, the revenue at risk, what you would measure before deciding, and the caveat that 18 months is this shop, not all shops.
4. The dry run: pretend it is next month — re-import a (copied, lightly edited: add 50 plausible new sales) CSV set, run the whole pack, and fix whatever breaks; write down what the dry run caught that the first run hid.
5. Update the portfolio: file the artefacts, write the 150-word note, and draft the STAR paragraph for "improved a reporting process" — while the details are vivid.

Further Reading

- Chapters 44–46 (the portfolio, assembled properly), Chapter 37 (the pack, automated)
- Appendix A (every pattern used here, ready to copy), Appendix E (the datasets)

PART IV

Statistics Without Fear

Chapters 22 – 29

Chapter 22: Numbers That Describe

Part IV — Statistics Without Fear

"The mean is a fact about arithmetic. The median is a fact about people. Knowing which sentence you are writing is half of statistics."

In this chapter you will learn

- The three "typical" numbers — mean, median, mode — as *theory*, not functions.
- When each one lies, and the shape-of-data reasoning that picks the honest one.
- Weighted means, and the market-share question that needs one.
- The first formal notation — read and write statistics like a native.
- The questions from Part III that statistics will now formalise.

22.1 Description Is a Claim

Part II computed the numbers; Part IV asks what they *mean* — and the first lesson is that every descriptive number is a claim about a whole distribution, compressed into one figure. A mean claims: "if every value were equal, here is the value that preserves the total." A median claims: "half of this data lies below me." A mode claims: "this value is the most common." Three different claims — all true, all partial, all capable of lying by selection. Statistics without fear is not statistics without care; it is knowing precisely which claim your number makes, and saying so.

22.2 The Mean: Arithmetic's Tyranny and Grace

The mean is the balance point of the data — physically, if you cut the histogram out of wood, it balances on the mean's edge. Its grace: it uses every value, and it is the only typical that plays correctly with totals ($\text{mean} \times n = \text{sum}$ — the reconciliation identity of Chapter 8, now understood as a *theorem*). Its tyranny: it is dragged by extremes. One 412-dollar whale moved Tariro's basket from 6.50 (median) to 8.42 (mean) — a 29% distortion from a single row.

The formal rule for when the mean lies: when the distribution is **skewed** — lopsided, with a long tail on one side. Money is almost always right-skewed (a floor at zero, no ceiling): incomes, basket sizes, city populations, website visits. The whale is not an aberration in money data; the whale is *what money*

data is. Hence the professional default: **medians for money headlines, means for totals arithmetic**, both reported when they disagree enough to matter — the Part II habit, now with its reasoning attached.

22.3 The Median and the Mode

The median is the middle value by rank — the 50th percentile (Chapter 24 builds the whole percentile family). Its virtues are the mean's vices: it ignores magnitude (the whale counts as one row), so it is **robust** — a word statisticians use to mean "liars cannot move it easily." Its vice: it ignores magnitude — for totals thinking it is useless ($\text{median} \times n$ is not the sum; try it and watch the reconciliation fail).

The mode — most common value — is the only typical that works on *categories*: the modal payment type (EcoCash), the modal shopping day (Saturday), the modal basket (5.00). It is underused and quietly valuable in retail, where "what do most people actually do" is often the question. Its failure mode: multimodal data (two humps — weekday shoppers and Saturday shoppers, each with their own typical) has no single mode, and reporting one anyway is a small lie with a straight face.

22.4 Which Number, Decided by Shape

The choosing rule, formalised:

Shape	Mean vs median	Honest headline
Symmetric (roughly mirror)	Close together	Either; report mean, sleep well
Right-skewed (long tail up: money)	Mean > median	Median for "typical"; mean only with totals
Left-skewed (long tail down: rare — ages at death, days-to-delivery)	Mean < median	Median again
Multimodal (two humps)	Either, misleadingly	Report the humps: "two kinds of shopper"

Draw the histogram (even a rough one) before choosing; the eye that sees two humps saves the mouth from one number. This is why Part II insisted on quartiles: the five-number summary is a *shape* description disguised as five values.

22.5 Weighted Means

One more typical, because a real question needs it: *what is the average price of cooking oil across all bottles sold?* Not the average of the prices (that treats a 30-bottle week and a 1-bottle week as equals) — the average *experienced by bottles*: weight each price by the bottles sold at it. The weighted mean:

weighted mean = $\Sigma(\text{price} \times \text{bottles}) / \Sigma(\text{bottles})$

Compute it once by hand on five weeks of data and you will never confuse it again: the naive mean of prices says 3.10; the bottle-weighted mean says 3.42 — because more bottles sold at the higher price. Every "average" you meet at work is secretly a choice of weights: average revenue per *store* vs per *transaction* vs per *customer* are three different weighted means of the same data, and reports that mix them up are quietly wrong. When someone says "the average", the professional reflex is now: *weighted by what?*

22.6 The Notation, Gently

Statistics has a small formal language, and reading it removes the foreigner's tax. The core, all of it used repeatedly in this book:

- $\bar{x}$ ("x-bar") — the sample mean. μ ("mu") — the population mean (Chapter 25 draws the line between them).
- n — the count of values. Σ ("sigma", uppercase) — "add these up": $\bar{x} = \Sigma x / n$ is the mean, written properly.
- Md or the tilde $\tilde{X}$ — the median (usage varies by book; this book writes "median" in words, always).
- s — sample standard deviation; σ ("sigma", lowercase) — population standard deviation (next chapter's star).

That is the entire notation kit for four chapters. When a formula in the wild looks alien, translate it back to these five symbols and it collapses into a sentence.

22.7 The Questions Returning

Part III ended owed: is Sunday's shortfall a pattern or a coincidence? Did the price rise *significantly* hurt? Did loyalty *cause* anything? Descriptive statistics — this chapter — sharpens the questions but cannot close them; that takes spread and shape (next), z-scores (24), sampling theory (25), and tests (26). The arc is deliberate: description → distribution → standardisation → sampling → inference. By Chapter 29 you will close every open question in the ledger, and the SPSS/Stata chapter will show you the menus where the same tests live in classic packages.

From Your Toolkit — one claim per number: mean, median, mode, weighted mean appear in every tool of this book — `AVG` and `PERCENTILE_CONT` in SQL (Appendix A), `.mean()/.median()` in pandas (Chapter 34), the "Average/Sum" toggle in Power BI (Chapter 40), Descriptives in SPSS (Chapter 29). What transfers is not the function name but the *claim*: which typical, why, weighted by what, with the shape checked first. That habit is the statistics.

Key Takeaways

- Every descriptive number is a claim: the mean preserves totals, the median preserves typicality, the mode preserves frequency — choose by the claim your sentence needs.
- Skew decides: money is right-skewed; medians headline money; means do totals; both when they disagree.
- Multimodal data has no honest single typical — report the humps.
- Weighted means are everywhere; the reflex question is "weighted by what?" (store, transaction, or customer — three answers, one dataset).
- Notation kit: $\bar{x}$, μ , n , Σ , s , σ — five symbols, four chapters, no more foreigner's tax.

Practice Lab

1. By hand (calculator or spreadsheet, no functions): compute mean, median, mode of these ten baskets — 2.50, 5.00, 5.00, 5.00, 6.50, 7.00, 8.00, 11.50, 19.00, 412.00. Then remove the whale and recompute. Write the two-sentence comparison that explains skew to a shop owner.
2. The choosing drill: for `amount`, `customer age` (from customers), and `items per sale`, sketch (or histogram) the distribution, name the shape, and choose the headline typical — with one sentence defending each choice.
3. The weighted trap: compute the naive and bottle-weighted average cooking-oil price from your data (or the five-week mini-dataset above); write the two numbers side by side and the sentence explaining the gap to Tariro.
4. Notation translation: write "the mean of the amounts is $\Sigma x/n$ " and "the median basket" using correct symbols; then find one formula in any statistics resource online and translate it into a sentence using your kit.
5. The ledger: open your questions log (from Chapter 2's habit); mark each open question with what it needs — description, spread, standardisation, sampling, or a test. You have just written Part IV's table of contents for

yourself.

Further Reading

- Chapter 23 (spread and shape — the rest of the portrait), *Naked Statistics* (Wheelan) chapters 2–3
- *How to Lie with Statistics* (Huff) — the 1954 classic; every trick is still in daily service

Chapter 23: Spread and Shape — the Bell Curve, Gently

Part IV — Statistics Without Fear

"Two shops, one average. The difference between thriving and closing is the spread — the shape around the middle."

In this chapter you will learn

- Range, IQR, variance, and standard deviation — the spread statistics, built by hand once.
- The bell curve (normal distribution) — what it is, why it appears, and what it promises.
- Skew, in the formal sense — and the right skew of money, again, understood.
- Outlier rules derived, not memorised — why the $1.5 \times \text{IQR}$ fence is 1.5.
- Reading a histogram like an analyst: shape words that mean something.

23.1 Spread, Built by Hand

Take five daily takings (dollars): 100, 110, 120, 130, 140. Mean = 120. The deviations from the mean: -20, -10, 0, +10, +20. Now the ladder of spread statistics, each answering "how far from typical?":

- **Range** = max – min = 40. Simple, honest, destroyed by one wild day.
- **IQR** = $Q3 - Q1 = 130 - 110 = 20$. The middle half's width — robust (Chapter 12's fence-builder).
- **Variance** = the average *squared* deviation = $(400+100+0+100+400)/5 = 200$. Squared units (dollars²!) — unintuitive, but mathematically the deep one, because squaring makes everything positive and treats big deviations severely.
- **Standard deviation** $s = \sqrt{\text{variance}} = \sqrt{200} \approx 14.1$ — *back in dollars*: "the typical distance from the mean", and the workhorse of everything that follows.

(For a *sample* — almost always your case — we divide by $n-1$ instead of n : $200 \times 5/4 = 250$, $s \approx 15.8$. Why $n-1$ is a Chapter 25 question about samples and populations; for now, note that spreadsheet's `STDEV.S` and the formulas ahead use $n-1$, and `STDEV.P` uses n . Choosing wrong on small data changes the answer visibly.)

Compare shop B: 60, 90, 120, 150, 180. Same mean (120), $s \approx 44.2$ — triple the risk. Spread is where the business lives, which is why every serious report pairs a typical with a spread, always.

23.2 The Bell Curve

Draw a histogram of a thousand *sums of things* — total daily takings (many customers' baskets added), a person's height (many genes and meals), a measurement error (many small nudges) — and the same shape emerges: symmetric, one hump in the middle, tails thinning both ways. The **normal distribution** (the bell curve) is not a law someone decreed; it is what happens when many small, independent causes add up — the Central Limit Theorem's promise (Chapter 25 makes it formal), and the reason the bell is the default shape of nature and industry.

The normal's magic property, worth memorising as a picture rather than a formula:

~~~~~		68% of data within $\pm 1s$ of the mean
~~~~~		95% within $\pm 2s$
~~~~~		99.7% within $\pm 3s$ ("the 68–95–99.7 rule")

On Tariro's daily takings (mean  $\approx 2,990$ ,  $s \approx 640$ , close enough to normal for the shape's promise): roughly 68% of days land between 2,350 and 3,630; a day below 1,710 ( $-2s$ ) happens about once in twenty days; a day below 1,070 ( $-3s$ ) happens about three days in a thousand. Suddenly "unusual" has a *number attached*, and the bank manager's anxiety can be calibrated: a 2,200-dollar Tuesday is a bad day, not a crisis — but a 900-dollar Tuesday is a three-sigma event, and *something happened* (a road closure, a competitor's opening, a till failure). This is the analytic superpower the bell delivers: **expected variation vs signal, on a ruler**.

### 23.3 When the Bell Is a Lie

The bell's danger is precisely its popularity: not everything is normal, and assuming it of money data is the classic error. Basket *amounts* — one customer, one purchase — are right-skewed (the whale's home); so are incomes, claim sizes, and website dwell times. The bell fits *sums and averages* of many small influences (daily takings = sum of  $\sim 35$  baskets), not the raw skewed ingredients — a distinction Chapter 25 will make precise and profitable (averages of skewed things behave normally even when the things don't).

**Skew, formally:** the third moment's sign — tail right = positive skew (mean  $>$  median), tail left = negative skew (mean  $<$  median). The histogram words that matter: **symmetric** (bell-ish), **right-skewed** (money), **left-skewed**

(rare), **bimodal** (two populations in one column — two kinds of shopper, weekdays and Saturdays; report the humps, never one typical). **Kurtosis** (tail heaviness) exists as a word; you may ignore it until an exam asks.

## 23.4 The Fence, Derived

Chapter 12's outlier rule — fences at  $Q1 - 1.5 \times IQR$  and  $Q3 + 1.5 \times IQR$  — deserved its constant. Here is where 1.5 comes from: for normal data,  $Q1$  and  $Q3$  sit about  $0.67s$  below and above the mean, so  $IQR \approx 1.35s$ , and the fence sits  $1.35s \times 1.5 / 2 \dots$  more usefully: the fence lands near  $\mu \pm 2.7s$  — just outside the 99.7% band. In other words, **for bell-shaped data, the  $1.5 \times IQR$  fence flags almost exactly the rarest 0.3% of values** — the rule is a significance test in costume, built from robust statistics so that whales do not move the goalposts they are judged by. (The z-score method of Chapter 24 — flag beyond  $\pm 3s$  — is the same idea computed from the mean and  $s$ , and *not* robust: the whale drags  $s$  and hides itself. When you must choose: fences for skewed data, z-scores for symmetric data you trust.)

That is the deeper pattern of this whole subject: **the same idea (how far is this value from typical, in units of typical variation?) reappears in fences, z-scores, t-tests, p-values, control charts** — one sentence wearing different costumes. Learn the sentence once, at whatever depth you are standing on, and every costume afterwards is a wardrobe change.

## 23.5 Reading a Histogram

The professional's shape-reading checklist — ten seconds, every numeric column, forever:

1. **One hump or two?** Two = two populations; stop, segment, report the humps.
2. **Symmetric or skewed?** Skew direction by tail; right-skew = money-shape = medians headline.
3. **Where does it end?** A hard floor at zero? A ceiling (closed shop, capped prices)? Ends carry business meaning.
4. **Any gaps or spikes?** A spike at exactly 5.00 (the mode — a psychological price point); a gap where a category has no sales — data *quality* story or business story, always worth one question.
5. **What would the fence flag?** Estimate  $Q1$ ,  $Q3$  by eye; where do the fences land; what population lives beyond them (bulk buyers — a segment, Chapter 12's lesson)?

**From Your Toolkit — the shape kit travels:** `PERCENTILE_CONT` + a spread in SQL (Appendix A), `.describe()` and `.hist()` in pandas (Chapters 34–35), the distribution line in Power BI (Chapter 42), Explore→Descriptives→Plots in SPSS (Chapter 29). The bell's 68–95–99.7 promise returns as confidence intervals (25), hypothesis tests (26), and control charts (Chapter 60) — same bell, three costumes. And the fence/z-score equivalence is the sentence that quietly unifies Part IV.

## Key Takeaways

- Spread ladder: range (fragile), IQR (robust), variance (squared units, deep),  $s$  (dollars again — the workhorse); sample vs population divisor ( $n-1$  vs  $n$ ) matters on small data.
- The bell appears wherever many small causes add up; 68–95–99.7 turns "unusual" into a number;  $-3s$  days are events, not weather.
- Money is right-skewed; bells fit sums and averages of money, not raw money; bimodal means two populations — segment, don't average.
- The  $1.5 \times \text{IQR}$  fence  $\approx$  the 99.7% band for bells, built from robust stats — a significance test in costume; fences for skewed,  $z$  for symmetric.
- Read every histogram with the five-question checklist — humps, tails, ends, spikes, fences.

## Practice Lab

1. By hand: the five-value ladder (range, IQR, variance,  $s$  with both divisors) on shop A and shop B above; confirm your  $s$  values against `STDEV.S/STDEV.P` in your spreadsheet; write the "same average, different risk" sentence with numbers.
2. The bell check: histogram of daily takings (a pivot by date, then a chart); estimate  $\mu$  and  $s$  by eye, mark  $\pm 1s$ ,  $\pm 2s$ ,  $\pm 3s$ , and count the days outside each band; how close to 68–95–99.7 does the shop's real data come? Write one sentence on why it is close but not exact.
3. The money-shape check: histogram of basket `amount`; name the shape; re-plot log-style if your tool allows (or simply describe the tail); write the "which typical" sentence this shape commands.
4. The fence derivation, verified: compute  $Q1$ ,  $Q3$ , IQR, fences on daily takings; list the flagged days; compare with the  $\pm 2.7s$  rule of thumb; explain any disagreement in one sentence (hint: is the data perfectly symmetric?).

5. The bimodal hunt: histogram of basket amounts *by weekday vs Saturday* (two series or two charts); if the humps differ, write the "two kinds of shopper" sentence — and note it in the questions ledger as a segmentation finding, description-level.

## Further Reading

- Chapter 24 (z-scores — the sentence standardised), Chapter 25 (why the bell keeps appearing — the Central Limit Theorem)
- *The Art of Statistics* (Spiegelhalter) chapters 4–5 — shape and spread, by a master teacher

# Chapter 24: Percentiles and Z-Scores — "Above Average", Properly

## Part IV — Statistics Without Fear

"'Above average' is a compliment paid by people who never asked how far above, or how rare that is."

### In this chapter you will learn

- Percentiles and quartiles, formally — and the "which method?" footnote that no one warns you about.
- The z-score: any value, restated in units of its own data's spread.
- What z-scores are for: flagging, comparing apples to oranges, and standardising columns.
- The empirical rule and Chebyshev's fallback — the bell's promise, and the promise that works on anything.
- Tariro's dashboards get a genuinely new number: how each day ranks, honestly.

## 24.1 Percentiles, Formally

The **p-th percentile** is the value below which p% of the data lies. The median is the 50th; quartiles are the 25th, 50th, 75th. The percentile *rank* of a value answers the reverse question: what percentage of the data lies below *this*? The two directions — value from rank, rank from value — are the same ruler read from opposite ends, and fluency is reading both without noticing.

Percentiles are the great democratic statistic: they compare a value to *its peers* without assuming any shape. "A 5,200-dollar Saturday is the 91st percentile of Saturdays" is true whether Saturdays are bell-shaped, skewed, or bumpy — no bell required. That shape-freedom is why percentiles dominate reporting where tails matter: exam scores ("90th percentile"), website latency ("p95 response time"), and income ("the 99th percentile") are all peers-relative-to-peers claims.

One footnote you need once and then forever: **there are several defensible ways to compute a percentile from finite data** (linear interpolation between ranks, nearest-rank, and variants) and tools differ — Excel's `PERCENTILE.INC` vs `PERCENTILE.EXC`, different defaults in SQL engines and pandas. On large data the differences vanish; on a 20-value column they can move a quartile noticeably. The professional rule: know your tool's default, state it

when precision matters, and never compare percentiles computed by different methods without checking. (Appendix D's cheat sheet lists the defaults.)

## 24.2 The Z-Score

The z-score is one of the smallest, most powerful formulas in this book:

$$z = (\text{value} - \text{mean}) / \text{standard deviation}$$

It restates any value as **a number of standard deviations from its own mean**. A 4,270-dollar day against daily takings with mean 2,990, s 640:  $z = (4,270 - 2,990)/640 = +2.0$  — a two-sigma day. The formula's quiet achievement: it *removes the units*. Dollars, seconds, kilograms, test scores — after z, everything is on the same dimensionless ruler, "typical distances from typical", and Chapter 23's 68–95–99.7 rule applies directly: a +2.0 day happens about once per twenty days (the 95% band's outside edge); a –3.0 day is a three-in-a-thousand event; a +1.6 day is merely a good day.

Negative z = below mean; the sign matters and beginners drop it. And z is exactly the machinery inside the IQR-fence derivation from Chapter 23.4 — the fence is a z-score of  $\pm 2.7$  built from robust ingredients. One sentence, three costumes, as promised.

## 24.3 What Z-Scores Are Actually For

**Flagging.** The daily-exceptions report: compute z for each day's takings against the trailing ninety days; flag  $|z| > 2$ . This is the analyst's on-call instrument — the same skeleton that powers fraud alerts (z of transaction size per customer), quality control (Chapter 60's control charts), and anomaly detection in its simplest honest form. Build it once as a spreadsheet or SQL query (Appendix A has it) and you own a professional tool.

**Comparing incomparables.** Which is more remarkable: Tariro's 4,270-dollar day, or her 61-customer Saturday (mean 35, s 9  $\rightarrow z = +2.9$ )? The dollars differ; the z's do not. "The Saturday *crowd* was more unusual than the Saturday *takings*" — a sentence no raw comparison can produce, and the seed of a real insight (what drove the crowd if not the spend?).

**Standardising columns.** Subtract the mean, divide by s — every column now sits on the z-ruler. This is the standard preparation for comparing many variables at once, and it is *literally* what lies under the hood of the z-tests of Chapter 26, the regression of Chapter 28, and machine learning's distance calculations (Chapter 58 gets a whiff of why unscaled columns bully scaled ones).



## 24.4 The Empirical Rule and Its Rival

Two rules, and knowing when each applies is half the exam and all of the craft:

- **The empirical rule** (68–95–99.7) holds for *normal* data. Beautiful, tight, conditional.
- **Chebyshev's inequality** holds for *any* shape: at least 75% of data lies within  $\pm 2s$ , at least 89% within  $\pm 3s$ , at least  $1 - 1/k^2$  within  $\pm k s$ . Weaker, unconditional — the floor that works even when the bell is a lie.

The pairing is a moral lesson in statistical honesty: *if you can assume the bell, you get tighter promises; if you cannot, you still get promises — just humbler ones*. Daily takings, near-bell: empirical rule,  $-2s$  once a month-ish. Basket amounts, skewed: Chebyshev only — and even Chebyshev is conservative, because percentiles (shape-free) are usually the better instrument on skewed data. The craft hierarchy on any column: **histogram first; bell-ish  $\rightarrow$  z and empirical; not bell  $\rightarrow$  percentiles and fences**.

## 24.5 The New Dashboard Number

Tariro's whiteboard gets its most useful line yet. Beside each day's takings, the day's **percentile among the last ninety days** — "Tuesday: 2,410 (14th percentile — quiet)"; "Saturday: 4,270 (99th — best day since opening)". The owner reads percentiles instantly ("one of the best days we've had") where z-scores would need translation; the analyst keeps z internally for flagging. Same ruler, two costumes, audience-chosen — a preview of Part VI's whole philosophy: *the number is not the deliverable; the sentence the reader can act on is*.

And with the ruler in hand, the Part III ledger's biggest question — *is Sunday's 28% shortfall a pattern or luck?* — is now one step from answerable: Sunday's mean takings against the spread of days, in s-units, is exactly the calculation Chapter 26's first hypothesis test will formalise. The z you built today is the t-test's embryo.

**From Your Toolkit — the standard ruler:** `PERCENTILE_CONT` and window-rankings in SQL (Appendix A),  $(x - \bar{x})/s$  as a one-liner in pandas (`((df.x - df.x.mean())/df.x.std())`, Chapter 34) and in Power BI measures (Chapter 40); SPSS's Descriptives panel prints z-scores on a checkbox ("Save standardized values"). Every tool you will ever touch computes both, because every analyst needs both — percentiles for shape-freedom, z for the bell's tight promises.

## Key Takeaways

- Percentiles compare a value to its peers with no shape assumptions — the democratic ruler; know your tool's method footnote.
- $z = (\text{value} - \text{mean})/s$ : units dissolved;  $|z| > 2$  is a once-in-twenty event *if* the bell holds.
- z's three jobs: flag (exceptions, fraud, control), compare incomparables, standardise columns for later machinery.
- Empirical rule if bell, Chebyshev if not (75% within 2s, always); hierarchy: histogram → z or percentiles/fences.
- Percentile for the audience, z for the engine — the same ruler in the costume the reader needs.

## Practice Lab

1. Hand-z: pick three days from your daily takings (an ordinary one, the worst, the best); compute z for each by hand; write the three sentences ("a -1.9 day: quiet, not alarming" etc.).
2. The exceptions report, built: z of takings vs trailing 90 days for every day (spreadsheet or SQL — Appendix A's pattern); list every  $|z| > 2$  day; annotate each with what you find when you look (holiday? payday weekend? nothing?). This artefact is portfolio-grade.
3. Incomparables, compared: z of the best day's takings vs z of the best day's customer count; which record is more remarkable? One sentence with both z's in it.
4. The method footnote, experienced: compute "median" three ways — `PERCENTILE.INC(..., 0.5)`, `PERCENTILE.EXC(..., 0.5)`, `MEDIAN` — on a 12-value column; report the (small) differences; write the one-line policy you will follow when precision matters.
5. The Sunday pre-analysis (ledger item): mean Sunday takings, mean weekday takings, the s of daily takings; express the Sunday gap in s-units; write your *informal* verdict ("about a -1.8s shortfall every week") and mark the ledger: "formal test pending Chapter 26". You are one chapter from closing your first real question.

## Further Reading

- Chapter 25 (sampling — where the population's  $\mu$  and  $\sigma$  enter the story), Chapter 26 (the t-test — your Sunday z grows up)
- Appendix D (the statistics cheat sheet: every rule in this Part, one page)

# Chapter 25: Samples and Populations

## Part IV — Statistics Without Fear

"You will never see the population. Everything you will ever do as an analyst is peek at a sample and argue, carefully, about the whole."

### In this chapter you will learn

- Population vs sample — the distinction that gives statistics its purpose.
- Sampling distributions: what a statistic *does* when you repeat it — the deep idea.
- The Central Limit Theorem, plainly stated, and why it is the luckiest fact in the field.
- Standard error: the standard deviation of a statistic; the ruler of "how wrong might this be".
- Confidence intervals and margins of error — built from the above, not memorised.

## 25.1 The Distinction That Runs Everything

Honesty Rule 3, from Chapter 2, now gets its theory. The **population** is every unit you *want* to speak about: every sale Tariro would ever make, every customer she will ever serve, every basket in every shop like hers. The **sample** is the part you can actually see: eighteen months of one shop's till. Statistics exists because these are different — and its entire architecture is the careful argument from the seen to the unseen.

The vocabulary you must own cold: **parameter** — a number about the population ( $\mu$ ,  $\sigma$ ; fixed, unknown); **statistic** — a number computed from the sample ( $\bar{x}$ ,  $s$ ; known, but it *varies* if you re-draw the sample). A statistic is an *estimator* of its parameter:  $\bar{x}$  estimates  $\mu$ . The professional's mental tag on every reported number: *this is a sample statistic standing in for a population parameter — with what care?*

Tariro's data is, technically, a *complete census* of her shop's past — but the moment she asks "will my Sundays stay weak?", the past becomes a sample of the shop's *ongoing behaviour* (the population). Almost every business question quietly converts history into a sample of the future; noticing the conversion is the analysis.

## 25.2 What a Statistic Does When You Repeat It

The deep idea, built by hand. Suppose we treat Tariro's 6,420 sales as a population, and we estimate the average basket from samples of 50 sales each. Take sample 1:  $\bar{x}_1 = 8.10$ . Sample 2:  $\bar{x}_2 = 8.95$ . Sample 3, 4, 5... each sample gives a *different* mean. Now — the pivot — collect those sample means and histogram *them*: they have their own distribution (the **sampling distribution** of the mean), their own centre (it clusters on  $\mu$ , the population mean — the estimator is *unbiased*), and their own spread (much tighter than the raw data: means wobble less than individuals).

You will never, in a job, actually draw a hundred samples. But every formula in the rest of this book is a shortcut *predicting* what that histogram would look like — so you must be able to picture it, and now you can: **the sampling distribution is the histogram of "what my statistic would say if I re-ran the world", and statistics is the science of its shape.**

## 25.3 The Luckiest Fact

The **Central Limit Theorem** (CLT): the sampling distribution of the mean approaches a **normal distribution** as the sample size grows — *whatever the shape of the raw data*. This is the luckiest fact in the field and the reason the bell runs the world: baskets themselves are viciously right-skewed (whales) — but *averages of baskets* are nearly normal, at  $n$  around 30+.

Verify it in your spreadsheet in ten minutes (this exercise is the single best ten minutes in Part IV): take 200 random samples of 50 sales each (sample rows with `=RANDBETWEEN` picks or your tool's sampling function), compute each  $\bar{x}$ , histogram the 200  $\bar{x}$ 's. The raw amounts histogram is a skewed mess; the sample-means histogram is a *bell*, centred on 8.42. You have just watched the CLT turn skew into symmetry — the raw material of Chapter 23's "bells fit sums, not money" rule, now demonstrated by your own hand.

## 25.4 Standard Error

The sampling distribution's spread has its own name: the **standard error** (SE), and for a mean:

$$SE = s / \sqrt{n} \quad (s = \text{sample standard deviation, } n = \text{sample size})$$

On Tariro's data:  $s \approx 12.10$ ,  $n = 50$ , so  $SE = 12.10/7.07 \approx \mathbf{1.71}$ . Read it as a sentence: "sample means of 50 baskets typically land about 1.71 dollars from the true mean." The formula's two levers are the whole of practical statistics: **bigger  $n$  shrinks error** (with the square root — quadruple your data, halve your error: the most important business fact about data collection); **noisier data (bigger  $s$ ) inflates it.**

Distinguish coldly: **standard deviation** describes individuals; **standard error** describes statistics. "Sales vary by about 12 dollars" (s) vs "our average-basket estimate is wrong by about 1.7 dollars" (SE). Mixing these two is the most common statistical sentence-error in business reporting, and now you cannot unsee it.

## 25.5 Confidence Intervals

The standard error plus the bell gives the **confidence interval** — the honest answer to "what is the average basket?":

$\bar{x} \pm 2 \times SE$  (the "95% interval"; 2 is the CLT's gift from the 95% rule)

$8.42 \pm 2(1.71) \rightarrow \mathbf{(5.00, 11.84)}$  for  $n = 50$ . With the full 6,420 sales ( $SE = 12.10/\sqrt{6420} \approx 0.15$ ):  $8.42 \pm 0.30 \rightarrow \mathbf{(8.12, 8.72)}$ . Same data, more of it, tighter honest claim — the interval *earns* its narrowness by  $n$ .

The correct reading, stated once and forever: a 95% CI means *the procedure* captures the true mean in 95% of its uses — not "95% probability that  $\mu$  is in this particular interval". Precision here is not pedantry; the sloppy reading is how CIs get abused into certainty they never promised (a Chapter 2 skepticism target, now with machinery). The **margin of error** is just the  $\pm$  part ( $2 \times SE$ , or  $1.96 \times SE$  exactly); the news's " $\pm 3$  points" is this chapter, every night, uncredited.

And the honest question every interval should provoke: *is my interval narrow enough to decide?* (8.12, 8.72) supports "charge for bags at 9.00 thresholds"; (5.00, 11.84) supports nothing but "collect more data". Statistics does not just measure uncertainty — it tells you when to go get more data, which is a budget decision.

**From Your Toolkit — the argument's engine:** SE and CI appear as `CONFIDENCE.T` in spreadsheets, confidence intervals printed by every SPSS/Stata procedure (Chapter 29), pandas' `.sem()` and statsmodels' full outputs (Chapter 36), and the error bars on every serious dashboard chart (Chapter 42). The A/B tests of Chapter 26, the survey margins of Chapter 27, and the forecast bands of Chapter 60 are all this chapter in costume — a statistic, its SE, and the bell's arithmetic.

## Key Takeaways

- Population (parameters, fixed, unseen) vs sample (statistics, seen, varying); almost every business question silently samples the future.
- The sampling distribution is "what my statistic would say across re-runs" — the picture behind every remaining formula.

- CLT: means of 30+ items are near-normal regardless of raw shape — skew in, bell out; verify it once by hand and own it.
- $SE = s/\sqrt{n}$ : bigger  $n$  tightens ( $\sqrt{n}$ ), noisier  $s$  loosens; SD describes individuals, SE describes statistics — never mix the sentences.
- $CI = \bar{x} \pm 2 \times SE$ , read as procedure reliability; and the decision question: is the interval narrow enough to act, or is more data the answer?

## Practice Lab

1. The ten minutes: the 200-samples-of-50 exercise; paste both histograms (raw amounts; sample means) into your log with the two caption sentences. This pair of pictures is the CLT, owned.
2. The lever drill: compute SE for  $n = 10, 50, 200, 1000$  on the basket data; write the sentence comparing how much data each quadrupling buys; and answer: to halve the error again after  $n=1000$ , how many sales do you need?
3. The interval ladder: 95% CIs for average basket at  $n = 50, 200$ , and the full dataset; write the three intervals and, for each, one decision it could and could not support.
4. The sentence audit: find three "average" claims in any report (Tariro's, a news article, your own Part II report — be honest) and restate each with its interval or an explicit "no interval computed"; note how the claims' persuasiveness changes.
5. The design question (ledger): Tariro wants to survey 30 customers about a planned Sunday promotion; using the  $s$  of individual baskets, what margin of error will her sample proportion carry (SE of a proportion:  $\sqrt{p(1-p)/n}$ , worst case  $p=0.5$ )? Write the "is 30 enough?" memo — this exact calculation returns in Chapter 27.

## Further Reading

- Chapter 26 (the tests — SE plus a null hypothesis), Chapter 27 (surveys, where sampling is the whole job)
- *The Art of Statistics* (Spiegelhalter) chapter 8 — sampling, from the master

# Chapter 26: Testing, Gently — Hypothesis Tests by Counting

## Part IV — Statistics Without Fear

"A hypothesis test is a courtroom for a number. The null hypothesis is 'nothing happened' — and you, the analyst, are the prosecution."

### In this chapter you will learn

- The logic of testing: null hypothesis, evidence, and how surprising is too surprising.
- p-values, finally defined without flinching — and the two wrong readings of them.
- The Sunday question, formally closed: a two-sample t-test, by hand and by tool.
- The before/after question: paired tests, and why pairing is a gift.
- Significance vs importance, and the three honest questions every test should end with.

## 26.1 The Courtroom

The ledger's first question — *is Sunday genuinely weaker, or is that the noise of ordinary weeks?* — becomes a test. The logic, which is a courtroom's:

- **Null hypothesis ( $H_0$ ):** nothing happened — Sunday's *true* mean takings equal the weekday's. (Innocent until proven guilty: difference = 0.)
- **Alternative ( $H_1$ ):** something happened — Sundays differ.
- **Evidence:** the data — Sunday mean 2,150 vs weekday mean 2,990.
- **The question the test answers:** *if nothing were really happening, how surprising is evidence this strong?* If "very" — stronger than chance serves up, say, one time in twenty — we convict: reject  $H_0$ , declare the difference real.

That last clause is the **significance level**,  $\alpha$ , chosen in advance (0.05 is the convention; 0.01 for stricter). It is the court's standard of proof, and choosing it *before* seeing evidence is not ceremony — it is the difference between testing and rationalising.

## 26.2 The p-Value, Without Flinching

The **p-value**: the probability of seeing evidence at least this strong, *if the null were true*. Not "the probability the null is true" (wrong reading #1 — it assumes the null to compute, so it cannot measure the null's own chances). Not "the probability the result is a fluke" (wrong reading #2, the same error in casual clothes). It is a *conditional surprise score*:  $p = 0.03$  means "in the nothing-happened world, a gap this big shows up 3% of the time."

The decision rule, mechanical and safe:  $p < \alpha \rightarrow \text{reject } H_0$ ;  $p \geq \alpha \rightarrow \text{fail to reject}$  (never "accept the null" — failing to convict is not proving innocence; absence of evidence is evidence of absence's *weakness*, nothing more). Two error types, named once, remembered by asymmetry: **Type I** = convicting the innocent (false alarm, false positive — a "significant" Sunday effect that isn't); **Type II** = acquitting the guilty (missed effect).  $\alpha$  controls Type I directly; sample size and effect size control Type II. Analysts guard Type I like a budget, because a false alarm sends the business chasing ghosts.

## 26.3 The Sunday Test, Closed

Two independent groups (Sunday days vs weekday days), comparing means: the **two-sample t-test** — the z-machinery of Chapter 24 wearing a small-sample correction (Student's t: slightly wider than the normal for small  $n$ , converging to it as  $n$  grows — every tool has it built in; Appendix D states the formula). The arithmetic skeleton, computed honestly:

```
Sunday days:  n1 = 78,  mean1 = 2,150,  s1 = 590
Weekday days: n2 = 390, mean2 = 2,990,  s2 = 655
difference = 840
SE of difference =  $\sqrt{(s_1^2/n_1 + s_2^2/n_2)} = \sqrt{(4468 + 1099)} \approx 45.7$ 
t = 840 / 45.7  $\approx$  18.4
```

A  $t$  of 18.4 against roughly 200 degrees of freedom:  $p < 0.0001$  — far past any  $\alpha$  you would ever set. **Verdict: reject  $H_0$ . Sunday's shortfall is real** — in the nothing-happened world, a gap this consistent across 78 Sundays essentially never occurs. The ledger's first formal closure, and notice what the test added to the Chapter 24 informal  $z$ : a *probability attached to the surprise*, which is what "significance" means — nothing mystical, just surprise, measured.

In a tool, the same test is three clicks (Analysis ToolPak → t-Test: Two-Sample; SPSS → Analyze → Compare Means → Independent-Samples T Test; Python/Excel functions in Appendix D) — but you can now *read the output*, which is the skill: the  $t$ , the  $df$ , the  $p$ , and the group means it was all computed from.



## 26.4 Pairing: Before and After

The price-rise question has a different shape: *the same items*, before and after month 9 — two measurements on one subject. That is the **paired** design, and pairing is a gift: each item serves as its own control, the week-to-week noise cancels, and the test sharpens. Cook oil quantities per month per item, before vs after: compute each item's *change*, then test whether the mean change is zero (a one-sample t-test on the differences — that is all a paired t-test is):

```
mean change in monthly quantity: -1.3 bottles per item
SE of change ≈ 0.28,  t = -1.3/0.28 ≈ -4.6,  df = 11  →  p ≈ 0.0008
```

Reject  $H_0$ : **the price rise cut quantities, really** — closing ledger question #2 (the quantity-led collapse from Q4, now with a p-value on it). The rule of craft: *if your data has a natural pairing (before/after, matched stores, same customer twice), never unpair it* — the unpaired test pays for its independence with noise, and noise is paid for in missed findings.

## 26.5 Significance Is Not Importance

The discipline that separates statisticians from significance-spitters:

- **A huge n makes trivial differences significant.** 6,420 sales will "significantly" detect a 2-cent basket difference. So what? The effect size — the difference itself (840 dollars per Sunday; -1.3 bottles per item) — is the business number; the p-value only certifies it is not noise. Report both, always: "*Sundays are 840 dollars lower (95% CI: 750–930),  $p < 0.001$ .*"
- **A tiny n hides real effects** (Type II): "not significant" on 12 observations is a shrug, not a verdict.
- **The three honest questions** ending every test: (1) *How big is the effect, with its interval?* (2) *Was  $\alpha$  chosen before looking?* (3) *Is this the first test of this question, or the fourth attempt to find something?* — the multiple-comparisons problem, probed gently: run 20 tests on nothing and one will be "significant" at 0.05 by luck (p-hacking's mechanism; Chapter 29's SPSS output makes it easy to commit accidentally; the professional defence is declaring the question before running the numbers).

**From Your Toolkit — the courtroom travels:** every tool in this book runs these tests from menus or one-liners — `T.TEST` in spreadsheets, SPSS/Stata dialogs (Chapter 29), `scipy.stats.ttest_ind/ttest_rel` in Python (Chapter 36), and the A/B-testing world of tech is this chapter industrialised (Chapter 57). The t, the p, the effect size, the CI: learn to read them once, and every output you ever meet is the same six lines.

## Key Takeaways

- Testing is a courtroom:  $H_0$  "nothing happened", evidence, and "how surprised should we be?" — with  $\alpha$  set in advance.
- p-value = probability of evidence this strong *if  $H_0$  is true*;  $p < \alpha$  rejects; failing to reject is not accepting.
- Type I (false alarm) is guarded by  $\alpha$ ; Type II (missed effect) by  $n$  and effect size; Sunday's  $t = 18.4$ ,  $p < 0.0001$  — closed.
- Paired data: test the differences — pairing cancels noise; the price rise cut quantity ( $t = -4.6$ ,  $p \approx 0.0008$ ) — closed.
- Effect size + CI +  $p$ , always; significance  $\neq$  importance; and beware the twentieth test.

## Practice Lab

1. By hand, then by tool: recompute the Sunday t-test on your own data (the group stats, SE,  $t$ ); then run `=T.TEST(...)` or the ToolPak and reconcile the p-value; paste both into the log with the ledger's question number it closes.
2. The paired version: before/after month-9 *revenue* per item (not just oil) as a paired test; write the finding with effect size, CI, and  $p$  — and one sentence on which framing (paired on items vs unpaired on months) you trust more, and why.
3. The significance-importance drill: invent two findings — one significant-but-trivial, one important-but-insignificant (small  $n$ ) — written exactly as you would report them; hand both to a friend and ask which decision they would change. (The correct answer: neither, until effect size is read.)
4. The p-hacking demonstration: take 20 random splits of weekday takings (e.g., odd vs even `sale_ids`) and test each; count your "significant" results; write the one-sentence lesson and pin it above your desk.
5. Ledger day: every question opened since Chapter 2 gets a status — closed with a test (cite it), closable with a test (name it), or descriptive-only (say so). This document is Part IV's real deliverable and Project 3's spine.

## Further Reading

- Chapter 27 (categories and surveys — chi-square and proportions), Chapter 29 (the same tests in SPSS and Stata)
- *The Art of Statistics* (Spiegelhalter) chapter 13 on p-values; the American Statistical Association's six principles on p-values (short, sobering)

# Chapter 27: Categories and Surveys

---

## *Part IV — Statistics Without Fear*

"Half the world's data is counts. It deserves better than being forced into averages."

### **In this chapter you will learn**

- Categorical data analysis: percentages done honestly, and the count-of-what trap.
- Cross-tabulations: the pivot table's statistical soulmate.
- Chi-square: the courtroom for counts, built by hand once.
- Surveys from first principles: questions, sampling, and margins of error.
- Tariro's customer survey — designed, executed, and analysed, in SPSS's menus step by step.

## **27.1 Counts Are Data Too**

Payment types, suburbs, loyalty tiers, weekdays, buy/didn't-buy — categorical data has no mean, and averaging it (or, worse, "averaging the codes") is a category error that production dashboards commit weekly. The honest statistics of categories are **counts, percentages, and cross-tabs** — with three rules that keep them truthful:

1. **Percent of what?** — every percentage declares its denominator. "23% are members" (of customers) vs "31% of revenue is members" (of dollars) differ because the denominators differ. The unlabelled percentage is the most common lie in business reporting.
2. **Count beside rate.** "Suburb X: 80% repeat-purchase rate" on  $n = 5$  is a headline made of air; the count rides along, always (Chapter 10's pairing habit, categorical edition).
3. **Rare categories and the small-denominator trap.** Percentages of tiny groups swing wildly; collapse small categories into "Other" (stated) or report them as counts only.

## 27.2 The Cross-Tab

A **cross-tabulation** is category-by-category counts — the pivot table (Chapter 10) with its statistical hat on. Tariro's loyalty question in cross-tab form: rows = loyalty tier (Gold/Silver/none), columns = "shopped this month?" (yes/no):

	Shopped	Not shopped	Row total
Gold	34	6	40
Silver	90	50	140
None (no card)	310	150	460
Column total	434	206	640

Read a cross-tab by **row percentages** (each row's counts as % of its row total): Gold 85% shopped, Silver 64%, none 67%. The eye's question: *do the rows differ?* Gold looks special; Silver looks like everyone else — and that pattern (loyal core, then nothing) is the planted design made visible. The cross-tab shows the pattern; the courtroom decides if it is real; enter chi-square.

## 27.3 Chi-Square, by Hand Once

**Chi-square ( $\chi^2$ )** tests whether two categorical variables are *associated* — whether the table's rows differ more than sampling noise explains. The logic is the courtroom's, applied to each cell: compare **observed** counts (O) against **expected** counts (E) — what each cell *would* contain if the variables were completely independent ( $E = \text{row total} \times \text{column total} / \text{grand total}$ ) — and total the standardized surprises:

$\chi^2 = \sum (O - E)^2 / E$	over every cell
Gold/Shopped:	$E = 40 \times 434 / 640 = 27.1$ ; $(34 - 27.1)^2 / 27.1 = 1.76$
Gold/Not:	$E = 40 \times 206 / 640 = 12.9$ ; $(6 - 12.9)^2 / 12.9 = 3.69$
Silver/Shopped:	$E = 140 \times 434 / 640 = 94.9$ ; $(90 - 94.9)^2 / 94.9 = 0.25$
... (all six cells summed) $\chi^2 \approx 5.5$	

Degrees of freedom =  $(\text{rows} - 1)(\text{cols} - 1) = 2$ ; the  $\chi^2$  table (or any tool) gives **p**  $\approx$  **0.064** — *fail to reject* at  $\alpha = 0.05$ . The honest verdict: the tier-vs-activity association is *suggestive but not established*; Gold's 85% is built on 40 customers, and counts that small wobble. (Note what the test did *not* say: "no difference exists" — it said the evidence is not yet strong enough to convict. Collect more Gold members, or more months, and retry.)

Two craft rules for  $\chi^2$ : **expected counts  $\geq 5$  per cell** (the approximation degrades; collapse categories or use the exact test — tools offer "Fisher's exact" for small cells); and **df is not a formality** — it is how many cells could freely vary, and the table's p-value hangs on it.

## 27.4 Surveys, First Principles

Tariro's Sunday-promotion question (Chapter 25's memo promised this) needs data nobody has: what customers *would* do. The survey is the instrument, and surveys fail at design long before analysis. The craft in five rules:

1. **Define the population and the frame.** Population: all shoppers. Frame: who you can actually reach — customers in the shop next week (which misses the Sunday-stayers at home; write that limitation down).
2. **Sample honestly.** Every shopper a chance, no hand-picking friends (convenience sampling is fine for exploration, poison for inference); aim for  $n$  from the Chapter 25 calculation ( $30 \rightarrow \pm 18\%$  at worst; for a real margin you want  $100+ \rightarrow \pm 10\%$  or better;  $385 \rightarrow \pm 5\%$  — the famous  $n$ ; diminishing  $\sqrt{n}$  returns beyond).
3. **Ask one thing at a time.** "Would you shop Sundays if we ran double points?" — not "...and would you also like longer hours?" Double-barrelled questions manufacture uninterpretable answers.
4. **Offer a scale with a neutral, and label it.** A 5-point from "Definitely would" to "Definitely would not" with a middle "Unsure" — balanced, labelled, and analysable as counts per level (not averaged into a "2.7 attitude").
5. **Pretest on five people.** Every question that five humans misread is a question fixed before it lies to you at scale.

## 27.5 The Survey, in SPSS

The chapter's deliverable: Tariro's survey, run for real ( $n = 112$  in-shop respondents over two Saturdays), analysed in **SPSS** — the classic statistics package you will meet in research, government, health, and market-research work (installation: the free trial or your institution's licence; Appendix D's tool tables carry the equivalents). SPSS's philosophy is the *opposite* of SQL's: no code visible, menus all the way — and behind every menu is a chapter you have already lived. The run-through:

1. **Enter the data** — Variable View: name each column (`sunday_intent`, `tier`, `usual_spend`), set Type (numeric/string) and, crucially, **Value Labels** (1 = "Definitely would", 2 = "Probably would"... 5 = "Definitely would not") — SPSS's equivalent of a data dictionary, and where categorical hygiene happens. Data View: one row per respondent.
2. **Frequencies** — Analyze → Descriptive Statistics → Frequencies: counts and percentages per question. The first honest look: 18% "Definitely would", 26% "Probably would", 22% unsure...

3. **Cross-tab with the test** — Analyze → Descriptive Statistics → Crosstabs: `sunday_intent` by `tier`; click Statistics → Chi-square; Cells → Row percentages. The output table is Chapter 27.2's cross-tab with the  $\chi^2$ , df, and p appended at the bottom — read exactly as this chapter taught: effect (the row percentages) first, then the surprise score, then the small-cell warnings SPSS prints honestly ("X cells have expected count < 5" — you know that sentence now).
4. **The finding, written properly:** "Among surveyed shoppers (n = 112), intent to shop Sundays was higher among loyalty members (51% definitely/probably would) than non-members (37%);  $\chi^2(4) = 9.8$ , p = 0.045 — a suggestive but modest association from a convenience sample; the promotion should be trialled and *measured in till data*, not assumed from the survey."

That last clause is the survey-analysis wisdom: surveys say what people *say*; tills say what people *do*; the professional uses the survey to design the experiment and the till to judge it — which is exactly the four-Sunday trial Project 4's dashboard will measure.

**From Your Toolkit — counts everywhere:** cross-tabs return as pandas' `crosstab` (Chapter 34), Power BI matrix visuals (Chapter 42), SQL's `GROUP BY two columns` (Chapter 16 — you have been cross-tabbing since Part III); chi-square lives in `scipy.stats.chi2_contingency` (Chapter 36) and every statistics package ever shipped. Survey margins of error are the Chapter 25 arithmetic on proportions — the "±3 points" of news polls, demystified forever.

## Key Takeaways

- Categorical data's honest statistics: counts, percentages (denominator declared), cross-tabs — count beside rate, always.
- Chi-square: observed vs expected per cell,  $\chi^2 = \Sigma(O-E)^2/E$ , df =  $(r-1)(c-1)$ ; expected  $\geq 5$  per cell; p  $\approx 0.064$  means suggestive, not established.
- Surveys fail at design: define the frame, sample honestly, one thing per question, labelled scales, pretest on five.
- SPSS is menus over this chapter: Variable View labels, Frequencies, Crosstabs + chi-square — you can read the output because you built the logic by hand.
- Surveys say; tills do. Design the trial with the survey; judge it with the till.

## Practice Lab

1. The hand  $\chi^2$ : complete the six-cell computation on the loyalty cross-tab above; reconcile your  $\chi^2$  against a tool (SPSS if installed, or a spreadsheet's CHISQ.TEST); write the verdict sentence with effect, statistic, and p.
2. The denominator audit: find five percentages in your Part II report or a news article; label each denominator; rewrite the two that were ambiguous into honest sentences.
3. Design Tariro's survey properly: write five questions (one at a time, labelled scales), the sampling plan with its stated limitation, and the n you would need for a  $\pm 5\%$  margin on the key proportion (Chapter 25's formula — show the arithmetic).
4. SPSS (or spreadsheet equivalent): enter a 30-row mini-survey, run Frequencies and Crosstabs with chi-square, and write the four-line finding in the Chapter's format; if SPSS is unavailable, build the identical outputs with pivots + CHISQ.TEST and note the translation.
5. The say/do memo: one page for Tariro — what the survey suggests, its three limitations (frame, n, self-report), and the four-Sunday trial design (what till metric will decide, what threshold counts as success). Portfolio-grade; keep it.

## Further Reading

- Chapter 29 (SPSS and Stata, properly), Chapter 36 (chi-square in Python)
- *Asking Questions* (Bradburn, Sudman, Wansink) — survey craft from the people who built the field

# Chapter 28: Relationships — Correlation, Regression, and the Trap

## Part IV — Statistics Without Fear

"Correlation is a fact about data. Causation is a claim about the world. The first is arithmetic; the second requires an argument."

### In this chapter you will learn

- Correlation: the scatter plot's number, built by hand once, read honestly.
- The line of best fit: regression as "the line that minimises squared wrongness".
- Prediction with a line — and prediction intervals, the honesty attachments.
- The three explanations of any correlation, and the causation trap's escape routes.
- Tariro's price-and-quantity question, answered the professional way.

## 28.1 The Scatter's Number

Chapter 11's sleeper chart — the scatter — now gets its arithmetic. **Correlation (r)** measures the strength and direction of a *straight-line* relationship between two numeric variables, on a ruler from  $-1$  (perfect negative) through  $0$  (no linear relationship) to  $+1$  (perfect positive).

Built by hand once, on five points of price vs bottles-sold (cooking oil, monthly): the trick is to standardise both variables (z-scores, Chapter 24) and multiply —  $r$  is the average of "x's z times y's z". When both are unusually high together (expensive month, few bottles — negative products) or unusually low together, the products run negative; months that agree in direction push  $r$ ; months that disagree pull it. Five points of oil:  $r \approx -0.87$  — a strong negative relationship: higher price, fewer bottles. (The formula, for the record:  $r = \Sigma(zx \cdot zy) / (n-1)$ .)

Reading rules that hold forever:  $|r| < 0.3$  weak (usually ignorable, in business data);  $0.3-0.7$  moderate (interesting, exploitable);  $> 0.7$  strong (rare, suspicious — check it is not an artefact);  $r \approx 0$  means *no straight-line* relationship — not "no relationship" (a U-shape has  $r \approx 0$  and a perfect relationship; plot before you conclude — the scatter, always the scatter).



## 28.2 The Line

The **line of best fit** (least-squares regression) answers the question the scatter begs: *what line summarises this cloud?* — specifically, the line minimising the sum of *squared* vertical misses (residuals — squared for the Chapter 23 reason: positivity plus severity). Two numbers define it, and both are sentences:

bottles  $\approx 6.2 - 1.1 \times \text{price}$  (on the oil months)  
 slope  $-1.1$ : each extra dollar of price  $\approx 1.1$  fewer bottles per month  
 intercept  $6.2$ : at price zero — extrapolation, not knowledge (no data there)

The **slope is the finding** — "a dollar costs a bottle a month" is a sentence a shop owner acts on (it prices the wholesaler's increase: at  $+0.80$ , expect  $\approx -0.9$  bottles/month per customer). The **intercept** anchors the line but means nothing outside the data's range. **R²** (r squared, here  $\approx 0.76$ ) is "the share of the y-variance the line explains" — 76% of month-to-month bottle swings track price; the other 24% is everything else (paydays, rivals, weather) — the residual world where the next question lives.

**Prediction, with honesty attached:** at price 3.60 the line says 2.24 bottles. That is a *point* prediction; the honest version carries an interval wide enough to matter (prediction intervals widen with distance from the data's centre and with the residual scatter — every tool prints them; Appendix D has the formulas). Lines are for interpolating within the data's range; extrapolation — price 9.00, predicted  $-3.7$  bottles (impossible, and the line does not care) — is where straight lines lie confidently. Extrapolate at your reputation's risk.

## 28.3 The Trap, Formally

Honesty Rule 2 becomes a taxonomy. You find a correlation — oil price and quantity,  $r = -0.87$ . Three explanations, *always all three, always in this order*:

1. **Coincidence** — two unrelated series drifting together ( $n = 18$  months is small; spurious correlations are industry on the internet). Defence: the surprise score (a significance test on  $r$  —  $p \approx 0.0005$  here, so not *pure* luck, though significance never converted coincidence to cause by itself).
2. **Confounding** — a third variable drives both (oil price and quantity both driven by the *harvest year*; ice creams and drownings both driven by *summer*). The most common truth in observational data; the defence is to *name and measure the confounder* — or hold it constant (compare within one harvest year only).
3. **Causation** — X actually moves Y. The claim requires an *argument*, and in business the argument's gold standard is the **experiment**: change the price deliberately, on one shelf, for one month, and watch quantity move

while everything else holds — the A/B logic of Chapter 57's world, available to Tariro at shelf scale.

The professional's phrasebook: "price and quantity are strongly negatively correlated ( $r = -0.87$ ,  $p < 0.001$ )" — always safe, always true. "The price rise cost us about 1.1 bottles per customer per month, pending a shelf trial" — a causal claim, honestly flagged as pending. What is never safe: sliding from the first sentence to the second without noticing. (Classic traps for your collection: summer  $\rightarrow$  both; health-conscious people both buy gym memberships and eat salad (self-selection — the loyalty-scheme caveat's family); richer suburbs buy more of everything — *everything correlates with money*.)

## 28.4 Tariro's Question, Answered Properly

The ledger's price question, closed in the manner Part IV has taught — every element of the arc in one finding:

*Cooking-oil price and monthly quantity per customer are strongly negatively correlated ( $r = -0.87$ ,  $p < 0.001$ ,  $n = 18$  months); the fitted line estimates  $\approx 1.1$  fewer bottles per month per dollar of price ( $R^2 = 0.76$ ). The month-9 rise of 0.80 therefore predicts  $\approx 0.9$  fewer bottles — matching the observed  $-1.3$  within the noise (paired test, Ch. 26:  $p \approx 0.0008$ ). Because price was not experimentally varied, the causal reading rests on the mechanism (a known wholesaler increase) and the size and consistency of the association; a one-shelf price trial would close the argument.*

Correlation to size the relationship, a test to rule out noise, an interval to bound the estimate, a mechanism to argue cause, an experiment to finish the job — that paragraph is the whole of Part IV assembled, and it is the calibre of finding that gets analysts hired (Chapter 46 will publish it as portfolio evidence).

## 28.5 Many Variables, a Glimpse

Real questions have several drivers — quantity by price *and* payday week *and* rival promotions. **Multiple regression** extends the line to several x's (quantity  $\approx a + b_1 \cdot \text{price} + b_2 \cdot \text{payday} + \dots$ ), each slope "the effect of that variable, *holding the others constant*" — the arithmetic of controlling for confounders. It is Part V-and-beyond territory (Chapter 36 runs it in Python; Chapter 58 meets its machine-learning descendants), but two warnings travel with the glimpse, today: a slope's meaning depends on *which other variables are in the model* (add/remove one, the story changes — "controlled for what?" is the reader's question); and more variables eat n (each one spends degrees of freedom; 18 months will not support eight predictors). When you meet a multiple regression in the wild, you now know the two questions to ask.

**From Your Toolkit — the relationship kit:** CORREL/LINEST in spreadsheets, `corr()` in SQL (some engines; else compute via the appendix recipe), pandas' `.corr()` and seaborn's scatter-matrix (Chapter 35), SPSS's Analyze → Correlate/Regression dialogs (Chapter 29), and Power BI's scatter visual with a trend line (Chapter 42). The *reading* — slope as finding,  $R^2$  as share, the three explanations, experiment as gold standard — transfers intact everywhere; it is the same paragraph in every tool.

### Key Takeaways

- r measures straight-line association: 0.3/0.7 as rough anchors;  $r \approx 0 \neq$  no relationship — plot the scatter, always.
- The line: slope is the finding ("a dollar costs a bottle"), intercept anchors only,  $R^2$  is the share explained; predict within range, with intervals.
- Any correlation has three explanations — coincidence, confounding, causation — and only an argument (ideally an experiment) licenses the third.
- The professional paragraph: correlation + test + interval + mechanism + proposed trial — Part IV's arc in five clauses.
- Multiple regression holds others constant; ask "controlled for what?" and watch n.

### Practice Lab

1. Hand-r: the five oil months (price, bottles) — standardise both columns, multiply, average; reconcile with CORREL; write the three-clause reading (strength, direction, significance).

2. The line, by tool: fit bottles  $\approx a + b \cdot \text{price}$  (spreadsheet trendline, SPSS, or Python); write the slope-sentence and the  $R^2$ -sentence; predict at the data's min, max, and  $1.5 \times$  the max — and annotate the third prediction "extrapolation, handle with care".
3. The confounder hunt: on Tariro's data, find two variables that correlate but are plausibly both driven by a third (e.g., revenue and baskets by month — both driven by customer count); name the confounder and write the "controlled for what?" question it provokes.
4. The trap collection: find two published spurious correlations (the internet's galleries are full of them); for each, write which of the three explanations is most likely and what evidence would separate them.
5. The trial design (ledger's next opening): design the one-shelf price trial — what price, which shelf, how long, what till metric decides, and what threshold counts as "the line was wrong". One page; it is Project 4's feedstock and portfolio evidence of experimental thinking.

## Further Reading

- Chapter 36 (regression in Python), Chapter 29 (SPSS/Stata's regression dialogs)
- *The Book of Why* (Pearl) — causation's deep theory, readable; *Spurious Correlations* (Vigen) — the comedy branch of this chapter

# Chapter 29: SPSS and Stata, Gently

## Part IV — Statistics Without Fear

"Some tools you choose. Some tools you inherit. SPSS and Stata are the family silverware of statistics — and you will be invited to dinner sooner than you think."

### In this chapter you will learn

- What SPSS and Stata are, where you will meet them, and how they differ in spirit.
- The SPSS workflow, complete: data view, variable view, menus, output, syntax.
- Stata for beginners: the command line, the do-file, and why it is loved.
- Reproducing Part IV's analyses in both — a translation table you keep.
- What employers mean by "SPSS/Stata experience" — and how to get it honestly.

### 29.1 The Classic Tools

Python (Part V) is the general-purpose language of modern data; **SPSS** and **Stata** are the *specialised* statistics packages — decades old, beloved in the fields that grew up with them, and still running the numbers in governments, hospitals, universities, central banks, and market-research houses across the world. The geography of encounter, so it never surprises you:

Tool	Where you'll meet it	Spirit
<b>SPSS</b> (IBM)	Social sciences, health research, market research, survey shops, psychology	Menus and dialogs; point-and-click statistics; output as pretty tables
<b>Stata</b>	Economics, epidemiology, policy evaluation, academia	Command line; everything scripted in "do-files"; reproducibility as religion
(SAS, R)	Pharma/clinical trials; statistics research & data science	Mentioned for completeness — R appears again in Chapter 57's world

The one-sentence contrast you can say in an interview: *SPSS is the tool that puts statistics in menus; Stata is the tool that puts statistics in sentences*. You have already used SPSS's menus (Chapter 27's cross-tab); this chapter completes the picture and gives Stata its first honest hour.

## 29.2 SPSS, the Complete Workflow

Everything SPSS does happens in four windows, and this book's habits map onto them one-to-one:

1. **Variable View** (the data dictionary, made visible): every column has Type, Label, **Values** (the value labels — 1 = "Gold", 2 = "Silver" — categorical hygiene lives here), Missing (declared missing codes — the Chapter 15 NULL discipline, GUI edition), and Measure (Nominal/Ordinal/Scale — you declaring what each column *is*, which SPSS uses to offer sensible menus). Fifteen careful minutes here saves every hour after; skipping it is how "averaged the suburb codes" is born.
2. **Data View**: the grid — one row per unit (respondent, sale, patient). SPSS will happily let you type in it; the professional reads it, cleans elsewhere, and documents.
3. **The dialogs** (Analyze menu): Descriptives and Frequencies (Ch. 22), Explore with plots (Ch. 23's histograms and the five-number summary), Crosstabs with chi-square (Ch. 27), Compare Means → T tests (Ch. 26), Correlate and Regression (Ch. 28). Each dialog is a chapter you have already lived; the buttons are in Appendix D's walkthrough.
4. **The Output Viewer**: results as formatted tables with significance stars and footnotes — the tables you now *read fluently* because this Part taught the six lines every test prints (statistic, df, p, effect, interval, n).

And the feature that elevates SPSS from calculator to profession: **Syntax**. Every menu run can Paste (instead of OK) its commands to a syntax window, saved as a .sps file, and re-run — SPSS's version of the query pack from Chapter 21. The working habit, identical to SQL etiquette: *menus to explore, syntax to deliver*. Your Chapter 27 cross-tab, pasted as syntax, is two lines that reproduce the finding on next month's data:

```
CROSSTABS TABLES=sunday_intent BY tier
/CELLS=COUNT ROW /STATISTICS=CHISQ.
```

## 29.3 Stata, the First Honest Hour

Stata has no menu-first culture; it has a **command line** and the **do-file** — and the culture is this book's culture (reproducible steps, named files, documented decisions) in its purest form. The canonical session:

```
* tariro_analysis.do -- monthly pack, Stata edition
import delimited "sales.csv", clear
describe
summarize amount, detail
* the Sunday test (Ch. 26): two-sample t
```

```
gen weekend = cond(dayname == "Sun", 1, 0)
ttest takings, by(weekend)
* the price relationship (Ch. 28)
regress bottles price
```

Read the shape of it: `*` comments (the why, as always), one command per line, each command an English verb with options after the comma. `summarize`, `detail` is Chapter 22's five-number summary; `ttest`, `by()` is the Sunday courtroom; `regress` is the line of best fit. Results print to a log; the do-file plus the log *is* the analysis — an auditor (or your successor) re-runs the do-file and watches your numbers re-emerge. That is Stata's beloved property, and it is the destination this book has been steering you toward since Chapter 8's log: **analysis as a document, not a performance**.

The beginner's Stata kit (Appendix D expands): `import delimited`, `describe`, `summarize`, `generate` (new variables), `keep/drop` (rows and columns), `tabulate` (cross-tabs, with `chi2`), `ttest`, `regress`, `save`. Every one maps to a chapter behind you — the translation table below is the proof.

### 29.4 The Translation Table

The deepest lesson of this chapter: you have already learned the analysis; the tools are accents. Keep this table; it is the Rosetta Stone of interviews (Chapter 52 will quiz it):

The task	Excel/Sheets	SQL	SPSS	Stata	Python (ahead)
Typical value	AVERAGE /MEDIAN	AVG, PERCENTILE_CONT	Descriptives/Frequencies	summarize	.mean()/median()
Five numbers	QUARTILE	PERCENTILE_CONT	Explore	summarize, detail	.describe()
Cross-tab	PivotTable	GROUP BY two cols	Crosstabs	tabulate x y, chi2	pd.crosstab
Two-group test	T.TEST	(CTE + arithmetic)	Independent-Samples T	ttest, by()	ttest_ind
Paired test	T.TEST paired	(CTE + arithmetic)	Paired-Samples T	ttest a=b	ttest_rel
Association	CORREL	(arithmetic)	Correlate	correlate/pwcorr	.corr()
The line	Trendline	(arithmetic)	Regression	regress	linregress/ols
Reproducible pack	Workbook +log	.sql files	Syntax .sps	Do-file .do	Notebook/script

Two columns you have not yet earned (Python's) fill in during Part V; the table is the syllabus's own map. When an interviewer asks "you know Stata, not

SPSS — how long to be useful?", the honest, winning answer is built from this table: *the statistics are the same six instruments; the syntax is a week, not a year.*

## 29.5 Getting Experience, Honestly

Employers in SPSS/Stata worlds ask for "experience with the package" — and the honest way to have it is to have *done the analyses*, because the package is 10% of the difficulty. The plan, all free or cheap:

1. **SPSS:** IBM's free trial (full package, time-limited) and many universities' free student licences; PSPP (the GNU clone) runs Frequencies/Crosstabs/T-test if licensing blocks you. Rebuild Chapter 27's survey analysis end-to-end, in syntax, saved.
2. **Stata:** no free full version, but the affordable student licence and the free **Stata tutorial datasets** (`sysuse auto`) make the first hour real; rebuild the Sunday test and the oil regression as a do-file, commented, logged.
3. **The portfolio line, earned:** "Statistics in Excel, SQL, SPSS and Stata: hypothesis tests, cross-tabs, chi-square, regression — same analyses, four tools" — and Projects 2–3's artefacts as evidence. That sentence, *backed by artefacts*, is worth more than any course certificate, and Chapter 46 will show you how to display it.

Part IV closes here. You can describe (typical values, spread, shape), standardise (z, percentiles), argue from samples (SE, CI), and conclude (tests, chi-square, correlation, regression) — in four tools, with the honesty rules enforced throughout. Tariro's ledger is closed: Sundays are real ( $t = 18.4$ ), the price rise bit ( $p \approx 0.0008$ ,  $\approx 1.1$  bottles per dollar), loyalty is suggestive-not-proven ( $\chi^2 p \approx 0.064$ ) and pending its trial. What remains is the *labour*: every month, the same cleaning, joining, testing, charting, reporting — by hand. Part V's Python automates exactly that.

**From Your Toolkit — the family silver:** SPSS and Stata are not detours from this book's path; they are the same road wearing older asphalt. Every habit — the dictionary before the analysis, syntax over menus, the do-file as deliverable, the six-line output read — is a habit you already own from spreadsheets and SQL. The table above is your certificate of transfer.

## Key Takeaways

- SPSS = menus and pretty output (surveys, social science, health); Stata = commands and do-files (economics, policy); both run the same six instruments you now command.



- SPSS workflow: Variable View dictionary → Data View → Analyze dialogs → Output; Paste-to-Syntax turns menus into the reproducible pack.
- Stata: verbs with options, do-file plus log = the analysis as document — reproducibility as religion, this book's habits in purest form.
- The translation table: statistics is the skill; tools are accents — a week, not a year, to convert.
- "Package experience" is earned by rebuilding the analyses you already understand, in syntax/do-file form, and filing the artefacts as portfolio evidence.

## Practice Lab

1. SPSS hour: install (trial/university/PSPP), rebuild the survey analysis of Chapter 27 end-to-end — variable labels, Frequencies, Crosstabs +  $\chi^2$  — then Paste it all to syntax, save `survey_ch27.sps`, and re-run it from a blank output viewer. Log the two minutes the re-run took.
2. Stata hour (or Stata-in-spirit): with `sysuse auto` (or any CSV), run `describe`, `summarize detail`, a `tabulate` with `chi2`, and one `regress`; save as `first_hour.do` with a header block and why-comments; save the log beside it. Two files, portfolio-grade.
3. Translation drill: pick three analyses from your Part IV log; write each in two tools from the table (spreadsheet+SPSS, SQL+Stata, any pair); reconcile the p-values and coefficients to three decimal places; note every discrepancy and its cause.
4. The conversion memo: a half-page to a hiring manager (Chapter 51's audience) — "my statistics are tool-agnostic; here is the translation table and three artefacts proving it"; attach the table. This is a CV paragraph rehearsed as a document.
5. Part IV retrospective: one page — the three findings you closed, the two habits that caught the most errors, and the one idea (sampling distributions? the three explanations?) you would explain to a colleague tomorrow. File it with the portfolio; Project 3's write-up begins here.

## Further Reading

- Part V (Python — the automation of everything this Part did by hand)
- UCLA's IDRE "Which test?" pages and Stata/SPSS learning modules — the internet's best free companions to this chapter

**PART V**

# **Python: From Zero to Dangerous**

Chapters 30 – 37

---

# Chapter 30: Why Python, and How to Start

---

## Part V — Python: From Zero to Dangerous

"Spreadsheets repeat. SQL re-queries. Python *works* — the same file, every month, forever, on any data you point it at."

### In this chapter you will learn

- Why Python earns a whole part of this book — the three jobs no other tool does.
- Python and Jupyter: what they are, and how to install in under an hour.
- The notebook way of working — code, output, and story in one document.
- Your first five cells, on Tariro's real data.
- How this part is structured, and how to practise so it sticks.

## 30.1 The Three Jobs

Part II answered questions with clicks; Part III saved the clicks as queries; Part IV gave the conclusions machinery. What is left? The *labour*: every month, Tariro's CSVs arrive, and the whole chain — import, cleaning checks, joins, tests, charts, report — runs again. In tools so far, that means re-doing. Python's pitch is exactly the remainder:

1. **Automation** — write the chain once; every month it is one command. (The monthly pack of Project 2 becomes a script that never sleeps.)
2. **Scale** — Python reads data bigger than spreadsheets (pandas, the star of this Part, works in the hundreds of thousands to millions of rows on a laptop) and connects to anything: CSVs, databases (your `tariros.db!`), APIs, the web.
3. **Everything else** — Python is a *general* language: statistics (Part IV's tests, one line each), machine learning (Chapter 58's door), web apps, glue between systems. Skills compound beyond the dashboard.

And the honest counter-pitch, so you choose tools like a professional: Python is *code* — typos are errors, not silent cells; the learning curve is real (a weekend, not an hour); and for a quick one-off look at 500 rows, the spreadsheet is still the right tool. The working analyst's stack is all five tools of this book, each doing what it is best at — Python's slot is *repeatable work at scale*.

## 30.2 Python, Anaconda, and the Notebook

**Python** is the language; a **Jupyter notebook** is where you will live: a document of *cells*, each cell a few lines of code with its output directly beneath — code, result, and prose interleaved. The notebook is the analyst's lab bench: try something, see it, explain it, move on. (If you have seen a data-science tutorial, you have seen a notebook.)

Installation, the recommended path — **Anaconda** (anaconda.com, free): a Python distribution that arrives with the entire scientific stack pre-installed — pandas, matplotlib, scipy, statsmodels, jupyter — no dependency archaeology. Download, install, launch **JupyterLab** from the Anaconda Navigator (or type `jupyter lab` in a terminal). Total time: under an hour; total decisions: nearly zero. (Already have Python? `pip install pandas matplotlib scipy statsmodels jupyterlab` — but Anaconda spares you the archaeology, and this book recommends it for your first year.)

The environment note that saves confusion: Anaconda Navigator also offers **Jupyter Notebook** (the older single-document interface) — this book's screenshots assume JupyterLab, but every cell works identically in both, and in VS Code's notebook mode, and in Google Colab (the no-installation, in-browser option — perfect if your machine is borrowed or locked down). Pick one; the cells do not care.

## 30.3 The First Five Cells

Open JupyterLab, File → New → Notebook, and save it as `tariro-first-five.ipynb` in your `tariro-data` folder. Then run these — each in its own cell, Shift+Enter to execute, and *watch the output appear beneath*:

```
# cell 1: the load -- pandas reads a CSV into a DataFrame
import pandas as pd
sales = pd.read_csv("sales.csv")
sales
```

A table appears — the whole sales history, neatly gridded, numbered. That object is a **DataFrame**: pandas' spreadsheet, and the centre of this entire Part. Note the last line: in a notebook, a bare name on the last line of a cell *displays* itself — the analyst's flashlight.

```
# cell 2: the shape and the first rows
sales.shape          # (6420, 6) -- rows, columns
sales.head()         # first five rows
```

```
# cell 3: the Part II statistics, wholesale
sales["amount"].describe()
```

One method, and the five-number summary plus count, mean, and standard deviation of the amount column print as a table — Chapter 22's entire chapter, one line. You will use `describe()` more than any other single line in this book.

```
# cell 4: a question answered
sales.groupby("payment_type")["amount"].agg(["count", "sum", "mean"])
```

The Chapter 16 pivot — payment types down the side, count/sum/mean across — in one line. If you slow down and read that line as a sentence (*take sales, group it by payment_type, take amount, aggregate count sum mean*), you have just read your first fluent pandas.

```
# cell 5: the chart
sales.groupby("date")["amount"].sum().plot(figsize=(10, 4), title="Daily takings");
```

A chart of daily takings renders in the notebook — Chapter 11's line, no menus, no spreadsheet, and (this is the point) re-runnable on next month's CSV by changing nothing.

Five cells, and the pitch is made: load, inspect, summarise, question, chart — the whole analytical arc, in a document you can re-run forever. Everything else in this Part is these five cells with more vocabulary.

## 30.4 The Way of Working

Notebook craft, learned early, saves months:

- **Run cells top to bottom.** A notebook has *state* — cell 4 remembers what cell 1 loaded. Running cells out of order is the classic notebook bug (numbers from stale data); the fix is the discipline, and *Restart & Run All* is the truth test — run it before trusting, sharing, or delivering any notebook.
- **Prose cells** (the + button, then switch the cell to Markdown) carry the story: what you are asking, what you found. A notebook that alternates prose and code is a *report that computes itself* — the deliverable format of Part VII's projects and of modern analytics teams.
- **Small cells, often.** Ten lines, run, look; not a hundred lines and a prayer. Errors (next chapter makes them friends) are cheap when cells are small.
- **Name files like the question** (`oil-price-analysis.ipynb`, not `untitled7.ipynb`) — Chapter 20's etiquette, unchanged by the tool.

## 30.5 How This Part Runs

Seven chapters, each a working session on Tariro's data, each ending in the Practice Lab you now expect: **31** variables and data types; **32** the analyst's control flow (loops, functions, and when *not* to loop — the pandas way); **33** pandas properly — the DataFrame grammar; **34** the analysis kit — groupby, join, time series (the whole book so far, re-expressed); **35** charts in matplotlib; **36** the statistics of Part IV, one line each; **37** the capstone — Tariro's monthly pack, automated, plus a professional project structure.

The method that makes it stick, the same method this book has used since Part I: *every concept arrives attached to Tariro's questions; every lab re-uses the artefacts you already built*. You are not learning Python and then applying it — you are re-doing work you understand, in a tool that makes it permanent. That is also the least frightening way to learn a language: the analysis is already in your head; only the accent is new.

**From Your Toolkit — the permanent bench:** the notebook becomes your default workspace for the rest of the book: Part VI feeds Power BI from Python-cleaned data; Chapter 58's machine learning lives here; the portfolio (Part VII) is notebook-driven; even this book's datasets (Appendix C includes the generator recipe) come with a starter notebook. The five cells of this chapter are the bench's five drawers — load, inspect, summarise, question, chart — and every chapter ahead opens them in different orders.

## Key Takeaways

- Python's three jobs: automation (the monthly chain, once), scale (beyond spreadsheets, into databases/APIs), and generality (statistics, ML, glue) — its slot in the stack is repeatable work at scale.
- Anaconda + JupyterLab: one free install, whole scientific stack; notebook = code, output, prose, one document.
- The five-cell arc — read_csv, head/shape, describe, groupby, plot — is the whole Part in miniature.
- Notebook craft: run top-to-bottom, Restart & Run All before trusting, prose cells tell the story, small cells always.
- Learning method: re-do the analyses you already understand, in the new accent — the analysis is in your head; only the accent is new.

## Practice Lab

1. Install: Anaconda (or Colab if installation is impossible), launch JupyterLab, and screenshot your first empty notebook for the log — the workspace milestone from Chapter 4, levelled up.
2. The five cells, typed by hand (no copy-paste — the fingers learn too), on `sales.csv`; caption each with a Markdown prose cell in your own words. Save as `tariro-first-five.ipynb`.
3. The `describe()` reconciliation: `sales["amount"].describe()` against your Part II five-number summary and STDEV.S values — every number must match to the decimal, and a one-line note certifies the bridge.
4. The groupby tour: re-run cell 4 for `till`, for `payment_type`, and for a date-derived month (hint: `sales["date"].str[:7]`) — three pivots, three sentences, same as Chapter 10's lab but re-runnable.
5. The permanence test: close the notebook, close Jupyter, reopen, *Restart & Run All* — everything must re-emerge untouched. Then add one Markdown cell at the top: what this notebook does, data source, date run. That header cell is the notebook edition of the query-pack README, and it is now your habit.

## Further Reading

- Chapter 31 (variables and types — the vocabulary begins)
- *Python for Data Analysis* (McKinney) — by pandas' creator; this Part's natural next step

# Chapter 31: Your First Lines — Variables, Numbers, Text

Part V — Python: From Zero to Dangerous

"Programming is talking to a very fast, very literal friend. It does exactly what you said — which is why you must learn to say what you meant."

## In this chapter you will learn

- Variables: labelled boxes, and the naming habits of professionals.
- Numbers and text (strings): the two workhorse types, with their operations.
- Booleans and comparisons: how Python decides things.
- f-strings: sentences with numbers inside — the analyst's reporting line.
- Errors, read properly — the red wall is a map, not a punishment.

## 31.1 Variables

A **variable** is a labelled box holding one thing:

```
shop_name = "Tariro's Grocery"
average_basket = 8.42
n_sales = 6420
```

The `=` is not mathematics' equality; it is an instruction: *put the right side in the box named by the left*. Read it right-to-left and it never confuses. Use a variable by name (`average_basket * 2`), change it (`n_sales = n_sales + 150` — take the box, add, put it back), and — the professional habits — **name it as what it means** (`average_basket`, not `x`, not `ab`); **snake_case** (lowercase, underscores — Python's universal convention); and let the name carry the unit when money is involved (`revenue_usd`, `price_per_bottle`) — the unlabelled-number rule of Chapter 27, applied to your own code.

## 31.2 Numbers

Two number types, and the difference is one you have met before:

```
n_sales = 6420          # int -- a count; no decimal point
average_basket = 8.42   # float -- a measurement; decimal point present
```

Python's arithmetic is the spreadsheet's: `+` `-` `*` `/`, parentheses for grouping, `**` for powers (`2**10`), and `//` with `%` for whole-division and remainder (`17 // 5 → 3`; `17 % 5 → 2` — the "how many full boxes, how many left over" pair, quietly



useful for weekday math: `day_number % 7`). Floats carry the Chapter 7 warning: `0.1 + 0.2` is `0.30000000000000004` — binary's small print, identical in every tool that uses binary floats, and the reason money is rounded for *display* and reconciled for *truth*. Integers never drift; when a number is a count, keep it an int.

### 31.3 Text

A **string** is text in quotes (single or double — pick one and stay consistent; this book uses double):

```
item = "Cooking Oil 2L"
len(item)           # 14 -- characters, spaces included
item.upper()        # "COOKING OIL 2L"
item.lower()        # "cooking oil 2l" -- the TRIM-family's cousins
item.strip()        # trims edge whitespace (the ghost-blank slayer)
item[:11]           # "Cooking Oil" -- slice: first 11 characters
"Oil" in item       # True -- membership test
```

Strings are Chapter 8's cleaning tools in code form: `.strip()` kills ghost blanks, `.upper()/lower()` unify case (three EcoCash spellings meet their final solution), and slicing takes substrings by position. Strings join with `+` ("`Daily` " + "`report`"), but the join you will actually use is `next` — because analysts do not concatenate; analysts *report*.

### 31.4 f-strings: The Reporting Line

The **f-string** is the single most-used line in the working analyst's Python — text with live values dropped in, formatted on the spot:

```
month = "2025-06"
revenue = 3401.55
n = 389
print(f"{month}: revenue ${revenue:,.2f} from {n} sales")
# 2025-06: revenue $3,401.55 from 389 sales
```

The recipe: an `f` before the opening quote; `{name}` drops a variable in; and after a colon, *format specifiers* — `,` for thousands separators, `.2f` for two decimals, `.0f` for none, `.1%` for percentages (`0.31` prints as `31.0%`). Every sentence of every report you generate in Part VII is an f-string at heart: the chart carries the shape, the f-string carries the number, in the format the reader needs. Practise until `f"{x:,.2f}"` is muscle.

## 31.5 Booleans and Decisions

A **boolean** is a fact: `True` or `False` (capitalised — Python is fussy). Comparisons produce them (`revenue > 3000`, `item == "Cooking Oil 2L"` — `double ==` compares, `single =` assigns; mixing them up is everyone's first bug), and they combine with `and/or/not` — the WHERE clause's combinators, spelled out in English:

```
is_weekend = day == "Sat" or day == "Sun"
big_sale = amount >= 50 and payment_type == "EcoCash"
is_member = customer_id != "none"
```

Booleans are the seeds of decisions (`if` statements, Chapter 32) and of *filters* — and here is the preview that makes Python click for an analyst: in pandas, `sales[sales["amount"] >= 50]` is a filter, and the inner part is exactly the boolean line above. The whole query language of this book — WHERE, filters, `if` — reduces to booleans. Learn them here, spend them forever.

## 31.6 Errors as Friends

Sooner than you plan, a red wall appears:

```
TypeError: can only concatenate str (not "float") to str
```

The instinctive response is dread. The professional response is a checklist, because Python's errors are *specifically, lovingly instructive*:

1. **Read the last line first** — the error *type* and its message. This one says: you used `+` between text and a number. (Fix: an f-string — the error was telling you what to do.)
2. **Read the arrow** — the traceback points at the exact line. Unlike a spreadsheet's silent wrong answer, Python's wrong answer *announces itself with a location* — a luxury, not an insult.
3. **Learn the four regulars**: `NameError` (typo — the box was never made or is misspelled), `TypeError` (mixing types — text plus number), `SyntaxError` (missing quote/colon/parenthesis — the sentence is not English), `KeyError/IndexError` (asking for a column or position that is not there — a data question, usually).

Copy the error's last line, search it verbatim, and the internet answers — errors are so standardised that the fix for yours is documented a million times. The reframe that makes you a programmer: *you are not a person who avoids errors; you are a person who reads them fast*.

**From Your Toolkit — the atoms:** every tool ahead is these atoms assembled: a pandas DataFrame is boxes of columns (variables), filters are booleans, report lines are f-strings, and cleaning is `.strip().upper()` at scale. SQL people: variable = alias, boolean = WHERE, f-string = CONCAT with manners. Nothing new arrives after this chapter that is not *combinations* — which is why the Practice Lab matters more than the reading.

## Key Takeaways

- Variables are labelled boxes; `=` assigns, names mean things, `snake_case`, units in money names.
- int vs float: counts vs measurements; floats round for display only; `//` and `%` for boxes and remainders.
- Strings carry the cleaning kit (`.strip()`, `.upper()`, slicing, `in`) — Chapter 8 as code.
- f-strings are the reporting line: `f"{revenue: ,.2f}"` is muscle to build; every generated sentence uses one.
- Booleans (`==`, `and/or/not`) are WHERE's atoms and pandas filters' seeds; errors are maps — last line first, arrow second, four regulars memorised.

## Practice Lab

1. The shop in variables: create `shop_name`, `revenue_usd`, `n_sales`, `best_day`; print a four-line summary with one f-string per line, all numbers formatted; then change `revenue_usd` and re-print — the boxes update, the format holds.
2. The float wall: compute `0.1 + 0.2`, see the binary truth; then `round(0.1 + 0.2, 2)` and print both with an f-string explaining the difference to a colleague; reconcile a real money sum against the spreadsheet to two decimals.
3. The cleaning atoms: take the three EcoCash spellings ("`EcoCash`", "`ecocash`", "`ECOCASH`") and, with `.strip().upper()` (or `.title()` if you prefer sentence-case), map all three to one; write the one-line before/after table in a Markdown cell.
4. The boolean bench: for six real sales rows (type them as variables), write booleans for "`weekend`", "`big basket`", "`member`", "`EcoCash weekend sale`"; print each with an f-string that states it in words.
5. The error museum: deliberately raise all four regulars (misspell a name, add text to a number, drop a quote, index a list at 99); for each, paste the last error line into a Markdown cell and write the fix underneath. Your museum is your future debugging speed.

## Further Reading

- Chapter 32 (control flow and functions — the atoms start combining)
- *Automate the Boring Stuff with Python* (Sweigart), chapters 1-2 — free online, the friendliest first-sixty-pages in the field

# Chapter 32: Loops, Functions, and the Pandas Way

Part V — Python: From Zero to Dangerous

"A loop repeats. A function is a reusable sentence. And pandas does to both what the combine harvester did to the scythe."

## In this chapter you will learn

- Lists and dictionaries: boxes that hold many things.
- `for` loops and `if` decisions — flow, in two keywords.
- Functions: your own verbs, defined once and used forever.
- The pandas way: *vectorised* operations — why analysts loop rarely.
- A complete mini-program: the daily exceptions report, from scratch.

## 32.1 Collections

Single variables hold one thing; analysis holds *many*. Python's two everyday containers:

```
# a list: ordered, numbered, editable -- the column's raw material
items = ["Maize meal 10kg", "Cooking Oil 2L", "Rice 2kg"]
items[0]          # "Maize meal 10kg" -- counting starts at ZERO
items[-1]         # "Rice 2kg" -- the last, from the end
items.append("Sugar 1kg")
# a dictionary: label -> value -- the row's raw material
sale = {"date": "2025-06-14", "amount": 11.50, "payment_type": "EcoCash"}
sale["amount"]    # 11.5 -- fetch by name, not position
sale["till"] = 2   # add a key
```

Zero-based counting is Python's first ambush: `items[1]` is the *second* item. It is not pedantry; it is how offsets work (position 1 means "one step from the start"), and every tool in this book secretly agrees (Excel's `INDEX`, SQL's `LIMIT ... OFFSET`). The dictionary is the more important analyst object: a table row *is* a dictionary (column names → values), and a `DataFrame` is a million of them — hold that mental image and pandas never feels arbitrary.

## 32.2 Flow: `for` and `if`

The **`for` loop** visits every item in a list; the **`if` statement** chooses:

```
daily_takings = [2980, 1520, 3410, 890, 3120, 4270, 2650]
for takings in daily_takings:
```

```

if takings < 2000:
    print(f"Quiet day: ${takings:,} -- check the log")
elif takings > 4000:
    print(f"Great day: ${takings:,} -- what happened?")
else:
    print(f"Ordinary day: ${takings:,}")

```

Read it aloud: *for each* takings in the list, *if* it is low say quiet, *else if* high say great, *else* ordinary. The colons and the indentation are not decoration — **indentation (four spaces) is how Python knows what belongs inside the loop and the if**. Get it wrong and `IndentationError` tells you; get it right and the structure of your code is visible in its shape, which is why Python reads like a recipe. (A `while` loop also exists — "repeat until a condition breaks"; rare in analyst code, recognised on sight in others'.)

## 32.3 Functions

A **function** is a sentence you define once and use forever — input in parentheses, answer from `return`:

```

def day_verdict(takings, low=2000, high=4000):
    """Classify one day's takings as quiet/ordinary/great."""
    if takings < low:
        return "quiet"
    elif takings > high:
        return "great"
    return "ordinary"

day_verdict(890)          # "quiet"
day_verdict(4270)         # "great"
day_verdict(1520, low=1200) # "ordinary" -- the threshold is an argument

```

The professional payoffs are exactly the query-pack's payoffs (Chapter 20's etiquette, now in code): **named once, tested once, reused everywhere; arguments make assumptions visible** (the thresholds are not buried, they are parameters); and the **docstring** (the triple-quoted sentence under the `def`) documents the *why*. Your Part II IQR fence, your z-score, your reconciliation check — all are functions in this book's `toolkit.py` by the end of this Part. Analysts who write functions stop repeating themselves; analysts who stop repeating themselves stop making copy-paste errors — the same argument as CTEs, the same argument as syntax files, the same argument as do-files. It is all one discipline.

## 32.4 The Pandas Way

Now the culture shock that defines working Python. You *could* loop over 6,420 sales and add up the EcoCash ones:

```

total = 0

```

```
for amount, ptype in zip(amounts, payment_types):
    if ptype == "EcoCash":
        total = total + amount
```

It works — and it is how you would explain a sum to a child. Python's (and pandas') native idiom is to say it *to the whole column at once*:

```
sales.loc[sales["payment_type"] == "EcoCash", "amount"].sum()
```

— a filter (a boolean, Chapter 31!) selecting rows, then `.sum()` over the selected column: **vectorisation** — one instruction applied to every row internally, at C speed. On 6,420 rows the difference is comfort; on a million it is the difference between a second and a coffee. The loop is not wrong — it is *for explanation*; the vectorised line is *for work*. The rule you will hear from every professional: **if you are writing a for loop over a DataFrame's rows, there is almost always a pandas one-liner you are missing** — and this book's job in Chapter 34 is to hand you the one-liners.

## 32.5 A Complete Mini-Program

Everything in this chapter, assembled into something real — the daily exceptions report (Chapter 24's flagging instrument, now as code). Type it into one cell; it is your first program, not your first exercise:

```
def exceptions_report(takings_by_day, mean=None, sd=None):
    """Flag days whose takings are more than 2 SDs from typical."""
    if mean is None:
        # compute if not supplied
        mean = sum(takings_by_day) / len(takings_by_day)
    if sd is None:
        sq = [(t - mean) ** 2 for t in takings_by_day]
        sd = (sum(sq) / (len(sq) - 1)) ** 0.5
    flags = []
    for day, t in takings_by_day:
        z = (t - mean) / sd
        if abs(z) > 2:
            flags.append((day, t, round(z, 1)))
    return flags
week = [("Mon", 2980), ("Tue", 1520), ("Wed", 3410),
        ("Thu", 890), ("Fri", 3120), ("Sat", 4270), ("Sun", 2150)]
for day, t, z in exceptions_report(week):
    print(f"{day}: ${t:,} (z = {z:+.1f}) -- investigate")
```

Find what is inside it: a list of tuples, a function with defaults and a docstring, a loop, an if, a z-score computed from scratch (with the  $n-1$  divisor — you know why), an f-string with a signed format, a filter list. Eleven meaningful lines; and when Chapter 34 rewrites it in three lines of pandas, you will know *exactly* what those three lines are doing — because you built the machine they replace. That is the honest way to learn a high-level tool: build the wheel

once, then accept the car.

**From Your Toolkit — flow is forever:** *for/if/functions* are the atoms of *all* code you will ever read — the *do-files* of Chapter 29, the SQL of Part III (a CTE *is* a function's cousin; WHERE *is* an if; GROUP BY *is* a loop with manners), and the automation scripts of Chapter 37 and Project 5. And the vectorised instinct returns as SQL's set-thinking: describe the *what*, let the engine do the *how*.

## Key Takeaways

- Lists are numbered columns (counting from zero); dictionaries are labelled rows; a DataFrame is a million dictionaries.
- *for* visits, *if* chooses, indentation *is* the structure — Python reads like a recipe because it is shaped like one.
- Functions: define once, use forever; arguments make assumptions visible; docstrings carry the why.
- The pandas way: vectorise — filters and methods over columns, not loops over rows; loops explain, one-liners work.
- The mini-program is the chapter: collections + flow + function + f-strings = the exceptions report, your first real artefact.

## Practice Lab

1. The tour of containers: build a five-sale list of dictionaries; print the third sale's amount, the last sale's payment type; add a sale; loop over all five and print an f-string line each ("sale 3: \$11.50 by EcoCash").
2. The weekday classifier: write `day_verdict(takings, low, high)` with a docstring; classify a full week; then classify with different thresholds and note in a Markdown cell *which business question* each threshold answers.
3. The fence, as a function: implement `iqr_fences(values)` returning the pair (lower, upper) — reuse it on `amount` data and reconcile against your Part II fence values exactly.
4. Loop vs pandas: compute EcoCash revenue both ways (the loop, then the one-liner) and time both with `%time` as the cell's first line; record the two numbers; write the sentence about what vectorisation buys.
5. Extend the mini-program: add a `summary` function that takes the flags and returns an f-string paragraph ("3 exceptional days this week: Tue (z = -2.1), Thu (z = -3.4), Sat (z = +2.0)"); wire it into `exceptions_report`; Restart & Run All; save as `exceptions-report-v1.ipynb` — version one of a tool you will rebuild in Chapter 34 and automate in Chapter 37.



## Further Reading

- Chapter 33 (pandas properly — the DataFrame grammar), Chapter 34 (the analysis one-liners)
- *Automate the Boring Stuff* (Sweigart) chapters 3–6 (functions, lists, dictionaries) for extra reps

# Chapter 33: DataFrames from Zero

## Part V — Python: From Zero to Dangerous

"A DataFrame is a spreadsheet that obeys orders exactly, remembers everything you did, and never detaches a column in a sort."

### In this chapter you will learn

- The DataFrame and Series: pandas' two objects, and how they relate to what you know.
- Loading Tariro's four CSVs — with the loading checks, now as code.
- Inspecting properly: head, info, describe, value_counts — the four instruments.
- Selecting: columns, rows by name and by boolean — the WHERE of pandas.
- Creating columns: the derived features that turn raw data into analysis material.

## 33.1 Two Objects

pandas has exactly two objects to learn, and both are old friends in new clothes:

- A **Series** is one column: a labelled list — `sales["amount"]` — with all of Python's arithmetic applied *to the whole column at once* (the vectorised way, Chapter 32).
- A **DataFrame** is the table: a dict of Series sharing an index — `sales` itself.

The **index** is the row-labels (0, 1, 2... by default) — a built-in row identity that survives sorts and filters and becomes genuinely powerful with dates (Chapter 34's time series). For now: know it exists, notice the bold numbers on the left, and let `.loc` (below) be your respectful way of addressing it.

```
import pandas as pd
sales = pd.read_csv("sales.csv")          # the load
customers = pd.read_csv("customers.csv")
stock = pd.read_csv("stock.csv")
suppliers = pd.read_csv("suppliers.csv")
```

Four loads, four tables, the whole shop in memory — and unlike a spreadsheet, these are *values*, not views: nothing moves unless you say so.

## 33.2 The Loading Checks, as Code

Project 2's loading liturgy — row counts, key census, orphan check, totals, NULL census — was a checklist in a log; in pandas it is *code that checks*, which means it runs itself every month:

```
# row counts -- the log writes itself
print(f"sales rows: {len(sales)}")
print(f"customers rows: {len(customers)}")
# key census: duplicate customer_ids must be zero
print(customers["customer_id"].duplicated().sum())      # 0
# orphan check: sales' ids that match no customer (non-members excluded first)
orphan = sales.loc[sales["customer_id"].notna()
                  & ~sales["customer_id"].isin(customers["customer_id"])]
print(f"orphan sales: {len(orphan)}")
# the totals bridge -- reconciles to Part II's workbook, to the cent
print(f"total revenue: {sales['amount'].sum():.2f}")    # 54,013.50
# NULL census, with meanings
print(sales["customer_id"].isna().sum())               # non-member sales
print(sales["amount"].isna().sum())                    # must be 0 -- or a story
```

Read each line and find the Part II-III habit it mechanises: `.duplicated().sum()` is the key census; `~.isin()` is the anti-join; `.isna().sum()` is the NULL census; the total is the bridge. **The checks you used to perform, your code now performs** — that is the whole meaning of "from zero to dangerous": the discipline becomes ambient.

## 33.3 The Four Instruments of Inspection

The four methods that open any new table, in the order you run them:

```
sales.head()          # the first five rows -- eyeball the columns, spot obvious trouble
sales.info()           # every column: type, non-null count -- the schema, free
sales.describe()       # numeric summary -- the five numbers plus count/mean/std
sales["payment_type"].value_counts() # the categorical summary -- counts per value
```

`info()` deserves its own sentence: it prints each column's **dtype** (`int64`, `float64`, object — the last is usually text, *sometimes a date that failed to parse, sometimes numbers with a stray letter in them*). "Column of 6,420 with 300 nulls and dtype object that should be float" — that one line of `info()` output is the entire data-quality conversation of Chapter 8, printed automatically. And `value_counts()` is the frequency table for *any* categorical column, normalising with `normalize=True` for shares — `sales["payment_type"].value_counts(normalize=True)` is the payment-mix pie, minus the pie.

### 33.4 Selecting: the WHERE of pandas

Selection has two gears, and both matter. **By name** (columns with [...], rows by label with .loc):

```
sales["amount"]                # one column (a Series)
sales[["date", "amount"]]      # two columns (note the double brackets)
sales.loc[0]                   # the row labelled 0
sales.loc[10:15, ["date", "amount"]] # rows 10-15, those columns
```

**By condition** (the boolean, powering a filter — the gear you will use daily):

```
sales[sales["amount"] >= 50]    # big baskets
sales[sales["payment_type"] == "EcoCash"] # one payment type
sales[(sales["amount"] >= 50) & (sales["till"] == 2)] # AND -- note the parentheses!
```

Two syntax landmines, both famous: **combine conditions with & / | (not and/or)** and **wrap each condition in parentheses** — and on Series is a `ValueError` with a helpful message, and you will meet it once, exactly once. And the date-range filter, pandas' daily bread:

```
sales["date"] = pd.to_datetime(sales["date"]) # parse once (see 33.5)
june = sales[(sales["date"] >= "2025-06-01") & (sales["date"] <= "2025-06-30")]
```

SELECT, WHERE, and BETWEEN — Chapter 15, wearing different brackets. (There is also `.query('amount >= 50 and till == 2')` — SQL-flavoured selection, almost a dialect pun; use whichever your team reads better.)

### 33.5 New Columns

Analysis begins when raw columns become *features*. A new column is an assignment over the whole column at once:

```
# parse the date properly (info() showed object -- fix it on load, always)
sales["date"] = pd.to_datetime(sales["date"])
# derived features, vectorised
sales["month"] = sales["date"].dt.to_period("M") # 2025-06 -- grouping key
sales["weekday"] = sales["date"].dt.day_name() # "Saturday"
sales["is_weekend"] = sales["weekday"].isin(["Saturday", "Sunday"])
sales["is_whale"] = sales["amount"] > 21.50 # the Part II fence
sales["basket_class"] = sales["amount"].apply(
    lambda a: "whale" if a > 21.50 else ("big" if a >= 10 else "small"))
```

Three idioms in that block, in ascending order of power: **arithmetic/boolean over columns** (`is_whale` — the fastest, use it whenever possible); **.dt and .str accessors** (`dt.day_name()`, `str.upper()`, `str.strip()` — the cleaning chapter *inside* the column: `sales["payment_type"].str.strip().str.upper()` is the three-EcoCash-solutions fix, permanently); and **.apply()** with a `lambda` — a per-row function when no column-level idiom fits (the loop of Chapter 32, wearing a suit; use it *last*, because it is the slowest and the least searchable).

The craft rule that ends the chapter: **every new column is an assumption made visible**. `is_whale` hard-codes the 21.50 fence — better written as a named constant and commented (the argument-defaults lesson): `WHALE_FENCE = 21.50 # 1.5 × IQR, Part II`, then `sales["amount"] > WHALE_FENCE`. Your columns are your analysis's load-bearing walls; label the load.

**From Your Toolkit — the table, finally programmatic:** the DataFrame is the object the whole rest of this book manipulates: Chapter 34 analyses it, Chapter 35 draws it, Chapter 36 tests it, Chapter 37 schedules it, Chapter 39 exports it to Power BI, and Chapter 58 learns from it. Everything a spreadsheet *is*, it is — minus the fragility, plus the audit trail: a notebook re-run is proof the table was built the same way twice.

## Key Takeaways

- Series = one labelled column; DataFrame = the table; the index is row identity.
- The loading checks become code: `duplicated()`, `~isin()` (anti-join), `isna().sum()`, the totals bridge — discipline mechanised.
- The four instruments: `head` (eyeball), `info` (dtypes and nulls — the schema free), `describe` (numbers), `value_counts` (categories).
- Select by name (`[...] / .loc`) or by condition (booleans; `&/|` with parentheses; `.query()` as the SQL pun); dates parse once with `to_datetime`.
- New columns are vectorised first (`.dt/.str` second, `.apply` last); every column is an assumption — name its constants and comment its loads.

## Practice Lab

1. The four loads + full loading-checks block on all four of Tariro's tables; reconcile revenue and row counts against your Project 2 log; paste the printed check output into a Markdown cell as the audit page.
2. The instrument sweep: `head/info/describe/value_counts` on `customers`; find one dtype surprise or NULL story per table (there are planted ones); write the four findings in four bullets.
3. Selection drills, all reconciled against SQL you wrote in Part III: (a) the ten biggest sales (`sort_values + head(10)`), (b) June EcoCash sales, (c) weekend member sales with `amount ≥ 50` — for each, also write the equivalent SQL in a Markdown cell and note the comparison.
4. The cleaning column: build `payment_clean` from `payment_type` with `.str.strip().str.upper()`, then `value_counts()` before and after — the three-EcoCash story in two output blocks; document which rows changed.

5. The feature shelf: add `month`, `weekday`, `is_weekend`, `is_whale` (with the named constant), and `basket_class`; then produce one finding per new column (e.g., weekend share of revenue; whale share of transactions vs revenue) — each as a sentence with its f-string.

## Further Reading

- Chapter 34 (groupby, merge, time series — the power moves), *Python for Data Analysis* ch. 5
- pandas' "10 minutes to pandas" — the official tour, painless after this chapter

# Chapter 34: pandas Power Moves — groupby, merge, time series

Part V — Python: From Zero to Dangerous

"Every analysis in this book so far — pivots, joins, running totals, month-over-month, top-N — arrives in this chapter as one line. All of them. One line each."

## In this chapter you will learn

- groupby: the pivot, the HAVING, and the aggregate habit — in one idiom.
- merge: Chapter 17's joins, with the fan-out defence built in.
- The time-series kit: resample, rolling, shift — running totals and month-over-month, natively.
- Pipes and method chains: the CTE habit, Python's accent.
- The full re-run: Part II's monthly report analysis, in fifteen lines.

## 34.1 groupby: The One Idiom

The single most-used line in pandas is also the single most-used idea in this book — "totals of something, for each something else":

```
sales.groupby("payment_type")["amount"].agg(["count", "sum", "mean"])
```

Read it as the sentence it is: *take sales, group by payment_type, take amount, aggregate count/sum/mean*. That is Chapter 16's query and Chapter 10's pivot, simultaneously. The variations you need, each one a Part II-III skill:

```
# multiple group keys -- the two-shelf pivot (rows x columns, stacked long)
sales.groupby(["month", "category"])["amount"].sum()
# named aggregations -- the query with AS aliases
(sales.groupby("category")
 .agg(revenue=("amount", "sum"),
      n_sales=("amount", "count"),
      avg_basket=("amount", "mean")))
# filter groups -- HAVING, verbatim
by_cat = sales.groupby("category")["amount"].agg(["sum", "count"])
by_cat[by_cat["count"] >= 100].sort_values("sum", ascending=False)
# the top-N-within-group pattern (Ch. 19), pandas edition
(sales.groupby(["category", "item"])["amount"].sum()
 .reset_index()
 .sort_values(["category", "amount"], ascending=[True, False])
 .groupby("category").head(3))
```

One aesthetic note that is also a professional one: pandas expressions *chain* — method after method, each feeding the next, usually wrapped in parentheses for line breaks. The chain reads top-to-bottom like a CTE pipeline (Ch. 18's story-shape), and that is not a coincidence: it is the same discipline — decompose, name (or chain), test each step — in a different accent. Run chains a link at a time while learning: build `groupby(...)`, run; add `["amount"]`, run; add `.agg(...)`, run. Each link's output is inspectable, exactly like running each CTE alone.

## 34.2 merge: Joins, With the Defence Built In

`merge` is Chapter 17 with the craft lessons as arguments:

```
# LEFT JOIN sales to customers, keeping non-members
enriched = sales.merge(customers, on="customer_id", how="left")
# the anti-join -- orphans and never-boughts
orphan = sales.merge(customers, on="customer_id", how="left", indicator=True)
orphan = orphan[orphan["_merge"] == "left_only"]
```

`how=` is the join direction you know ("left", "inner"; "right" and "outer" exist and are rarely right); `on=` is the key; and `indicator=True` adds the `_merge` column ("left_only" / "both" / "right_only") — the anti-join as an argument. And the fan-out defence is *easier* than SQL's, because pandas will often warn you ("merging on duplicate values" behaviour) — but do not rely on warnings; rely on the liturgy, which is two lines:

```
print(len(sales), "->", len(enriched))          # row count before/after
print(sales["amount"].sum() == enriched["amount"].sum()) # totals reconcile
```

Count before, count after, reconcile the totals — Chapter 17's habit, mechanised. If the row count grew on a left merge, the right side repeated keys and every aggregate downstream is inflated; you now catch it in one printed line, forever.

## 34.3 Time Series: The Shop's Calendar, Natively

Dates parse once (Ch. 33) and then pandas becomes a calendar engine. The three moves that cover working time-series analysis:

```
daily = (sales.groupby("date")["amount"].sum()
         .sort_index())          # a dated Series: takings per day
monthly = daily.resample("ME").sum() # 1: daily -> monthly totals
daily["rolling_7"] = daily["amount"].rolling(7).mean() # 2: the moving average
monthly["mom_pct"] = monthly["amount"].pct_change() * 100 # 3: month-over-month
```

- `resample` is `groupby-for-time`: "ME" is month-end; "W" week; "Q" quarter — the pivot's date-grouping with calendar awareness (months of unequal length handled correctly).



- `rolling` is the window function's frame: `rolling(7).mean()` is Chapter 19's moving average; `.rolling(7).sum()` is the rolling total; `.cumsum()` the running total — one method each.
- `shift` and `pct_change`: `shift` moves a column down a row (LAG, verbatim); `pct_change()` is  $(x - x.shift()) / x.shift()$  pre-assembled — month-over-month growth, the requested metric of Chapter 19, as one call.

The lag-then-compare idiom extends anywhere: `monthly["amount"] - monthly["amount"].shift(12)` is year-over-year change, and you did not write a loop or a window clause to get it. The time-series kit is the single biggest practical upgrade pandas gives Tariro's shop: the whole Chapter 10 drag-dance per month is now `daily.resample("ME").sum().plot()`.

## 34.4 The Full Re-Run

Proof of the chapter's opening claim — Part II's monthly report analysis, the numbers behind all three findings, in fifteen lines (run it; then read it):

```
sales = pd.read_csv("sales.csv", parse_dates=["date"])
sales["month"] = sales["date"].dt.to_period("M")
sales["weekday"] = sales["date"].dt.day_name()
# headline: monthly revenue
monthly = sales.groupby("month")["amount"].sum()
# finding 1: top items
top_items = (sales.groupby("item")["amount"].sum()
              .sort_values(ascending=False).head(10))
# finding 2: the oil collapse -- price vs quantity, monthly
oil = sales[sales["item"] == "Cooking Oil 2L"]
oil_monthly = oil.groupby("month")["amount"].agg(["count", "sum"])
# finding 3: Sundays (traffic, not basket) + loyalty share
by_day = sales.groupby("weekday").agg(n=("amount", "count"),
                                      revenue=("amount", "sum"),
                                      avg_basket=("amount", "mean"))
member_share = (sales.assign(member=sales["customer_id"].notna())
                 .groupby("month")["member"].mean() * 100)
print(f"Top item: {top_items.index[0]} ({top_items.iloc[0]:.2f})")
print(f"Sunday avg basket: ${by_day.loc['Sunday', 'avg_basket']:.2f} "
      f"vs weekday ${by_day.drop('Sunday')['avg_basket'].mean():.2f}")
print(f"Member share of sales, latest month: {member_share.iloc[-1]:.1f}%")
```

Every line is a chapter you have lived: the parse, the derived columns, the pivot, the filter, the groupby-with-names, the assign-and-group, the f-strings with formats. Nothing here is new — that is the chapter's entire point. **You already know the analysis; you are now holding it as an artefact.** Next month's CSV arrives, this runs unchanged, and the report's numbers regenerate: that is "dangerous".

**From Your Toolkit — the analytical engine:** groupby/merge/resample are the load-bearing walls of everything ahead — Chapter 36's tests consume their outputs, Chapter 37 automates this very block, Chapter 39's Power BI dataset is often *prepared* by exactly these lines before import, and Project 4's churn analysis is groupby-plus-shift wearing a business suit. The spreadsheet gave you the concepts; SQL gave you the grammar; pandas gives you the *engine* — the same machine, now yours to program.

## Key Takeaways

- groupby is pivot + GROUP BY in one idiom: `df.groupby(keys)[col].agg(...)`; named aggregations are your AS aliases; group-filtering is HAVING.
- merge is the join: `on=`, `how=`, `indicator=True` for anti-joins — and the two-line fan-out defence (row counts, totals) runs after every merge, always.
- Time series: resample (calendar groupby), rolling (frames), shift/pct_change (LAG and MoM) — running totals to year-over-year, one call each.
- Chains read like CTE pipelines; build them link by link and inspect each stage.
- Fifteen lines re-run Part II's report analysis — the concepts were yours; now the artefact is.

## Practice Lab

1. The pivot tour, re-run: rebuild Chapter 10's five pivots (category × revenue, month × revenue, weekday counts, payment mix, member share) as one chained block each; reconcile every total against the Part II workbook; paste reconciliations as a Markdown table.
2. The merge gauntlet: LEFT JOIN sales→customers with the two-line defence; then the anti-join with `indicator=True`; confirm orphan counts match Project 2's documented orphans exactly; write the one-sentence difference between doing this in SQL and in pandas.
3. The time-series shelf: from daily takings build the 7-day rolling mean, month-over-month pct change, and year-over-year change; chart the three (next chapter's `.plot()` is allowed early); annotate the biggest MoM jump with its cause (a planted pattern, by now an old friend).
4. The top-N drill: top 3 items by revenue within each category (the Section 34.1 pattern); reconcile the grand total; write the tie-policy note (which method breaks ties, and how you know).

5. The re-run, owned: extend the fifteen-line block with a fourth finding of your choosing (whales, suburbs, or the oil price question); wrap the whole thing in a function `monthly_numbers(sales)` that returns a dict of the key figures — Chapter 37 will thank you, because this function *is* the monthly pack's engine.

## Further Reading

- Chapter 35 (charts — the lines get pictures), *Python for Data Analysis* ch. 8-10
- pandas' groupby and merging user guides — the official deep dives, readable after this chapter

# Chapter 35: Charts in Python — matplotlib, Honestly

*Part V — Python: From Zero to Dangerous*

"The grammar of honest charts is finished (Chapter 11). What remains is learning a second pen."

## In this chapter you will learn

- matplotlib's mental model: figure, axes, plot — and the three lines that cover 80% of needs.
- The five charts of Chapter 11, redrawn: line, bar, scatter, histogram, stacked bar.
- The one-liner vs the craft version — when `.plot()` is enough and when it is not.
- Saving charts as files — the report's exhibits, generated.
- When to stay in Excel — the honest tool-choice rule.

## 35.1 The Model

matplotlib feels baroque until you hold its one mental model: a chart is a **figure** (the page) containing **axes** (one chart's frame — its x, its y), on which you **plot** marks, then decorate:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(10, 4))      # one page, one frame
ax.plot(monthly.index, monthly.values)      # the marks
ax.set_title("Monthly revenue")
ax.set_ylabel("Revenue (USD)")
plt.show()
```

The `fig, ax = plt.subplots()` incantation is the doorway you walk through a thousand times; everything else is marks plus decoration. (You will also meet — and use freely — pandas' shortcut `series.plot()` and `df.plot(kind="bar")`, which builds figure-and-axes for you and hands back the `ax` for decorating: `ax = monthly.plot(); ax.set_title(...)`. Use the shortcut for speed, the full form for control; they are the same machine.)

## 35.2 The Five, Redrawn

Chapter 11's five shapes, each in its working form on Tariro's data — find the same arguments hiding in different clothes:

```
# 1. LINE -- "how does it move through time?"
ax = daily.resample("ME").sum().plot(figsize=(10, 4))
ax.set_title("Monthly revenue: steady growth, one dip"); ax.set_ylabel("USD")

# 2. BAR (ranked, horizontal) -- "which is biggest?"
top10 = sales.groupby("item")["amount"].sum().nlargest(10)
ax = top10.sort_values().plot(kind="barh", figsize=(8, 5))
ax.set_title("Top 10 items by revenue"); ax.set_xlabel("USD")

# 3. SCATTER -- "are these two related?"
ax = sales.plot(kind="scatter", x="amount", y="hour_of_day", alpha=0.2, figsize=(8, 5))
ax.set_title("Basket size vs time of day")

# 4. HISTOGRAM -- "what is the shape of this column?"
ax = sales["amount"].plot(kind="hist", bins=50, figsize=(8, 4))
ax.set_title("Basket sizes: right-skewed (whales live in the tail)")

# 5. STACKED BAR -- "what is it made of, over time?"
mix = sales.pivot_table(index="month", columns="payment_type",
                        values="amount", aggfunc="sum")
ax = mix.plot(kind="bar", stacked=True, figsize=(10, 4), width=0.8)
ax.set_title("Payment mix by month: EcoCash share rising")
```

Details that carry the craft: `alpha=0.2` makes scatter points translucent (overplotting defeated — 6,420 solid dots is a black rectangle); `bins=50` tunes histogram resolution (too few hides shape, too many makes noise); `nlargest(10)` is the top-list; `sort_values()` before a horizontal bar makes the ranking read top-down like a league table; and `pivot_table` is the pivot — `index/columns/values/aggfunc` are the four shelves by name.

### 35.3 The Craft Version

The one-liner gets a chart on screen; the craft version gets a chart *into a report* — and the difference is exactly the Chapter 11 rules, now as code:

```
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(monthly.index.astype(str), monthly.values, color="#0F766E", linewidth=2)
ax.set_title("Monthly revenue grew 31% over 18 months\n(dip after the month-9 oil price\nrise)",
            fontsize=13, fontweight="bold")
ax.set_ylabel("Revenue (USD)")
ax.set_xlabel("Month")
ax.spines[["top", "right"]].set_visible(False)           # delete the furniture
ax.grid(axis="y", alpha=0.3)                             # subtle guides only
ax.tick_params(axis="x", rotation=45)
fig.tight_layout()
fig.savefig("exhibit-monthly-revenue.png", dpi=150)      # the file, for the report
```

Line by line, it is Chapter 11's checklist compiled: the finding title (with the number in it), labelled axes with units, furniture deleted (spines off), subtlety in the grid, and — the part one-liners cannot do — `savefig`: the exhibit as a file, at report resolution, ready for the PDF. A chart that regenerates from data

every month *and* lands in the report automatically is the whole automation story in one function call. (Colours: name them ("teal") or hex them ("0F766E"); a consistent two-or-three-colour palette per report is the professional tell — Tariro's reports use teal and amber, matching this book's cover.)

## 35.4 The Notebook Dialogue

The chart's greatest notebook power is not beauty — it is *iteration*: see it, adjust, see it again, three cells a minute. The dialogue that produces good charts is only possible when the loop is seconds long, which is why analysts chart *inside* the analysis (spreadsheet too: the pivot and the chart are adjacent tabs) rather than exporting data to a separate charting tool. And a seaborn glimpse, honestly flagged as a taste of Chapter 58's world: `import seaborn as sns; sns.scatterplot(data=sales, x="amount", y="hour_of_day", hue="payment_type")` — statistical charts (with regression lines, distributions, categorical splits) in one line, built *on* matplotlib. When you need the statistics drawn, seaborn; when you need control, matplotlib; both ship with Anaconda.

## 35.5 When to Stay in Excel

The honest tool rule, stated once and applied forever: **the chart's audience decides the tool**. Exploring, iterating, one-off questions → the notebook (speed of iteration). A monthly PDF exhibit that must regenerate identically → matplotlib + `savefig` (automation). A chart the boss wants tweaked *in the meeting* ("make it blue, show March only") → the spreadsheet, because drag-and-drop beats re-running code when the requirement is changing live. And Part VI adds the fourth case: charts that must *filter themselves* when the reader clicks → Power BI. One grammar, four pens, chosen by the audience — the analyst who insists on a single tool for every chart is choosing their comfort over the reader's need, and Chapter 2's "says who?" applies to tools too.

**From Your Toolkit — the report pipeline completes:** load (33) → analyse (34) → **chart (35, `savefig`)** → report. Chapter 37 wires these stages into the monthly pack, and Project 6's dashboard inherits the same exhibits. The five shapes and three lies never change; the pen does — and you now hold two of the four pens this book teaches.

## Key Takeaways

- Model: figure (page) → axes (frame) → marks → decoration; `fig, ax = plt.subplots()` is the doorway; pandas' `.plot()` is the shortcut.
- The five shapes transfer directly — line, ranked barh, scatter (alpha!), histogram (bins), stacked bar via `pivot_table` (the four shelves by name).
- Craft version = Chapter 11 compiled: finding title, units, spines off, and `savefig` — the exhibit as a regenerating file.
- The notebook's iteration speed is charting's real superpower; seaborn for statistical charts when needed.
- Audience picks the pen: notebook to explore, matplotlib to automate, spreadsheet for live tweaks, Power BI for interactive — one grammar, four pens.

## Practice Lab

1. The five, redrawn: run all five working-form charts on your data; then pick the two best and upgrade them to craft versions (finding titles, units, furniture deleted, `savefig`) — the PNGs are report exhibits; file them in your project folder.
2. The lie, reproduced in code: rebuild the truncated-axis lie from Chapter 11's lab (`ax.set_ylim(9000, 9500)` does it); save both versions; write the caption pair for the log/audit trail.
3. The alpha lesson: scatter of amount vs hour with `alpha=1` then `alpha=0.2` on the same data; paste both; write one sentence on what the translucent version reveals that the solid one hides.
4. The palette: re-colour your two craft exhibits with a consistent two-colour palette (hex codes in a constants cell at the top of the notebook — the named-constant habit, applied to design); note the palette in the header Markdown.
5. The regeneration test: change one month's data in the CSV (copy it first), re-run the notebook Restart & Run All, and confirm the exhibit PNG changed accordingly — then restore. You have just demonstrated the monthly-report promise: charts that rebuild themselves.

## Further Reading

- Chapter 36 (statistics in Python — the tests join the charts), Chapter 37 (the pack, assembled)
- matplotlib's official gallery — copy the nearest example, adjust to your data (the professional's method, openly admitted)

# Chapter 36: Statistics in Python — the Same Answers, in Six Lines

## Part V — Python: From Zero to Dangerous

"Part IV gave you the courtroom. `scipy` is the courtroom, motorised — with every case you have already argued, one line each."

### In this chapter you will learn

- `scipy` and `statsmodels`: the two statistics libraries, and their division of labour.
- Every Part IV test, re-run: t-tests, chi-square, correlation, regression — in lines, not menus.
- Reading the outputs: p-values, statistics, and confidence intervals as printed objects.
- One new capability statistics alone couldn't give: the bootstrap, intuitively and in five lines.
- The three-tools answer: SPSS vs Python vs spreadsheets — when each, honestly.

## 36.1 The Two Libraries

**`scipy.stats`** is the workhorse: the standard distributions and tests, fast and minimal. **`statsmodels`** is the report-writer: the same models with full statistical output — coefficients, intervals, diagnostics, the six lines every test prints (Chapter 29's SPSS tables, in text form). The working division: `scipy` to *test*, `statsmodels` to *read everything about the model*. Both ship with Anaconda:

```
from scipy import stats
import statsmodels.api as sm
```

## 36.2 The Re-Runs

Every analysis of Part IV, now as code. Run them against your own Part IV numbers — the reconciliation is the point (same data, same statistics, three tools; when all three agree, you have *verified* your verification).

**The Sunday test (Ch. 26)** — two independent groups:

```
sun = daily[daily["weekday"] == "Sunday"]["amount"]
week = daily[daily["weekday"] != "Sunday"]["amount"]
```



```
t_stat, p_value = stats.ttest_ind(sun, week, equal_var=False)
print(f"Sunday t = {t_stat:.1f}, p = {p_value:.6f}")
# Sunday t = -18.5, p = 0.000000 -- the ledger's closure, reproduced
```

(`equal_var=False` is Welch's *t* — the version that does not assume equal spreads; the professional's default. Note the sign: Sunday first means negative *t*. Statistics identical to Part IV's, tool irrelevant.)

**The paired test (Ch. 26)** — before/after the price rise, per item:

```
t_stat, p_value = stats.ttest_rel(before_bottles, after_bottles)
# t = -4.6, p = 0.0008 -- the oil verdict, reproduced
```

**Chi-square (Ch. 27)** — the survey cross-tab:

```
table = pd.crosstab(survey["tier"], survey["shopped_this_month"])
chi2, p, dof, expected = stats.chi2_contingency(table)
print(f"chi2 = {chi2:.2f}, dof = {dof}, p = {p:.3f}")
# chi2 = 5.5, dof = 2, p = 0.064 -- suggestive, not established; reproduced
```

And the expected-count warning, mechanised: `expected.min() >= 5` is now an assert in your code — the cell-count rule enforced by the machine, not the memory.

**Correlation and the line (Ch. 28):**

```
r, p = stats.pearsonr(oil["price"], oil["bottles"])
slope, intercept, r_value, p_value, se = stats.linregress(oil["price"], oil["bottles"])
print(f"r = {r:.2f} (p = {p:.5f}); slope = {slope:.2f} bottles per dollar")
model = sm.OLS(oil["bottles"], sm.add_constant(oil["price"])).fit()
print(model.summary()) # the SPSS-grade table: CIs, R-squared, everything
```

`linregress` gives the slope sentence ("a dollar costs a bottle"); `model.summary()` gives the full argument — including the 95% confidence interval on the slope that Part IV's honest paragraph demanded ( $[-1.6, -0.7]$ , say). Read that summary block once, unhurriedly: you now know every line in it (coef, std err, *t*,  $P > |t|$ ,  $[0.025, 0.975]$  — statistic, surprise, interval: the courtroom's paperwork, complete).

## 36.3 The Bootstrap: A New Superpower

One Part IV idea gets *easier* in code than in theory: the confidence interval. The **bootstrap** — resample your data with replacement, recompute the statistic, watch its spread — is the sampling distribution of Chapter 25, *demonstrated instead of derived*:

```
import numpy as np
means = [daily.sample(frac=1.0, replace=True)["amount"].mean()
          for _ in range(2000)]
lo, hi = np.percentile(means, [2.5, 97.5])
print(f"mean daily takings: {daily['amount'].mean():.0f} "
```

```
f"(95% CI {lo:.0f} to {hi:.0f})")
```

Two thousand re-runs of the world (Chapter 25's phrase, now literal), and the interval falls out of the spread of re-run means — no formula, no normality assumption, the same logic working on medians, on ratios, on any statistic a formula finds difficult. The bootstrap is how modern statistics handles the awkward cases, and it is yours in five lines because you own the *concept*: the sampling distribution is a picture of "what my statistic would say across re-runs" — and the computer can simply *have the re-runs*.

### 36.4 Three Tools, Honestly

The question every learner asks: *if Python does all this, why learn statistics in SPSS or spreadsheets at all?* The honest answer, tool by tool — and it is a professional's answer, not a tribal one:

Tool	Statistics for...	Its edge	Its cost
Spread sheet	first looks, one-offs	zero friction, everyone has it	no audit trail, silent errors
SPSS/Stata	governed/research workflows	validated procedures, standard tables	licensing, point-and-click walls
Python	automated/repeatable analysis	free, programmable, scales, bootstrap	you must know what you're asking for

The deep point is the last cell of the third row: **menus protect you from choices you don't know you're making; code exposes every choice you make.** `equal_var=False` is a decision you *saw*; the SPSS dialog's default is a decision you *missed*. Neither culture is safe by default — safety is knowing the six lines every test prints, whichever tool prints them. That fluency is now yours, three times over; Chapter 52's interview questions will test precisely it.

**From Your Toolkit — the sixth instrument, everywhere:** this chapter is the bridge to everything remaining: Project 4's churn analysis is a t-test plus a groupby; Chapter 37's automated report embeds these exact lines; Chapter 58's machine learning *is* `model.summary()`'s descendants at scale; and the A/B-testing culture of tech companies (Chapter 57) runs these six lines at industrial size. The tests are six; the tools are many; the fluency is one.

### Key Takeaways

- `scipy.stats` to test, `statsmodels` to read everything; both ship with Anaconda; both print the six lines you already know.
- Every Part IV test reproduced in 1–3 lines (t, paired t, chi-square, correlation, regression) — reconcile against Part IV's numbers to complete

the three-tool verification.

- Welch's `t` (`equal_var=False`) is the professional default; assert expected  $\geq 5$  for chi-square — decisions visible, assumptions mechanised.
- The bootstrap: 2000 re-runs, take the spread — CIs for anything, no formulas; Chapter 25's picture, computed.
- Menus hide choices, code exposes them; spreadsheet/SPSS/Python divide by workflow, and fluency — not tribe — is the safety.

## Practice Lab

1. The reconciliation run: re-run all four Part IV analyses (Sunday `t`, paired oil, chi-square, oil regression) on your data; build a Markdown table: Part IV result | SPSS-or-sheet result | Python result — three tools, one column of numbers; certify it.
2. The `summary()` read: from `model.summary()`, write the four-sentence finding (slope, its CI,  $R^2$ ,  $p$ ) in Chapter 28's professional-paragraph format; underline which sentence each summary number funded.
3. The bootstrap, twice: CI of the mean daily takings (above), then of the *median* basket — a statistic formulas find awkward, the bootstrap doesn't care; write one sentence on why the median's interval is the honest one for money.
4. The assumption audit: re-run the Sunday test with `equal_var=True` vs `False`; note the (small) difference; then write the one-paragraph "which did I choose and why" for the report — the visible-choice habit, practised.
5. The test-picker quiz (self-made): write five business questions from Tariro's world, each answerable by exactly one of the six instruments; shuffle them; answer them a week later without notes; grade yourself. This quiz is Chapter 52's interview, rehearsed.

## Further Reading

- Chapter 37 (the tests join the automated report), Chapter 58 (regression's descendants)
- *Think Stats* (Downey) — statistics *through* Python, free online; the natural continuation of this chapter

# Chapter 37: Project 3 — The Report That Builds Itself

Part V — Python: From Zero to Dangerous

"The best monthly report is the one you read with coffee, not the one you build with overtime."

## In this chapter you will learn

- The automation mindset: pipeline, not performance — inputs → checks → analysis → outputs.
- Building Tariro's automated monthly report, end to end, in one script.
- The professional project structure: files, folders, config, README.
- Scheduling: cron and Task Scheduler — the report that runs while you sleep.
- Failure handling: what a good automation says when data is missing.

## 37.1 The Pipeline Mindset

Every analysis so far in this book was a *performance* — you, present, running cells. Automation converts the performance into a **pipeline**: a fixed sequence — *inputs* → *loading checks* → *analysis* → *outputs* — that runs identically every time, unattended, and says something sensible when the world misbehaves. The mindset shift is the analyst's rite of passage: from *doing the analysis* to *building the thing that does the analysis*. Employers pay generously for the second (Chapter 50's job listings will say "automate recurring reporting" almost verbatim), because a pipeline's work compounds: build once, collect every month.

Tariro's monthly report — the one you assembled by hand in Project 1 (Chapter 13) and re-analysed in SQL (Project 2) — is the target. It will now build itself: fresh CSVs in, checked; analysis computed; charts drawn; a text report written; everything saved to a dated folder, with a log of exactly what happened.

## 37.2 The Structure

Professional Python projects live in folders, not lone notebooks — the structure now, because Project 4–6 and your portfolio (Chapter 49) inherit it directly:

```

tariro-monthly/
  README.md          -- what this is, how to run it, the data contract
  config.py          -- paths, thresholds, the fence, all constants
  toolkit.py         -- the reusable functions (fences, z-scores, checks)
  build_report.py    -- THE PIPELINE: one command, everything happens
  data/
    raw/             -- CSVs as received (never edited by hand)
    processed/       -- cleaned parquet/CSV, written by the pipeline
  output/
    2025-06/         -- one dated folder per run
      report.md      -- the findings, written by the machine
      exhibits/      -- the PNG charts
      run_log.txt    -- what was checked, what was found, what failed

```

Three conventions carry all the professionalism: **raw data is sacred** (the pipeline reads `data/raw/`, never modifies it — cleaning happens *to a copy*, in code, reproducibly; hand-edited raw data is the original sin of Part II, never again); **constants live in config** (`WHALE_FENCE = 21.50` in one place — the named-constant habit, now load-bearing); and **every run writes its own dated folder plus log** — the audit trail Chapter 20 demanded of SQL, generated automatically.

### 37.3 The Toolkit and the Checks

`toolkit.py` — the functions you have already written in Practice Labs, gathered to be imported forever:

```

# toolkit.py
import pandas as pd

def iqr_fences(values):
    q1, q3 = values.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - 1.5 * iqr, q3 + 1.5 * iqr

def loading_checks(sales, customers):
    """The liturgy: row counts, key census, orphans, totals, NULLs.
    Returns a list of check-result strings for the run log."""
    lines = []
    lines.append(f"sales rows: {len(sales):,}")
    lines.append(f"duplicate customer ids:
    {customers['customer_id'].duplicated().sum()}")
    orphan = sales.loc[sales["customer_id"].notna()
        & ~sales["customer_id"].isin(customers["customer_id"])]
    lines.append(f"orphan sales: {len(orphan)}")
    lines.append(f"total revenue: {sales['amount'].sum():.2f}")
    lines.append(f"null amounts: {sales['amount'].isna().sum()} (must be 0)")
    return lines, len(orphan)

def z(series):
    return (series - series.mean()) / series.std()

```

And the pipeline's first stage — the checks with *consequences*: a run whose totals do not reconcile with last month's bridge, or whose null amounts are non-zero, must **stop and say so**, not produce a beautiful report from broken data. Automation without failure-handling is a rumour generator:

```
# inside build_report.py -- the guard
lines, n_orphan = loading_checks(sales, customers)
if sales["amount"].isna().sum() > 0:
    raise SystemExit("ABORT: null amounts in sales -- investigate before reporting")
if n_orphan > 20:
    # config: the documented tolerance
    raise SystemExit(f"ABORT: {n_orphan} orphan sales (tolerance 20)")
```

`raise SystemExit` stops the run with a message — crude, honest, and exactly right for a first pipeline. (The professional pattern is logging with levels and email alerts; you will meet `logging` in Chapter 58's world. The principle is already here: **fail loudly, never report from data you don't trust.**)

## 37.4 The Pipeline

The heart of `build_report.py`, abridged to its moves (the full listing is in Appendix C; every line is a chapter you own):

```
"""Tariro's monthly report -- builds itself. Run: python build_report.py"""
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
import toolkit, config

# 1. INPUTS
sales = pd.read_csv(config.RAW / "sales.csv", parse_dates=["date"])
customers = pd.read_csv(config.RAW / "customers.csv")
sales["month"] = sales["date"].dt.to_period("M")
latest = sales["month"].max()

# 2. CHECKS (fail loudly -- see 37.3)
# 3. ANALYSIS -- the Chapter 34 re-run, as functions
monthly = sales.groupby("month")["amount"].sum()
by_weekday = sales.groupby(sales["date"].dt.day_name())["amount"].agg(["count", "sum",
    "mean"])
member_share = sales.assign(m=sales["customer_id"].notna()).groupby("month")["m"].mean()
prev, this = monthly.iloc[-2], monthly.iloc[-1]

# 4. EXHIBITS
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(monthly.index.astype(str), monthly.values, color=config.TEAL)
ax.set_title(f'Monthly revenue: {this:,.0f} this month ({(this/prev-1)*100:+.1f}% MoM)')
fig.savefig(config.OUT / str(latest) / "exhibits" / "revenue.png", dpi=150)

# 5. THE REPORT -- f-strings assemble the findings
report = f"""# Tariro's Grocery -- Monthly Report {latest}
Revenue: ${this:,.2f} ({(this/prev-1)*100:+.1f}% vs last month).
Member share of sales: {member_share.iloc[-1]*100:.1f}%.
Sunday takings averaged ${by_weekday.loc['Sunday', 'mean']:,.0f}
```

```

vs weekday ${by_weekday.drop('Sunday')['mean'].mean():,.0f}.
_Caveats: member share is association, not causation (Ch. 28);
18 months is one shop, not all shops (Ch. 25)._
"""
(config.OUT / str(latest) / "report.md").write_text(report)
print(f"Built: output/{latest}/report.md")

```

Run it: `python build_report.py`. A dated folder appears with an exhibit and a report whose numbers you did not type. **Read the report critically anyway** — automation relieves the typing, never the judgment; the caveats are in the template *because* the machine won't add them unprompted. That sentence is the ethics of automation, and it matters more every year.

## 37.5 Scheduling and the Compounding

The last step removes you entirely. **Windows:** Task Scheduler → Create Basic Task → weekly → `python C:\...\tariro-monthly\build_report.py`. **macOS/Linux:** cron — `crontab -e`, add `0 7 1 * * cd /path/tariro-monthly && python build_report.py` (the syntax: minute hour day-of-month month day-of-week — this line runs at 07:00 on the first of each month). On the first of next month, the report builds itself before Tariro's coffee. (Chapter 57's world industrialises this same idea with workflow orchestrators; the principle is identical and starts here.)

And so the promise of Part V closes: the analysis that took a spreadsheet afternoon (Part II) became a SQL pack of queries (Project 2) and is now a *program* — checked, charted, written, scheduled, logging itself. The before/after timing — an afternoon vs zero minutes — is the portfolio's most persuasive number, and Project 3 is its artefact.

**From Your Toolkit — the pipeline is the profession:** this exact structure — config, toolkit, pipeline script, sacred raw data, dated outputs, run logs, failure guards — is how every serious analytics deliverable is built, from a corner shop to a data platform. Projects 4–6 reuse it; Chapter 49 publishes it; employers' "automate recurring reporting" is it. You have not just learned Python; you have learned *how the work works*.

## Key Takeaways

- Automation = pipeline: inputs → checks → analysis → outputs, identical every run, sensible when the world misbehaves.
- Structure carries the professionalism: README, config constants, toolkit functions, sacred raw data, dated output folders, run logs.

- Checks must have consequences: fail loudly (abort with a message) rather than report from data you don't trust.
- The pipeline is every chapter assembled: loading checks (33), analysis (34), exhibits (35), tests (36), findings with caveats as f-strings.
- Scheduling (Task Scheduler / cron) completes the compounding: build once, collect monthly — an afternoon becomes zero minutes.

## Practice Lab

1. Build the full structure and get `build_report.py` running end-to-end on your data: raw CSVs, checks, three findings, two exhibits, the dated output folder, the run log. (Appendix C's listing is your scaffold; type it, don't paste it.)
2. The failure drill, twice: (a) corrupt a copy of `sales.csv` (delete an amount, duplicate a `customer_id`) and watch the guard abort; (b) add a new month's plausible rows and watch the report roll forward unchanged in effort. Log both runs' messages — the pair is the automation's character reference.
3. The human review: read the machine's report and mark every place the numbers are right but the *telling* needs you (missing caveats? a finding the template doesn't know to look for?); extend the template by one finding — your judgment, encoded.
4. The scheduling, done for real: schedule the script (or simulate it: run it twice a day for three days) and collect the run logs; write the one-paragraph runbook ("if it fails, do this") in the README — the handover document you would leave a successor.
5. Portfolio evidence #3: README with before/after timing, the folder tree, two exhibits, a screenshot of the run log; update the 150-word case-study note and the STAR paragraph ("tell me about something you automated"). Three projects in the ledger — the portfolio (Chapter 49) is nearly ready to assemble.

## Further Reading

- Chapters 38–43 (Power BI — the report becomes a dashboard), Chapter 49 (the portfolio)
- Appendix C (the full pipeline listing + the Python cookbook)



**PART VI**

# **Dashboards and Storytelling with Power BI**

Chapters 38 – 43

---

# Chapter 38: What Power BI Is

---

## Part VI — Dashboards and Storytelling with Power BI

"A report tells you what happened. A dashboard waits to be asked — and answers in the time it takes to wonder."

### In this chapter you will learn

- What Power BI is (and is not), and the free-vs-paid line that matters.
- The three surfaces: report, dashboard, and the workflow that connects them.
- Why dashboards change the analysis job — from answering to *anticipating*.
- The Power BI workflow in one page — get data, model, measure, visual, publish.
- Installing Power BI Desktop and taking the ten-minute tour.

## 38.1 The Tool and Its Place

Power BI is Microsoft's business-intelligence platform: load data, build a **data model** (tables and relationships — you know these), define **measures** (calculations with names — you know these too), arrange visuals into an interactive **report**, and publish it so readers can explore with filters and slicers instead of waiting for next month's PDF. Its siblings in the wider world — Tableau, Looker, Qlik, Metabase, Superset — share the same skeleton; Power BI is the most common in Microsoft-world business, and the one this book teaches (Chapter 61 maps the free alternatives).

The free/paid line, stated precisely because it decides your learning path: **Power BI Desktop is completely free** — the full authoring experience, unlimited local reports, everything this Part needs. What costs money is **sharing**: the Power BI service (publishing to the web, app workspaces, scheduled refresh, row-level security) is where licences enter. Learning, building, and demonstrating a portfolio on Desktop: free. Publishing to an organisation: your employer's licence problem, solved on day one by IT. Download Desktop from [microsoft.com](https://microsoft.com) (Windows; Chapter 61 covers the macOS and browser paths honestly).

## 38.2 What Changes When You Build Dashboards

Everything so far in this book answered *questions you were asked*: this month's revenue, the oil collapse, Sunday's verdict. A dashboard inverts the relationship — you must *anticipate* the questions. The reader (Tariro, her bank manager, a department head) will open the page at any moment, click any filter, and expect an answer *now*. Three consequences, and they are thePart's real syllabus:

1. **The numbers must be right for every slice.** Revenue filtered to March, to EcoCash, to Tuesdays — every combination must compute correctly, which means the *model* must be right (Part III's keys and fan-out lessons, now structural) and the *measures* must handle every context (DAX's specialty, Chapter 40).
2. **The page must answer in five seconds.** Not performance (though that too) — *reading speed*: the reader glances, clicks, glances. Chapter 11's chart grammar becomes Chapter 42's page grammar: layout, hierarchy, the headline that survives a skim.
3. **The data must refresh itself.** A stale dashboard is a lie with a date on it. Part V's automation mindset returns as scheduled refresh — the pipeline feeds the dashboard.

And the honest boundary, said now and repeated in Chapter 43: **a dashboard does not replace the report**. Dashboards answer the recurring questions ("how are we doing?"); reports answer the deep ones ("why did oil collapse and what should we do?"). The monthly report of Project 3 survives; the dashboard absorbs the questions it was being asked repeatedly.

## 38.3 The Workflow in One Page

The whole of Part VI is this loop, six stages, every one an old friend in a new accent:

- |             |                                                                       |
|-------------|-----------------------------------------------------------------------|
| 1. GET DATA | -- CSV, database, web: the loading liturgy (Ch. 39)                   |
| 2. SHAPE    | -- Power Query: clean, typed columns, the Chapter 8 work, recorded    |
| 3. MODEL    | -- tables + relationships: the star, the keys (Ch. 39)                |
| 4. MEASURE  | -- DAX: named calculations like revenue, MoM %, member share (Ch. 40) |
| 5. VISUAL   | -- charts on a page, filtered, interactive (Ch. 41–42)                |
| 6. PUBLISH  | -- to the service, scheduled refresh, shared (Ch. 43)                 |

Notice what is *not* in the loop: ad-hoc cell edits, hand-typed numbers, copy-paste between files. The dashboard is compiled from a model, the model from sources, the sources checked by queries — the audit trail Chapter 20 demanded, embodied in a tool. When your manager asks "where does this number come from?", the answer is a *lineage view*: visual → measure → table → source, clickable. That is the professional difference between a dashboard

and a spreadsheet emailed on Mondays.

## 38.4 The Ten-Minute Tour

Install Desktop (free, from Microsoft's site), open it, and take the tour before Part VI begins properly — hands on the wheel for ten minutes:

1. **Get data** → **Text/CSV** → your cleaned `sales.csv`. The preview appears; click **Load**. (Transform Data — Power Query — waits for Chapter 39; Load is fine today.)
2. The **Fields** pane (right) lists your table and columns. Drag `amount` onto the blank page: a table visual appears. Now drag `payment_type` onto it — a pivot, instantly.
3. With the visual selected, open the **Visualizations** pane and click the clustered-column chart icon: it becomes a bar chart of revenue by payment type. You have just built the pivot → chart pipeline of Chapters 10–11 with two drags.
4. Drag `date` into the visual's **Axis** area and watch it group by month automatically — the time intelligence beginning.
5. **Insert** → **Slicer**, drag `payment_type` into it: a clickable filter. Click `EcoCash`; the chart obeys. *That click is the entire meaning of "interactive" — the reader is now querying your model themselves.*

Ten minutes, five moves, and the shape of the Part is in your hands. Everything remaining is depth: shaping properly (39), measuring properly (40), pages that read fast (41–42), and telling the story when the dashboard's job is done (43).

## 38.5 Tariro's Brief

The Part's running brief, from Tariro and her bank manager, gives the next six chapters their spine — a real deliverable, specified like a work request:

*"We want a dashboard the bank manager can open the morning of the quarterly meeting, and Tariro can open any evening: revenue and how it's moving, the payment mix, the Sunday situation, the oil story, the loyalty share, and the top items — with the ability to look at any month, any till, any payment type. And it must show its own data age — no stale numbers."*

Read the brief and see the syllabus: movement (time intelligence), mix (percent-of-total), the Sunday situation (a test's verdict, displayed), top-N (ranking), slicing (filters), and *data age* — the refresh stamp, the honesty requirement. By Chapter 43 it exists; Project 6 (Chapter 48) extends it into the operations dashboard — the portfolio piece that shows employers you can

build for a *reader*, not just compute for yourself.

**From Your Toolkit — the fifth tool, the same atoms:** Power BI is the book's fifth language, and its nouns are all borrowed: relationships are Chapter 17's keys (the model view *is* the diagram you learned to draw); measures are named formulas with Chapter 34's semantics; the slicer is a WHERE the reader types with their hand; the refresh is Chapter 37's cron wearing a GUI. Nothing new to understand — only a new accent to speak, and a genuinely new discipline: designing for readers you will never watch.

## Key Takeaways

- Power BI Desktop is free and complete for learning, building, and portfolio; only *sharing* (the service) costs licences.
- Dashboards invert the job: anticipate questions, be right for every slice, answer in five seconds, refresh yourself — and they complement, not replace, reports.
- The workflow: get data → shape (Power Query) → model → measure (DAX) → visual → publish — the audit trail embodied, lineage clickable.
- The ten-minute tour already covers pivot→chart→slicer: the reader is querying your model with clicks.
- Tariro's brief is the Part's deliverable: movement, mix, Sunday, oil, loyalty, top-N, slicing — plus the data-age stamp.

## Practice Lab

1. Install Power BI Desktop (or set up the browser alternative from Chapter 61's notes); take the ten-minute tour on `sales.csv`; save as `tariro-tour.pbix`.
2. Slicer drill: build the payment-type slicer plus a month slicer; click combinations and watch the chart recompute; write down three questions a reader could now answer *without you* — the inversion, experienced.
3. The lineage hunt: with your visual selected, use View → Performance/lineage features (or Model view) to trace visual → table → source; screenshot the lineage for your log — the "where does this number come from" answer, embodied.
4. The brief, annotated: re-read Tariro's brief; for each requirement, write which chapter of Part VI you expect to deliver it; keep the page — Chapter 43 checks it off.
5. The competitor skim (30 minutes): look at Tableau Public and Metabase's demo pages; write one paragraph: what all three tools share (the skeleton), and one thing each does differently — the tool-agnosticism of Part III's

Chapter 20, applied to BI.

## Further Reading

- Chapter 39 (getting data in and the model — where correctness is decided)
- Microsoft's "Get started with Power BI Desktop" learning module — free, and painless after this chapter

# Chapter 39: Getting Data In — Shaping and the Model

Part VI — Dashboards and Storytelling with Power BI

"The dashboard is only as honest as the model beneath it — and the model is only as honest as the shaping."

## In this chapter you will learn

- Power Query: the recorded, replayable cleaning pipeline — Chapter 8 with a memory.
- Loading Tariro's four tables, shaped properly.
- The data model: the star schema, relationships, and cardinality.
- The fan-out guard, structurally: how the model view makes it visible.
- Date tables: the one special table every serious model has.

## 39.1 Power Query: Cleaning That Remembers

Click **Get data** → **Text/CSV**, and before Load, choose **Transform Data** — the door into **Power Query Editor**, and the most under-taught superpower in the whole Microsoft stack. Power Query is Chapter 8's cleaning liturgy, *recorded*: every step you take (remove blank rows, change a type, trim text, split a column) is written into an **Applied Steps** list on the right, and the whole chain replays automatically on every refresh. The part-time cashier's detached-sort disaster (Chapter 14's story) is structurally impossible here: raw data arrives, steps transform a *copy*, the original is never touched, and next month's CSV gets the same treatment by replay. It is exactly Chapter 37's pipeline — *data/raw* sacred, cleaning reproducible — as a visual editor.

The working session on Tariro's *sales.csv*, in Power Query's own buttons (each step lands in Applied Steps, and you should *name* steps by double-clicking them — the why-comments of Chapter 20, wearing a list):

1. **Home** → **Remove Rows** → **Remove Blank Rows** — the ghost rows.
2. Right-click *payment_type* → **Transform** → **Trim** (whitespace), then **Format** → **Capitalize Each Word** — the three-EcoCash-spellings fix, replayed monthly forever.
3. Right-click *amount* → **Change Type** → **Decimal Number** (and *date* → Date). Power Query *guesses* types on load — the Chapter 33 lesson: check every guess.

4. **Add Column → Column From Examples** → type "Mon" next to a Monday date: the weekday column, inferred.
5. Right-click a column → **Rename**; and **Choose Columns** to drop what the model does not need.
6. **Close & Apply** — the steps run, the table lands in the model.

Do the same for `customers`, `stock`, `suppliers` — four queries, four named step-lists, and a shaping layer you could hand to a successor with the words "read the steps top to bottom."

## 39.2 The Model

With data loaded, open **Model view** (the icon on the left edge): your tables float in a diagram — and here Power BI draws Chapter 17's boxes-and-lines picture *for you*. The work is arranging relationships by dragging keys onto keys: `customers.customer_id` → `sales.customer_id`; `stock.item` and `suppliers.item` → an items dimension (or a small `items` table if the design needs one). Three properties per relationship, and they are the Chapter 17 vocabulary exactly:

- **Cardinality** — One-to-many (1:): *the safe default (one customer, many sales). Many-to-many (:): Power BI permits it, flags it, and you should treat the flag as Chapter 17's fan-out warning** — a many-to-many in your model means the design is asking for a bridge table.
- **Cross-filter direction** — Single (filters flow one way, from the "one" side) or Both. Single is the safe default; Both is for special patterns and quiet ambiguity.
- **Activate/Inactive** — multiple date relationships (order date vs ship date) exist; advanced DAX picks between them (beyond this book's brief).

The **star schema** is the shape you are aiming at: one **fact table** in the middle (sales — the events, many rows, the numbers) surrounded by **dimension tables** (customers, items, a date table — the context, fewer rows, the descriptors). Star schemas are why dashboards are fast and sane: measures aggregate the fact; dimensions filter and label it; every question is a path from a dimension through the fact. (Tariro's four tables are a small star already; most of your working life will be recognising, or building, this shape in bigger clothes.)

## 39.3 The Fan-Out Guard, Structurally

The old enemy returns, with a new weapon. In a spreadsheet, fan-out hides; in SQL, it is a doubled total; **in a model, it is visible in the diagram** — a relationship arrow whose wrong cardinality or whose many-to-many badge is the smoking gun. The defensive ritual, once per model, forever:



1. **Model view:** check every line's cardinality badge; any *:~ is investigated (usually: the dimension has duplicate keys — fix in Power Query with a Group By/dedup step, or the relationship's key is not actually unique).
2. **The reconciliation card:** a card visual with `SUM(sales[amount])`, compared to the known bridge total (54,013.50). It must match *exactly* before any other visual is built — the same cent-for-cent bridge from every Part of this book, now standing guard on the dashboard itself.
3. **The no-relationship test:** a table visual of `customers` names next to `sales` amounts that computes *wrong* (repeating, inflated) is almost always a missing or misdirected relationship — the amputation/fan-out pair, visible at a glance.

## 39.4 The Date Table

One special table deserves its own section because serious models always have one: a **date table** — a calendar, one row per date, spanning (and exceeding) your data's range, with the columns every time question needs: date, year, month name, month number, weekday, weekend flag, quarter. Build it in Power Query (Home → Enter Data is too small; better: a small query generating dates, or `calendar.csv` from Python — Appendix C's generator emits one), relate it to `sales.date`, then **Mark as date table** (Table tools → Mark as date table).

Why the ceremony? Because "month" sliced from a raw date column is *text* ("2025-06") — it sorts, but it does not *know* things: it cannot tell you which months are which quarter, whether a day was a weekend, or what "the previous month" means across years. A marked date table gives DAX's time intelligence (Chapter 40's month-over-month and running totals) a true calendar to compute against — the difference between dates-as-strings and *time as a dimension*. The professional tell in any .pbix you inherit: look for the date table; its absence predicts every date-related bug you are about to find.

## 39.5 Where You Are

The model is the deliverable of this chapter, even though nothing visual exists yet: four shaped, replayable queries; a star with correct cardinality; the bridge reconciled; the calendar marked. Everything Chapters 40–43 build — measures, visuals, pages, the story — stands on this, and the industry's hardest BI bugs live exactly here, which is why this chapter spent its length on keys, badges, and the bridge. Tariro's brief has its foundation; the next chapter teaches it to calculate.

**From Your Toolkit — the model is forever:** the star schema is the same shape behind every BI tool (Tableau's relationships, Looker's views, a warehouse's dims and facts — Chapter 57's world is this chapter industrialised); Power Query's step-list is Chapter 37's pipeline and Chapter 20's header block in visual form; and the date table is the calendar that pandas' `.dt` accessor was quietly being for you all along. Learn it once here, recognise it everywhere.

## Key Takeaways

- Power Query records and replays every cleaning step: raw data sacred, steps named, next month's CSV transformed identically — the pipeline as an editor.
- The model is Chapter 17 drawn: relationships with cardinality and direction; star schema (facts in the middle, dimensions around); treat many-to-many badges as fan-out warnings.
- The reconciliation card (SUM vs the known bridge) guards every model before the first visual is built.
- Build and mark a date table: time as a dimension — the precondition for time intelligence.
- The hardest BI bugs live in the model; an afternoon here prevents a quarter of confusion later.

## Practice Lab

1. Shape all four tables in Power Query with named steps (trim, case, types, weekday-from-example, renames); write the step-list for `sales` into your log as prose ("what the pipeline does, in order") — the successor's document.
2. Build the star: relate customers→sales, items→sales, date→sales; screenshot the model diagram with cardinality badges visible; annotate the screenshot with the one-to-many arrows in your own words.
3. The bridge card: total revenue visual vs 54,013.50; then break it on purpose (delete the customers relationship, note what changes; mis-set a cardinality to `:`, note the badge) — the fan-out museum, BI wing.
4. The date table: build or import the calendar, mark it, and confirm the weekday column matches `sales`' own weekday derivation for ten sample dates.
5. The refresh rehearsal: add next month's rows to a copy of `sales.csv`, point the query at it (or replace the file), hit **Refresh**, and watch the steps replay and the bridge card update — then write the two-paragraph runbook ("how

this dashboard's data refreshes") for the README you will publish in Chapter 49.

## Further Reading

- Chapter 40 (DAX measures — the model learns to calculate)
- Microsoft Learn: "Model data in Power BI" — the official depth behind this chapter

# Chapter 40: Measures and DAX — the Ten Formulas

## Part VI — Dashboards and Storytelling with Power BI

"A measure is a formula with a name and a home. DAX is the dialect it speaks — and you already speak three of its parent languages."

### In this chapter you will learn

- Measures vs calculated columns — the distinction that separates DAX users from DAX sufferers.
- The evaluation context: the one idea that makes DAX make sense.
- The ten formulas that cover 90% of real dashboards, each on Tariro's data.
- Time intelligence: month-over-month, running totals, period comparisons.
- The filter-propagation insight: why a measure is right in *every* slice.

## 40.1 Measures, Not Columns

Your first DAX decision is architectural, and beginners get it wrong by reflex. A **calculated column** computes row by row at refresh time (`amount * 1.15` for every row — a new column stored in the table): use it for *attributes* (weekday flag, price band). A **measure** computes at *query* time, over whatever rows the current context selects (SUM of `amount` — a formula with a name, evaluated live for every visual, filter, and slicer). The rule the professionals follow: **if the number answers "how much/how many", it is a measure; if it labels a row, it is a column**. Revenue, count, share, growth: measures. Weekend flag, suburb, tier: columns.

Create a measure: right-click the `sales` table in the Fields pane → New measure:

```
Revenue = SUM(sales[amount])
```

Then drag it into a card visual: the bridge number again (54,013.50 — reconciled at the moment of its birth, as Chapter 39 required). Rename tables' display names, keep measures in a dedicated "Measures" home table if you like (a professional habit for navigation), and format each measure once (Format → Currency/Percentage) — formatting is part of the measure's definition, not a per-visual chore.

## 40.2 The One Idea: Evaluation Context

DAX is confusing for exactly one reason, and understanding it dissolves 80% of the confusion: **every measure evaluates inside a context** — the set of rows currently visible — and the *same measure* computes a different (correct) number in each context. Revenue in a card with no filters: 54,013.50. The same [Revenue] in a table visual by payment_type: three numbers, each the sum over that type's rows. The same [Revenue] under the EcoCash slicer: only EcoCash rows. This is **filter context**, and it propagates through the model's relationships: click Avondale in a suburb slicer, and the filter flows customers → sales, and [Revenue] recomputes — *the reader's click becomes a WHERE clause executed against your star schema*. Nothing in DAX makes sense except in light of filter context — and everything in DAX becomes manageable once you hold it: you never write "revenue for March" — you write [Revenue] and let March be the context. (Row context, its sibling, is calculated-column territory; you will feel it when you meet it.)

## 40.3 The Ten

The working set — genuinely, on real dashboards, these ten (and their obvious mutations) cover almost everything. Type each on Tariro's model, and meet old friends:

```
-- 1. Total revenue
Revenue = SUM(sales[amount])
-- 2. Count (of rows = of sales)
Sales Count = COUNTROWS(sales)
-- 3. Average basket
Average Basket = DIVIDE([Revenue], [Sales Count])
-- 4. Share of total (% of grand total -- the Ch. 18 share pattern, live)
Revenue Share =
DIVIDE([Revenue], CALCULATE([Revenue], ALL(sales)))
-- 5. Distinct customers (the loyalty denominator)
Distinct Customers = DISTINCTCOUNT(sales[customer_id])
-- 6. Members' revenue share (filter context, steered by hand)
Member Revenue =
CALCULATE([Revenue], sales[customer_id] <> BLANK())
-- 7. Month-over-month (needs the marked date table)
Revenue MoM % =
VAR CurrentMonth = [Revenue]
VAR PrevMonth =
    CALCULATE([Revenue], DATEADD('Calendar'[Date], -1, MONTH))
RETURN DIVIDE(CurrentMonth - PrevMonth, PrevMonth)
-- 8. Running total (the cumulative year, Ch. 19 again)
Revenue YTD = TOTALYTD([Revenue], 'Calendar'[Date])
-- 9. Same period last year (the comparison every manager asks for)
Revenue LY =
```

```

CALCULATE([Revenue], SAMEPERIODLASTYEAR('Calendar'[Date]))
-- 10. The Sunday verdict, as a measure
Sunday Avg Basket =
CALCULATE([Average Basket], 'Calendar'[Weekday] = "Sunday")

```

Four vocabulary notes, and they are the grammar's core: **DIVIDE** is the division that survives zeros (returns BLANK instead of an error — the IFERROR of dignity). **ALL(table)** removes the filters — "the whole thing, regardless of context" — which is how shares get their denominators (measure 4 is literally Chapter 18's CTE pair). **CALCULATE** is DAX's crown jewel: *evaluate this measure in a context I modify* — a WHERE clause you write by hand, composed with the reader's clicks. And **VAR ... RETURN** names intermediate steps — the CTE habit, DAX's accent; name your thinking.

## 40.4 Time Intelligence, Properly

Measures 7–9 are **time intelligence** — DAX's biggest practical gift, all of it dependent on the marked date table from Chapter 39. The patterns to recognise on sight (and reuse endlessly): **DATEADD** (shift the period: –1 month, –1 year — month-over-month, year-over-year); **TOTALYTD/MTD** (cumulative within a boundary — the running total, the bank-manager line); **SAMEPERIODLASTYEAR** (the comparison that makes trends meaningful: this month against its own past, not against last month's season). Note the deep honesty these enable: "revenue is down" becomes "revenue is down 4% against the same month last year but up 11% year-to-date" — context, not vibes, the Chapter 2 habit compiled into formulas.

One classic trap, named so you never fall in: time intelligence compares *dates*, and a partial current month (data to the 14th) will "collapse" against full comparison months. The professional's fix is a measure like `Is Current Month Complete` or reporting MTD with a data-age stamp (Chapter 41 implements the stamp) — the stale-dashboard lie, prevented at the measure layer.

## 40.5 Right in Every Slice

The chapter's closing claim, and thePart's heart: a measure built on a correct model is right in *every* context — every filter, every slicer, every combination — because it computes from the rows the context selects, not from a number you froze. That is the qualitative jump from the spreadsheet world: the pivot you re-dragged monthly (Part II), the query you re-ran monthly (Part III), the report that rebuilt itself monthly (Part V) — all of them are now **one measure, evaluated live, for a reader you will never watch**. The craft obligation follows: you must *test* the slices (does [Revenue MoM %] behave in the January boundary? does the member share survive the suburb slicer?) — the quality gate of Chapter 13, applied to contexts instead of pages.

**From Your Toolkit — the measure mindset:** a DAX measure is a named formula evaluated in context — the same object as a SQL aggregate over a GROUP BY you cannot see, a pandas `.agg()` whose groupby the reader chooses, an Excel named formula with a filter attached. CALCULATE is WHERE; ALL is "remove the WHERE"; VAR/RETURN is the CTE. You have been training for DAX since Chapter 9 — the accent is new; the sentences are yours.

### Key Takeaways

- Columns label rows (refresh-time); measures answer "how much" (query-time, in context) — choose by the question, not by reflex.
- Filter context is the one idea: the same measure is a different, correct number in every visual/slicer combination; clicks are WHERE clauses.
- The ten: SUM, COUNTROWS, DIVIDE average, ALL-share, DISTINCTCOUNT, CALCULATE-filters, DATEADD MoM, TOTALYTD, SAMEPERIODLASTYEAR, steered verdicts.
- DIVIDE survives zero; CALCULATE steers context; VAR/RETURN names your thinking — CTE habits, DAX accent.
- Test the slices (boundaries, slicer combos); stamp the data age — a measure is a promise to readers you will never watch.

### Practice Lab

1. Build all ten measures on your model, each formatted once at creation; drag each into a card or table and reconcile the unfiltered numbers against your Part II/III bridges (Revenue, count, average basket, distinct customers — four cent-for-cent matches logged).

2. The context experiment: one table visual (payment type × [Revenue] and [Revenue Share]), then add suburb and month slicers; click ten combinations and verify two of them by hand against SQL or pandas — the "right in every slice" promise, personally audited.
3. The boundary test: place [Revenue MoM %] in a month table and check the January row (does it reach for last *December*? does your date table span that far?); fix the calendar if needed and note the fix — this is the trap, sprung and defused.
4. The steer drill: write three custom measures with CALCULATE — weekend revenue, whale-only revenue (amount > 21.50), till-2 revenue; display beside [Revenue]; verify one against a pandas equivalent.
5. The tenth, extended: a small "verdict table" — Sunday vs weekday average basket and revenue, using measures 10 and a weekday variant; caption it with Part IV's formal finding ( $t = -18.5$ ,  $p < 0.001$ ) — the dashboard now *displays the test's verdict*, the analysis-to-dashboard bridge this Part keeps building.

## Further Reading

- Chapter 41 (the dashboard itself — the measures get their page)
- *The Definitive Guide to DAX* (Russo & Ferrari) — when you outgrow the ten, this is the mountain



# Chapter 41: Your First Dashboard

## Part VI — Dashboards and Storytelling with Power BI

"A dashboard is not a chart collection. It is a page that answers a reader's next question before they finish asking it."

### In this chapter you will learn

- Page architecture: the grid, the reading order, the five-second rule.
- Building Tariro's dashboard page by page — every element from the brief.
- Bookmarks, drillthrough, and tooltips: the interactions that make a page feel alive.
- The data-age stamp and the reconciliation card — honesty, embedded.
- Performance basics: why the page is slow, and the three usual fixes.

### 41.1 The Architecture of a Page

Dashboard layout is a design discipline with rules you can learn in an afternoon (Chapter 42 deepens the craft). The core: **readers scan top-left to bottom-right, big to small** — so the page is a *deliberate* hierarchy:

+-----+   TITLE + DATA-AGE STAMP + the slicers (month, till, payment)   +-----+	
THE HEADLINE NUMBER(S)	THE MOVEMENT
Revenue (big card)	Revenue by month (line)
MoM % (with LY comparison)	
+-----+	
THE MIX	THE STORIES
Payment type (100% bar)	Sunday verdict   Oil trend
Top items (ranked bar)	Member share trend
+-----+	
THE DETAIL (small, bottom): the table for the person who	
wants exact numbers	
+-----+	

The **five-second rule** governs every placement: glance at the page for five seconds; what did you learn? (Revenue, whether it's up, one thing that needs attention — if not, the top of the page is wrong.) The detail table at the bottom serves the reader who wants precision — the appendix instinct from Chapter 13, transposed. And nothing decorates: every visual earns its pixels by answering part of the brief, or it goes.

## 41.2 Building It, Page by Page

Open your .pbix from Chapter 40 — model built, measures named — and build the brief in order. The session, element by element (each element names the measure or chapter that powers it):

**The header.** A text box: "Tariro's Grocery — Trade Dashboard". Beside it, the **data-age stamp**: a card visual on a measure —

Data Age =

```
"Data through " & FORMAT(MAX('Calendar'[Date]), "d MMM yyyy")
```

— the brief's honesty requirement, implemented: a reader instantly knows whether they are looking at this month or last quarter's ghost. (This measure is text, so it lives as a card; it is also your staleness *monitor* — see 41.4.)

**The slicer rail.** Three slicers across the top (month from `Calendar`, till, payment type); Format → Single select off (readers combine); give them a consistent style. Every visual below respects them automatically — the model's filter propagation doing the work.

**The headline cards.** `[Revenue]`, `[Revenue MoM %]`, `[Revenue LY]` beside it — three cards, one row: *what, which way, against what baseline*. (Card visual, callout value formatted; the MoM card coloured conditionally — green above 0, red below — via Conditional formatting on the card: the glance-friendly version of Chapter 11's finding-title rule.)

**The movement.** Line chart: Axis = `Calendar[Month]`, Values = `[Revenue]`. Add `[Revenue LY]` as a second line — the year-over-year comparison the bank manager actually wants, on the chart itself.

**The mix.** A **100% stacked bar** of payment type by month (Axis = month, Legend = payment type, Values = `[Revenue Share]`-style percentage) — the EcoCash-share-rising story as a visible current; or simpler first pass: stacked bar with the mix measure. Beside it, **top items**: a ranked horizontal bar (Top N filter on the visual: Top 10 by `[Revenue]`) — Chapter 34's `nlargest`, clicked.

**The story tiles.** Two or three small visuals carrying the named stories: the Sunday verdict (weekday bar with Sunday highlighted — a weekday table + `[Revenue]`, Sunday's bar coloured via data labels), and the oil trend (line of oil revenue by month, showing the month-9 cliff). These are the dashboard's *sentences* — the brief's items that are stories, not statuses.

**The detail floor.** A matrix visual: rows = items, values = `[Revenue]`, `[Sales Count]`, `[Average Basket]`, sorted by revenue. The precision layer, for the reader who wants to quote a number in a meeting.

Then the polish pass, ten minutes that double the page's quality: align everything to a grid (Format → Align); consistent fonts and two colours

(Tariro teal + amber, matching the report exhibits); visual headers phrased as findings-in-waiting ("Revenue by month — growth and the month-9 dip", not "Sum of amount by Month"); and the page named "Trade Overview" (a dashboard with more than one page is a navigation problem — see 41.3).

### 41.3 The Interactions

Three Power BI features turn a static page into a tool — implement at least the first two:

- **Tooltips.** Hover on any bar/point: the default shows values. Better: Format → Tooltip → place a small extra visual (the payment mix for the hovered month) — a reader's question answered *on hover*.
- **Drillthrough.** Right-click any month → Drill through → "Item detail" — a second page, pre-filtered to that month, showing the item-level table and the top-N bars. The Chapter 10 double-click drill-down, reborn: summary to evidence in one right-click. Build the target page once, set its drillthrough field (`Calendar[Month]`), and every visual gains the path.
- **Bookmarks.** Saved states of the page (filters, visibility) — "Reset" (clears slicers; the small kindness every reader appreciates) or toggled focus views. Optional craft; the Reset bookmark alone is worth it.

### 41.4 The Honesty Layer

Two small elements, permanent policy on every dashboard you ever ship: the **data-age stamp** (built above) and the **reconciliation card** — the `[Revenue]` card, checked once against the bridge total whenever the data refreshes (Chapter 39's guard, kept visible or at least kept in a "Checks" hidden page). Together they answer the two questions every honest dashboard owes its readers: *how fresh is this?* and *can I trust the total?* A third belongs in your publishing checklist: **the checks page** — a hidden page with the bridge, the null counts, the orphan count as card visuals — so that *you* can open the file on refresh day and see the whole liturgy at a glance. (Chapter 37's fail-loudly guards, transposed: the pipeline aborts on broken data; the dashboard *displays* its health.)

## 41.5 Performance, the Short Version

If the page is slow (spinner on every click), the causes are almost always three, in this order: **visuals showing too much** (a table of 6,420 rows rendered — top-N it or aggregate); **measures that scan repeatedly** (the same ALL/CALCULATE pattern in ten visuals — simplify the model, or pre-aggregate in Power Query); **too many visuals** (each one is a query; twenty visuals is twenty queries per click — cut to the ones that answer the brief). The debugging tool is **View → Performance analyzer**: record a refresh, see per-visual timings, fix the slow one — the EXPLAIN habit of Chapter 20, pointing at pixels instead of queries.

**From Your Toolkit — the deliverable template:** this page — header/stamp, slicer rail, headline cards, movement, mix, stories, detail floor, checks page — is the skeleton of every operational dashboard you will ever build (Project 6 extends it; your first job's "can you improve our weekly dashboard?" inherits it). The visuals change; the architecture — hierarchy, honesty layer, drill paths — is the craft that transfers.

### Key Takeaways

- A page is a hierarchy, not a gallery: five-second headline, movement, mix, stories, detail floor — reading order is design.
- Build the brief in order: stamp and slicers first, cards, line with LY, mix, top-N, story tiles, matrix floor; then the polish pass (grid, palette, finding-phrased headers).
- Tooltips, drillthrough, and the Reset bookmark are the interactions that make a page feel alive — implement at least tooltips and drillthrough.
- The honesty layer ships always: data-age stamp, reconciliation card, hidden checks page — freshness and trust, visible.
- Slow pages: too much data shown, repeated heavy measures, too many visuals — Performance analyzer is EXPLAIN for pixels.

### Practice Lab

1. Build the full Trade Overview page on your model — every element from the brief, the polish pass included; save `tariro-trade.pbix`. Screenshot the finished page for the log and the portfolio.
2. The five-second test: hand the file (or screenshot) to a friend for five seconds; ask what they learned. Whatever they missed, move up or enlarge. Iterate until the headline survives the glance.

3. The drillthrough build: an "Item detail" page (matrix + top-N bars + month context card), drillthrough on month; test the path from three different visuals; write the two-line note a reader would need (or does the page explain itself?).
4. The checks page, built: bridge card, null-count cards, orphan count, data age — hidden but complete; open it after a refresh rehearsal (Ch. 39's lab data) and log what you see.
5. Performance analyzer run: record, sort by duration, screenshot the slowest visual, and fix it (top-N, simplify, or cut); record before/after — a portfolio-ready micro-story: "dashboard interaction from 3.2s to 0.4s by removing two visuals and top-N-ing the table".

## Further Reading

- Chapter 42 (chart craft at dashboard scale — the rules, intensified), Chapter 43 (presenting the story)
- Microsoft Learn: "Create and use analytics reports in Power BI" — the official companion

# Chapter 42: Chart Craft, Again — the Dashboard Edition

## Part VI — Dashboards and Storytelling with Power BI

"Chapter 11 taught you to draw one honest chart. A dashboard is twenty honest charts agreeing with each other — which is a different, harder honesty."

### In this chapter you will learn

- The dashboard editions of the three chart lies — new costumes, same crimes.
- Visual consistency: the system of colour, sort, and scale that makes a page readable.
- Choosing the visual by the question — the chooser's table, extended for BI.
- Small multiples and the "compare across" moves dashboards make possible.
- Accessibility: the craft rules that include every reader.

## 42.1 The Three Lies, Dashboard Edition

Chapter 11's three lies return wearing business-casual, and dashboards give them new room to work:

1. **The truncated axis** now arrives *by default*: BI tools auto-scale axes to the data's range, so a revenue line from 9,100 to 9,400 auto-truncates without anyone choosing to lie. Fix deliberately: start value axes at zero for bars (always) and consider it for lines (state the choice in the visual's header if not); the auto-scale is a decision you inherit — and you are responsible for inherited decisions (the Chapter 20 rule, now about pixels).
2. **The unlabelled scale** multiplies: twenty visuals is twenty sets of axes, and a reader cannot remember which is dollars and which is counts. Fix with a *system*: currency to two decimals with thousands separators on every money visual, percentages to one decimal, counts as integers — set once in the measure's formatting (Chapter 40), inherited everywhere. When formats are part of measures, this lie becomes impossible.
3. **The dual-axis trick** reappears as "combo charts" (columns + line, two axes) — legitimate for genuinely different units (revenue and *count*), treacherous when both axes are money scaled to cross at flattering angles. The dashboard rule: dual axes only for different units, labelled on both

sides, and never the chart carrying the page's main claim.

## 42.2 Consistency Is a System

One page, one visual language. The craft rules, each worth adopting permanently:

- **Colour means something, once.** EcoCash is teal *everywhere on the page and in every tooltip*; Sunday is amber in every weekday visual. Colour-as-category must be stable across visuals, or the reader re-decodes each chart from scratch — the palette is a dictionary, not a decoration. (Tariro's dashboard: teal = primary series, amber = attention/comparison, grey = context. Three colours; a fourth requires a meeting with yourself.)
- **Sort is a statement.** Ranked bars sort by value; time charts run left-to-right by date; category tables sort by the page's primary measure. "Alphabetical" is a valid choice only when the reader looks things up (a matrix of *items*) rather than compares them.
- **Scale is comparable.** If two charts on a page show the same measure, their axes should agree — or the disagreement is a lie by juxtaposition. Small multiples (42.4) are the systematic fix.
- **White space is load-bearing.** The reader's eye needs rest stops; cramming is not density, it is noise. Twelve clean visuals beat twenty cramped ones on every measure that matters, including speed.

## 42.3 The Chooser's Table, Extended

Chapter 11's question→shape table, extended with the visuals BI adds (and the traps each carries):

The question	The visual	Trap to avoid
"How does it move?"	Line	Truncated auto-axis; too many lines (>4 = spaghetti)
"Which is biggest?"	Ranked bar	Alphabetical sort (says nothing)
"What's the mix, and how does it shift?"	100% stacked bar	Legends hiding the shift — label the ends
"Same measure, many groups, compared?"	Small multiples	Inconsistent per-tile scales
"Where does the total come from?"	Waterfall	Random order — sort contributions by size
"Two numbers per group (risk vs size)?"	Scatter (bubbles)	Unlabelled outliers — label the interesting dots
"Exactly what are the numbers?"	Matrix/table	No total row; thousand-row detail with no top-N

The question	The visual	Trap to avoid
"Where in the journey do they drop?"	Funnel	Unequal stage definitions
"Is this number okay?"	KPI card <b>with sparkline</b>	The naked number: no baseline, no trend

That last row is the dashboard-era addition: the **KPI card with context** — a big number alone says nothing (compared to what? says who? — the Chapter 2 questions, miniaturised). The craft pattern: number + comparison (+8.4% vs LY) + sparkline (the last 12 months) + target marker if a target exists. Every headline card on a serious dashboard is that quartet, because a number without context is furniture pretending to be a finding.

### 42.4 Small Multiples and the Compare Moves

The move dashboards make possible that single charts cannot: **small multiples** — the same chart repeated once per group (revenue by month, one tile per suburb, one per till, one per payment type), sharing axes so comparison is honest *by construction*. Power BI: the "Small multiples" well on line/bar charts (drop `suburb` into it). It is the most underused first-class visual in the tool, and it is Chapter 2's "compared to what?" answered structurally — the baseline is built into the grid. Use it whenever the question contains the words "across" or "each of the".

Its siblings: **decomposition tree** (drill a total into its contributions interactively — the explorer's funnel), and the humble **scatter with play axis** (two measures over time, animated — occasional, memorable). Each has exactly one question it answers best; the craft is matching, not collecting.

### 42.5 Accessibility Is Chart Craft

The rules that make charts work for *every* reader, which is simply professionalism: **colour is never the only carrier of meaning** — pair it with labels, position, or shape (the ~8% of men with colour-vision deficiency read your dashboard too; teal-vs-amber is a colourblind-safe pair, one reason this book chose it); **contrast** — dark text on light ground, 12pt minimum, which conveniently is also the "readable in a meeting-room projector" setting; **alt text** on visuals (Power BI: Alt text property — screen readers and the same text serves your PDF exports); and **keyboard paths** (Tab through slicers, Enter to apply — test it once). None of this is decoration; it is the same honesty as labelled axes, extended from the chart to the reader.



**From Your Toolkit — the craft layer, permanent:** these rules are tool-blind: they govern matplotlib exhibits (Chapter 35 — where you set the palette in a constants cell for exactly this reason), SPSS output charts, the Tableau dashboards you will meet, and every slide in Chapter 53's presentation. Chart craft was learned once in Chapter 11; what this chapter added is *system* — consistency, context, and the recognition that a page of charts is itself one chart, read as a whole.

## Key Takeaways

- The three lies, dashboard edition: auto-truncated axes (inherit decisions deliberately), format confusion (fix formats in measures, once), and combo-chart crossings (different units only, both axes labelled).
- Consistency is a system: colour-as-dictionary, sort-as-statement, comparable scales, load-bearing white space.
- The extended chooser: 100% stacked for shifting mixes, waterfalls for contributions, small multiples for "across", KPI cards only with context (number + comparison + sparkline + target).
- Small multiples make comparison honest by construction — use for every "each of the" question.
- Accessibility is craft: never colour alone, contrast and size, alt text, keyboard paths — honesty extended to every reader.

## Practice Lab

1. The audit, inward: run this chapter's rules against your Chapter 41 page — every axis (zero-baseline where owed), every format (is it in the measure?), every sort (statement or lookup?), colour dictionary (stable across visuals?); fix everything found and log the before/after screenshots.
2. The KPI quartet: upgrade your three headline cards to number + comparison + sparkline + (where real) target; screenshot the difference a sparkline makes to "Revenue: 3,401".
3. Small multiples, deployed: revenue by month as small multiples of suburb (or till); write the sentence the grid makes visible that a single line chart hid; check every tile shares the axis scale.
4. The colourblind check: view your page through a colour-vision simulator (many free online; or squint-test the greyscale); find any meaning that colour alone was carrying; fix with labels or position.
5. The one-page critic: find one public dashboard (Power BI gallery, Tableau Public, a news site's data page) and write a ten-line audit — the three lies,

consistency system, chooser matches, accessibility — praising what deserves it. Critiquing outward sharpens building inward.

## Further Reading

- Chapter 43 (the story — presenting the page and its findings), *Storytelling with Data* (Knaflitz) — the craft classic, whole-book version of this chapter
- The Power BI visuals reference (Microsoft Docs) — every visual, one line each

# Chapter 43: Telling the Story

---

## Part VI — Dashboards and Storytelling with Power BI

"The dashboard shows everything. The story says what matters. The craft is delivering both in one meeting."

### In this chapter you will learn

- The BI publishing path: Desktop file → service → shared, with scheduled refresh.
- The narrative layer: annotations, the summary page, and the guided path.
- Presenting to non-analysts: the five-minute walkthrough that works.
- The dashboard-vs-report decision, made consciously.
- Part VI's close: the full toolkit, assembled around one shop.

## 43.1 Publishing, the Honest Path

The last technical mile: getting the dashboard to its readers. In Power BI's world: **Publish** (Home → Publish, from Desktop) sends the `.pbix` to the **Power BI service** — Microsoft's cloud — where the report lives at a URL, viewable in a browser and (with the mobile app) on a phone; readers need licences (free viewers, in the common organisational setup). **Scheduled refresh** connects the service to the data source and refreshes on a timer — the Chapter 37 cron, managed by clicks; a **gateway** (a small local agent) is needed when the source is on-premises (Tariro's laptop CSVs, a company database behind the firewall).

For this book's scope — and many small-team real cases — the honest publishing ladder is simpler: (1) the `.pbix` file itself, shared (readers open it in Desktop, free — the *file* is the deliverable, version-controlled like any artefact); (2) export: Power BI can export pages/visuals to PDF or PowerPoint for the meeting pack — the dashboard's findings, frozen for the record; (3) the service, when an organisation is already licensed — which is your first job's likely path, and a day-one skill after this Part. (Chapter 61's free stack covers the no-licence alternatives: publishing the same star-schema to Metabase, or a scheduled HTML report from Part V's pipeline — the *architecture* of this Part transfers entire.)

## 43.2 The Narrative Layer

A dashboard shows everything; a reader needs to be *told* what matters. Three narrative devices, all built into the file, all turning the page from data into communication:

- **The summary page** — the *first* page of the report: three finding-phrased text boxes ("Revenue grew 31% over 18 months"; "Sundays are 28% below weekdays — a traffic problem ( $p < 0.001$ )"; "EcoCash share rising: 31% → 44%"), each beside its supporting visual in miniature, each with a page-navigation button to the full story. Chapter 13's one-page report, reborn as the dashboard's front door — and it satisfies the five-second rule at the *file* level, not just the page level.
- **Annotations on the chart** — the month-9 oil cliff, labelled on the line chart itself ("cooking-oil price rise +0.80"); the payday sawtooth, noted. Power BI: text boxes positioned precisely, or the "Smart narrative" visual (generated text you then *edit* — the machine drafts, you verify; always verify). The annotation is the finding-title rule from Chapter 11, placed at the exact coordinate where the reader will wonder.
- **The guided path** — buttons that navigate ("See the oil story →", "Drill to items →") so a first-time reader is walked through the argument you would walk them through in person. Bookmarks (Chapter 41) plus buttons equal a *tour* of the data: the storytelling of a presentation, encoded in the artefact.

## 43.3 The Five-Minute Walkthrough

The meeting — bank manager, boss, client — where you present the dashboard. The format that works (it is Chapter 13's report structure, spoken):

1. **The headline, first** (30 seconds): "Trade is up 31% over eighteen months, with one real problem — cooking oil since the price rise — and one opportunity, Sundays. I'll show you all three in five minutes." The audience now has the story's shape; everything after is evidence they have been promised.
2. **The movement** (60s): the revenue line with LY comparison; point at the dip *when you reach it*, name its cause, move on. Never narrate every point — you are the analyst, not the chart's tour guide; you pre-read the chart (Chapter 11) and deliver its three findings.
3. **The stories** (2 minutes): Sunday (the verdict tile — and the *caveat in the sentence*: "...which is a traffic problem, not basket size — that's a tested finding, not a hunch"), oil (the cliff, the quantity story, the pending shelf trial from Chapter 28), loyalty (the share trend, with the self-selection

caveat attached as naturally as the number).

- 4. **The interaction, once** (60s): hand over the mouse or click the slicer yourself — "here's March, here's EcoCash only" — showing the audience the dashboard answers *their* follow-ups without you. That moment converts the dashboard from your presentation into their tool.
- 5. **The ask** (30s): what should happen next — the recommendations, measurable, with owners and dates (the shelf trial; the four-Sunday promotion test; the top-5 order increase). End on a decision, not on "any questions?" (there will be questions; the ask survives them).

Two presenting rules that carry the whole craft: **never show a number without its comparison or cause** (the naked number is the amateur's tell), and **say the caveat before the listener finds it** (Chapter 2's "says who?", delivered proactively — the trust dividend compounds across every future meeting).

43.4 Dashboard or Report?

The conscious decision, made on every deliverable for the rest of your career:

Reach for a dashboard when...	Reach for a report (PDF/notebook/deck) when...
The question recurs ("how are we doing?")	The question is deep ("why did this happen?")
Readers need different slices	Readers need one argument, end to end
Data refreshes on a schedule	The analysis is a snapshot in time
The audience is many and varied	The audience is specific and known
Speed of answer matters most	Rigor and narrative matter most

Tariro's shop needs both, and has both: the Trade Overview dashboard for the recurring pulse, Project 3's automated monthly report for the argued findings. The professional failure mode is format fundamentalism — the analyst who dashboards everything (deep findings crushed into tiles) or reports everything (recurring questions re-answered monthly by hand). The brief decides; the toolkit serves.

## 43.5 Part VI Closes: the Toolkit Assembled

Stand back from the shop. One data story now flows through five tools, each doing its natural job: the **spreadsheet** (Part II) for first contact and one-off questions; the **database** (Part III) as the governed source of truth; **statistics** (Part IV) as the meaning machinery; **Python** (Part V) as the automated engine; and the **dashboard** (Part VI) as the always-on window for readers. The same shop, the same numbers, the same honesty rules — reconciled at every hand-off, caveated at every claim, tested where testing was owed. That is the working stack of a data analyst, complete, and *you built all five layers on one real dataset* — which is precisely the portfolio story Part VII now turns into a career asset.

**From Your Toolkit — communication is the last mile:** every artefact of Parts II–VI exists to be *read*: the report page, the chart's title, the dashboard's summary screen, the five-minute walkthrough. The pattern is one pattern — headline, evidence, caveat, ask — from Chapter 13's one-pager to this chapter's meeting. Master the pattern and the tools become interchangeable instruments behind it.

### Key Takeaways

- Publishing ladder: share the .pbix (free, version-controlled) → export PDF/PPT for the record → the service with scheduled refresh and gateway when licensed; the architecture transfers to free alternatives.
- The narrative layer ships in the file: summary page (findings as front door), chart annotations at the wondering-point, guided navigation.
- The five-minute walkthrough: headline → movement → stories (with caveats spoken first) → one interaction → the ask; never a naked number.
- Dashboard vs report is a conscious decision by question type, audience, refresh, and depth — both live in the professional stack.
- Part VI completes the five-tool stack around one dataset — the portfolio story Part VII monetises.

### Practice Lab

1. Build the summary page: three finding-phrased text boxes with mini-visuals and navigation buttons; reorder pages so it is first; run the five-second test at the *file* level (open, glance, close: what did you learn?).
2. The annotation pass: label the oil cliff, the payday rhythm, and the best week on the charts where they occur; add one Smart-narrative visual and *edit its text* until every sentence would survive your own skepticism audit.

3. The walkthrough, rehearsed: script your five minutes (headline, movement, stories, interaction, ask — timing pencilled in the margin); deliver it to a friend or your phone's camera; watch it back once; note the two fixes.
4. The export pack: export the dashboard pages to PDF and the summary findings to one slide; compare what survives the export (interactivity does not — does the static version still tell the story? fix what does not); file both in the portfolio folder.
5. The reflection page (portfolio feedstock): 200 words — "one dataset, five tools, what each was best at" — with the Trade Overview screenshot at the top. Chapter 49 will publish this page almost verbatim.

## Further Reading

- Chapters 44–49 (projects and the portfolio — the story becomes a career), *Storytelling with Data* (Knaflic) ch. 7–8
- Microsoft Learn: "Manage workspaces" — the service-side skills of this chapter, official form

**PART VII**

# **Real Projects and Your Portfolio**

Chapters 44 – 49

---



# Chapter 44: Where Practice Data Comes From

Part VII — Real Projects and Your Portfolio

"Tariro was invented so you could learn honestly. The world's real data is sitting right there, free, waiting for the same treatment."

## In this chapter you will learn

- The public-data landscape: where real, free practice data actually lives.
- How to choose a dataset that builds a career (and how not to).
- Generating your own data — the honesty of planted patterns, reversed.
- Reading a new dataset cold: the first hour, systematic.
- Building the data-habit that outlasts this book: one dataset a month.

## 44.1 The Public Landscape

The excuse that dies in this chapter is "I don't have data". Real, current, free data — government statistics, open city records, health data, sports, transport, World Bank indicators — is published deliberately for public use. The landmarks, with what each is *good practice for*:

- **Kaggle** ([kaggle.com/datasets](https://kaggle.com/datasets)) — the supermarket of datasets: enormous variety, ratings, and (be careful) many over-cleaned "perfect" files. Good for: joining real-world mess with community notebooks to compare against.
- **Government open-data portals** — [data.gov](https://data.gov) (US), [data.gov.uk](https://data.gov.uk), and nearly every national statistics agency (ZimStat for Zimbabwe's official figures; Statistics South Africa; the UK's ONS). Censuses, prices, trade, health: authoritative, documented, and often gloriously messy. Good for: *serious* credibility — a portfolio project on national data reads professional.
- **The World Bank / UN / WHO** — indicators across countries and decades, clean APIs, well-documented. Good for: time-series and cross-country comparison projects (the gapminder tradition).
- **Open city data** — transport timetables and delays, air quality, property records, budgets. Good for: dashboards with a story a city-dweller instantly understands.
- **Blogs and academics who share** — FiveThirtyEight's archived datasets (politics, sports, culture), Tidy Tuesday's weekly community datasets, university data libraries. Good for: chart-practice reps with interesting

questions attached.

- **APIs and scrapes** — weather, exchange rates, sports fixtures. Good for: the Python skills of Part V feeding a live dashboard (Chapter 61's stack makes this free).

The licensing note that keeps you professional: nearly all of the above are published under open licences permitting analysis and republication with attribution; check the licence page once per dataset, cite the source in your project README (the "says who?" habit, structural), and you are clean.

## 44.2 Choosing Data That Builds a Career

Not all practice data is equal. The selection criteria — the same criteria a hiring manager applies to your portfolio (Chapter 49):

1. **A question, not a dataset.** "Did the 2008 crisis change what Zimbabwe exports?" (World Bank trade data) is a project; "analysis of a CSV I found" is not. Choose data because it can answer something you actually wonder.
2. **Local relevance is gold.** A project on your *own city's* transport or your *own country's* prices stands out in a pile of US-flight-delays clones — it says "I look at the world around me", which is the analyst's actual job description.
3. **Messiness earns trust.** A dataset with real flaws (missing years, three spellings of one province — you know this world intimately from Part II) demonstrates cleaning craft. Perfect data demonstrates nothing.
4. **Two-table minimum.** Single-table projects cannot show joins — the model (Part III) and the relationships (Part VI) need at least two connected tables to exist. Prefer data with dimensions: who/what/where/when.
5. **A number a decision could rest on.** The best projects end in "therefore, X" — a recommendation, a verdict, a quantified gap. If you cannot imagine the last sentence of the analysis, the dataset may only be a toy.

## 44.3 Generating Your Own — the Reversal

You have spent a book learning on *generated* data (Tariro's), and there is a craft worth naming in that choice — because generating data is itself an analyst's skill, used for testing pipelines, teaching, and prototyping. The reversal of this chapter's title: when you generate, you *plant the truth*; when you analyse, you *hunt it*. The deepest lessons of this book came from that arrangement: you always knew the Sunday effect existed, so every method could be *scored against truth* — did the pivot find it? did the test close it? did the dashboard show it?

The generator's recipe (Appendix C carries the full code; the shape matters here): draw the honest noise first (amounts right-skewed around a lognormal; weekday counts varying), then *plant* the effects (Friday/Saturday  $+1/3$ ; payday spikes; a month-9 price rise cutting oil quantity; a loyal core slowly churning), then record the truth in a `truth.md` file — "Sunday effect:  $-28\%$ , planted" — so every analysis can be graded. That structure — noise + effects + truth file — is how professionals build test data for pipelines (does the anomaly detector fire on the planted anomaly *and only* on it?), and it is why Tariro's shop taught better than any open dataset could: **you were always scoring your methods against a known reality.** (One rule, ethical and absolute: generated practice data is labelled as generated, in the README, in capital letters if necessary. Synthetic data passed off as real is the portfolio's mortal sin — Chapter 59's territory.)

## 44.4 The First Hour on a New Dataset

The systematic approach to a dataset you have never seen — the skill this chapter actually installs, in the order the moves happen:

1. **Read the documentation first** (the data dictionary, the collection methodology, the licence). Ten minutes here saves an hour of misreading a column — and the methodology section is where you learn the *sample's* biases (Chapter 25's questions, asked of the source).
2. **The four instruments** (Ch. 33): `head()`, `info()`, `describe()`, `value_counts()` — types, nulls, ranges, categories, and the surprises each one surfaces.
3. **The reconciliation sweep**: row counts vs the documented totals ("the file says 12,400 households; the data has 12,382 — why?"); duplicates on the key; orphans if multi-table. The loading liturgy, run cold.
4. **One plotted column of everything numeric** — histograms at scale; shape surprises (bimodal ages? a spike at round numbers?) become the first questions.
5. **Write the questions log** — the open questions the first hour raised, in a file that becomes the project's spine. Analysis is what happens between a questions log and its closure.

## 44.5 The Habit: One Dataset a Month

The compounding practice that separates the hired from the trained: **one new dataset a month, minimum** — an evening's first-hour, then whatever depth it earns. Twelve a year; by your first interview (Part VIII), you will have touched more real datasets than many working analysts touch in three years, and every tool-skill from this book will have been exercised on data that fought back. Keep a `data-journal.md`: date, dataset, one question asked, one thing learned, one surprise. The journal itself becomes portfolio evidence — of curiosity, which is the trait every interviewer probes for and none can teach.

**From Your Toolkit — the supply chain:** every project in this Part (and the two jobs of Part VIII) is fed by this chapter: Project 4 and 6 run on Tariro's generated truth (scored!); Project 5 on a survey *you* collect; and your portfolio's "wild" project — the one that makes it three — comes from the public landscape here. The habit outlasts the book: the analyst with a supply of questions never lacks for work.

### Key Takeaways

- Free real data is abundant: Kaggle, national statistics agencies, World Bank/UN, city portals, community datasets — check the licence, cite the source.
- Choose by question, local relevance, messiness, multi-table shape, and decidable endings — the criteria hiring managers apply.
- Generated data = noise + planted effects + truth file; it scores your methods against known reality — and it must always be labelled synthetic.
- The first hour: documentation → four instruments → reconciliation sweep → plotted columns → questions log.
- One dataset a month, journaled — the compounding habit that makes every tool in this book permanent.

### Practice Lab

1. The first-hour drill, run on public data: pick a national-statistics dataset (ZimStat, ONS, or World Bank); run the full first hour including the questions log; save as `first-hour-<dataset>.md` — this file is the artefact.
2. The evaluation: score your chosen dataset against the five selection criteria, honestly; write the verdict — is it a project, a toy, or neither? If a toy, find a better one; if a project, open its questions log and leave it open for Project-time.

3. The generator, read: open Appendix C's Tariro generator; find each planted pattern in the code (payday, oil, churn); write the `truth.md` you would write for a generator of your own — then change one parameter and describe what an analyst would now find.
4. The journal, started: create `data-journal.md` with entries for every dataset this book has already made you touch (Tariro's four tables, the survey, your public pick); note the one-surprise line for each.
5. The question bank: write down five questions about your own city or country that public data might answer; rank them by how much the answer would matter to someone; the top one is your portfolio's wild-card project, queued.

## Further Reading

- Chapter 45 (the anatomy of the analysis these datasets feed), Appendix C (the generator)
- Tidy Tuesday ([github.com/rfordatascience/tidytuesday](https://github.com/rfordatascience/tidytuesday)) — a new dataset every week, community practice

# Chapter 45: The Anatomy of an Analysis

## Part VII — Real Projects and Your Portfolio

"Every good analysis you will ever do is the same five moves. The projects change; the moves do not."

### In this chapter you will learn

- The five-move anatomy: question → data → clean → analyse → communicate.
- The checklist that runs underneath each move — this book, condensed to one page.
- Time budgeting: the 80/20 of real projects, and where projects die.
- The case-study method: writing 150-word notes that get interviews.
- How Projects 1–6 map onto the anatomy — and what your own projects must add.

### 45.1 The Five Moves

Strip away the tools and the domains, and every analysis that ever changed a decision has the same skeleton:

1. QUESTION	-- a decision or curiosity, stated, with its audience named
2. DATA	-- sourced, licensed, understood (the first hour, Ch. 44)
3. CLEAN	-- the liturgy: types, dupes, orphans, NULLs, the bridge
4. ANALYSE	-- describe, compare, test; effect sizes with intervals
5. COMMUNICATE	-- headline, findings with caveats, recommendations, artefacts

You have lived these moves nine times now (six Part-projects plus the Part IV and VI deliverables), always in this order, and the order is the anatomy's first law: **a question written before data is touched, and a communication designed before the analysis is finished**. Beginners run data → analysis → (maybe) a question that fits; the professional runs question-first and designs the deliverable *backwards from the reader* (Chapter 13's planning trick — which is this law, applied). The second law: **the moves iterate but never skip** — you will loop question→data when the data cannot answer the question (that is a *finding*: "unanswerable with current data"), but you never jump to 4 without 3, and you never start 5 without something worth saying.

## 45.2 The Checklist

This book, compressed to the page working analysts keep pinned (mentally) above every project — each line is a chapter, and the parentheses give you the trail back:

**QUESTION** ☐ One sentence, with a decision in it ☐ Audience named (who reads this?) ☐ Success state defined (what answer ends the project?) ☐ Scope fenced (18 months, one shop, this segment) (Ch. 2, 13)

**DATA** ☐ Licence checked, source cited ☐ Documentation read; collection method noted — its biases asked (Ch. 44) ☐ First hour run: instruments, reconciliation, plotted columns ☐ Questions log opened (Ch. 33, 44)

**CLEAN** ☐ Types verified; dates parsed ☐ Duplicates counted and resolved ☐ Orphans found and documented ☐ NULL census with meanings ☐ Bridge total reconciled against a second source ☐ Log of every cleaning decision (Ch. 8, 14, 21, 33)

**ANALYSE** ☐ Described before tested (five numbers, shapes) ☐ Compared against a baseline ☐ Tested where testing is owed — right test, assumptions checked,  $\alpha$  pre-chosen ☐ Effect size + interval with every p-value ☐ Correlations read through the three explanations (Ch. 22–28, 36)

**COMMUNICATE** ☐ Headline that survives alone ☐ Findings with caveats attached ☐ Recommendations measurable, with owners ☐ Right format: report vs dashboard vs walkthrough, decided (Ch. 13, 43) ☐ Artefacts filed, named, versioned, with the log (Ch. 20, 37)

**THE THREE RULES, EVERYWHERE** ☐ average  $\neq$  the answer ☐ correlation  $\neq$  causation ☐ sample  $\neq$  population

Print this page. It is the book's entire argument, operationalised — and it is the review checklist for every project remaining, including your own.

## 45.3 Time: the 80/20 of Real Projects

The time distribution nobody believes until their first job: **question and cleaning consume the majority of every real project; the glamorous analysis is the minority; communication is under-estimated by everyone.** A working budget for a two-week stake: framing and data 30%, cleaning and checking 30%, analysis 20%, communication 20% — and when something goes wrong (it will), the overruns eat analysis time, *never* cleaning or communication time. The corollary discipline: the cleaning 30% is where shortcuts poison everything downstream; the communication 20% is where good analyses go to die unnoticed. Projects fail at the edges, not the middle.

Where projects die, the three morgue drawers: **the unowned question** (no decision attached — the analysis wanders until abandoned); **the data that cannot answer it** (discovered at move 4 instead of move 2 — the first hour exists to kill this early); and **the unfinished tell** (analysis done, communication deferred, nothing shipped — the "portfolio" of dead notebooks). Each drawer has a door: a written question, a first hour, a delivery date.

### 45.4 The Case-Study Method

The portfolio's atomic unit is not the project — it is the **150-word case-study note**, and its anatomy is the project's anatomy, compressed to five sentences (you have written three already, in the Part projects; here is the method, deliberate):

[SITUATION]	One line: who, what data, why it mattered.
[WORK]	One-two lines: the moves, with tools named.
[FINDING]	One line: the number and its verdict -- the headline that survives alone.
[IMPACT]	One line: the decision or the improvement, timed or measured ("an afternoon became twenty minutes").
[LIMITS]	One line: the honest caveat (sample, association, synthetic labelling).

Why 150 words: recruiters skim (Chapter 51 gives the eye-tracking reality); interviewers *lift* these notes as their questions ("tell me about the 31% growth finding" — and you are in your STAR story, Chapter 53, on ground you prepared). Write the note the day the project ships, while the numbers are vivid; a note written a month later is a different, worse artefact. And file the note with the artefacts it points to (repo, dashboard, PDF) — the note is the shop window; the artefacts are the shop.

### 45.5 The Ledger So Far, and What's Next

Where the six projects stand against the anatomy — this table is the portfolio's table of contents, and Chapter 49 publishes it:

#	Project	Tools	The finding it owns	The craft it proves
1	Monthly report	Excel	Growth 31%, oil -40%, Sundays traffic-poor	The one-page deliverable
2	Shop database	SQL	Ten questions as a query pack	Schema, joins, reproducibility
3	Report that builds itself	Python	An afternoon → twenty minutes	The pipeline, automated
4	Customers and churn	Python + stats	The loyal core is drifting	Cohorts, survival thinking



#	Project	Tools	The finding it owns	The craft it proves
5	The survey	SPSS/tools	Intent ≠ behaviour, measured	Survey craft, chi-square
6	Operations dashboard	Power BI	The always-on shop	Model, measures, story

Projects 1–3 are done (Parts II, III, V); this Part builds 4, 5, and 6 on the running case, then Chapter 49 assembles the portfolio. Your own seventh project — the public-data wild card from Chapter 44 — completes the set with data nobody planted for you, and the anatomy you now hold is the method for all of it.

**From Your Toolkit — the anatomy is the job:** job postings (Ch. 50) are this checklist in prose; interviews (Ch. 52–53) test its moves; your first 90 days (Ch. 54) will be scored by how naturally you run it. The tools were five languages; this — the five moves, the checklist, the honest rules — is the *profession*, tool-blind and permanent.

### Key Takeaways

- Five moves, in order: question → data → clean → analyse → communicate; question written first, deliverable designed backwards; iterate, never skip.
- The one-page checklist is the book condensed — run it on every project, including your own.
- Time's truth: framing+cleaning ≈ 60%, analysis 20%, communication 20%; projects die at the edges — unowned questions, unanswerable data, unshipped tellings.
- The 150-word case-study note (situation/work/finding/impact/limits) is the portfolio's atom; write it on shipping day, file it with artefacts.
- Six projects on one running case + your public-data wild card = the portfolio; the anatomy is the method for all seven.

### Practice Lab

1. The audit, retroactive: run the full checklist against Project 3 (the automated report); mark every box you genuinely earned and every one you skipped; fix the two most serious gaps in the artefact itself.
2. The anatomy, pre-written: for Project 4 (next chapter — customers and churn), write the question (one sentence, decision attached), the audience, the success state, and the planned deliverable *before reading further*; compare with the chapter's brief when you get there.

3. The note, retro-fitted: write the 150-word case-study note for Projects 1 and 2 (they shipped before this chapter formalised the method); lift the numbers from your logs; attach the artefact paths.
4. The morgue tour: recall (honestly) any analysis in your life that died in a drawer — a course assignment, a work file, a personal project; name the drawer (unowned question / unanswerable data / unfinished tell); write the one-line door that would have saved it.
5. The checklist, printed: format this chapter's checklist into your own one-page reference (your words, your trail-backs); pin it — physical or digital — where projects happen. It will be in use in Chapter 49 and in your first job's first week.

## Further Reading

- Chapters 46–48 (the three projects, built on this anatomy), Chapter 49 (the portfolio assembled)
- *The Art of Statistics* (Spiegelhalter) — the PPDAC cycle (Problem, Plan, Data, Analysis, Conclusion) — this anatomy's academic twin

# Chapter 46: Project 4 — Customers and Churn

Part VII — Real Projects and Your Portfolio

"Revenue tells you what happened. Customers tell you what happens next."

## In this chapter you will learn

- Churn, cohorts, and retention: the vocabulary of customer analysis.
- Building the cohort grid: customers by join-month, tracked month by month.
- Survival thinking and the metric that beats averages: lifetime by cohort.
- Finding Tariro's drifting loyal core — the planted pattern, hunted with everything you know.
- The deliverable: a churn report with a save-campaign recommendation.

## 46.1 The Question

Chapter 5 planted it; the design's truth file holds it: *a loyal core is slowly churning*. The business translation, written the Chapter 45 way — one sentence, decision attached: **"Which customers are drifting away, how fast, and what would it cost to lose them — so Tariro can decide whether a save-campaign is worth running?"** Audience: Tariro, this month. Success state: a named list of at-risk valuable customers, a quantified drift, and a recommendation with a budget attached.

The vocabulary first, because customer analysis has a small precise dictionary: **churn** — a customer who stops buying (defined, always: no purchase in N days; we will use "no purchase in 60 days" for a grocery, stated); **cohort** — a group sharing a start (customers who joined in the same month); **retention** — the share of a cohort still active after k months; **CLV** (customer lifetime value) — what a customer is worth across the whole relationship. The reason cohorts rule this domain: averages hide time. "Average customer lasts 9 months" blends the one-visit tourist with the decade-loyal neighbour; cohorts unblend them — and Chapter 22's "which typical" lesson becomes, here, *whose* typical.

## 46.2 The Cohort Grid

The core artefact of customer analysis — a grid: rows = join-month cohorts, columns = months since joining, cells = the share of the cohort still shopping. Build it in pandas (Part V's kit, assembled; every line is a chapter you own):

```
import pandas as pd
sales = pd.read_csv("sales.csv", parse_dates=["date"])
customers = pd.read_csv("customers.csv", parse_dates=["joined"])
# last purchase per customer: the "active until" stamp
last_seen = sales.groupby("customer_id")["date"].max()
# month of join and month of last purchase -> lifetime in months
cust = customers.set_index("customer_id")
cust["last_seen"] = last_seen
cust["life_months"] = ((cust["last_seen"] - cust["joined"])
                      .dt.days / 30.4).round()
# the cohort grid: retention by join cohort and age
cust["join_cohort"] = cust["joined"].dt.to_period("M")
active = sales.assign(month=sales["date"].dt.to_period("M")).merge(
    cust["join_cohort"], left_on="customer_id", right_index=True)
active["age"] = (active["month"] - active["join_cohort"]).apply(lambda x: x.n)
grid = (active.drop_duplicates(["customer_id", "age"])
        .groupby(["join_cohort", "age"])["customer_id"].count()
        .unstack(fill_value=0))
grid_pct = grid.div(grid[0], axis=0)          # % of cohort still active
```

Read `grid_pct` as the shop's leak diagram: column 0 is everyone (100%); column 3 is who remained after a quarter; the *slope down the columns* is churn, cohort by cohort. On Tariro's data the pattern Chapter 5 planted shows up as it was designed to: early cohorts (the founding neighbours) hold flat at high retention for their first year — then, from around month 12, the top rows thin faster than the rows below them. The loyal core is drifting, and the grid caught it in the only way it *could* be caught: by keeping time separate instead of averaging it away.

## 46.3 Survival Thinking and CLV

The cohort grid's more rigorous sibling, one idea deeper: **survival analysis** — the study of *time until an event* (here, churn; in medicine, relapse; in machinery, failure — the same statistics across worlds). The beginner-friendly version: the **survival curve** — for each month  $k$ , the share of customers still active — plotted as one line per cohort, or one overall line (the Kaplan-Meier estimator is its formal name; the intuition is the grid's last row read as a curve). Why it beats averages: it answers the question the business actually asks — "*if we acquire a customer today, what fraction survives to month 12?*" — directly, from data, without assuming everyone is average.

CLV, computed honestly on the same frame: average monthly spend (members only, by cohort)  $\times$  expected lifetime (from the survival curve, say 11 months for recent cohorts vs 14 for the founding one) — a number per cohort, with its caveat spoken: *computed from past behaviour, assuming the future resembles the past, churn-styled by definition of the 60-day rule* (Chapter 2's "says who?", aimed at our own arithmetic). The finding this yields on Tariro's data: the founding cohort's CLV is roughly  $1.4\times$  the newer cohorts' — *losing one founding customer costs what losing one-and-a-half new ones does* — which reframes the whole save-campaign question: the drift is small in *customer count* and large in *value*.

## 46.4 The Drift, Found and Named

The analysis converges on the deliverable the question demanded — the named list. Definition (stated in the report, as always): a customer is **at-risk** if they were ever a monthly shopper ( $\geq 3$  purchases/month for 3+ months) and have not purchased in 45 days. In pandas: filter customers by their historical monthly frequency, join to last-seen, compute days-silent, filter — and out comes the list: on the generated data, about 60 customers, average lifetime value 40% above the shop's mean, concentrated — as the design planted — among the founding cohort. Each row: name, suburb, tier, lifetime value, days silent. That list is the save-campaign's mailing list, and its existence is the difference between "some churn is happening" (a vibe) and "these sixty people, worth this much, are leaving" (a decision).

The recommendation, written the Chapter 13 way: *run a targeted win-back for the 60 at-risk high-value customers (personal SMS + a voucher worth ~2% of their annual value); measure redemption after 3 weeks; success = 25% return rate, which on the CLV arithmetic pays for the campaign roughly  $8\times$* . Measurable, budgeted, with a kill-line — and the caveat that this is *association observed, not cause established*: the drift's *cause* (a competitor? prices? life events?) is unknown, which is exactly why the campaign is a test with a measurement, not a certain fix (Chapter 28's discipline, applied to a business action).

## 46.5 The Deliverable and the Evidence

Ship it as the anatomy prescribes: `churn-report/` — the notebook (`churn-analysis.ipynb`, prose-celled, Restart & Run All clean), the cohort grid as a chart exhibit (a heatmap: cohorts down, age across, retention shaded — one picture that tells the whole story), the survival curves, the at-risk list as CSV, the one-page report with the recommendation, the 150-word case-study note ("Situation: grocery shop, 18 months, 640 customers... Work: cohort retention, survival curve, CLV by cohort... Finding: founding cohort churning 2× fast, 60 named at-risk worth 1.4× mean... Impact: save-campaign designed, 8× payback if 25% return... Limits: synthetic-labelled, cause unknown, 60-day churn rule stated").

And the interview line it funds, added to the ledger from Project 2: "tell me about a time your analysis changed a planned action" — the campaign design, the CLV arithmetic, the honest unknown-cause caveat. Employers in *every* industry run churn questions (telecoms, banks, streaming, retail — the vocabulary transfers whole), and Project 4 is the proof you have run one end-to-end.

**From Your Toolkit — time in the rows:** cohort grids and survival curves are the general answer to "how does a group move through time?" — retention in product analytics, dropout in education, default in lending, relapse in health: same grid, different nouns. `pandas'` `groupby-unstack` built it; SQL's window functions could have (Appendix A carries the pattern); even a spreadsheet can, painfully. The *idea* — never average across time when time is the question — is the transferable instrument.

### Key Takeaways

- Churn needs a stated definition (no purchase in N days); cohorts unblend the averages that hide time.
- The cohort grid (join-month × age → retention) is customer analysis's core artefact; build it with `groupby + unstack`; read the slopes.
- Survival thinking answers "what fraction survives to month k?" directly; CLV = monthly spend × expected lifetime, per cohort, caveats spoken.
- The deliverable is a named list with values and a measurable, budgeted recommendation — association observed, cause unknown, so the campaign is a test.
- Ship the folder: notebook, exhibits, at-risk CSV, one-pager, 150-word note — the portfolio's fourth evidence and a story every industry recognises.

## Practice Lab

1. Build the cohort grid and the retention heatmap on your data; read the founding-cohort thinning with your own eyes; write the three-sentence story the heatmap tells (include the numbers).
2. The survival curves: one line per join-year cohort (or founding vs later), retention vs months-since-join; annotate where the founding line bends; state the 12-month survival for each cohort with its interval (bootstrap, Ch. 36 — you own it).
3. CLV by cohort with the arithmetic shown; then the sensitivity check: recompute at  $\pm 20\%$  monthly spend and note how the campaign payback changes — the "is my interval narrow enough to decide?" question, applied.
4. The at-risk list: implement the stated definition; reconcile the count by hand for five customers (their last purchase dates, their frequency history); export the CSV and check it opens clean in a spreadsheet.
5. The report and the note: assemble `churn-report/` complete; run the Chapter 45 checklist against it; write the 150-word note and the STAR paragraph; update the portfolio ledger — two projects remain.

## Further Reading

- Chapter 47 (surveys — asking the customers who are leaving), Chapter 58 (churn's machine-learning sequel)
- *Survival Analysis* (Kleinbaum) ch. 1-2 — if the Kaplan-Meier idea grabbed you

# Chapter 47: Project 5 — The Survey

---

## Part VII — Real Projects and Your Portfolio

"Chapter 27 taught the mechanics. A project is when the mechanics meet real humans — including you."

### In this chapter you will learn

- Running a real survey end-to-end: design, field, clean, analyse, report.
- Analysing survey data honestly: the *n* that arrives vs the *n* you dreamed of.
- The say/do gap: connecting survey answers to till behaviour.
- The report: findings, limitations stated loudly, the decision it feeds.
- Portfolio evidence #5, and the ethics review that comes with human data.

## 47.1 Design and Field

The brief, written before anything else (the anatomy's law): *Tariro needs to know whether a Sunday double-points promotion would bring shoppers in — and who would come. Decision: run the promotion or not. Audience: Tariro. Success: an estimate of Sunday intent by customer segment, with its margin of error, in two weeks.*

The design inherits Chapter 27's rules, now under project pressure. The instrument: seven questions, one per card — visit frequency ("How often do you shop with us?" — labelled bands, never "how many times per month on average", which humans answer badly); Sunday intent (the 5-point definitely/probably/unsure/probably-not/definitely-not scale); what would change the answer (multi-choice: nothing, double points, a hot-food counter, later hours); payment preference; suburb (to join to till data!); membership; and one open text ("anything you'd want us to stock?") — because one open question humanises the dataset and occasionally delivers a product idea in the customer's own words. The frame, stated and written down: shoppers in the shop, two Saturdays and two weekdays, every third customer approached (a system, not a vibe — convenience-with-rules, its limitation declared). The target *n*: 150 (the Chapter 25 arithmetic:  $\pm 8$  points on a proportion — adequate for a run/don't-run decision, stated in the report).

Fielding reality, which is the education: refusals happen (~30% — logged, because refusal rate is itself a bias finding: who refuses? hurried weekday lunchers, mostly); the question that confuses ("double points *of what?*" — re-read the scale's wording, fix in the field, note the fix); and the response



distribution that surprises (weekday respondents are *more* Sunday-interested than Saturday respondents — a selection story before the analysis even begins). Field notes are data: the log carries them.

## 47.2 Clean and Analyse

Responses arrive — 168 of them, because fielding overshot (good). The cleaning liturgy, survey edition: one row per response (no duplicate timestamps); the labelled scale kept as *categories* (1–5 with value labels, never averaged into a "2.7 attitude" — Chapter 27's rule, now load-bearing); the multi-choice parsed into separate yes/no columns; the open text read *by a human* (you) and thematically coded (three categories emerged: hot food, bulk staples, baby items — each becomes a countable column). Every cleaning decision logged, as always.

The analysis, in the order the anatomy prescribes:

1. **Describe:** the intent distribution (definitely 17%, probably 24%, unsure 21%, probably-not 23%, definitely-not 15%) — shown as counts *beside* percentages, *n* declared on every table.
2. **Segment:** the cross-tabs — intent by tier (members: 51% definite/probable vs non-members 33%), by suburb, by usual payment. Chi-square on each, expected-counts check, effect sizes read from the row percentages, not the p-values (Chapter 27's full method, now fluent).
3. **Margin of honesty:** every headline percentage carries its interval — 41%  $\pm$  7.5% for the pooled definite/probable share; the segment splits are wider (*n* = 61 members:  $\pm$ 12%), and the report says which comparisons the data can and cannot support.
4. **The weighting question, considered and answered:** weekday/Saturday balance is off (by fielding design); rather than statistically reweighting (defensible but fragile at this *n*), the report presents both field-days separately and lets the reader see the selection — transparency over sophistication, at this sample size, is the right professional call (stated as such).

### 47.3 The Say/Do Join

The project's signature move — the one that lifts it above a classroom exercise — is joining what people *said* to what people *do*: respondents' suburbs and tiers (collected precisely for this) are joined to their till-history segments from Project 4's frame. The finding that matters: among respondents who *said* "definitely would" shop Sundays, the ones who are *already* Sunday shoppers (in till data) are 2.3× more frequent than the ones who are not. Read carefully: stated intent is strongest exactly where behaviour already agrees — the survey is partly *measuring enthusiasm*, not forecasting change. That is the say/do gap, quantified on your own two datasets, and it converts the recommendation from "run it, 41% are interested!" to the honest form: *"interest is real but concentrated among existing Sunday shoppers; the promotion's incremental pull is likely nearer 15-20% than 41%; the four-Sunday trial with till-based measurement (baseline already in the dashboard) is the decisive, cheap test."* A survey that checks itself against behaviour is worth two surveys that don't.

### 47.4 The Report

survey-report/ ships with: the instrument (questions + field protocol, so a successor could re-run it); the cleaned data; the analysis notebook; the four-exhibit findings (intent distribution; intent × tier; say/do table; the limitations box); and the one-page decision memo. The limitations box, loud and early rather than buried: convenience frame (in-shoppers only — the Sunday-stayers-at-home are the population the promotion targets and the survey *cannot see them*); n and margins by segment; self-report bias (measured, 47.3); no causal claims (intent is association with behaviour). Then the recommendation with its decision logic: run the four-Sunday trial if the cost of the trial is under the expected value of a 15-20% Sunday lift; measure in till data; kill or scale at the pre-agreed threshold (Chapter 26's pre-registered  $\alpha$ , applied to a business test).

The 150-word note, written on shipping day: *Situation: promotion decision for a grocery, two weeks, 168 in-shop respondents. Work: designed and fielded the survey; cleaned and cross-tabbed; joined stated intent to till behaviour. Finding: 41% ± 7.5% expressed Sunday intent, but intent concentrates among existing Sunday shoppers — incremental pull nearer 15-20%. Impact: decision reframed from launch to a measured four-Sunday trial. Limits: convenience frame, self-report, survey-on-generated-data labelled.* — and note the last clause: this project runs on the book's synthetic till data plus *real human responses only if you truly fielded it*; either way the labelling is explicit (Chapter 44's rule, and Chapter 59's ethics preview).

## 47.5 The Ethics Review

Human data carries obligations, and this project is where the book makes them concrete: **consent** — respondents approached as such, told the purpose, free to refuse (the 30% refusals are the system working); **anonymity by design** — no names collected, only the join-columns (suburb, tier) the analysis needs, the minimum-data principle; **purpose limitation** — the data answers the Sunday question and nothing else; no repurposing into a "marketing list" beyond the consent given; **honest reporting** — the limitations box *is* an ethical artefact, not just a statistical one. At work, the formal versions arrive as review boards, data-protection law (GDPR-style regimes, POPIA, Zimbabwe's Cyber and Data Protection Act), and company policy; the professional habit starts here: *when humans are the data, the checklist grows a conscience section*. Chapter 59 builds the full map.

**From Your Toolkit — asking, everywhere:** surveys are the only instrument that reaches the futures inside people's heads — intentions, satisfactions, reasons — and every analytics team eventually runs one (product NPS, employee pulse, customer exit). The craft constants: frame declared, scales labelled and kept categorical, counts beside rates, intervals attached, behaviour joined where possible, limitations loud. SPSS ran this project's cross-tabs; pandas or Stata could have — the *method* is the asset.

## Key Takeaways

- Design under project pressure: one question per card, labelled scales, frame and refusal rate logged, target n from the margin arithmetic — and field notes are data.
- Clean survey-style: categories stay categories; multi-choice splits to columns; open text human-coded; every step logged.
- Report counts beside percentages,  $\chi^2$  with effect sizes, intervals on every headline — and prefer transparency to fragile reweighting at small n.
- The say/do join is the signature: intent measured against behaviour reframes the recommendation (41% interest → 15-20% incremental) — surveys that check themselves are worth two that don't.
- Ethics is a checklist section: consent, minimum data, purpose limitation, honest limitations — the conscience the human-data projects require.

## Practice Lab

1. Field it for real (recommended): the instrument on friends, family, a class, a workplace — 30+ responses minimum; run the full pipeline and note where reality diverged from this chapter's fielding (it will; that divergence is the education).
2. Or simulate honestly: Appendix C's survey generator (labelled synthetic); run the identical pipeline; write the one-paragraph memo on what a simulated survey *cannot* teach (refusals, confusion, the field's surprises) — the meta-finding.
3. The say/do join, built: respondent segments  $\times$  till behaviour; the 2.3 $\times$  table with its counts and its chi-square; write the three sentences a decision-maker reads.
4. The limitations box, drafted twice: once where this chapter puts it (early, loud) and once where most reports put it (buried); hand both versions to a friend and ask which report they would trust more — then keep the loud version forever.
5. Portfolio evidence #5: the folder, the exhibits, the note, the STAR paragraph ("a time you changed a recommendation with better analysis"); the ledger now reads five — one project remains.

## Further Reading

- Chapter 48 (the operations dashboard — the trial's measurement instrument), Chapter 59 (ethics, the full map)
- *Asking Questions* (Bradburn et al.) — the survey craft bible, at project scale

# Chapter 48: Project 6 — The Operations Dashboard

---

## Part VII — Real Projects and Your Portfolio

"The trade dashboard asked 'how are we doing?'. The operations dashboard asks 'what do we do on Monday morning?'"

### In this chapter you will learn

- Operations analytics: the shift from describing trade to running the shop.
- Stock analytics: days-of-cover, the reorder signal, and the missing-week problem.
- Staffing analytics: the weekday×hour heat map, and Sundays, finally acted upon.
- Building the operations dashboard: the page, the alerts, the drill paths.
- Portfolio evidence #6: shipping the full stack, end to end.

## 48.1 The Brief

Tariro's Part VI dashboard serves the quarterly meeting. This project serves the *back office* — the weekly operational rhythm: what to order, how to staff, what to watch. The brief, anatomy-first: *build the page Tariro opens on Sunday evening to plan the week: stock risk (what runs out before the next delivery), staffing (how many hands, which hours), and the live trials (the Sunday promotion and the oil shelf test, both from earlier chapters) with their pre-agreed thresholds. Audience: Tariro, weekly. Success: order and roster decisions made from the page in fifteen minutes.*

Note what changed from the trade dashboard: the reader's *action horizon* collapsed from a quarter to a week — so the page's centre of gravity moves from trends to *thresholds*: numbers against lines, alerts against amber, "days remaining" against "days until delivery". Operations analytics is description converted into triggers.

## 48.2 Stock: Days of Cover

The core stock metric, small and mighty: **days of cover** = quantity on hand ÷ average daily quantity sold. Forty-five bottles of oil against an average 4.2/day  $\approx$  10.7 days of cover; the next delivery is in 7 days; therefore order enough for the *gap plus* a safety margin (cover target 14 days: order =  $(14 - 10.7) \times 4.2 \approx 14$  bottles). One formula, one reorder logic — and in pandas, one more pipeline (the stock table joined to item daily averages; a `days_cover` column; a `reorder_flag` where `cover < days-to-delivery + safety`):

```
stock["avg_daily"] = stock.merge(
    item_daily, on="item", how="left")["qty_sold_daily"]
stock["days_cover"] = stock["qty_on_hand"] / stock["avg_daily"]
stock["reorder"] = stock["days_cover"] < (days_to_delivery + 2)
```

And the missing weeks — the design's planted data flaw (three stock weeks absent) — return as an *operations* lesson, not just a cleaning one: the Sunday stock count is a *process*, and processes skip. The dashboard's stock page therefore carries a **count-freshness flag** (weeks since last count, from the data's own timestamps — the data-age stamp of Chapter 41, applied to a business process): a stale count must not masquerade as a current position. The professional insight this plants: *operational dashboards must monitor their own inputs' health* — the missing weeks are a finding about the shop, surfaced as a visible alert rather than silently interpolated away.

## 48.3 Staffing: the Heat Map

The staffing question — how many hands, which hours — is answered by one visual: the **weekday × hour heat map** of customer counts (transactions per hour, from the till timestamps). Build it in pandas (`pivot_table` weekday × hour, matplotlib `imshow/heatmap`) and then as the Power BI matrix with conditional formatting — the shop's whole weekly rhythm in one rectangle: the Saturday 10:00–13:00 block glowing; the weekday 16:00–18:00 after-work shoulder (school runs, dinner ingredients); the Sunday flatness, now quantified *hour by hour*.

The staffing rules that fall out (with numbers attached): two hands for any block averaging >30 customers/hour (the after-work shoulder needs them too — the finding the monthly average hid); one hand suffices Sundays 14:00+ (traffic, not baskets — Project 1's verdict, operationalised); and the payday Friday exception (the + $\frac{1}{3}$  spike): the roster template carries a payday variant. Each rule is a threshold on the heat map — and the honest caveat rides along: 18 months of averages recommend; the actual Saturday (a funeral, a football crowd, rain) decides. Averages staff the *usual*; humans staff the day.

And the Sunday promotion — the running trial from Projects 4–5 — gets its tile: this Sunday's revenue against the pre-agreed baseline weeks, the four-Sunday threshold visible as a target line, the kill/scale decision date printed. The dashboard closes the loop this book opened in Chapter 5: a question became a test; the test's instrument is now a live tile; the decision will be made on data, on schedule.

## 48.4 Building the Page

The operations page in Power BI, on the Part VI model plus the stock table (a second fact table — the model grows a constellation: sales facts and stock facts around shared dimensions — the shape you will inherit at every employer):

- **Header:** "Operations — week of [date]" + the input-health strip (last stock count age, last till sync, data through).
- **Left third — stock risk:** table of items by days-cover ascending, `reorder` flag as conditional-format colour, a suggested-order column (the 48.2 arithmetic, as a measure), the count-freshness flag on top.
- **Centre — the heat map:** weekday × hour matrix of customer counts; drillthrough to the item-level transactions for any hot block.
- **Right third — staffing rules + trials:** the threshold tiles (blocks needing two hands, this week's payday variant), and the Sunday-trial tile with its target line and decision date.
- **The checks page,** inherited and extended: bridge totals, nulls, orphans, stock-count age — the honesty layer, watching a busier shop.

The build session is Chapter 41's method repeated at higher fluency — and if you follow the labs, the page ships in a long evening, because every component is a chapter you have already lived. That fluency is the portfolio claim: not "I have used Power BI" but "I can take a warehouse-shaped model to an operational page with alerting thresholds in a day."

## 48.5 Shipping the Stack

Portfolio evidence #6, and the capstone of the project set. The folder: `operations-dashboard/` — the `.pbix` (with the checks page), the stock-analysis notebook (the days-of-cover pipeline, restart-clean), the heat-map exhibit, the staffing-rules memo (one page, thresholds with numbers), and the 150-word note: *Situation: a grocery's weekly order/roster decisions, made monthly in a spreadsheet. Work: days-of-cover reorder logic; weekday×hour heat map; trials with pre-agreed thresholds; BI page with input-health alerts. Finding: two-handed staffing needed on the weekday shoulder the averages hid; stock reorder signal with count-freshness honesty. Impact: weekly planning in fifteen minutes on one page. Limits: synthetic-labelled; averages staff the usual, humans staff the day.*

The six-project ledger is complete — Excel, SQL, Python, statistics, SPSS-adjacent survey craft, Power BI — one shop, one story, every tool, every honesty rule, every artefact versioned and logged. Chapter 49 assembles it into the portfolio that gets you hired; Part VIII arms you for the hunt itself.

**From Your Toolkit — thresholds, everywhere:** operations analytics — stock, staffing, queues, capacity, on-call alerts — is the same move in every industry: a metric, a threshold, a freshness flag, and a drill path to the evidence. Retail runs it on days-of-cover; hospitals on bed occupancy; factories on downtime; SaaS on error rates. The dashboard you just built is a *pattern* with retail nouns; substitute the nouns and it is every operations page you will ever be paid to build.

## Key Takeaways

- Operations analytics = description converted into triggers: metrics against thresholds, alerts against amber, freshness flags on inputs.
- Days of cover (on hand ÷ daily rate) + delivery gap + safety margin = the reorder signal; missing stock counts become input-health alerts, not silent interpolations.
- The weekday × hour heat map answers staffing; thresholds with numbers, plus the payday variant; averages staff the usual, humans staff the day.
- The page: stock risk (left), heat map (centre), staffing + live trials with target lines and decision dates (right), checks page inherited.
- Six projects, one dataset, every tool — the portfolio's raw material is complete; Chapter 49 builds the shop window.



## Practice Lab

1. The stock pipeline: days-of-cover for every item, the reorder flag, the suggested-order column; reconcile three items by hand (bottles, math, log); export this week's order list as CSV — an artefact a real shop could act on.
2. The heat map, both ways: pandas/matplotlib exhibit *and* the Power BI matrix with conditional formatting; compare the two builds in five lines of log notes (effort, fidelity, refresh).
3. The staffing memo: one page — the thresholds, their numbers, the payday variant, the Sunday decision (with the trial tile's current status); write it for Tariro's fridge door, not for an analyst.
4. The page, built: the full operations dashboard with input-health strip, drillthroughs, and the trials tile; screenshot; five-minute walkthrough rehearsed (Ch. 43) with the order/roster decisions as the ask.
5. The ledger closed: all six 150-word notes in one file, STAR paragraphs beside them; the Chapter 45 checklist run once more across the project set. Read the ledger top to bottom — that hour is the fastest career-confidence you will ever buy.

## Further Reading

- Chapter 49 (the portfolio — the ledger becomes a shop window), Chapter 57 (the industrial versions of these patterns)
- "Retail ops analytics" case studies on any BI vendor's blog — recognise your own work in enterprise clothing

# Chapter 49: Your Portfolio

## Part VII — Real Projects and Your Portfolio

"Nobody hires you for what you know. They hire you for what you have *done* — and the portfolio is the proof you keep where they can see it."

### In this chapter you will learn

- GitHub from absolute zero: account, repository, commits — the analyst's minimum viable git.
- The three-project portfolio: what to include, what to cut, and how to order it.
- The README as shop window: writing the page that gets you the interview.
- The walkthrough: a guided tour a stranger can follow in ten minutes.
- Keeping it alive: the portfolio as a career habit, not a job-hunt artefact.

## 49.1 GitHub in One Hour

GitHub is where working analysts (and every hiring manager) look for evidence — a public shelf for your artefacts. The minimum-viable workflow, genuinely from zero:

1. **Account:** github.com → Sign up (free). Your username is professional-facing: a variant of your real name beats `dataninja420`.
2. **Repository:** New repository → name it `data-analytics-portfolio` → Public → Add a README → Create. A *repository* is a project folder that tracks every change.
3. **Files:** the **+** → **Upload files** page handles your first uploads (the web interface is a fine start; the command line arrives when a job demands it). Upload a project folder's contents, **Commit changes** (the commit message: "Project 4: customer churn analysis" — a labelled save point; the habit Chapter 20 taught, now on the world's shelf).
4. **Edit in the browser:** the pencil icon on any file — the README you will rewrite in section 49.3 is edited exactly like a document.

That is the whole requirement: an account, a repository, files, commits. (Branches, pull requests, `git` itself — the professional's tools — arrive in your first job or in an evening of learning from GitHub's own tutorials; the *portfolio* needs none of them to start. If you already know `git`, use it properly — commit per project, meaningful messages; if you do not, the web path is honest and

sufficient.)

Two house rules: **every project folder gets its own README** (what, why, how to run, the finding, the limits — the header-block habit of Chapter 20, grown to a page); and **data goes in with care** — small generated datasets in-repo (labelled synthetic), large files linked rather than committed, never anything you have no right to publish (Chapter 44's licences; Chapter 59's ethics).

## 49.2 The Three-Project Portfolio

Six projects exist in your ledger; the *portfolio* shows three to four. Selection is the craft — a portfolio is an argument, not an archive:

- **Include:** (1) the **wild-card public-data project** (Chapter 44's queue — real data, real mess, no planted answers: your proof you work outside the book); (2) **one deep analysis project** with statistics (Project 4 churn, or the wild-card if it runs tests) — proof you conclude, not just describe; (3) **one automation or BI project** (Project 3's pipeline or Project 6's dashboard) — proof you *ship tools*, the rarest skill at junior level. A fourth slot (the survey, if you fielded it for real — human data + ethics discipline) if it is genuinely yours.
- **Cut:** near-duplicates (Projects 1–2 are chapters of the same story as 3 — fold their findings into 3's README); unfinished work (a dead notebook reads as unfinished work, because it is); anything you cannot defend line-by-line in an interview — the portfolio is an *invitation to be questioned*, and you will be (Chapter 52).
- **Order:** lead with the wild card (real data) or the project with the strongest *finding* — the first project a recruiter opens gets the only unmixed attention you will receive.

The book's projects 3, 4, 6 + your wild card is the recommended four — tools covered (Python ×3, statistics, BI), every finding real-or-labelled, each with the full artefact trail (code, exhibit, report, note).

## 49.3 The README as Shop Window

The repository's front page is written for a stranger with 90 seconds: a hiring manager, skimming, phone in hand. The structure that earns the second minute:

```
# Your Name – Data Analytics Portfolio
One paragraph: who you are, what you do (Excel, SQL, Python,
statistics, Power BI), and the one-line story of the portfolio.
*(Links: LinkedIn, email, CV.)*
## Projects
```

```

### 1. [Wild-card title] – one sentence, the finding with its number
Real data from [source]. Question, method, finding, the
recommendation it produced. **Stack: Python, pandas, scipy.**
→ notebook · report · dashboard (links)
### 2. Customer churn – founding cohort drifting, 60 at-risk named
... (same shape, each 3-4 lines)
## What I practise
The working stack, one line each: the reconciliation habit,
testing over vibes, caveats attached to claims. (Yes – say the
*habits*. Everyone lists tools; habits are what you actually sell.)
## Data & honesty
All synthetic datasets are labelled as generated (generator
included). Public data cited to source. No data I have no right
to publish.

```

The craft rules, all of them this book's rules in miniature: **findings with numbers in the project blurbs** ("founding cohort churning 2× fast", not "explored churn"); **links that work** (dead links are the most common portfolio failure — click every one, monthly); **the honesty section present** (it reads as senior; most candidates have none); and *spell your own name right* — the portfolio, CV, and LinkedIn must match exactly (this book's author has seen a CV misspelling its own candidate's name; so has every hiring manager).

## 49.4 The Walkthrough

Some interviews (and most serious recruiters) will open the portfolio *with you watching*. Prepare the ten-minute guided tour, three projects, three minutes each — the Chapter 43 five-minute walkthrough's sibling, applied to your own shelf:

1. **The map** (1 min): the README, top to bottom — who, the four projects, the honesty section. "Everything here is reproducible from the repo."
2. **One deep project** (3–4 min): the wild card or churn — question first (the anatomy's law, spoken), the one exhibit that carries the finding, the statistical verdict with its caveat, the recommendation. Close on the *limits* — voluntarily, before asked; it is your strongest move.
3. **One tool project** (2–3 min): the pipeline or the dashboard — what it automates (the before/after timing), the checks that guard it, a live click or re-run if the room allows.
4. **The invitation** (1 min): "Happy to go deeper on any of these — the code, the decisions, or what I'd do differently." Then *stop talking*. The interview takes the invitation.

Rehearse it once aloud with a timer (the Chapter 43 lab method); ten minutes is the ceiling, and stopping on time is itself evidence of the communication

craft Part VI built.

## 49.5 Keeping It Alive

The portfolio's final rule: it is a *living shelf*, not a job-hunt artefact — the Chapter 44 dataset habit feeds it (one dataset a month; the best become projects), each new project gets its note-and-README on shipping day, and the "What I practise" section evolves as you do (in your first job, it grows employer-relevant lines: their warehouse, their domain). A repository with a commit last month is read as *an analyst at work*; one with a burst of activity two years ago is read as *a course certificate with extra steps*. You are the first — the compounding career of Chapter 62 begins on this shelf.

**From Your Toolkit — the shelf is the career:** Part VIII is about to send you into the market, and every move there — the CV (Ch. 51), the interviews (52–53), the freelancing (55) — links back here: the CV's projects section is three lines pointing at this repository; the technical tests are passed with the skills these projects evidence; the case interviews are *your walkthrough, retold*. The portfolio is not the last chapter of learning; it is the first chapter of working.

## Key Takeaways

- GitHub minimum: account, public repository, files, commits, per-project READMEs — web interface is an honest start; git proper can wait for the job.
- Show three or four projects, chosen as an argument: real data (wild card), deep statistics, a shipped tool (automation/BI); cut duplicates, dead work, and anything you cannot defend.
- The README is written for a 90-second stranger: findings with numbers, working links, the habits section, the honesty section.
- The ten-minute walkthrough: map → one deep project (limits stated first-hand) → one tool project → the invitation; rehearse with a timer, stop on time.
- Keep it alive monthly — a living shelf reads as an analyst at work; the dataset habit feeds it forever.

## Practice Lab

1. The hour: create the GitHub account and `data-analytics-portfolio`; upload Projects 3, 4, and 6 with their folders; write per-project READMEs (what/why/how-to-run/finding/limits) — commit messages meaningful.

2. The front page: write the main README in section 49.3's shape; make your four project links; verify every link by clicking; align name spelling across CV, LinkedIn, and GitHub.
3. The wild card, begun (if not yet built): take Chapter 44's top-ranked local question; run the full anatomy (Ch. 45) at portfolio depth — question log, first hour, liturgy, analysis, one-page report; ship it as project #1 with its note.
4. The walkthrough, rehearsed: ten minutes, timer, camera (phone); watch it back; note the two worst moments (usually: mumbling the finding, or over-running); re-rehearse once; book it done.
5. The liveness system: set a monthly reminder (first Friday); the agenda is already written — one dataset (Ch. 44), link-check the shelf, update the notes; write the agenda into the reminder so the habit survives enthusiasm's departure.

## Further Reading

- Part VIII (the hunt: CV, interviews, offers), Chapter 61 (the free stack that keeps the shelf fed)
- GitHub's "Hello World" guide — 10 minutes, the git concepts in miniature

**PART VIII**

# **Getting the Job**

Chapters 50 – 56

---

# Chapter 50: The Job Landscape

## Part VIII — Getting the Job

"'Data analyst' is one job title wearing a dozen different job descriptions. Reading the difference is the first skill of the hunt."

### In this chapter you will learn

- The titles and what they actually mean — analyst vs scientist vs engineer vs BI.
- What employers actually want, decoded from real postings.
- Where the jobs are: industries, company sizes, and the entry paths.
- The reality of requirements lists — and how to read them like a professional.
- The market in your region and the remote world — hunting where you stand.

### 50.1 The Titles

The family tree, in plain words, with the overlaps stated honestly:

Title	The daily job	Your book covered
<b>Data Analyst</b>	Answer business questions with data: reports, dashboards, ad-hoc analysis, the five working shapes	All of it — this is your target
<b>Business/Reporting Analyst</b>	Analyst + process: KPIs, weekly packs, stakeholder wrangling	All of it, minus the deepest stats
<b>BI Analyst / Developer</b>	Dashboards, data models, semantic layers — Power BI/Tableau-centric	Parts II-III, VI strongly
<b>Data Scientist</b>	Analyst + modelling + experimentation (prediction, causal inference, ML)	Most of it + Ch. 57-58 as the door
<b>Data Engineer</b>	Build the pipelines and warehouses analysts query	Part V's architecture, then a specialisation
<b>Analytics Engineer</b>	The modern bridge: dbt, versioned SQL, modelling in the warehouse	Parts III, V, and 57's world
<b>Research/Statistician</b>	Surveys, experiments, formal inference — SPSS/Stata heartland	Part IV strongly

The practical reading: junior titles blur — a "Junior Data Analyst" at a retailer and a "Reporting Analyst" at a bank may be the same job; a "Data Scientist (Junior)" may be an analyst job with a grander name (check the *duties*, never the title). Your positioning, straight from this book's stack: **analyst who can**



**automate** — Excel, SQL, statistics, Python, BI — is the profile that fits the widest set of postings in the family.

## 50.2 Decoding Real Postings

Read the *duties* section of a posting as the honest job and the *requirements* as the wishlist (next section). The duties that recur, decoded into this book's chapters:

- "Provide insights and reporting to support decision-making" — the five moves of Chapter 45, on request.
- "Develop and maintain dashboards" — Parts III + VI: model, measures, pages, refresh.
- "Write SQL to extract and transform data" — Parts III + V: the working shapes plus pipelines.
- "Clean and validate data from multiple sources" — the liturgy, in every Part.
- "Present findings to stakeholders" — Chapters 13, 43, 49: the one-pager, the walkthrough, the caveat-first habit.
- "Support A/B testing / experimentation" — Chapter 26's courtroom at industrial scale (57).

The skills-frequency reality check (consistent across market surveys for years): **SQL and Excel appear in the overwhelming majority of analyst postings; a BI tool (Power BI or Tableau) in most; Python in roughly half, rising; statistics fluency valued everywhere but listed least** — because interviewers *test* it rather than list it. This book's stack is ordered the way the market orders itself; that was not an accident.

## 50.3 Where the Jobs Are

- **Industries:** retail and e-commerce (inventory, pricing, customer — Tariro's world, scaled); banking, insurance and fintech (risk, fraud, product analytics — heavy SQL, heavy regulation); health and public sector (research-flavoured, SPSS/Stata heartland — Part IV's tools earn their keep); telecoms (churn, network, revenue assurance); consulting and agencies (many domains fast; client-facing). Match the *domain* to your background when you can — a retail background + retail analytics is a story that interviews itself.
- **Company size, honestly: Large companies** — structured graduate/internship pipelines, mentors, narrow first roles ("the SQL person for revenue reporting"), slower decisions. **Small companies and startups** — no pipeline, wider roles ("the data person" — the whole stack of this

book), faster learning, less support, and titles that mean whatever the founder means. **Agencies/consultancies** — many clients, many tools, steep communication curve. For a first job, apply widely across all three; the "which is best" question answers itself per offer (Chapter 53's decision table).

- **The entry paths:** internships and graduate programmes (apply even if you think you are "not the type" — the portfolio levels that field); junior/entry postings (the direct route); **adjacent-entry** — the inside move: taking an ops/customer-service/marketing role at a company with a data team and becoming "the person who does the numbers" internally, then transferring — *statistically the most under-rated path into analytics*, and your portfolio makes it deliberate rather than lucky.

## 50.4 Reading Requirements Like a Professional

The requirements list is a *wishlist*, assembled by committee, aspirational by nature. The decoder: **"required" means screened, "nice-to-have" means bonus, and the years-of-experience number is a proxy for competence, not a law** — a posting asking "3+ years, SQL, Python, Power BI" is describing the *work at the level they hope for*; candidates with strong evidence of doing that work get interviews at 1 year, and candidates with 4 undifferentiated years often do not. The professional's response to a requirements list: score yourself honestly against the *duties* (can I do this work? could I learn it in weeks?), then apply if you clear most duties and half the wishlist — and let the portfolio (Chapter 49) carry the argument the CV's years cannot. The one hard filter to respect: genuinely required legal/security clearances, and specific work-authorisation needs in your market; everything else is negotiable with evidence.

## 50.5 Your Region and the Remote World

Hunting from Zimbabwe (this book's home ground) or anywhere outside the big tech corridors, the map is three markets at once: **local** — banks, telecoms (Econet, NetOne, Telecel ecosystems), retailers, NGOs and development agencies (MEAL officer roles are *analyst* roles with a social-sector name — surveys, indicators, reporting: Part IV + the survey project map directly), government statistical agencies; **regional/remote-Africa** — South African and pan-African fintech/retail hiring remotely, in your timezone; **global remote** — the fully-remote junior market is thinner (remote favours mid-level) but real, and grows every year — the portfolio is *more* important there, because it is the only interview you get.

Two honest strategic notes for the region: the NGO/development sector values exactly this book's statistics-and-survey half (SPSS/Stata fluency is a listed requirement in MEAL postings — Chapter 29 was not a detour), and the local salary reality differs from global-remote for identical work — the practical play, for many, is local experience for 12–24 months while building the global-remote portfolio, then choosing deliberately. Chapter 55's freelancing adds a third lane from day one.

**From Your Toolkit — the hunt's map:** this chapter is the market's anatomy; the next chapters are its crafts (CV, interviews, negotiation). Keep the decoded-duties table beside you as you read postings this week — ten postings decoded by hand (the lab) teaches more than any guide, including this one.

## Key Takeaways

- Titles blur; duties are the job: analyst/reporting/BI are your bullseye family, and "analyst who can automate" is the widest-fit profile.
- Posting duties decode to this book's Parts: the stack is market-ordered — SQL and Excel everywhere, BI in most, Python half and rising, statistics tested rather than listed.
- Hunt three company sizes and every entry path — and remember adjacent-entry (join a company, become its data person, transfer): the under-rated route your portfolio supercharges.
- Requirements are wishlists scored by duties; evidence beats years; only clearances and work rights are hard filters.
- Hunt three markets at once (local — where NGO/MEAL roles value the SPSS/Stata half; regional-remote; global-remote), and let the portfolio carry what the CV cannot.

## Practice Lab

1. The decoder drill: take ten live analyst postings (LinkedIn, local boards, company sites); for each, list the top three duties decoded into chapters, and score yourself 1–5 on each; total your honest readiness — this is your gap list for Part VIII's remaining chapters.
2. The market map: for your city/country, list the twenty most plausible employers by sector; note which have data teams (LinkedIn search: company + "data analyst"); ten cold targets for Chapter 51's outreach.
3. The title study: find one posting each for analyst, BI developer, analytics engineer, and data scientist at a similar company; write the two-paragraph

comparison of what differs *in duties* — your positioning statement grows from it.

4. The requirements histogram: from your ten postings, count how many list each skill (SQL, Excel, Python, Power BI, Tableau, statistics, SPSS/Stata, communication); compare your histogram to this chapter's market claims; note the local differences.
5. The path choice, provisional: draft your hunt plan — target titles, three markets, the adjacent-entry candidates, and the timeline; mark it provisional (Chapter 53 will sharpen it) and store it with the portfolio ledger.

## Further Reading

- Chapter 51 (the CV and LinkedIn — the decoded duties become bullet points)
- Local job boards and LinkedIn's "Jobs" filters — set the alerts tonight, not "when ready"

# Chapter 51: Your CV and LinkedIn

## Part VIII — Getting the Job

"A CV is not read; it is skimmed. Write for the skim, and the interview invitation is what survives it."

### In this chapter you will learn

- The analyst's CV, section by section — with the lines that earn interviews.
- Bullet craft: achievements with numbers, not duties with verbs.
- The screening reality: ATS, keywords, and the 30-second human skim.
- LinkedIn as the passive pipeline: the profile that works while you sleep.
- The cover note that gets answered — short, specific, evidence-linked.

## 51.1 The CV, Section by Section

One page (two, absolute maximum, if experience demands it), and the shape:

```
YOUR NAME                                city · phone · email · LinkedIn · portfolio
      Data Analyst | Excel · SQL · Python · Power BI · Statistics
SUMMARY (3 lines)
Analyst who automates: [X] projects shipped on real and public data,
from cleaning to tested findings to dashboards. [Domain] background;
strong written communication; portfolio: [link]
PROJECTS (the heart – yes, above experience)
Customer churn analysis – Python, pandas, scipy
  · Found founding-cohort churn at 2× the shop average; named 60
    at-risk customers worth 1.4× mean CLV; designed a save-campaign
    with 8× projected payback
  · Built cohort retention grids and survival curves; 150-word
    case study + reproducible notebook at [link]
[Pipeline project] – an afternoon of monthly reporting → 20 minutes
[Public-data wild card] – [the finding, with its number]
EXPERIENCE
[Role], [Employer], [dates]
  · [achievement with a number – 51.2]
  · ...
EDUCATION & CERTIFICATES
[Degree/diploma], [institution], [year]
[Certificates – only ones with exams: e.g. Microsoft PL-300 if taken]
SKILLS (honest, scannable)
SQL (joins, CTEs, windows) · Excel (pivots, lookups, ToolPak)
Python (pandas, matplotlib, scipy) · Power BI (model, DAX, publish)
Statistics (t-tests, chi-square, regression, surveys) · SPSS/Stata
```

The structural argument, for projects above experience: at entry level, your projects *are* your experience — six shipped artefacts beat six months of unnumbered duties, and the recruiter who skims for evidence finds it in the first scroll. When a real analyst job arrives, the sections swap naturally.

## 51.2 Bullet Craft

The formula that converts duties into evidence: **[action] + [what] + [number/measure]**. Before and after, from this book's own work:

- □ "Responsible for monthly reports" → □ "Rebuilt the monthly reporting pack in Python: an afternoon's manual work became 20 automated minutes, with data-quality checks that abort on broken inputs"
- □ "Worked with SQL" → □ "Wrote a 10-query monthly analysis pack (joins, CTEs, window functions) reconciled cent-for-cent against the source system"
- □ "Skilled in statistics" → □ "Closed the shop's Sunday question with a two-sample t-test ( $t = -18.5$ ,  $p < 0.001$ ) and attached the confidence interval to the finding"
- □ "Created dashboards" → □ "Built a Power BI operations dashboard (star schema, 12 DAX measures, drillthrough paths) that weekly planning now runs on"

Every bullet: a verb that did something, an artefact or method, and a number wherever one exists (time saved, rows processed, accuracy, effect size). Numbers are not decoration — they are *evidence*, and after this book you have dozens to choose from. The one law: **never a number or a claim you cannot defend for five minutes under questioning** (Chapter 52 will question exactly these lines).

## 51.3 The Screening Reality

How a CV actually dies: the **ATS** (applicant tracking system) parses it into fields and matches keywords — so mirror the posting's *own words* where honestly true ("Power BI" if the posting says "Power BI", not only "BI tool"; "SQL" spelled exactly so); avoid tables/text-boxes/graphics that parse badly (the pretty-design CV is the ATS's prey); and name the file `Firstname-Lastname-Data-Analyst-CV.pdf`. Then a human — 30 seconds on the first pass: name, current role, top three bullets, portfolio link, done. Both gates are passed by the same craft: *keywords honestly present, evidence in the top third, numbers in every bullet*. (Keyword stuffing — listing tools you cannot use — fails the *interview's* parse; the honesty rules apply to yourself.)

## 51.4 LinkedIn as Pipeline

The profile that works while you sleep, an evening's work: **headline** — "Data Analyst | Excel · SQL · Python · Power BI" (searchable terms, not "aspiring data enthusiast"); **about** — the portfolio's opening paragraph, first person, with the link; **featured** — pin the portfolio repo, the best project's exhibit, the dashboard screenshot; **experience/projects** — the CV's bullets, mirrored; **skills** — the honest list, endorsements gathered from colleagues/classmates (recruiters filter by them); **open-to-work** (visible to recruiters, or publicly if bold — the toggle is yours). Then the habit: one engagement a week — a short post on a project ("built a churn analysis; here's the cohort grid and what surprised me"), a thoughtful comment on someone's analysis post. The algorithm rewards consistency over volume; recruiters *do* read activity feeds; and the discipline costs twenty minutes a week.

## 51.5 The Note That Gets Answered

Applications with a short specific note out-perform naked applies. The four-sentence template:

Dear [Name] — I'm applying for [exact role]. I fit the [duty X] closely: my [project] did [finding with number], reproducible at [portfolio link]. My background in [domain] adds [one relevant sentence]. I'd welcome a conversation about the team's [something specific from the posting — the dashboard refresh, the experiments].

Specific, evidenced, and *about their posting*, not about your need. Send it to a human where possible (the recruiter named on the posting, or the hiring manager via LinkedIn — one polite message, not a campaign); address hiring managers by name; and never mass-blast identical notes — specificity is the entire mechanism, and its absence reads instantly.

**From Your Toolkit — one evidence chain:** CV bullet → portfolio project → notebook/report → the code and the log. The chain is what this Part keeps calling *evidence*, and every artefact of it was built by the habits of Parts I-VII: the numbered findings (the report pages), the reproducibility (the packs and pipelines), the honesty labels (the notes' limits lines). The CV is simply the chain's first, shortest link.

## Key Takeaways

- One page, projects above experience at entry level; the summary states the profile ("analyst who automates") and the portfolio link.

- Bullets = action + artefact + number; numbers are evidence; nothing you cannot defend for five minutes.
- ATS: mirror honest keywords, clean layout, named PDF; human skim: evidence in the top third.
- LinkedIn: searchable headline, featured portfolio, mirrored bullets, one honest engagement a week.
- The four-sentence note: specific duty, evidenced fit, portfolio link, one detail from *their* posting — to a human where possible.

## Practice Lab

1. Write the CV from the template — every project bullet carrying a number lifted from your project notes; have a friend skim it for exactly 30 seconds and report what they retain; fix what they missed.
2. The ATS pass: paste your CV into a plain-text reader; check the keywords from your Chapter 50 decoder drill all appear honestly; rename and export the PDF properly.
3. The LinkedIn hour: headline, about, featured (repo + two exhibits), skills, open-to-work decision; then write and schedule (drafts are fine) your first engagement post on the churn project's surprise.
4. The note, written three ways: one application note for a real posting (send it); one cold message to a hiring manager from your Chapter 50 target list (polite, one shot); one referral request to a former colleague or classmate (specific, low-pressure).
5. The chain audit: pick your three strongest CV bullets; click through each to its portfolio artefact; fix any broken link, missing exhibit, or defence-gap you find — the interview will do exactly this, with questions.

## Further Reading

- Chapter 52 (the technical interview — where these bullets get defended), Chapter 53 (cases, behaviour, offers)
- Any recruiter's "what I read in 30 seconds" post on LinkedIn — find three; note how consistent they are with this chapter



# Chapter 52: The Technical Interview

Part VIII — Getting the Job

"The technical interview is not an exam about tools. It is a simulation of the job — with witnesses."

In this chapter you will learn

- What technical interviews actually contain — the four formats, decoded.
- SQL tests: the classic question shapes, and how to pass them thinking aloud.
- Excel and case tests: what the spreadsheet exercise is really grading.
- Statistics questions: the six concepts tested, and the honest answers.
- The practice system: how to train for tests the way you trained this book.

## 52.1 The Four Formats

Technical interviews for analyst roles come in four shapes, often combined within one process. Knowing the shape tells you what is being graded — which is rarely what it looks like:

Format	Looks like	Actually grades
Live SQL/code screen	"Write the query for top 3 per category"	Fluency <i>and</i> the thinking: clarifying, structure, checking
Take-home task	"Here's a dataset; send findings in 48h"	Craft judgement: scope, cleaning, honesty, communication — <i>not</i> model sophistication
Spreadsheet exercise	"In this sheet, find why revenue dropped"	Investigation process: hypothesis, pivot, verify — under time
Rapid-fire knowledge	"Inner vs left join? What's a p-value?"	Precision and honesty — can you say it in two sentences

The universal truth across all four: **they are grading what it would be like to work with you** — do you ask sensible questions, state assumptions, check your work, admit uncertainty, communicate clearly? The candidate who reasons aloud, catches their own mistake, and fixes it *beats* the silent one who writes a perfect answer — because the first is showing the actual job.

## 52.2 The SQL Shapes

Live SQL screens draw from a small, stable repertoire. You have written every pattern in this book; here they are as questions:

1. **Top-N per group** — "top 3 selling items per category": aggregate → rank with `ROW_NUMBER() OVER (PARTITION BY)` → filter  $rn \leq 3$  (Ch. 19). *The most common screen question in existence.*
2. **The second-highest** — "second-largest sale": `ORDER BY + OFFSET/LIMIT`, or `MAX` with a `WHERE < the max`, or `DENSE_RANK = 2` — they are testing whether you know *two ways* and can say why one is safer (ties).
3. **Join with a twist** — "customers with no orders": `LEFT JOIN + WHERE right IS NULL` (the anti-join, Ch. 17) — or "count customers *and* their orders even if zero" (`LEFT + COUNT(column)` vs `COUNT(*)`, Ch. 15-16's two counts).
4. **The cumulative/running** — "running total by month": `SUM() OVER (ORDER BY)` (Ch. 19).
5. **Share of total** — "each category's % of revenue": grouped CTE + total, or `SUM() OVER ()` (Ch. 18).
6. **The dedup** — "remove duplicate rows, keep latest": `ROW_NUMBER()` partitioned by key, ordered by date `DESC`, keep  $rn = 1$  (Ch. 19 + 8 combined).

The *method* for the live version, as important as the answer: repeat the question in your own words; state one assumption out loud ("assuming 'selling' means revenue, not units — I'll note the alternative"); write it in readable steps (CTEs — the interviewer reads them like paragraphs); then *self-check*: "row counts before and after the join, and the total should reconcile." The candidate who says "I'd check the total didn't double" is performing Chapter 17's law, live, and every interviewer recognises it.

## 52.3 Excel and Case Tests

The spreadsheet exercise (usually 30–60 minutes, a "messy" workbook): a sales sheet with three spellings of a region, blanks, mixed date formats, and a question like "did the March dip hit all regions?". What they grade, in order: **do you profile before you touch?** (counts, duplicates, the shape — Chapter 8's liturgy); **do you build the checkable?** (a cleaned copy, formulas traceable, no hard-typed numbers); **do you answer as a sentence with a number and a caveat?** The case version (verbal, sometimes in the spreadsheet): "revenue is down 8% — walk me through how you'd investigate" — graded on hypothesis structure: *segment the fall* (region? channel? product? — the decomposition instinct), *find the movement* (trend vs step vs level shift), *check the data first* (a classic trick: the "drop" is a missing week's data — Chapter 44's reconciliation instinct catches it, and mentioning that possibility unprompted is the single strongest move available in a case interview).

## 52.4 Statistics Questions

The six concepts that account for nearly all statistics screening, with the two-sentence honest answers (all Part IV; the discipline is brevity *plus* correctness):

- **p-value:** "The probability of seeing evidence this strong if nothing were really happening. Small p means 'surprising under the null', not 'probably true'."
- **Statistical vs practical significance:** "Significance says it's not noise; practical says it's worth money. With big n, 2-cent differences are 'significant' — I always report effect size with the p."
- **Correlation vs causation:** "Three explanations — coincidence, confounding, causation — and only an argument (ideally an experiment) licenses the third. I name confounders before claiming causes."
- **Type I vs II:** "False alarm vs missed effect.  $\alpha$  sets the first; sample size drives the second. In business, false alarms send teams chasing ghosts, so I guard Type I."
- **Which test:** "Compare means  $\rightarrow$  t-test (paired if the same units measured twice); compare proportions/counts  $\rightarrow$  chi-square; relationship between two numbers  $\rightarrow$  correlation, then regression for the line. I check assumptions before running."
- **Sample vs population:** "My data is a sample of the process I care about — 18 months is not forever, this shop is not all shops. Intervals state how wrong the estimate might be."

Interviewers probe these with follow-ups ("so is 0.05 magic?" — no, convention; "what if the  $p$  is 0.06?" — same evidence, weaker label; pre-decide  $\alpha$  and report the effect honestly). The candidates who answer *like analysts* — with the caveat attached, unprompted — are distinguishable within two questions.

## 52.5 The Practice System

Train as you trained this book — reps on real shapes, not passive rereading:

1. **SQL reps:** LeetCode/HackerRank SQL sets (the database medium tier  $\approx$  this book's level; StrataScratch and DataLemur mirror real interview questions) — 30 minutes daily, *thinking aloud even alone*, and always the self-check line after each solve.
2. **The mock spreadsheet hour:** rebuild Project 1's cleaning-and-question flow on a fresh messy dataset weekly until it fits inside 45 comfortable minutes.
3. **The statistics drill, spoken:** a friend (or camera) asks the six questions randomly; you answer in two sentences; the grade is honesty + brevity, not vocabulary.
4. **The take-home rehearsal:** Project 4 is one — reread its deliverable folder as an interviewer would, 30 seconds per artefact, and note the first impression; that impression is what your next take-home must produce.
5. **Appendix E:** one hundred interview questions across tools and topics, with guidance — the practice bank; work it in batches of ten, aloud, and let it find your gaps the way this book's labs found them.

**From Your Toolkit — the interview is the job, compressed:** every tested shape is a Part of this book performed under observation: the SQL screen is Part III with an audience; the spreadsheet case is Part II on a clock; the statistics questions are Part IV in two sentences; the take-home is Part VII's anatomy with a deadline. You have been rehearsing this interview since Chapter 6 — what remains is format fitness, and format fitness is bought with reps.

## Key Takeaways

- Four formats, one grade: what you'd be like to work with — clarify aloud, state assumptions, self-check, admit limits.
- SQL screens draw from six shapes; know two routes to each and always say the reconciliation check.

- Spreadsheet/case exercises grade the process: profile first, build checkable work, answer in sentences with caveats — and suspect the data before the story.
- Statistics screening is six concepts; two-sentence honest answers beat five-minute lectures.
- Train with reps and aloud-ness: daily SQL sets, weekly mock-spreadsheet hour, spoken stats drill, and Appendix E's bank in batches.

## Practice Lab

1. The six SQL shapes, timed: solve all six from memory in one sitting (your own data or DataLemur); note where you hesitated; drill that shape daily for a week.
2. The aloud drill: record yourself solving two SQL questions *narrating* the method (repeat, assume, build, check); listen back once; note the silences — those are the interview's dead air.
3. The case rehearsal, twice: "revenue is down 8%" — once as a spoken investigation (10 min), once with a sabotaged dataset where the drop is missing data; did you check the data first? (Now you always will.)
4. The statistics gauntlet: a friend/camera asks the six plus follow-ups; two sentences each; repeat weekly until the caveats arrive unprompted.
5. The take-home, redone: pick any Chapter 50 target posting's *plausible* take-home (or invent one from its duties); execute it in four hours with the full folder craft; score yourself against Chapter 45's checklist — this rehearsal is the closest thing to the real thing that exists.

## Further Reading

- Chapter 53 (cases, behaviour, and the offer), Appendix E (the 100-question bank)
- DataLemur and StrataScratch's free question sets — real interview shapes, explained

# Chapter 53: Cases and Behaviour — the Offer

## Part VIII — Getting the Job

"The technical round proves you can analyse. The case and behaviour rounds prove they can put you in a room with their stakeholders — and the negotiation proves you can be in one with them."

### In this chapter you will learn

- The case interview: the metric-decomposition framework that fits in your pocket.
- Behavioural rounds: STAR stories from your own ledger, told in 90 seconds.
- The take-home presentation: defending your analysis to a panel.
- Questions *you* ask — the underrated scoring round.
- Negotiation, politely: the script, the silence, and the decision table.

## 53.1 The Case Interview

The analyst case ("engagement is down 15% — what do you do?") is graded on *structure*, and the structure is Chapter 45's anatomy with a clock. The pocket framework — five moves, spoken aloud:

1. **Clarify the metric** — "Engagement — how is it defined here, and who defines it? Over what baseline?" (Never accept a fuzzy noun; the clarification *is* the first analytical act.)
2. **Segment the fall** — "I'd break the 15% down: by channel, by user group, by platform, new vs returning. A 15% average is usually 0% somewhere and -40% somewhere else." (Decomposition — the pivots of Part II, the GROUP BYs of Part III, as instinct.)
3. **Data first** — "Before believing the story, I'd check the data: tracking changes, missing days, a definition shift. Twice in my work, the 'drop' was a measurement artefact." (Says the candidate this book built — and it lands.)
4. **Hypothesise and test** — "Say mobile-new-users are the whole fall: I'd look for what changed — release, pricing, a competitor, seasonality — and distinguish coincidence, confounding, cause." (Chapter 28's three explanations, named.)

5. **Recommend with measurement** — "The fix depends on the segment; I'd propose the cheapest reversible action and define the metric that decides in two weeks." (Recommendation + kill-line, the Chapter 13 discipline.)

Cases are *conversations*: the interviewer feeds facts as you probe ("actually, desktop is up") — and the grading favours adaptation over a recited ladder. If a hypothesis dies, say so and pivot: "then seasonality is out; let me look at the release calendar." The fatal case errors are two: silence (thinking invisibly) and sureness (leaping to cause without the data check).

53.2 Behavioural Rounds: Your Ledger, Spoken

"Tell me about a time you..." — and the answer is already written, in your project ledger. **STAR** in 90 seconds: **S**ituation (10s), **T**ask (10s), **A**ction (40s — the majority, and the only part that is about you), **R**esult (30s — with the number). From the six projects, prepare six stories, mapped to the six questions every behavioural round asks:

The question	Your story (examples from this book's ledger)
"...a time you found something others missed"	The say/do gap (Project 5): 41% intent reframed to 15-20% by joining survey to behaviour
"...you made a mistake / data was wrong"	The fan-out or the missing weeks: totals doubled, caught by reconciliation, never repeated
"...you had to explain something complex"	The five-minute dashboard walkthrough to a non-analyst (Ch. 43); the Sunday verdict in shop-owner sentences
"...you disagreed with a stakeholder"	The loyalty-scheme caveat: presenting the association-not-causation limit to an owner who wanted "proof"
"...you worked to a deadline"	The 48-hour take-home: scoped to the strongest finding, shipped the honest limits
"...you improved a process"	An afternoon became twenty minutes (Project 3): the automation story with its before/after

Tell them honestly, including the limits — "the finding was real but the cause was unknown, so we designed a trial" is a *stronger* story than a clean fake, because it is the shape of real work and interviewers know it. Rehearse aloud, timed, once each — polished but not robotic; the 40-second action segment should carry the *decisions* ("I chose a paired test because the same items were measured twice"), because decisions are what they are buying.

### 53.3 The Take-Home Presentation

The final round of many processes: your take-home, presented (10 minutes + questions) to a panel. The format that wins is Chapter 43's walkthrough, applied to their data: **headline first** (the finding, with its number, in the first 30 seconds); *then* the method in three steps (question, what the data needed, the analysis); *then* the finding properly (evidence, effect size, the chart that carries it); *then* limitations and next steps, offered voluntarily; then questions. The Q&A is where offers are won: slow down, answer the question asked, say "I don't know — here's how I'd find out" where true (the most under-rated sentence in interviewing), and treat the hostile question ("isn't this just seasonality?") as the gift it is — a chance to show the three explanations and the data check *in action*: "Good challenge — that's exactly the confounder I checked second; here's the seasonal split." Panels remember composure under challenge longer than any slide.

### 53.4 The Questions You Ask

"Any questions for us?" is a scored round, not a courtesy. The questions that score — because they are *analyst questions*:

- "What does the team's data stack look like end-to-end — from source to the dashboards stakeholders actually open?"
- "How do decisions actually get made here — has an analysis changed a decision recently? What happened?" (You are interviewing their data culture; a team that cannot name a recent analysis-driven decision will teach you bad habits.)
- "What does success look like for this role in the first 90 days?" (Chapter 54 is your answer script, prepared.)
- "How is data quality handled — is there a checking culture, or do analysts discover surprises in production?"

Ask two or three, genuinely (you *are* gathering decision data — Chapter 53's whole subject applied to your own career). The questions you do not ask: salary (that round has its own moment, next section), "what does the company do" (you knew before applying), and anything the interviewer already answered.

### 53.5 The Offer and the Negotiation

The moment an offer exists, the power balance shifts: they have decided they want you; everything from here is *expected* to be discussed. The polite script, in three moves:



1. **Thank, and buy time** — "Thank you — I'm genuinely excited about this. I'd like two or three days to review the full package. Could you send the complete terms in writing?" (Written terms matter: bonuses, review timing, leave, study support, remote days — the full package, not the headline salary.)
2. **The counter, once, evidence-backed** — "Based on [market range for the role — Glassdoor/Payscale/local peers] and the portfolio evidence I bring [automation from day one], I was hoping for [number]. Is there flexibility?" Then **stop talking** — the silence is the negotiation; whoever fills it first concedes ground. One counter, polite and specific; not a duel.
3. **The decision table** — when offers or options compete, score them like an analyst (you are one): scope of learning (which tools/domain will this role grow?), mentorship (is there a senior analyst to learn from?), data culture (the "changed a decision" answer), comp (total, including growth trajectory), stability and location. Weight the rows by *your* current priority — junior years should weight learning heavily — and let the table decide, then honour it (the Chapter 2 discipline, applied to yourself: the analysis is finished when the decision is made).

A declined negotiation is not a lost offer (in the overwhelming majority of professional processes, a polite counter changes the number or changes nothing); an accepted-first-number is a permanent anchor. And the rejections that will still come — they come to everyone, at every level — are data for the pipeline, not verdicts on your worth: log them (stage, what was asked, what you'd strengthen), adjust, continue. The hunt is a funnel, and funnels are measured, not mourned.

**From Your Toolkit — the interview is the analysis:** case = the five moves; behaviour = the ledger with numbers; presentation = the walkthrough; your questions = the data-culture audit; the offer = the decision table. You are not performing a different skill in interviews — you are *running the discipline of this book on yourself as the dataset*. That reframe (and it is fully true) is the calm that wins rooms.

## Key Takeaways

- The case framework: clarify the metric → segment the fall → data check first → hypothesise with the three explanations → recommend with a kill-line; talk, adapt, and let hypotheses die loudly.
- Six STAR stories from your project ledger, 90 seconds each, action-majority, numbers in results, limits told honestly.

- Take-home presentations: headline in 30 seconds, method in three steps, limits volunteered, "I don't know — here's how I'd find out" as a strength.
- Your questions are scored: ask about the stack, the decision culture, and the 90-day success definition — you are auditing them.
- Negotiate politely once with market evidence, then silence; decide with a weighted table; log rejections as funnel data.

## Practice Lab

1. The case, run twice: "engagement down 15%" spoken to a camera (10 min); then a friend plays interviewer feeding you a twist every two minutes; review both recordings against the five moves.
2. The six stories, drafted and timed: write each STAR in under 120 words; speak each in 90 seconds; polish the action segment to carry decisions; store in `interview-stories.md` with the numbers bolded.
3. The hostile-question gym: brainstorm the ten hardest questions your projects invite ("why not a regression here?", "isn't this just seasonality?", "wouldn't the real fix be X?"); answer each in three sentences; rehearse the composure, not just the words.
4. The mock offer: role-play the script with a friend playing a friendly-but-firm recruiter (counter, silence, decision); write your real decision table now — weights chosen — so the real moment has a prepared analyst in it.
5. The funnel log, started: every application gets a row (date, company, stage, what was asked, lesson); the metrics of your own hunt — applications → screens → technicals → finals → offers — are conversion rates you can improve deliberately, which is the exact skill this book taught.

## Further Reading

- Chapter 54 (the first 90 days — the job you are about to start), Chapter 55 (the freelance lane)
- *Decode and Conquer* (Mei) — case-interview patterns, marketing flavour but transferable

# Chapter 54: Your First 90 Days

## Part VIII — Getting the Job

"You spent a book learning to analyse. The first ninety days are the analysis of you — run them like a project."

### In this chapter you will learn

- Days 1–30: learning the terrain — the data, the people, the questions that already exist.
- Days 31–60: the first owned deliverable, and the quick win that buys trust.
- Days 61–90: the systematisation that makes you indispensable.
- The stakeholders: whom to meet, what to ask, how to say no while new.
- The traps of the new analyst — the five ways beginners quietly fail.

### 54.1 Days 1–30: Learn the Terrain

The beginner's error is proving yourself with immediate cleverness; the professional's opening move is a *listening tour* with a notebook. Your first month is three maps:

- **The data map:** where does data live, really? (Warehouse, vendor exports, the finance director's personal spreadsheet that is somehow the source of truth.) What are the tables, the refresh schedules, the *known* flaws — and who knows them? Ask every person the same question: "*if a number looks wrong, who would you ask?*" — the answers draw the org chart of trust. Then run Chapter 44's first hour on the main datasets, for real: types, nulls, the reconciliation sweep. You are not fixing anything yet; you are building the map every future fix will use.
- **The people map:** your manager (what does *their* boss ask *them* monthly? — that question is the job), the senior analyst (your mentor-in-place; bring specific questions, not general neediness), the stakeholders (who consumes numbers; who blocks; who is quietly right), and the engineers (the relationship that pays for years: they know why the data is the way it is). One coffee, one genuine question, each — the *notes* are the deliverable.
- **The questions map:** what reports already exist (get them all — the weekly pack, the dashboard nobody opens, the ad-hoc slide decks)? What questions do people repeat? Every repeated question is either an automatable answer or an unresolved argument — and both are your 90-day material.

## 54.2 Days 31-60: First Deliverable and the Quick Win

Choose the first owned deliverable with Chapter 45's anatomy and one new criterion — *visibility*: a recurring deliverable somebody already wants, ideally one the previous analyst delivered slowly. Take the monthly sales pack (every company has one): rebuild it with the full craft — loading checks, reconciliation to the known totals, tested findings with caveats, the one-page front, the log. Ship it *early* (before the deadline, once), and the message lands: this one is safe to hand bigger things.

Then the **quick win**: a fix so obviously valuable it cannot be argued with, sized to a week. The classic shapes, all from this book: the reconciliation report that catches the double-counted source before month-end (Ch. 17's law, productised); the 90-minute report that becomes 10 automated minutes (Ch. 37, the story you already own); the dashboard whose broken filter nobody had time to fix; the top-ten query pack for the stakeholder who emails "can you pull..." weekly. The quick win's mechanics matter: *state the before* ("this takes Sue 3 hours monthly"), *deliver the after with the number* ("it now takes 10 minutes"), and *credit the people who helped* — the win buys trust for the system, not the ego.

## 54.3 Days 61-90: Systematise

The month that converts "promising new hire" into "load-bearing analyst": you stop *doing* the work and start *building the way the work is done*. The moves: **document** — the READMEs, runbooks, and query-pack headers of Chapters 20 and 37, applied to everything you touched; a successor should run your monthly pack from your runbook alone (test that claim — hand it over for one cycle). **Automate the second time** — anything done twice gets scheduled; the pipeline habit, applied to your own calendar. **Take the recurring question** — the one from your questions map with the most askers — and answer it *permanently*: a dashboard tile, a saved query, a weekly mail. And **measure yourself**: at day 90, write your own review — what ships faster because of you, what breaks less, what does the team know now that it didn't — the Chapter 51 bullet craft, applied to the job itself. (That document, kept quarterly, is the raw material of every future promotion case — Chapter 56's ladder is climbed on these.)

## 54.4 Saying No While New

The trap nobody warns juniors about: the requests. "Can you just pull this quickly?" (a day's work, undefined), "can you make the dashboard show the numbers we want?" (a request to break the data), "can you take on this side project?" (someone else's abandoned headache). The polite-refusal toolkit, learnt early: **clarify before committing** ("happy to look — what decision is this for? deadline? who else has asked?" — half of all requests dissolve at this question, because they were exploratory); **negotiate sequence, never just say yes** ("I can take this on after the month-end pack on the 5th, or sooner if we bump X — which would you prefer?"); and **the two words that protect your integrity forever**: "let me check the data first" — the sentence that makes you *slow once* and trusted always. New analysts who say yes to everything build the reputation of a vending machine; the ones who clarify and sequence build the reputation of an analyst. Ninety days decides which.

## 54.5 The Five Traps

The quiet failure modes, named so you can check yourself against them monthly:

1. **Tool peacocking** — leading with clever code on problems a pivot solves. The tools serve the question; nobody credits a scissor lift used to change a lightbulb.
2. **Skipping the reconciliation** — inheriting data, assuming it is clean; the previous analyst's numbers wrong *becomes* your numbers wrong the moment you touch them. Bridge-check everything you inherit, on day one.
3. **Going dark** — working two weeks on the perfect thing nobody asked for. Ship small, show early, course-correct in daylight (the small-cells habit, applied to careers).
4. **Caveat-free confidence** — presenting findings without the limits; the first time a stakeholder finds your hole, the trust debt is enormous. The Part IV discipline is a career-long one.
5. **Becoming the report machine** — letting the job collapse into producing the weekly pack. The pack is the floor, not the ceiling; the questions map keeps pointing at the ceiling, and 61–90 is when you claim it.

**From Your Toolkit — the first project is you:** notice what the ninety days actually are: the five moves of Chapter 45, run on yourself — *question* (what does this team need?), *data* (the terrain maps), *analysis* (the deliverables), *communication* (the coffees, the numbers-with-names), *checking* (the self-review, the traps audit). The book's final reframe: you are the dataset now — profile yourself honestly, clean your habits, ship your improvements, and reconcile your promises against your delivery.

## Key Takeaways

- Month 1: three maps (data, people, questions) and the first-hour liturgy on the main datasets; listen before fixing; ask "who would you ask if a number looked wrong?"
- Month 2: a visible recurring deliverable rebuilt with full craft and shipped early; then a one-week quick win with a stated before/after and shared credit.
- Month 3: systematise — runbooks a successor could run, automate-the-second-time, permanently answer one recurring question, and write your own 90-day review.
- Requests: clarify (half dissolve), negotiate sequence, and "let me check the data first" — the vending-machine reputation is the one to avoid.
- The five traps: tool peacocking, skipping reconciliation, going dark, caveat-free confidence, becoming the report machine — audit yourself monthly.

## Practice Lab

1. The 90-day plan, written: adapt this chapter's arc to the role you are hunting (Chapter 50's postings tell you the deliverables); one page, the three months, the measurable self-review dated.
2. The question kit: write your ten terrain questions (two per map — data, people, questions, requests, traps); memorise the three you will ask on day one; they are now yours in any job, forever.
3. The refusal rehearsal: script polite responses to the four classic trap requests; role-play with a friend playing an urgent stakeholder; note where you caved and rebuild the sentence.
4. The traps audit, on yourself: score yourself 1–5 on the five traps *today* (from your projects, study, or work history); write the one-line antidote for your two worst; schedule the monthly self-audit.

5. The self-review template: create the quarterly document now (what ships faster, what breaks less, what do they know now) and fill it once for your job-hunt itself — you have been running a 90-day project since Part I, and it deserves its review.

## Further Reading

- Chapter 55 (the freelance lane — the same 90-day logic, compressed into small paid engagements)
- *The First 90 Days* (Watkins) — the leadership classic; this chapter is its analyst translation

# Chapter 55: Freelancing and Side Income

---

## Part VIII — Getting the Job

"A freelance analyst is what you become the moment someone pays you for one project. The craft is making that moment deliberate, priced, and repeatable."

### In this chapter you will learn

- The honest economics of analyst freelancing — what sells, what doesn't.
- Finding the first three clients: the market that already surrounds you.
- Pricing small jobs: the three models, and the beginner's mistake with each.
- Delivering like a pro: scope, contracts-lite, and the handover.
- Building the side practice without wrecking the day job (or the ethics).

## 55.1 What Actually Sells

The honest market scan: strangers do not buy "data analysis" (too abstract); they buy **finished business outcomes that happen to be analytical**. What sells at small scale, in rough order of demand:

- **The spreadsheet rescue** — the broken workbook, the slow report, the VLOOKUP that stopped matching after someone inserted a column. (Part II, productised: diagnosis, rebuild, a walkthrough for the owner.)
- **The monthly-report automation** — exactly Project 3: "your three hours every month become ten minutes." Easiest sale in the market because the ROI is arithmetic.
- **The first dashboard** — small shops, NGOs, school associations: one Power BI/Metabase page on data they already collect. Part VI, miniaturised.
- **The survey and its analysis** — NGOs and researchers needing instruments, field protocol, and the stats: Part IV + Project 5, and in development-sector work this is a *listed procurement category* (MEAL consultancies).
- **The one-question analysis** — "should we stock X? did the price change hurt us? which branch is lying?" — Project-style scoped engagements on the client's own data.

What does *not* sell at small scale: machine learning, "insights", anything whose value you cannot state as a sentence with a number in it. If you cannot



tell the client what they will *have* at the end that they lack now, it is not a product.

## 55.2 The First Three Clients

You do not find the first clients; you *notice* them — they are the market that already surrounds you:

1. **Your current or former workplace** (if employed): the adjacent departments with spreadsheet pain. The internal side-practice: solve one visible problem in your own time on terms you negotiate (and clear with your employer — see 55.5).
2. **Your network's businesses**: the cousin's shop, the church's accounts, the friend's transport company, the NGO where someone you know volunteers. Your first paying engagement is one conversation away and probably already complains about its numbers. (Tariro's Grocery began exactly here — a relative's shop, a first free-ish project, a case study.)
3. **Small local orgs and associations**: schools (fees analysis), clinics (patient flows), farmer's cooperatives, savings clubs, church networks — organisations with real data and no analyst, reachable by one genuine conversation, priced accessibly (55.3).

Then the platforms — Upwork/Fiverr and their kin — as the *fourth* channel, with honest expectations: global competition on price, slow first jobs, but a real training ground for scoping and communication once you have three reviews. The portfolio (Chapter 49) is your profile there; lead with the wild-card real-data project.

The **first engagement, structured**: offer it deliberately cheap or free *by design, not by accident* — "I'll do this one for [small fee/free] because I'm building my practice in this niche; in exchange, may I use the (anonymised) case study and would you refer me if it's good?" — that is a marketing spend with a defined return, and it converts satisfied clients into the referral engine that becomes your pipeline.

## 55.3 Pricing

Three models, the beginner mistake with each, and the fixes:

- **Hourly** ("\$/hour"): the mistake — beginners price at *low* hourly to win, then discover that getting good makes each job pay *less* (faster = fewer hours). The fix: hourly only for open-ended diagnosis ("exploratory, capped at N hours"), priced at your day-job equivalent  $\times 1.5$  minimum (self-employed carries its own costs), and always with a cap stated.

- **Fixed per deliverable** ("300 for the automated monthly report"): the mistake — scoping by optimism ("it's just a pivot!") and eating the overrun. The fix: fixed price *only after* a paid or free scoping look at the actual data (the first hour of Chapter 44 is the scoping product), with the deliverable written as a checklist and a round of revisions counted.
- **Value/retainer** ("80/month: the dashboard kept fresh, questions answered, quarterly review"): the mistake — never offering it, leaving recurring value on the table. The fix: propose the retainer at handover of every successful project ("what happens next month when the data changes?") — recurring revenue is the freelancer's compounding asset, exactly like Chapter 62's career habit.

The universal beginner error: **under-pricing to win**, which attracts the worst clients (price-shoppers churn and complain), devalues the work, and sets an anchor you will spend a year raising. The counter-intuitive truth of small-market freelancing: at accessible-but-confident prices, with a portfolio and a scoping conversation, you *close more* than at desperation prices — because the price signals the seriousness. Start at the price that makes you slightly uncomfortable; raise it every third engagement.

## 55.4 Delivering Like a Pro

The craft of the small engagement, all of it this book's habits compressed:

- **The one-page scope** — not a contract; a plain agreement: the question, the deliverable checklist, the timeline, the price, what is *not* included ("data cleaning beyond X hours is separate"), and the revision policy. Both parties keep a copy. This single page prevents 90% of freelance misery (the other 10% is scope creep the client genuinely didn't intend — handled by "happy to; that's outside our scope page; I'll send the add-on price today").
- **The working discipline** — the anatomy (Ch. 45) runs every engagement: question written, loading checks, the analysis, caveats. Small clients get *the same rigour* as the book's projects — that is precisely why they keep you.
- **The handover** — deliver the artefacts *plus the runbook* (how to refresh, what the checks mean, what to do when something breaks), a 30-minute walkthrough (Chapter 43's five-minute format, expanded), and the retainer offer. The client who can run it without you and *still* keeps you — that is the relationship worth building.
- **The paper trail** — an invoice for every engagement (even the free one, marked "no charge — case-study arrangement"), a folder per client, a simple ledger (engagement, dates, fee, hours, margin, referral source). The ledger is your pricing data: after six engagements, *your own numbers* tell

you your real hourly rate and which work earns it — the analytics habit, applied to the practice itself.

## 55.5 The Ethics and the Day Job

The two clearances that keep the side practice safe: **your employment terms** — many contracts require disclosure or limit outside work; ask, in writing, before the first paid engagement; the conversation is normal and the alternative is a career hazard. And **your clients' data** — the Chapter 47 conscience, now contractual: minimum data (collect only what the engagement needs), explicit permission for any case study (anonymise to the bone), no client data on personal unsecured devices, and no work you cannot defend (if a client wants you to "find the numbers that justify the decision" — the request will come — that is a refusal, politely: "I can find what the data says; I can't decide what it says first"). The freelancer's brand is the honesty rules; there is no employer's reputation to hide behind, only yours.

**From Your Toolkit — the practice is the book, sold:** every freelance product in this chapter is a Part of this book wearing a price tag: the spreadsheet rescue (II), the query pack (III), the tested finding (IV), the automation (V/37), the dashboard (VI), the survey (47). You are not learning a new trade — you are packaging the one you have, and the packaging (scoping page, ledger, handover runbook, retainer) is the only genuinely new craft here.

## Key Takeaways

- Sell outcomes, not analysis: rescues, automations, first dashboards, surveys, one-question projects — if the value has no sentence with a number, it has no market.
- First clients are your surroundings: workplace (cleared), network's businesses, local orgs — then platforms as channel four; structure the first engagement as deliberate marketing with a case-study return.
- Price: hourly (capped,  $\times 1.5$ ) for diagnosis; fixed only after scoping; always offer the retainer at handover — and never under-price; start slightly uncomfortable, raise every third job.
- Deliver with the one-page scope, the full rigour, the handover runbook, and the ledger — your own numbers become your pricing analytics.
- Clear the day-job terms in writing; take minimum data; anonymise case studies; refuse the "find me the right numbers" request — the honesty rules are the brand.

## Practice Lab

1. The product list: write your three sellable products as one-paragraph offers (outcome, deliverable checklist, timeline band, price band) — the spreadsheet rescue, the automation, the dashboard; these paragraphs become your profiles and your pitch.
2. The market scan: list ten real prospects in your surroundings (workplace-adjacent, network, local orgs), each with the product that fits and the one person you would talk to; mark the three warmest.
3. The scope page, drafted: take your strongest product and write its one-page scope agreement template — question, checklist, exclusions, revisions, price blank; keep it; you will use it.
4. The pricing rehearsal: price your automation product three ways (hourly capped, fixed post-scope, retainer) with your real numbers from the project ledger (hours it took you, honestly); note which model you would defend and why.
5. The first pitch, made: approach one warm prospect this week with a specific offer ("I can turn your monthly [X] into a ten-minute job — a look at the data is free; the build is [price]"); log the outcome whatever it is — the pipeline metric, started.

## Further Reading

- Chapter 56 (the ladder — where the practice and the career combine), Chapter 61 (the \$0 tool stack that keeps overhead at zero)
- *Company of One* (Jarvis) — the small-on-purpose mindset, friendly to analyst practices

# Chapter 56: The Career Ladder

---

## Part VIII — Getting the Job

"The ladder is not a ladder. It is a widening of the question — from 'what did the data say?' to 'what should we ask?' to 'what should we never have done?'"

### In this chapter you will learn

- The stages: junior → analyst → senior → lead/principal — what actually changes.
- The two ladders: management vs craft (IC), and choosing honestly.
- Specialising: the domains and niches, and when to commit.
- Staying current cheaply: the filter that beats the feed.
- The compounding career: what the second decade looks like from the first.

## 56.1 The Stages

What changes at each step — the honest version, from the job's inside:

- **Junior analyst (year 0-1):** given questions, told the data. Success = trustworthy execution: the reconciliation habit, shipped work, stakeholders who say "easy to work with". Your 90-day chapter is the whole syllabus.
- **Analyst (1-3):** given *goals*, find the questions. Success = ownership: you scoped the engagement yourself, pushed back on the fuzzy brief, chose the method, defended the caveat in the meeting. The Chapter 45 anatomy is now yours to run unsupervised.
- **Senior (3-6):** given *problems*, find the goals. Success = judgement: you know which of the five requests matter; your estimates are trusted before the data arrives; you design the experiments, not just run them; juniors get better because of you. The tell that you are here: people ask you "*should* we look at this?" rather than "*can* you?"
- **Lead/Principal (6+):** given *directions*, find the problems. Two forms — the **manager's** path (people, hiring, priorities, the team's output as your product) and the **principal's** path (the organisation's hardest analytical problems, methodology, the standard others follow). The compensation flattens between them; the work is simply different, and the choice is about *you*, not status.

The mechanics of moving up, identical at every step: **scope you can prove** (the self-review document of Chapter 54, kept quarterly, becomes the promotion case — "here is what I now own that I didn't"), a

**successor-shaped gap** (you get promoted into bigger work when your current work survives without you — the runbooks and handovers pay off here, literally), and **sponsorship** (a senior person who has *seen* your work argue for it in the room you are not in — which is why credit-sharing and stakeholder craft are career skills, not manners).

## 56.2 Two Ladders, One Honest Choice

Somewhere around senior, the paths fork, and pretending they don't is how careers stall: **management** — people, plans, hiring, the meeting-heavy calendar; your output becomes the team's output, and the craft skills convert into coaching and judgement calls. **Craft (IC)** — deeper specialisation, the hardest problems, methodology ownership; your output remains the analysis, at higher stakes and wider scope. Both are careers, both pay, and the honest test is where your energy goes: a day of one-on-ones and roadmaps that ends with you energised → management; a day of stakeholder wrangling that ends with you longing for the notebook → craft. (The test is not which you are *good* at — by senior you are good at both, that is what senior means.) And the choice is reversible for a while, then hardens; try the adjacent path *inside* your role first (mentor a junior for a quarter; lead one project) before you bet a title on it.

## 56.3 Specialising

The generalist stack of this book is the *entry* ticket; somewhere in years 2–5, a specialisation compounds faster than breadth. The honest map of niches — each a combination of *domain* (retail, health, finance, development/NGO, telecoms) and *function* (pricing, churn/retention, experimentation/A-B, fraud/risk, supply chain, survey research, data visualisation, analytics engineering):

- Choose by **accumulated advantage**: the domain you keep ending up in, the stakeholder questions you answer best, the projects in your portfolio that generated the follow-on work. Specialisation chosen from evidence beats one chosen from lists — it is a Chapter 28 argument about yourself (mechanism, not vibes).
- The **development-sector niche** deserves a flag for this book's readers: MEAL (monitoring, evaluation, accountability, learning) is a career *and* a consultancy market where Part IV's statistics, the survey craft, SPSS/Stata, and honest reporting against logframes are the daily trade — and where a freelancer-to-consultant ladder (Chapter 55's practice, grown) is a well-worn path.

- **When NOT to specialise:** before you have seen three domains. Early over-commitment (the "I only do churn in fintech" of year one) is betting the career on one hand you haven't been dealt.

## 56.4 Staying Current, Cheaply

The field's firehose is a manufactured anxiety; the professional's filter is small and stable: **one weekly newsletter** (a general data one — e.g. a widely read analytics or data-engineering digest — not five), **one community** (a local analytics meetup or an online forum where you *answer* as well as ask — answering is where learning compounds), **the release notes of your actual stack** (Power BI monthly updates, pandas releases, your warehouse's changelog — thirty minutes a month, because the tools you use daily are the ones whose changes matter), and **one deliberate annual stretch** (a new tool, a small course, a conference — chosen in December against the year's gap list, not joined in January from guilt). Everything else — the daily posts, the "you're behind" content economy — is someone else's funnel; the Chapter 2 questions, applied to the feed itself: *says who, compared to what, what's missing, is it even true?*

## 56.5 The Compounding Career

What the second decade looks like, so the first is spent well: skills compound — the reconciliation habit that made you trustworthy as a junior makes your estimates trusted as a senior; the runbooks that got you promoted become the data culture you eventually lead. Reputation compounds — the analyst who says "let me check the data" in 2026 is the consultant whose word settles arguments in 2032. And the portfolio compounds — Chapter 49's shelf, kept alive, becomes the speaker bio, the consulting credential, the book of case studies (some analysts literally publish theirs; you have, in a sense, read one). The one-sentence version of this entire chapter, and of this Part: **the career is a dataset with a long tail — collect honestly, reconcile often, and let the compounding do what compounding does.**

**From Your Toolkit — the long game:** the ladder's stages are the anatomy's moves at increasing scope (execute → own → judge → direct); the two ladders are the tool-choice chapter (Ch. 35's audience rule: management serves the team, craft serves the question); specialisation is a regression on your own history; staying current is the first-hour discipline applied to the field. This book has one method; the career is where you get to run it for decades.

## Key Takeaways

- The stages widen the question: juniors execute, analysts own, seniors judge, leads direct — and promotion mechanics are provable scope, a successor-shaped gap, and sponsorship.
- The fork is real (management vs craft/IC); test adjacent inside the role before betting a title; choose by where energy goes, not talent.
- Specialise from accumulated advantage around years 2-5 (domain × function); MEAL is a real ladder for this book's stack; not before three domains seen.
- Current cheaply: one newsletter, one community (where you answer), your stack's release notes, one deliberate annual stretch — the feed is a funnel; audit it.
- The career compounds like a long-tail dataset: habits → reputation → portfolio → culture; collect honestly, reconcile often.

## Practice Lab

1. The stage map: write your current stage and its success definition in your own words, then the two behaviours that would evidence the *next* stage within six months — the promotion case, started early.
2. The fork test, taken honestly: log your energy for one week (days that ended energised vs drained, and what those days contained); write the one-paragraph verdict; note the adjacent-path experiment you will run first.
3. The specialisation regression: list every project you have touched (this book's and life's); tally domain and function; write the "accumulated advantage" reading — what does the evidence say you keep becoming?
4. The current-stack budget: choose your one newsletter, one community, and stack release-notes half-hour now; calendar the annual stretch decision for December; unsubscribe from the rest this week — the filter, installed.
5. The decade letter: write one page to yourself at year ten — what the compounding should have produced; seal it in the portfolio repo (a file named `letter-to-year-ten.md`); the commit message is part of the artefact.

## Further Reading

- Part IX (the maps beyond: ML, big data, ethics, the free stack, your decade)



- *So Good They Can't Ignore You* (Newport) — the craft-compounding argument, full length

**PART IX**

# **Beyond the Basics**

Chapters 57 – 62

---

# Chapter 57: A Gentle Map of Machine Learning

---

*Part IX — Beyond the Basics*

"Machine learning is not a new career. It is your regression chapter, fed more data, pointed at prediction instead of explanation."

## In this chapter you will learn

- What machine learning actually is — the one-paragraph demystification.
- The vocabulary map: supervised/unsupervised, classification/regression, training/testing.
- The three ML questions a shop like Tariro's actually has.
- When you need ML and (mostly) when you do not.
- The learning path from this book's base — without the course-industrial complex.

## 57.1 The Demystification

Strip the mystique: **machine learning is fitting a function to data so it can make predictions on new data.** You fit a function in Chapter 28 — a line through oil prices and bottles; give it a new price, it predicts bottles. That *was* machine learning in embryo: a model (the line), parameters (slope, intercept), fitted (least squares), used for prediction. ML is the same skeleton with (a) models flexible enough to capture curves and interactions a straight line cannot, (b) data big enough to justify them, and (c) a culture obsessed with proving predictions work on data the model hasn't seen. Nothing in this sentence requires a PhD; it requires the discipline you already have — plus one new idea, next section, which is the culture's core.

Why "learning"? Because the parameters are not written by a human (nobody typed the slope); they are *estimated from data* — the machine "learned" them. The honest framing: it is statistics, industrialised, pointed at prediction — and its dangers are your book's dangers, amplified: overfitting (the model that memorises noise — Chapter 26's p-hacking, at scale), data leakage (the test sneaking into the training — a broken reconciliation), and the causation trap wearing a robot suit.

## 57.2 The Vocabulary Map

The terms, translated into chapters you own:

- **Supervised learning** — the model learns from *labelled* examples (past sales with their outcomes; past customers with their churn outcomes). Two flavours: **regression** (predict a number — next month's revenue; Chapter 28's line, generalised) and **classification** (predict a category — will this customer churn: yes/no; the t-test's question, answered per-customer).
- **Unsupervised learning** — no labels; find structure. **Clustering** (customers who resemble each other — the segmentation your cross-tabs did manually, automated), **dimensionality reduction** (many columns → few, losing little — the `describe()` instinct, compressed).
- **Training/testing split** — hide part of the data; fit on the rest; grade against the hidden part. *The single most important practice in the field*, and it is Chapter 25 in costume: the test set is the sample of the future, and grading on data the model has seen is evaluating a student on the exam they were allowed to memorise.
- **Features** — the input columns (Chapter 33's derived columns — the feature shelf you built *is* feature engineering).
- **Overfitting / underfitting** — too flexible (memorised noise) vs too rigid (missed the pattern): the bias-variance trade-off, which is "which typical?" (Ch. 22) applied to models.
- **Cross-validation** — the train/test idea rotated several times, so the grade doesn't depend on one lucky split: the bootstrap's cousin (Ch. 36).
- **The metrics** — for classification: accuracy (misleading on imbalanced data — 99% accuracy by predicting "no churn" always, when 1% churn), precision/recall (the trade-off that matters), the confusion matrix (a cross-tab of predicted vs actual — literally Chapter 27's table). For regression: RMSE (typical size of the miss — standard deviation's costume), MAE (the median's costume).

### 57.3 Tariro's Three ML Questions

The shop's genuine ML-grade questions, each mapped to its method and its trap:

1. **"Which customers will churn next month?"** — classification on Project 4's data: features = recency, frequency, tenure, basket trend (the columns you built); label = churned-in-the-following-month; method = logistic regression (the honest, interpretable start) or a random forest (more power, less interpretable). The traps: class imbalance (few churners — accuracy lies; use precision/recall), and *leakage*: if a feature knows the future ("days since last purchase" computed *after* the prediction date), the model scores brilliantly and predicts nothing. The professional's question

about any impressive ML result: *what did the features know, and when?*

2. **"How much will we sell next week?"** — regression/time-series forecasting: next week's revenue per item from lags and seasonality (your `shift/pct_change` features, Chapter 34). Traps: the split must respect *time* (train on the past, test on the future — random splits leak tomorrow into yesterday), and the baseline to beat is embarrassingly strong: "same as last week" wins more often than fancy models deserve. Always report the naive baseline beside the model's.
3. **"What kinds of shoppers do we have?"** — clustering on customer features (basket size, frequency, category mix, weekday pattern): the segments your eye found in cross-tabs (the humps of Chapter 23!) found automatically. Trap: clusters are *descriptive* — "segment 3" needs a human to name it ("Saturday bulk buyers") and a decision to serve it; the algorithm finds the groups, the analyst finds the meaning.

## 57.4 When You Need It (Mostly You Don't)

The professional's filter, applied honestly: you need ML when **(a) the question is genuinely predictive** (a per-entity forecast that recurs: churn, demand, risk), **(b) there is enough labelled data** (hundreds of examples minimum, realistically thousands), **(c) the pattern is too complex for rules and regressions you can read**, and **(d) a small improvement is worth real money** (1% better demand forecast at scale = real money; 1% better understanding of 18 months of one shop = a pivot table). Most business questions — *what happened, why, what should we do* — are this book's questions, and a t-test with a caveat beats a black box with a dashboard. The analyst who reaches for ML last, not first, is the senior one; the junior who reaches for it first is announcing tools before questions (Chapter 54's first trap, at scale).

## 57.5 The Learning Path

From this book's base, the honest sequence (months, not weekends): (1) **consolidate** — pandas, scipy, regression fluently (Parts V-VI plus Chapter 28; you have this); (2) **scikit-learn fundamentals** — the `fit/predict` API, train/test splits, the metrics, on *your own* project data (churn on Project 4 is the perfect first dataset — labelled, meaningful, small enough to understand every error); (3) **one course or book, done properly** — the classic applied-ML courses (Andrew Ng's; the *Hands-On Machine Learning* book; fast.ai for the top-down temperament) — *one*, finished, with notes, not five, sampled; (4) **one project, shipped to the portfolio** with the honesty section the field respects most: the baseline comparison, the leakage audit, the error analysis (which customers does the model get wrong — and yes, you will find it over-predicts churn for the founding cohort, because class balance shifts over time; that finding is worth more than the model). Then decide — Chapter 56's fork has a data-science branch, and this chapter is its trailhead.

**From Your Toolkit — the constant is the honesty:** every ML practice is a habit you own wearing new clothes: the train/test split is sampling discipline; leakage is the broken reconciliation; the baseline is "compared to what?"; the metrics debate is "which typical?"; the error analysis is the drill-down. The models change; the analyst's conscience — check it, reconcile it, caveat it — is the same instrument, and it is rarer in ML than it should be, which makes it *your edge*.

## Key Takeaways

- ML = fitting a function to data for prediction on new data; your Chapter 28 line was the embryo; the culture's core is grading on unseen data.
- Vocabulary: supervised (regression/classification) vs unsupervised (clustering); features = your derived columns; metrics are your statistics in costume — accuracy lies on imbalanced data.
- Tariro's three: churn (classification; leakage and imbalance), demand (time-series; the naive baseline), segments (clustering; humans name the groups).
- The filter: predictive question + enough labels + complexity beyond readable models + money in the margin — most questions are this book's questions.
- Path: consolidate → scikit-learn on your own churn data → one course finished → one shipped project with the honesty section; then the fork, knowingly.

## Practice Lab

1. The vocabulary bridge, written: define each ML term of 57.2 in one line *and* its Chapter costume in a second line; keep the two-column table — it is your interview answer for "do you know ML?" (the honest answer: "the fundamentals, from statistics up").
2. The leakage hunt, hypothetical: for the churn model, list five features and audit each — which could know the future? Rewrite the leaky ones as of prediction-date; write the one-sentence audit rule you just invented.
3. The baseline drill, on real ground: forecast last month's revenue with "same as last month" and "same weekday average"; compute the misses; that number — the one every fancy model must beat — goes in your log with its formula.
4. The clustering, run lightly: k-means on customer features (three lines of scikit-learn in the notebook: `KMeans(n_clusters=4) → fit → label` the customers); plot the segments; *name* each cluster in business words; write the decision each name suggests — the human half of unsupervised learning.
5. The path, chosen: write your ML sequence with dates (consolidate-by, course-by, project-by); the project is Project 4's data with the error analysis as the deliverable; commit the plan to the portfolio repo as `ml-path.md`.

## Further Reading

- Chapter 58 (the other side of "beyond": big data), *The StatQuest Guide to Machine Learning* (Starmer) — the friendliest correct map in print
- scikit-learn's own tutorials — start with the "getting started" page, not the API

# Chapter 58: A Gentle Map of Big Data

---

## Part IX — Beyond the Basics

"'Big data' is not a size. It is the moment your laptop stops being the right tool — and that moment arrives later than the brochures claim."

### In this chapter you will learn

- When data is *actually* big: the honest thresholds, in laptop terms.
- The scaling ladder: what changes at each rung, tool by tool.
- The cloud, in plain words — and what it costs to know it.
- The modern data stack you will inherit at work: named and demystified.
- The skill that matters most in the big-data world — and you have it.

## 58.1 The Honest Thresholds

Numbers, not vibes. A laptop with 8-16 GB RAM handles, comfortably: **spreadsheets to ~1 million rows** (which is a spreadsheet's hard ceiling and past its comfort), **pandas to a few million rows** (tens of millions with the memory tricks: dtypes, chunking, only-needed-columns), **SQLite to gigabytes, a local Postgres to tens of gigabytes** before you notice. For calibration: 18 months of Tariro's shop is ~6,400 rows; a mid-size retailer's transactions, ~10-100 million; a national bank's card authorisations, ~billions; a social platform's events, ~trillions *per day*.

The honest mapping onto careers: **most analyst jobs sit in the thousands-to-millions band** — the laptop band, this book's band — and the tools that matter there are exactly yours (SQL against a warehouse, pandas where it fits, BI on top). "Big data" proper — the distributed kind — is a *specialist* world (data engineering, platform teams) that most analysts touch only through its interface: a query box. The professional's rule: **scale problems are diagnosed, not assumed** — measure the data (rows × columns × bytes), compare with the machine, and only then choose the tool. The beginner who says "we need big data tools" about a 400,000-row CSV has announced tools before questions; the file is 40 MB and fits in RAM eleven times over.

## 58.2 The Scaling Ladder

What actually changes as data grows — the rungs, and the move at each:



1. **Laptop ( $\leq$  a few million rows):** everything in this book, unchanged.  
When pandas stumbles *before* the data is truly large: read only needed columns, downcast dtypes (`int64`→`int32` halves memory), process in chunks, or aggregate in SQL first and analyse the summary (the "aggregate, then join" instinct, scaled).
2. **A database server (tens of GB):** Postgres/SQL Server on a machine that is not yours — already the reality of most jobs. Nothing changes but the connection string; your Part III SQL is the whole skill.
3. **The warehouse (hundreds of GB to petabytes): BigQuery, Snowflake, Redshift, Databricks** — databases that live in the cloud, store data in columns (analytics-friendly), and scale by *renting* more machines per query. The changes that matter: SQL dialect quirks (Chapter 20's table), **cost per query** (you are now paying rent by the terabyte-scanned — the sargability chapter becomes a *budget* chapter), and table *partitioning* (data pre-sliced by date so your query reads March, not everything). The analyst's warehouse skill is: SQL that respects the bill.
4. **The distributed world (truly big):** Spark and its kin — thousands of machines, one computation. The analyst rarely writes Spark itself; you meet **Databricks notebooks** (pandas-like APIs on big data) or the lakehouse tables underneath a warehouse. Know the name, read the lineage, let the engineers own the tuning.

The meta-lesson of the ladder: **each rung changes the infrastructure, not the analysis**. GROUP BY at 10 rows and at 10 billion is the same idea; the joins, the reconciliations, the caveats — identical. The ladder climbs under you; the method rides on top unchanged.

### 58.3 The Cloud, in Plain Words

Demystified in one paragraph: "the cloud" is **renting computers by the hour** — AWS, Azure, GCP are hotels for computation: you check in (upload/store), use rooms (compute), check out, pay for what you used. Nothing mystical runs there; the same Postgres, the same Python, the same Power BI — in someone else's building, at scale, with a bill. What the cloud *adds* for analysts: warehouses that fit any data (58.2's rung 3), scheduled pipelines without a server under your desk (Chapter 37's cron, hosted), and integrations (Power BI talks to Azure as natively as it talks to a CSV). What it *costs*: money that scales with use — hence the one skill every cloud-age analyst needs and almost none are taught: **reading the bill** (which queries scanned what, which schedule ran wild), which is this book's reconciliation habit pointed at money. The certification note (Chapter 51's "only exams" rule): cloud *fundamentals* certificates (AWS Cloud Practitioner, Azure Fundamentals) are cheap, real, and signal seriousness for cloud-heavy markets — useful, optional, never a substitute for the portfolio.

### 58.4 The Stack You Will Inherit

Named, so the job's first week reads familiar. The modern data stack, in pipeline order, with each layer's incumbents and its chapter:

SOURCES	tills, apps, CRM, spreadsheets	(Ch. 5's world, scaled)
INGESTION	Fivetran/Airbyte (connectors)	(the pipelines, bought)
WAREHOUSE	Snowflake / BigQuery / Redshift	(Ch. 16's engine, rented)
TRANSFORM	dbt – SQL, versioned, tested	(your query packs, grown up)
ANALYTICS	Power BI / Tableau / Looker	(Part VI)
EXPERIMENT	A/B platforms, feature flags	(Ch. 26 industrialised)
GOVERNANCE	lineage, catalogs, access control	(Ch. 20's audit trail, as product)

The layer worth a paragraph is **dbt** ("data build tool"): analysts writing SELECT-statements-as-modules — your CTEs and query packs, but version-controlled, tested, and documented *as the standard practice* — and its rise is the clearest career signal in the modern stack: the industry has decided that the analyst's craft is code (SQL, versioned, tested — exactly Part III's discipline), not spreadsheets exchanged by email. **Analytics engineering** (Chapter 50's title table) is the job that owns that layer, and it is the most natural senior-path for analysts who loved Part III.

## 58.5 The Skill That Matters Most

Ask the people who run big platforms what limits analysts there, and the answer is not distributed-computing theory: it is **asking the right question of the right table and knowing when the answer is wrong** — the anatomy, the reconciliation habit, the caveats. At scale, errors scale too: the fan-out that doubled one shop's revenue doubles a billion-row aggregate just as silently, and nobody at warehouse scale eyeballs every table — the *checks* are all that stand between a query and a wrong board report. The rarest big-data skill is the one this book drilled 62 times: **run the liturgy, state the limits**. Everything else — Spark, the cloud, the stack — is learnable in weeks from that base; the base is the years.

**From Your Toolkit — one method, every rung:** the ladder's truth deserves its final restatement: your five moves, your liturgy, your three honesty rules are *scale-free*. A million rows or a trillion, the analyst adds the same value — trust. The brochures sell infrastructure; the career sells the conscience. You have spent a book acquiring the scarcer asset.

### Key Takeaways

- Big data is diagnosed, not assumed: laptop band = most jobs (thousands-millions); measure rows × bytes before claiming a tool.
- The ladder: laptop → database server → warehouse (dialects, cost-per-query, partitioning) → distributed; infrastructure changes, analysis does not.
- The cloud is rented computers by the hour; the analyst's cloud skill is SQL that respects the bill — reconciliation, pointed at money.
- The inherited stack: sources → ingestion → warehouse → dbt (your query packs, grown up) → BI → experiments → governance; analytics engineering is Part III's natural senior path.
- The scale-free skill is the rarest one: the question, the checks, the limits — trust at any size.

### Practice Lab

1. The size audit: for each dataset you own (Tariro's four, your wild card, any work data you may touch), compute rows × columns × approx bytes; place each on the 58.1 ladder; write the tool decision each placement commands — the diagnosis habit, practised on your own shelf.
2. The memory lab: load your biggest CSV in pandas; print `info(memory_usage="deep")`; downcast and select columns; measure the

savings; write the three lines you would teach a colleague.

3. The warehouse dialect, skimmed: take three of your saved queries (Ch. 20's professional file) and rewrite for BigQuery or Snowflake dialect (LIMIT→FETCH, date functions); note each change in the dialects file you started in Chapter 20.
4. The dbt hour: install dbt (free, local), point it at your SQLite or a DuckDB file, and convert two of your monthly-pack queries into dbt models with a test (`unique`, `not_null` on the key); the concept lands in ninety minutes — and the résumé line becomes honest.
5. The cost instinct, built: in the warehouse of your choice's free tier (BigQuery's sandbox is free), run a query with and without a date-partition filter on a public dataset; read the bytes-scanned in each job's stats; write the one-sentence rule you will carry to any billed warehouse.

## Further Reading

- Chapter 59 (ethics — the other thing that scales), Chapter 61 (the free versions of this whole stack)
- *Fundamentals of Data Engineering* (Reis & Housley) — the pipeline book, if rung 4 called to you

# Chapter 59: Ethics and Privacy, Essentials

---

## Part IX — Beyond the Basics

"The ethics chapter is not at the end because it is a conclusion. It is at the end because by now you can understand every example."

### In this chapter you will learn

- The five principles that govern working analysts — short enough to memorise.
- Privacy law's common core: consent, purpose, minimisation — across borders.
- Bias: where it actually enters (data, measurement, modelling) and what you owe.
- The honest-presentation duties: cherry-picking, dark patterns, the numbers that harm.
- The refusal list — the work you decline, and how to decline it.

## 59.1 The Five Principles

Data ethics as practiced — not as a philosophy seminar, but as the working rules that keep people safe and analysts employable:

1. **Consent and legitimate use** — people's data is used with permission or another lawful, honest basis; "we have it, so we may" is not a basis.
2. **Purpose limitation** — data collected for X is used for X (Chapter 47's rule, generalised: the survey about Sundays is not a marketing list).
3. **Minimisation** — collect and keep the least data that answers the question; every column is a liability as well as an asset.
4. **Honesty in representation** — the analysis says what the data says, with its limits attached (Parts I-VIII's discipline, now a *duty* and not just a craft).
5. **Do no predictable harm** — before shipping, ask who could be hurt by this number being wrong *or right*: the segment it profiles, the worker it schedules to zero hours, the neighbourhood it redlines.

These five are the common core of every serious framework — from the ACM's code to corporate policies to law (next section). Memorise them; they compress to: *used with permission, for the stated purpose, the least of it, told honestly, harming no one.*

## 59.2 Privacy Law's Common Core

You will meet data-protection law by name: **GDPR** (Europe — and the world's de facto template), **POPIA** (South Africa), Zimbabwe's **Cyber and Data Protection Act**, and their siblings. The names differ; the skeleton is the same five principles, in legal costume, plus operational rights you must be able to recognise when a stakeholder asks: **access** ("show me the data you hold on me"), **rectification** ("correct it"), **erasure** ("delete it" — the right to be forgotten, with its exceptions), **portability**, and **objection**. The analyst-relevant realities, in practice: personal data has a *lawful basis* or it does not get processed; there are **special categories** (health, religion, ethnicity, sexuality, biometrics) where the bar is higher and the purposes narrower; **anonymisation** that actually works is harder than it looks (a "anonymised" dataset of postcode + birthdate + purchases re-identifies embarrassingly often — aggregation and k-anonymity thinking exist because naive masking fails); and *someone in the room is responsible* — increasingly, by name, with training and sign-off. The career-craft version: the analyst who can say "this needs a lawful basis check before we build it" is not the blocker in the room; they are the one saving the team from the conversation nobody wants with the regulator.

## 59.3 Where Bias Enters

Bias, in the analyst's terms, is Chapter 25 wearing consequences. It enters at three doors, and the duties differ at each:

- **Data bias** — the sample is not the population: the survey that only saw in-shoppers (Ch. 47, stated), the hiring data of a biased past, the fraud data that only caught the fraudsters your current rules found. Duty: *know your sampling frame, declare its limits, and never let the quiet omission (who is missing?) pass unexamined* — the missing are rarely missing at random.
- **Measurement bias** — the instrument distorts: the churn definition that reads poverty as disloyalty (customers with less money buy less often — a 60-day silence rule flags the budget before the loyalty), the satisfaction survey only the satisfied answer, the crime statistics that measure policing, not crime. Duty: *interrogate what the number actually measures* — the "says who?" question, asked of the metric itself.
- **Modelling bias** — the model amplifies the past into the future: trained on biased history, it recommends the bias (the classic: résumé screens that learned to discount women because history did). Duty at analyst scale: *test the model's errors by group* (the error analysis of Ch. 57, disaggregated — if it fails more for one segment, say so loudly), and never let a proxy (postcode for income, purchases for health) launder a sensitive attribute

through an innocuous column. The statistical vocabulary you own — base rates, false positives, selection — is precisely the vocabulary in which bias is visible; that is no coincidence.

## 59.4 Honest Presentation

The communication duties, beyond the craft you already practise: **no cherry-picking** — the window, the segment, the metric chosen because it flatters (Chapter 11's truncated axis and Chapter 26's twentieth test were rehearsals; at work, the choice of *which* finding to show is the same crime, committed with a spotlight); **no dark analytics** — the dashboard engineered to hide the declining line, the "average" chosen because the median was worse, the chart that resets its axis per quarter; and the **harms of accuracy** — the number that is *right* and shouldn't be shown: the individual-level productivity ranking (statistically fragile, humanly corrosive), the neighbourhood risk score, the list of "likely pregnant shoppers" (a real, notorious case — correct prediction, indefensible use). The analyst's question before shipping anything about people: *would I be comfortable explaining this chart, to the people in it?* — the Chapter 2 questions, finally pointed at yourself.

## 59.5 The Refusal List

The work you decline, stated now — while it is abstract — so you can recognise it when it arrives wearing a deadline and a job title: findings pre-decided ("we need the numbers to show X" — you can find what the data says; you cannot decide what it says first); deception by omission (the report that leaves out the disconfirming segment); surveillance beyond consent (the tracking whose subjects don't know); profiling of the vulnerable (predicting pregnancy, sexuality, illness — or pricing against desperation); and the re-identification of data you were told was anonymised ("just join it with this public list"). The refusal craft — because careers are built on *how*: offer the honest alternative ("I can't build a case for a pre-decided finding, but I can test whether the data supports it — and if it doesn't, you'll want to know before the board does"), escalate once, in writing, and document your dissent politely. The one-sentence spine, worth its weight in decades: **the analyst's signature is the reason anyone believes the number — guard what it is worth.**

**From Your Toolkit — the conscience is the career:** every principle here is a chapter you already lived, generalised: consent (47), minimisation (44's licensing and 47's minimum-data), honesty (all of it), the missing-data question (8, 25, 60), the error analysis (57). The tools will change employers; the ethics travel with the signature — and in a decade of practice, they are what the reputation is made of (Ch. 56's compounding, at its longest horizon).

## Key Takeaways

- Five principles: permission, purpose, minimisation, honesty, no predictable harm — the common core of codes and law alike.
- Privacy law's skeleton is those principles plus the data-subject rights; special categories are a higher bar; naive anonymisation fails — someone is responsible by name.
- Bias enters three doors (sample, measurement, model); duties: declare the frame, interrogate the metric, disaggregate the errors — and watch the proxies.
- Presentation duties: no cherry-picking, no dark analytics, and the question "could I explain this to the people in it?"
- The refusal list and craft: offer the honest alternative, escalate once in writing, document — the signature is why anyone believes the number.

## Practice Lab

1. The principles audit, on your own shelf: run Projects 1–6 against the five principles; write the two-paragraph ethics section your portfolio README deserves (Ch. 49 promised one; here is its content).
2. The re-identification experiment: take your customers file; "anonymise" it by dropping names; then show that suburb + join-date + rough spend identifies specific individuals (join against a hypothetical public list — reason it through); write the k-anonymity note: how many people share each combination?
3. The measurement-bias gallery: find three metrics in your own work (or public life) that measure the instrument rather than the thing (school league tables, response-time metrics, engagement); for each, one sentence on what it really measures.
4. The disaggregation drill: on the churn project's error analysis, split the model's misses (or the rule's flags) by suburb and tier; report honestly whether the rule fails unevenly; write the one-line caveat the report needs.



5. The refusal rehearsal: script your response to the two requests most likely in your first year ("can you make the dashboard show growth?" and "just pull the numbers that support the proposal"); role-play both; keep the scripts — the wording, rehearsed, is what holds under deadline pressure.

## Further Reading

- Chapter 60 (messy reality — where ethics meets deadlines), *Weapons of Math Destruction* (O'Neil) — the essential book on this chapter, no prerequisites
- Your jurisdiction's data-protection act — read the principles section once; you will recognise every line

# Chapter 60: Messy Reality — Missing Data, Biased Samples, Deadlines

Part IX — Beyond the Basics

"The book's data was planted so every method could find its answer. This chapter is about everything that happens when nobody planted anything."

## In this chapter you will learn

- Missing data, properly: the three mechanisms, and what to do for each.
- Working under deadline: the 80/20 delivery that survives scrutiny.
- The stakeholder realities: changing questions, missing owners, politics.
- The audit of reality: a checklist for the messy-project archaeology.
- How to keep learning from projects that go wrong.

## 60.1 Missing Data, Properly

Chapter 8 dropped rows and logged it; Chapter 15 counted NULLs; Part IV said "the missing are rarely missing at random" — now the full theory, compressed to what analysts need. The three mechanisms (Rubin's taxonomy, demystified):

- **MCAR** (missing completely at random) — the reason is pure chance (a till crashed Tuesday afternoon). Rare, and the least harmful: complete-case analysis (drop the missing) is *unbiased*, just smaller. What you did all book, and now you know when it is safe.
- **MAR** (missing at random — misleading name: missing *given* other data) — the reason is visible in your other columns: EcoCash sales missing more on Sundays; survey income missing more among older respondents. The common case; the honest tools: model the missingness (imputation by regression on the observed columns — "fill each gap with what similar rows show"), or better, *weight or stratify* by what predicts missingness. Dropping rows now *biases* the result (you systematically lose Sunday-EcoCash), and the bias is silent.
- **MNAR** (missing not at random) — the reason is the value itself: the biggest baskets unrecorded (voided to hide theft); dissatisfied customers never answering. The dangerous case: no statistical trick fully fixes it, because the data cannot tell you what it never saw. The honest responses: bound the damage (sensitivity analysis — "if missing sales average  $1 \times$  the mean, revenue is  $X$ ; if  $2 \times$ , it's  $Y$ "), find a second source to triangulate, and *state*

*the direction of the bias in the report* ("our estimate is conservative; theft-voided baskets are systematically large").

The working liturgy for any NULL-heavy column: **mechanism first, decision second** — chart the missingness (missing by day? by segment? by value of neighbours?) against every column that might explain it; classify MCAR/MAR/MNAR with your reasons written; then choose (drop / impute+flag / bound+declare). The imputation craft notes: impute with a *flag column* (`was_imputed`) so downstream honesty survives; prefer simple (median by segment) over clever (fancy models imputing fiction) unless the imputation itself is validated; and never, ever let imputed values silently become "real" in a report — the fence around them is a sentence in every finding: "12% of this column was imputed; the estimate without imputation is also shown."

## 60.2 The Deadline 80/20

The real project ships Friday, and the data arrived Thursday with 4,000 rows missing in the middle. The craft of delivery under constraint — the triage order that preserves both the deadline and the integrity:

1. **Scope to the question that matters:** not "the full analysis" but the decision this meeting makes; cut every branch that does not feed it (Chapter 13's kindness, weaponised against time).
2. **The honest core first:** the number the decision needs, computed on the *clean* subset — declared ("first 6 months only; the rest arrives verified Monday"). A verified partial beats an unverified whole.
3. **The checks that survive compression:** three survive every deadline — the bridge total, the missingness statement, and the known-flaw line. Twenty-minute checks that have saved careers; never traded away.
4. **The staged delivery:** the honest core Friday; the completed analysis Monday; the deep questions next cycle. Stakeholders *accept* staged truth remarkably well when offered it proactively and dated — what they never forgive is the confident Friday number that quietly changes.

The rule underneath: **under deadline, you cut scope, never rigor** — scope is negotiable (with the stakeholder, out loud), rigor is not (with reality, silently). The analyst who ships the small-true thing on Friday and the whole-true thing on Monday has a career; the one who ships the big-maybe thing has an incident.

## 60.3 The Stakeholder Realities

The human weather systems of real projects, with the working responses:

- **The question that changes mid-flight** ("actually, can we look at it by region?") — respond with scope arithmetic: "yes — that's a new cut; it adds two days, or replaces the price analysis; which would you prefer?" The question-changer is not the enemy; the *undisclosed* change in cost is. Make costs visible and you can absorb any number of new questions.
- **The missing owner** — the decision-maker who commissioned the analysis and then vanished; the finding lands in a meeting where nobody has authority to act. Response: name the decision in the first email ("this supports the pricing call — who owns it?"), and deliver to the decision, not the request.
- **The politically inconvenient finding** — your analysis is correct and unwelcome (the flagship product's numbers are bad; the director's project harmed revenue). The craft: the *finding* is delivered exactly as a welcome one would be — same rigour, same caveats, same channel — because the moment you soften it, you own the softening forever. The *framing* can serve the reader ("three options the data supports") without bending the data. And Chapter 59's refusal line applies when "reframe" means "reverse".
- **The repeat ad-hoc** — the same "quick question" weekly, from the same stakeholder, about the same data. This is not a workload problem; it is Chapter 37's pipeline problem wearing a person — and solving it (the tile, the saved query, the scheduled mail) is precisely the kind of quick win that builds a first-90-days reputation (Ch. 54).

## 60.4 The Reality Checklist

The messy-project audit — run it on any inherited or half-alive project, in place of the despair that usually accompanies both:

1. **What question was this trying to answer?** (Reconstruct from artefacts; if nobody can say, that is the first finding — and usually the reason the project died.)
2. **What data exists, what's missing, and which mechanism?** (60.1's liturgy, run as archaeology.)
3. **What checks were run — and what do they say now?** (Re-run the bridges; a stale reconciliation is the fastest autopsy.)
4. **What is actually finished?** (Notebooks that Restart-And-Run-All; queries with headers; reports with dates — the Ch. 45 anatomy, applied as triage.)
5. **What is the smallest honest deliverable?** (80/20: cut to a question someone owns, ship the verified core, stage the rest — resuscitation, not heroics.)

## 60.5 Learning From the Ones That Go Wrong

The failed projects are tuition paid — collect the learning: a **post-mortem in three paragraphs** (what happened, honestly; what I'd check earlier; what I'd say no to), filed in the same ledger as the successes, because Chapter 52's interviewers ask about failure deliberately and "tell me about a project that went wrong" is answered *best* by someone with a written archive of the answer ("the data arrived 40% missing at MNAR — I bounded it, delivered the honest core, and learned to ask about data completeness *before* accepting the timeline; here's what I ask now"). The careers that compound (Ch. 56) are built exactly this way — not on unbroken success, which does not exist, but on failures that were *processed*: what this book has done in miniature with every lab and every log, now as a lifelong habit.

**From Your Toolkit — reality is the final tool:** the whole book has been rehearsal in a controlled world; this chapter is the patch notes for the uncontrolled one. Every item here is still the same instruments — the liturgy, the bridge, the caveat, the anatomy, the staged truth — played at speed, under fire, with humans in the room. The mess does not change the method; it is why the method exists.

### Key Takeaways

- Missingness: classify MCAR/MAR/MNAR with reasons written; drop only MCAR; impute with flags and simple models; bound MNAR and declare the direction of bias — never let imputed become silently real.
- Deadline craft: scope to the deciding question, ship the verified core with its three survival checks, stage the rest with dates — cut scope, never rigor.
- Stakeholder weather: make change-costs visible, deliver to the named decision, deliver inconvenient findings with unchanged rigour, and treat repeat ad-hocs as pipeline problems.
- The reality checklist: question, data state, checks re-run, finished-vs-dead triage, smallest honest deliverable.
- Post-mortem in three paragraphs, filed — processed failure is the raw material of compounding.

## Practice Lab

1. The missingness liturgy, run for real: on a dataset with genuine gaps (your stock table's missing weeks, or a public dataset's holes), chart missingness by every plausible predictor; classify the mechanism with reasons; apply the matching response; write the report sentence that discloses it.
2. The imputation clinic: impute the gaps two ways (median-by-segment; regression) with `was_imputed` flags; compare downstream findings across: original-dropped, imputed-A, imputed-B — three numbers, honestly different; write the disclosure line you would ship.
3. The Friday drill: simulate the deadline — a stakeholder question, a dataset with a hole, four hours on a clock; deliver the staged package (Friday core + Monday plan); note where the deadline pressure tried to eat a check, and that you refused.
4. The archaeology: find a dead project (yours, or a public abandoned notebook); run the reality checklist; write its three-paragraph post-mortem even though — especially because — it is uncomfortable.
5. The script library: from this chapter, write your four standing sentences (scope arithmetic, decision-naming, finding-delivery, no-with-alternative); rehearse them aloud once; they are now as portable as your SQL.

## Further Reading

- Chapter 61 (the free stack — practising all of this for \$0), Chapter 62 (the decade view)
- *Missing Data* (Allison) — the little orange book; everything an analyst needs in 90 pages

# Chapter 61: The Free Stack — \$0 Toolkit, \$200 Upgrade

## Part IX — Beyond the Basics

"Every tool in this book is free, or has a free twin. The profession's gate is not a credit card — it is the habits, and you have those."

### In this chapter you will learn

- The complete \$0 analyst stack, tool by tool, matched to this book's Parts.
- The honest gaps in free — and the workarounds that hold until income.
- The \$200 upgrade path: what to buy first, second, never.
- Working offline and on thin hardware: the craft that outlasts bandwidth.
- Tool-agnosticism as the permanent skill — switching accents, cheaply.

### 61.1 The \$0 Stack

The full inventory, mapped to the Parts it serves — print this section if money is the constraint; it removes the excuse permanently:

Part / need	The \$0 tool	Notes
Spreadsheet (Part II)	<b>LibreOffice Calc</b> , Google Sheets, Excel for the web	Calc is genuinely sufficient: pivots, lookups, ToolPak-equivalents (Data → Statistics); this book was written with it in mind
Database (Part III)	<b>SQLite + DB Browser</b>	A complete relational engine and workbench; zero install drama (Ch. 14). <b>DuckDB</b> — the modern analytical cousin — is the \$0 upgrade for warehouse-scale SQL on a laptop, and runs the same chapters
Statistics GUI (Part IV)	<b>PSPP (GNU)</b> , <b>jamovi</b> , <b>JASP</b>	PSPP speaks SPSS syntax; jamovi is the friendliest menu-driven stats tool ever built, and free forever; JASP adds Bayesian options
Python (Part V)	<b>Python + Anaconda</b> , or lighter: plain Python + pip	Everything (pandas, matplotlib, scipy, statsmodels, jupyter) is free <i>by design</i> ; Google Colab runs it on borrowed computers — no machine of your own required

Part / need	The \$0 tool	Notes
BI / dashboards (Part VI)	<b>Power BI Desktop</b> (free), <b>Metabase</b> (free, self-hosted), <b>Apache Superset</b> , <b>Looker Studio</b> (Google, free), <b>Tableau Public</b>	Desktop is free and complete for building (Ch. 38); the others publish for free — Metabase on a \$5 VPS or even a Raspberry Pi runs a real department dashboard
SQL at work-scale (Part III/58)	<b>PostgreSQL</b> (free, forever), DuckDB, <b>BigQuery sandbox</b>	Postgres is the industry's favourite open engine; the BigQuery sandbox is a real warehouse with free limits
Publishing / portfolio (Part VII)	<b>GitHub</b> free tier, <b>GitHub Pages</b> , <b>Streamlit Community</b>	The portfolio (Ch. 49), a personal site, and even Python-built interactive dashboards — hosted free
Data (Ch. 44)	The public landscape	Governments and agencies publish for free <i>on purpose</i>

The point of the table is not the individual tools; it is the column header: **the profession runs on free software at its core** — Python, SQL engines, Git, the BI free tiers — because the expensive products (SPSS licences, Tableau Desktop, cloud at scale) are *employers'* purchases, not *practitioners'*. You can be fully, certifiably employable — portfolio, projects, dashboards, statistics — on \$0. The book you just read was assembled that way.

## 61.2 The Honest Gaps

Where free genuinely falls short, stated so you plan rather than discover: **SPSS and Stata themselves** are paid (the trials are time-limited; PSPP covers the core menu statistics — Frequencies, Crosstabs, T-tests, regression — but not every procedure; Stata has no free twin: learn the concepts in jamovi, the syntax on the job, the "week not a year" argument of Ch. 29); **Power BI service sharing** needs a licence (Desktop files and export work around it — Ch. 43's ladder); **cloud scale** bills per use beyond the sandboxes (the sandbox tiers are enough to *learn* the dialect and the cost instinct — Ch. 58's lab); **collaboration platforms** (Slack, Notion, the office suites) are free-tier limited but fine for one. None of these gaps touches learning, practising, portfolio-building, or freelancing — they touch enterprise convenience, which is what salaries are for.

## 61.3 The \$200 Upgrade Path

When income (or a first job, or a first client — Ch. 55) puts ~\$200 in the toolkit budget, the order that buys the most capability per dollar:

1. **A comfortable second-hand laptop with 16 GB RAM** (~\$150–250 if your current machine is the constraint) — RAM is the analyst's laptop spec: it is the difference between pandas struggling and pandas shrugging (Ch.



58's thresholds). Nothing else on this list matters if the machine swaps.

2. **A domain name (~\$15/yr)** — `yourname.dev` pointed at GitHub Pages: the portfolio stops being a `github.com` path and becomes a destination (Ch. 49's shop window, upgraded).
3. **A VPS (~\$5-10/mo)** — Metabase or Superset publicly visible, plus scheduled scripts (Ch. 37's cron, hosted): a *running* dashboard in your portfolio that a recruiter can click.
4. **One exam certificate** (Ch. 51's rule: only with exams — e.g. Microsoft's PL-300 Power BI Data Analyst, the one certification this book's Part VI maps to, exam fee ~\$165 — often less with vouchers) — *after* the portfolio exists, never instead of it.
5. **Never:** paid course subscriptions as a first resort (the free moocs and vendor learning paths cover this book's entire territory), "premium" data (the public landscape exists), or tool licences a job will buy for you on day one.

## 61.4 Working Offline and on Thin Hardware

Constraints this book's readers may know well — intermittent power, metered data, shared machines — and the craft that keeps the profession open: **local-first tools** (Calc, SQLite/DuckDB, Python — all work fully offline; only the cloud chapters need a connection, and the sandboxes are patient); **small-footprint workflows** (Jupyter is heavier than plain `.py` scripts; VS Code runs lighter than full IDEs; DuckDB queries gigabytes where a spreadsheet cannot even open them — *light hardware with DuckDB outperforms heavy hardware with the wrong tool*); **batch the connectivity** (download datasets and documentation in one café session; git commits queue offline and push when the signal returns); **version everything** (the habit that makes shared machines safe: your work is the repo, not the machine — pull to any borrowed laptop and continue). And the perspective that is also a truth: **intermittent infrastructure builds better habits than abundant infrastructure** — the analyst who logs, scripts, and versions because re-doing is expensive has accidentally adopted the professional discipline that offices pay consultants to teach.

## 61.5 Tool-Agnosticism, the Permanent Skill

The final word on tools, from a book that taught five of them: **the tool is an accent; the language is the method.** Evidence: the translation table of Ch. 29 (six instruments, five dialects); the anatomy of Ch. 45 (tool-blind); the interview chapters (the tests probe the method, and the method transfers). The professional who *cannot* switch tools is a liability the day the employer switches; the professional who can, reads a new stack's documentation like a menu ("ah — this is GROUP BY with different brackets"). Practise the switch deliberately, once a year (Ch. 56's annual stretch): rebuild one portfolio project in a tool you have never used, in a weekend. The rebuild is always faster than the original — and the speed of that rebuild, felt firsthand, is the permanent, portable, \$0 confidence this book has been building toward.

**From Your Toolkit — the stack is free; the career is built:** the constraints chapter closes with its own inversion — you were never waiting on software licences. The stack above plus the habits of Parts I–VIII is a *complete* professional toolkit: Tariro's shop went from CSV to dashboard on it, and so did this book. What remains — the tenth part of nothing, the everything of it — is the decade, next chapter.

### Key Takeaways

- The \$0 stack covers every Part: Calc/Sheets, SQLite/DuckDB, jamovi/PSPP, Python, Power BI Desktop + free publishers, GitHub — the profession's core is free by design.
- The honest gaps are employers' purchases: SPSS/Stata licences, service sharing, cloud scale — none touch learning or the portfolio.
- \$200 order: RAM-comfortable laptop first, domain, VPS, then one exam certificate (PL-300) — never paid courses as first resort.
- Offline craft: local-first, light-but-right tools (DuckDB), batched connectivity, the repo as your working identity — constraints build the professional habits.
- Tool-agnosticism is permanent: rebuild one project in a new stack yearly — the method is the career; the accents are cheap.

### Practice Lab

1. The stack audit: build your personal \$0 inventory table (what you have, what you'd add, from 61.1); install the two gaps; run one operation in each tool to verify — the shelf, verified.

2. The DuckDB hour: install it (single binary), point it at your biggest CSV, and run three of your saved SQL queries on it directly; note what "analytical database engine" means for a file you used to open in Calc.
3. The jamovi rebuild: redo Chapter 27's survey analysis in jamovi (menus only); confirm the identical  $\chi^2$  and p; write the three-line translation note (SPSS → jamovi) in your dialects file.
4. The publish, for free: get one interactive thing publicly online at \$0 — Metabase/Superset screenshot export, a Looker Studio report, or a Streamlit app on the Community tier; link it from the portfolio README (and verify the link from someone else's phone).
5. The annual switch, rehearsed early: pick one tool you have *never* used from 61.1 (say, Superset or R); rebuild Project 1's one-pager in it over a weekend; time the rebuild vs the original; write the one-paragraph "switch cost" note — your personal evidence for Ch. 61.5's claim.

## Further Reading

- Chapter 62 (the decade — where this stack compounds), the tools' own sites (all linked from the free-tier philosophy above)
- [duckdb.org](https://duckdb.org)'s "why DuckDB" essay — fifteen minutes that will change your CSV life

# Chapter 62: Your Next Decade

---

## Part IX — Beyond the Basics

"A book ends so the work can begin. This last chapter is not a summary — it is a launch sequence."

### In this chapter you will learn

- The roadmap: what to deepen in years 1, 2-3, and 5+ — sequenced, not listed.
- The community habit: finding your people, giving before taking.
- The reading life of an analyst: the dozen books, the one rule.
- The identity shift: from learning analytics to *being* an analyst.
- The last page: what this book set out to do, and the baton handover.

## 62.1 The Roadmap

Not a syllabus — a *sequence*, because sequencing is what makes learning compound instead of sprawl:

**Year 1 — convert skills to evidence.** You have the stack; the year's job is the working identity: the first job or first three freelance engagements (Parts VII-VIII, executed); the 90-day discipline (Ch. 54) run for real; the monthly dataset habit (Ch. 44) uninterrupted; the portfolio alive (Ch. 49) with one *deep* addition — a project with real stakes and a real stakeholder, whatever its size. Depth check at year's end: a stranger reads your GitHub for five minutes and can tell you what you are good at. That is the whole year, and it is enough.

**Years 2-3 — the specialisation and the scale.** The fork (Ch. 56) taken honestly; the niche (domain × function) chosen from accumulated advantage; the scale ladder climbed one rung — a warehouse dialect (BigQuery/Snowflake) learned on sandboxes, dbt added to the SQL craft, one cloud-fundamentals certificate if the market rewards it. The communication compounder: *teach something publicly* — a talk at a local meetup, a blog post series on your niche, an internal workshop. Teaching is the highest-yield learning there is (you discover what you only half-knew), and it is the reputation engine that later becomes the consulting practice (Ch. 55, grown) or the principal path (Ch. 56).

**Years 5+ — the compounding years.** Whatever was built deliberately now grows on its own: the niche becomes recognised expertise (people forward

your name for *that* problem); the teaching becomes speaking or writing; the judgement becomes sought — the "should we even look at this?" calls. New responsibilities arrive (team, methodology, standards); the habit that keeps them honest is the same one this book installed in Chapter 2: the questions, especially about your own certainty. And somewhere in these years, most analysts face the version-choice the field now poses — deepening craft (principal), multiplying people (management), or compounding independence (consulting, Chapter 55 scaled) — and by then you will have the data about yourself to choose well, which is the entire point of having kept the ledger.

## 62.2 The Community Habit

Nobody compounds alone; the community skill, stated as craft: **find three rooms** — the local one (analytics/data meetups, university events, the meetups that become conferences; if none exists where you are, that is not a gap, that is *your first leadership role*: start one, announce it on LinkedIn, host six people in a café — the founding of a community outranks attendance at ten), the online one (one forum or community where you *answer*, not just ask — Stack Overflow, a data community Discord, a local WhatsApp/Telegram group for analysts), and the professional one (the people from your jobs and projects — the alumni of every engagement; keep the ledger of who you have worked well with, because opportunity travels these edges). The rule that makes rooms work: **give before and more than you take** — answer the beginner questions (you were the beginner one hundred pages ago), share your projects and their failures, credit loudly. Communities route opportunity to the people who make the room better; that is not idealism, that is how rooms function.

## 62.3 The Reading Life

The dozen books that make an analyst, reread-able for a decade (this book's Further Readings, distilled to the canon): *How to Lie with Statistics* (Huff) and *How Charts Lie* (Cairo) — the skepticism pair; *Naked Statistics* (Wheelan) and *The Art of Statistics* (Spiegelhalter) — the intuition pair; *Storytelling with Data* (Knafllic) and *The Pyramid Principle* (Minto) — the communication pair; *Python for Data Analysis* (McKinney) — the reference; *Weapons of Math Destruction* (O'Neil) — the conscience; *Designing Data-Intensive Applications* (Kleppmann) — the systems horizon; *Thinking, Fast and Slow* (Kahneman) — the mind you are working inside; *The First 90 Days* (Watkins) and *So Good They Can't Ignore You* (Newport) — the career pair. Twelve books, two or three a year beside the work — that is the entire programme. The one rule that governs it: **read to answer live questions, not to hoard answers** — the book that finds you wrestling a real problem teaches ten times the book read "for background" (you have felt this in every Further Reading you actually followed up).

## 62.4 The Identity Shift

Somewhere in year one — usually quietly, on an ordinary afternoon — the shift happens: you stop being *someone learning analytics* and become *an analyst* — the person the room looks at when a number is in dispute, the one who says "let me check the data first" and is believed. You cannot schedule the shift, but you can *feed* it, and the feeding is three habits this book can claim to have installed: **ship things** (the identity is built of delivered artefacts, not of confidence); **say the numbers aloud** (the first time you defend a caveat in a meeting — the real meeting, with stakes — something clicks that no chapter could click for you); and **keep the logs** (the analyst's diary — the questions ledger, the post-mortems, the data journal — is the identity's paper trail; on the days you doubt you belong, it is the evidence that you do, because it is a record of work no imposter could have faked).

And with the identity arrives its obligation, stated here as the book's last piece of standing instruction: **you are now the skepticism in somebody's room**. The world you will work in is filling with confident numbers — dashboards that mislead, models that launder bias, statistics deployed as ammunition. The person who asks "compared to what?", "says who?", "is it even true?" — calmly, with the method to check — is not the room's cynic; they are the room's *analyst*, and increasingly its rarest asset. The three honesty rules were never exam material. They were a job description.

## 62.5 The Last Page

This book set out to do one thing: take a person with no background, no budget, and no credentials, and deliver them — with evidence — to the first job of a working career. The route was a story on purpose: a real-shaped shop, real-shaped mess, real-shaped questions, because analytics is not learned from features lists but from *wanting to know* and being equipped to find out. You know the five tools — but more, you know the one method that runs through them: the question before the data, the liturgy before the analysis, the test before the claim, the caveat attached to the number, the artefact instead of the memory.

Tariro's shop is yours now — the ledger closed, the dashboard live, the Sunday question answered with a p-value and a business decision, which is this profession in one sentence. The decade ahead will swap her shop for warehouses and warehouses for whatever comes after them; the method will not swap. It is yours; it cost you a book; it is worth a career.

Go and ship something.

— *Tatenda Makuvaza, Harare*

**From Your Toolkit — the last tool is the door:** Appendix E's question bank is the final rehearsal; the portfolio is the interface to the market; this chapter's roadmap is the maintenance schedule. The book ends; the checklist does not. Keep the questions sharp, the bridges reconciled, and the caveats attached — and welcome to the profession.

## Key Takeaways

- Year 1: convert skills to evidence — first roles, the monthly habit, one deep stakeholder project; the five-minute-stranger test.
- Years 2-3: the fork and the niche; one warehouse rung, dbt; teach publicly — the compounding engine.
- Years 5+: expertise compounds into reputation; choose the version with your own data; keep the Chapter 2 questions pointed at your own certainty.
- Three rooms (local, online, professional), giving first — and if the room doesn't exist, founding it is your first leadership role.
- The dozen-book canon read beside live questions; the identity built of shipped artefacts and defended caveats; and the final obligation: be the skepticism in the room — that is the job description.

## Practice Lab

1. The decade plan, one page: years 1 / 2–3 / 5+ in your own words, dated, with the fork question and the niche hypothesis marked *provisional*; store beside the letter to year ten (Ch. 56's lab).
2. The rooms, joined or founded: identify the three rooms (one local or to-be-founded, one online where you will answer, one professional ledger started); take the first action in each this week — including, if needed, posting the founder's message.
3. The canon, staged: pick the next three books by the live-question rule (what are you actually wrestling?); order or download them; calendar them — two chapters a week is a book a month.
4. The identity audit: list every artefact you shipped during this book (projects, reports, dashboards, notes); read the list aloud once; that is the paper trail — add the date you first defended a number in front of someone, when it happens.
5. The baton: write the 150-word case study of *this book as your first project* — the situation (reader, zero background), the work (62 chapters, six projects, one running case), the finding (what you now know you can do), the impact (what shipped), the limits (what you still owe yourself: the live question, the real stakeholder). It is portfolio item zero — the one that proves the method works on you.

## Further Reading

- Everything you followed up on, all book long — the reading life, already begun
- Appendix E — the 100 questions, and the door



## APPENDICES

# Reference Section

Quick answers, cheat sheets, solutions and practice data

---

# Appendix A: SQL Cookbook

## Appendices — The Cookbook

"Thirty-five queries, commented line by line — the 25 patterns this book used, plus the advanced set promised in Chapters 18–19. Replace table and column names, keep the shape."

Conventions: queries run on SQLite unless noted (dialect notes at the end); the running tables are `sales(sale_id, date, till, customer_id, payment_type, amount)`, `customers(customer_id, name, suburb, joined, loyalty_tier)`, `stock(week, item, qty_on_hand)`, `suppliers(item, supplier, unit_cost, last_price_change)`.

## Reading and Filtering

### 1. First look — the safe preview

```
SELECT *                -- asterisk for exploring ONLY; name columns for real work
FROM   sales
LIMIT 5;                -- never meet a new table without LIMIT
```

### 2. Named columns, chosen order

```
SELECT sale_id,
       date,
       amount,           -- SELECT order is your presentation order
       payment_type
FROM   sales;
```

### 3. Filtering rows — the core comparisons

```
SELECT *
FROM   sales
WHERE  amount >= 50      -- numeric: no quotes
      AND payment_type = 'EcoCash' -- text: quotes, case-sensitive
ORDER BY amount DESC;
```

### 4. Date ranges — two safe forms

```
SELECT *
FROM   sales
WHERE  date >= '2025-06-01'
      AND date < '2025-07-01'; -- half-open range: survives timestamps
                                     -- ("2025-06-30 14:33") that BETWEEN can miss
```

### 5. Membership and exclusion

```
SELECT *
FROM   sales
WHERE  payment_type IN ('Cash', 'Card') -- the readable OR chain
```

```

AND customer_id IS NOT NULL      -- NULLs never match; test by name
AND suburb NOT IN ('Avondale');  -- careful: NOT IN + NULL = empty result

```

## 6. Top-N

```

SELECT sale_id, date, amount
FROM   sales
ORDER BY amount DESC    -- sort first
LIMIT 10;              -- then cap

```

## 7. Text matching

```

SELECT DISTINCT item
FROM   stock
WHERE  item LIKE '%oil%'        -- % = any characters
      AND item NOT LIKE '%sample%'; -- LIKE is case-insensitive in SQLite,
                                     -- case-sensitive in Postgres (use ILIKE)

```

# Grouping and Aggregating

## 8. The whole table in one line

```

SELECT COUNT(*)      AS n_sales,      -- rows
       SUM(amount)   AS total_revenue,
       AVG(amount)   AS avg_basket,   -- NULLs ignored
       MIN(amount)   AS smallest,
       MAX(amount)   AS largest
FROM   sales;

```

## 9. The two COUNTs

```

SELECT COUNT(*)      AS all_sales,    -- every row
       COUNT(customer_id) AS member_sales -- non-NULL only
FROM   sales;          -- the difference IS an analysis

```

## 10. Totals of something, for each something else

```

SELECT  payment_type,
        COUNT(*)    AS n,
        SUM(amount) AS revenue
FROM    sales
GROUP BY payment_type    -- one row per distinct value
ORDER BY revenue DESC;

```

## 11. Groups with conditions — WHERE chooses rows, HAVING chooses groups

```

SELECT  category,
        COUNT(*)    AS n_sales,
        AVG(amount) AS avg_basket
FROM    sales
WHERE   amount <= 21.50    -- exclude whales BEFORE averaging
GROUP BY category

```

```
HAVING COUNT(*) >= 50      -- then keep busy categories
ORDER BY avg_basket DESC;
```

## 12. Month extraction and grouping (SQLite / portable-ish)

```
SELECT substr(date, 1, 7) AS month,      -- '2025-06'; Postgres: date_trunc('month',
    date)
    SUM(amount)          AS revenue
FROM sales
GROUP BY substr(date, 1, 7)
ORDER BY month;
```

## 13. Percentile (the robust typical)

```
SELECT AVG(amount)          AS mean,
    (SELECT amount FROM sales ORDER BY amount
     LIMIT 1 OFFSET (SELECT COUNT(*)/2 FROM sales)) AS approx_median
FROM sales;
-- Postgres/Snowflake/BigQuery: PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)
```

## Joining

### 14. Enrichment join (LEFT — keep all sales)

```
SELECT s.sale_id, s.amount,
    c.name, c.suburb      -- qualify columns when tables share names
FROM sales s
LEFT JOIN customers c
    ON c.customer_id = s.customer_id;  -- unmatched -> NULLs on the right
```

### 15. The fan-out defence (run after EVERY join)

```
SELECT (SELECT COUNT(*) FROM sales) AS rows_before,
    COUNT(*) AS rows_after,      -- must equal rows_before
    SUM(s.amount) AS revenue      -- must equal the bridge
FROM sales s
LEFT JOIN customers c ON c.customer_id = s.customer_id;
```

### 16. The anti-join — rows with no match

```
SELECT s.customer_id, COUNT(*) AS orphan_sales
FROM sales s
LEFT JOIN customers c ON c.customer_id = s.customer_id
WHERE c.customer_id IS NULL      -- keep only the unmatched
GROUP BY s.customer_id;
-- "customers who never bought" = same pattern, tables swapped
```

### 17. Aggregate, then join (the fan-out-proof pattern)

```
WITH member_value AS (
    SELECT customer_id, SUM(amount) AS lifetime_revenue, COUNT(*) AS n_sales
    FROM sales
    WHERE customer_id IS NOT NULL
```

```

        GROUP BY customer_id          -- one row per customer FIRST
    )
    SELECT c.name, c.suburb,
           v.lifetime_revenue, v.n_sales
    FROM   customers c
    JOIN   member_value v ON v.customer_id = c.customer_id
    ORDER BY v.lifetime_revenue DESC
    LIMIT 20;

```

## 18. Join with condition on the right table (the classic trap)

```

-- WRONG: WHERE c.suburb = 'Avondale' silently turns LEFT into INNER
SELECT s.sale_id, c.suburb
FROM   sales s
LEFT JOIN customers c
       ON c.customer_id = s.customer_id
       AND c.suburb = 'Avondale';  -- condition INSIDE ON: non-matching kept, NULL
suburb

```

## Layering (CTEs and Windows)

### 19. Share of total

```

WITH by_payment AS (
    SELECT payment_type, SUM(amount) AS revenue
    FROM   sales
    GROUP BY payment_type
)
SELECT payment_type,
       revenue,
       ROUND(100.0 * revenue / SUM(revenue) OVER (), 1) AS pct_share
                                -- OVER () = whole table, rows not collapsed
FROM   by_payment
ORDER BY revenue DESC;        -- 100.0 forces decimal division

```

### 20. Top-N within each group

```

WITH item_rev AS (
    SELECT item, SUM(amount) AS revenue
    FROM   sales GROUP BY item
),
ranked AS (
    SELECT item, revenue,
           ROW_NUMBER() OVER (ORDER BY revenue DESC) AS rn_all,
           DENSE_RANK() OVER (ORDER BY revenue DESC) AS dr_all
    FROM   item_rev
)
SELECT * FROM ranked WHERE rn_all <= 10;
-- per-category: PARTITION BY category inside the OVER

```

### 21. Running total and moving average

```

SELECT date,
       SUM(amount) OVER (ORDER BY date, sale_id) AS running_total,
       AVG(amount) OVER (ORDER BY date
                        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS avg_7day
FROM   sales;
-- tie-break with sale_id: same-date rows need a deterministic order

```

## 22. Month-over-month with LAG

```

WITH monthly AS (
  SELECT substr(date,1,7) AS month, SUM(amount) AS revenue
  FROM   sales
  GROUP BY substr(date,1,7)
)
SELECT month,
       revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / LAG(revenue) OVER (ORDER BY month), 1) AS pct_change
FROM   monthly;

```

## 23. Deduplicate — keep the latest row per key

```

WITH ranked AS (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                           ORDER BY date DESC) AS rn
  FROM   messy_contacts
)
SELECT * FROM ranked WHERE rn = 1;

```

## 24. Gap check — the data-quality sweep

```

SELECT 'duplicate sale_ids' AS check_name, COUNT(*) AS n
FROM   (SELECT sale_id FROM sales GROUP BY sale_id HAVING COUNT(*) > 1)
UNION ALL
SELECT 'null amounts', COUNT(*) FROM sales WHERE amount IS NULL
UNION ALL
SELECT 'orphan customer refs', COUNT(*)
FROM   sales s LEFT JOIN customers c ON c.customer_id = s.customer_id
WHERE  s.customer_id IS NOT NULL AND c.customer_id IS NULL;

```

## 25. The professional header (not a query — the wrapper)

```

-- monthly_revenue_by_till.sql
-- Purpose: monthly revenue split by till, for the ops meeting
-- Author: <you>, created 2025-06; source: sales (till system export)
-- Expected when last run: 18 rows, total 54,013.50 -- reconcile before trusting
WITH monthly AS (
  SELECT substr(date,1,7) AS month, till, SUM(amount) AS revenue
  FROM   sales
  GROUP BY substr(date,1,7), till

```

```
)
SELECT * FROM monthly ORDER BY month, till;
```

## Beyond the 25 — The Advanced Patterns (promised in Chapters 18-19)

### 26. Gaps and islands — find runs of consecutive days

```
-- Which stretches of days was each item out of stock (or on promotion)?
WITH dated AS (
    SELECT item, week,
           -- days since a fixed epoch, numbered
           CAST(julianday(week) AS INTEGER) AS d    -- Postgres: week - DATE
           '2024-01-01'
    FROM    stock
),
grouped AS (
    SELECT item, d,
           d - ROW_NUMBER() OVER (PARTITION BY item
                                   ORDER BY d) AS island_id
           -- consecutive days share the same (d - row_number): the gap trick
    FROM    dated
)
SELECT item,
       MIN(week) AS run_start,
       MAX(week) AS run_end,
       COUNT(*) AS run_length
FROM    grouped
GROUP BY item, island_id
HAVING COUNT(*) > 1          -- runs longer than one
ORDER BY item, run_start;
```

### 27. Sessionisation — group events into visits

```
-- A new "visit" starts when a gap exceeds 30 minutes (web logs, till events)
WITH stamped AS (
    SELECT customer_id, occurred_at,
           LAG(occurred_at) OVER (PARTITION BY customer_id
                                   ORDER BY occurred_at) AS prev_at
    FROM    events
),
flagged AS (
    SELECT *,
           CASE WHEN prev_at IS NULL
                OR (julianday(occurred_at) - julianday(prev_at)) * 24 * 60 > 30
                THEN 1 ELSE 0 END AS new_visit    -- Postgres: EXTRACT(EPOCH FROM ...)
    FROM    stamped
),
visited AS (
    SELECT *,
```

```

        SUM(new_visit) OVER (PARTITION BY customer_id
                             ORDER BY occurred_at
                             ROWS UNBOUNDED PRECEDING) AS visit_no
    FROM    flagged
)
SELECT customer_id, visit_no,
       COUNT(*)           AS events,
       MIN(occurred_at)   AS started,
       MAX(occurred_at)   AS ended
FROM    visited
GROUP BY customer_id, visit_no;

```

## 28. Cohort retention grid in SQL (Project 4's shape)

```

WITH first_purchase AS (
    SELECT customer_id, MIN(date) AS first_date
    FROM    sales
    WHERE   customer_id IS NOT NULL
    GROUP BY customer_id
),
activity AS (
    SELECT s.customer_id,
           substr(f.first_date, 1, 7) AS join_cohort,
           CAST((julianday(substr(s.date,1,7)) || '-01')
                - julianday(substr(f.first_date,1,7) || '-01')) / 30.4 AS INT)
               AS age_months,    -- months since first buy
           1 AS active
    FROM    sales s
    JOIN    first_purchase f USING (customer_id)
    GROUP BY s.customer_id, join_cohort,
           substr(s.date, 1, 7)    -- one row per active month
)
SELECT join_cohort,
       age_months,
       COUNT(*) AS active_customers
FROM    activity
GROUP BY join_cohort, age_months
ORDER BY join_cohort, age_months;
-- read as a grid: cohort down, age across -- the retention heatmap, in SQL

```

## 29. Year-over-year same-period comparison

```

SELECT month,
       revenue,
       LAG(revenue, 12) OVER (ORDER BY month) AS same_month_ly,
       ROUND(100.0 * (revenue - LAG(revenue, 12) OVER (ORDER BY month))
            / LAG(revenue, 12) OVER (ORDER BY month), 1) AS yoy_pct
FROM    monthly_revenue;
-- LAG with an offset of 12 -- the year-over-year version of query 22

```

## 30. Conditional aggregation (the pivot-in-SUM trick)



```

SELECT substr(date, 1, 7) AS month,
       SUM(amount) AS total,
       SUM(CASE WHEN payment_type = 'EcoCash'
                THEN amount ELSE 0 END) AS ecocash,
       ROUND(100.0 * SUM(CASE WHEN payment_type = 'EcoCash'
                            THEN amount ELSE 0 END)
              / SUM(amount), 1) AS ecocash_pct
FROM   sales
GROUP BY substr(date, 1, 7)
ORDER BY month;
-- one pass, one table, columns-as-conditions -- the cross-tab without pivoting

```

### 31. The running-total reconciliation (a check worth saving)

```

-- Does the ledger's running total land exactly on the closing total?
WITH daily AS (
  SELECT date, SUM(amount) AS day_total
  FROM   sales GROUP BY date
),
checked AS (
  SELECT date, day_total,
         SUM(day_total) OVER (ORDER BY date) AS running,
         SUM(day_total) OVER () AS grand
  FROM   daily
)
SELECT MAX(running) AS final_running, -- must equal grand
       MAX(grand) AS grand_total
FROM   checked;

```

### 32. Sampling — N random rows (for audits and quick checks)

```

SELECT * FROM sales ORDER BY RANDOM() LIMIT 100; -- SQLite
-- Postgres: TABLESAMPLE SYSTEM (1) BigQuery: TABLESAMPLE SYSTEM (1 PERCENT)
-- the audit habit: pull 100 random rows and eyeball them, monthly

```

### 33. The histogram in SQL (bins before charts)

```

SELECT CAST(amount / 5 AS INTEGER) * 5 AS bin_floor, -- $5-wide bins
       COUNT(*) AS n,
       printf('%s', substr('█', 1,
                          MIN(20, COUNT(*) / 20))) AS bar -- ASCII bar, SQLite
FROM   sales
GROUP BY bin_floor
ORDER BY bin_floor;
-- shape-checking in a terminal: the histogram, no chart tool required

```

### 34. Medians per group (the robust typical, SQLite edition)

```

WITH ranked AS (
  SELECT category, amount,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY amount) AS rn,
         COUNT(*) OVER (PARTITION BY category) AS n

```

```
FROM sales
)
SELECT category, AVG(amount) AS median
FROM ranked
WHERE rn IN ((n + 1) / 2, (n + 2) / 2) -- middle one, or middle two averaged
GROUP BY category;
-- Postgres/Snowflake: PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)
```

35. The pre-aggregation pattern (warehouse etiquette)

```
-- Never join a 40-million-row fact to a dimension for one number:
-- aggregate the fact to the grain you need FIRST, then join.
WITH daily_region AS (
  SELECT region_id, date, SUM(amount) AS revenue
  FROM fact_sales
  WHERE date >= '2025-01-01' -- sargable: filter before the work
  GROUP BY region_id, date -- aggregate before the join
)
SELECT r.region_name, SUM(d.revenue) AS revenue
FROM daily_region d
JOIN dim_region r ON r.region_id = d.region_id
GROUP BY r.region_name;
-- smaller intermediate, smaller bill, same answer (Ch. 58's cost instinct)
```

Dialect Notes

Task	SQLite	PostgreSQL / BigQuery / Snowflake
First N rows	LIMIT 10	LIMIT 10 / FETCH FIRST 10 ROWS ONLY
Substring	substr(x,1,7)	SUBSTRING(x,1,7) or date_trunc
Case-insensitive text	LIKE (already)	ILIKE
Median	hand-rolled (Q13)	PERCENTILE_CONT(0.5) WITHIN GROUP (...)
Booleans	0/1	true/false
Date math	date(d, '+1 month')	d + INTERVAL '1 month'

*Keep the shapes; translate the dialect. The query you cannot read is the query you cannot trust — comment the why, header the expectation, reconcile the total.*

# Appendix B: Excel Cookbook

## Appendices — The Cookbook

"Twenty-five formulas and tricks — the spreadsheet craft of Part II, compressed to a reference. Formulas shown for Excel; LibreOffice Calc and Google Sheets agree on nearly everything (notes where they don't)."

Conventions: `sales` refers to a clean table with headers in row 1: A date, B item, C category, D `payment_type`, E amount, F `customer_id`; `customers` is a second sheet. Row 2 is the first data row. Structured-table references (`Table1[amount]`) are the professional habit where available; plain ranges shown for universality.

## The Essentials

### 1. The anchoring dollar — copy-safe ranges

```
=SUM($E$2:$E$6421)    -- $ = lock: column, row, or both (F4 cycles)
                        -- ranges without $ drift when copied
```

### 2. SUMIF / COUNTIF — one-condition aggregation

```
=SUMIF(D:D, "EcoCash", E:E)    -- revenue for EcoCash
=COUNTIF(C:C, "Staples")      -- how many staple sales
=AVERAGEIF(D:D, "Cash", E:E)  -- average cash basket
```

### 3. SUMIFS / COUNTIFS — multiple conditions

```
=SUMIFS(E:E, D:D, "EcoCash", A:A, ">=2025-06-01", A:A, "<2025-07-01")
-- criteria in (range, criterion) PAIRS; dates as ISO strings
```

### 4. SUMPRODUCT — the anything-condition (incl. weighted means)

```
=SUMPRODUCT((D2:D6421="EcoCash") * E2:E6421)    -- SUMIFS without the limits
=SUMPRODUCT(prices, bottles) / SUM(bottles)    -- the weighted mean (Ch. 22)
```

## Looking Up and Joining

### 5. XLOOKUP — the modern join

```
=XLOOKUP(F2, customers!A:A, customers!C:C, "no match")
-- find F2, return the suburb, fail politely
```

### 6. VLOOKUP with error handling (the inherited-code version)

```
=IFNA(VLOOKUP(F2, customers!$A:$E, 3, FALSE), "no match")
-- always FALSE (exact); always IFNA; count the "no match"es
```

### 7. INDEX-MATCH — the column-insertion-proof join

```
=INDEX(customers!C:C, MATCH(F2, customers!A:A, 0))
-- MATCH finds the row; INDEX returns the value; survives inserted columns
```

## 8. The dup check

```
=IF(COUNTIF(F:F, F2) > 1, "DUP", "")    -- fill down; filter for DUP
```

## 9. The TRIM/CLEAN stack — ghost blanks and hidden characters

```
=TRIM(CLEAN(UPPER(D2)))    -- whitespace, non-printables, case
-- run on a helper column, then paste-as-VALUES over the original, delete helper
```

## Dates and Text

### 10. Date hygiene

```
=DATEVALUE("2025-06-14")    -- text date -> real date
=TEXT(A2, "yyyy-mm")        -- real date -> month key "2025-06"
=TEXT(A2, "ddd")            -- "Sat" (weekday name, Ch. 10)
=EOMONTH(A2, 0)             -- month end; -1 = previous month end
=DATEDIF(B2, A2, "d")       -- days between (quietly useful)
```

### 11. Text splitting

```
=LEFT(B2, FIND(" ", B2) - 1)    -- first word
=TEXTSPLIT(B2, " ")             -- modern: splits to adjacent cells
=TEXTJOIN(" ", TRUE, C2, D2)    -- joins, skipping blanks
```

### 12. The id-building concat

```
=TEXT(A2, "yyyymmdd") & "-" & TEXT(ROW()-1, "0000")
-- deterministic row id from date + row number (dedupe/reconciliation key)
```

## Statistics (Part II's set)

### 13. The typicals

```
=AVERAGE(E2:E6421)    =MEDIAN(E2:E6421)    =MODE.SNGL(E2:E6421)
=TRIMMEAN(E2:E6421, 0.05)    -- mean dropping top/bottom 5% (robust-ish)
```

### 14. The five-number summary

```
=MIN(E2:E6421)    =QUARTILE.INC(E2:E6421,1)    =MEDIAN(E2:E6421)
=QUARTILE.INC(E2:E6421,3)    =MAX(E2:E6421)
```

### 15. Spread

```
=STDEV.S(E2:E6421)    -- sample (n-1): your default
=STDEV.P(E2:E6421)    -- population (n): only for complete censuses
=QUARTILE.INC(E2:E6421,3) - QUARTILE.INC(E2:E6421,1)    -- IQR
```

### 16. The 1.5×IQR fence

```
=QUARTILE.INC(E2:E6421,3) + 1.5 * (QUARTILE.INC(E2:E6421,3) - QUARTILE.INC(E2:E6421,1))
-- flag: =IF(E2 > that_cell, "whale", "")
```

## 17. Z-score per row

```
=(E2 - AVERAGE($E$2:$E$6421)) / STDEV.S($E$2:$E$6421) -- anchored, fill down
```

## 18. Rank and percentile rank

```
=RANK.EQ(E2, $E$2:$E$6421) -- 1 = largest
=PERCENTRANK.INC($E$2:$E$6421, E2) -- 0.91 = 91st percentile
```

## 19. Correlation and the line

```
=CORREL(x_range, y_range) -- r
=INTERCEPT(y_range, x_range) =SLOPE(y_range, x_range)
=FORECAST.LINEAR(new_x, y_range, x_range) -- predict on the line
=RSQ(y_range, x_range) -- R-squared
```

## 20. The tests (Part IV in cells)

```
=T.TEST(range1, range2, 2, 3) -- two-tailed, unequal variance (Welch)
=T.TEST(range1, range2, 2, 1) -- paired
=CHISQ.TEST(observed_range, expected_range)
=CONFIDENCE.T(0.05, STDEV.S(range), COUNT(range)) -- the +/- of a 95% CI
```

# Craft and Guardrails

## 21. The error handlers

```
=IFNA(VLOOKUP(...), "no match") -- lookup misses
=IFERROR(1/0_cell, 0) -- last resort; IFNA first (hides less)
```

## 22. Rounding — display vs truth

```
=ROUND(E2 * 1.15, 2) -- round the RESULT once, for money
=TEXT(E2, "#,##0.00") -- format for display; never destroys value
```

**23. Conditional formatting as a check** — Home → Conditional Formatting → "Duplicate values" on the key column; "Greater than" the fence cell on amounts; a red-fill rule on `=ISBLANK(A2)`. *Three rules, the whole loading liturgy, visible.*

**24. Data validation — the guard at the door** — Data → Data Validation → List (payment types from a named range); whole-number bounds on amounts. *Prevents the flaw at entry, which beats cleaning it at load (Ch. 8's deepest lesson, as a feature).*

**25. The audit trio (trace your inheritance)** — Formulas ribbon: **Trace Precedents** (what feeds this cell), **Trace Dependents** (what breaks if I change it), **Evaluate Formula** (watch the calculation step by step — the EXPLAIN of spreadsheets). Plus Ctrl + ] (jump to precedents) and the

whole-book habit: **Ctrl + T** (make it a Table — structured references, auto-extension, one-click totals).

## Calc / Sheets Notes

- Calc and Sheets support everything above; XLOOKUP exists in recent versions (Calc 24.8+, Sheets yes); TEXTSPLIT is Excel/Sheets-only (Calc: SPLIT() in newer builds or Text→Columns).
- Sheets' QUERY() function (a mini-SQL on ranges) is a legitimate bridge to Part III: =QUERY(A:F, "select D, sum(E) where E > 0 group by D", 1).
- The ToolPak equivalents live in Calc under Data → Statistics; Sheets needs the formulas themselves.

*The cookbook's one law: every formula is a step in a story someone else must read — anchor the ranges, label the helpers, and reconcile the total before you believe the sheet.*

# Appendix C: Python Cookbook

## Appendices — The Cookbook

"Twenty-five pandas snippets — Part V compressed to a reference — plus two bonus sections this book promised: the full monthly-report pipeline, and the recipe that generated Tariro's data."

Conventions: `import pandas as pd` assumed; `sales` is a `DataFrame` with columns `date` (`datetime`), `item`, `category`, `payment_type`, `amount`, `customer_id`, `till`; `customers` has `customer_id`, `name`, `suburb`, `joined`, `loyalty_tier`.

## Loading and Inspecting

### 1. The load, with the things you always forget

```
sales = pd.read_csv("sales.csv",
                    parse_dates=["date"],      # dates parsed at load
                    dtype={"customer_id": "string"}) # ids stay text
```

### 2. The four instruments

```
sales.head()                # eyeball
sales.info()                # dtypes + nulls (the schema, free)
sales["amount"].describe()  # the five numbers plus mean/std
sales["payment_type"].value_counts() # categorical frequencies
```

### 3. The loading liturgy (the checks, as code)

```
assert sales["amount"].isna().sum() == 0, "null amounts -- stop"
assert customers["customer_id"].duplicated().sum() == 0, "dup keys -- stop"
orphans = (sales.loc[sales["customer_id"].notna()
                    & ~sales["customer_id"].isin(customers["customer_id"])])
print(f"{len(sales):,} rows | total {sales['amount'].sum():, .2f} | "
      f"{len(orphans)} orphans")
```

## Selecting and Cleaning

### 4. Boolean selection (WHERE)

```
big_ecocash = sales[(sales["amount"] >= 50)
                    & (sales["payment_type"] == "EcoCash")] # parentheses + & |
june = sales[sales["date"].between("2025-06-01", "2025-06-30")]
```

### 5. The string cleaners (Ch. 8 as methods)

```
sales["payment_type"] = (sales["payment_type"]
                        .str.strip().str.title()) # ghost blanks + case
sales["suburb"] = sales["suburb"].fillna("Unknown")
```

## 6. New columns, vectorised

```
sales["month"] = sales["date"].dt.to_period("M")
sales["weekday"] = sales["date"].dt.day_name()
sales["is_weekend"] = sales["weekday"].isin(["Saturday", "Sunday"])
sales["is_whale"] = sales["amount"] > 21.50          # name the constant in real code
```

## 7. Conditional labelling (the IF chain, ordered)

```
sales["basket_class"] = pd.cut(sales["amount"],
                               bins=[0, 5, 10, 21.5, np.inf],
                               labels=["tiny", "small", "big", "whale"])
```

## 8. Deduplicate, keep latest

```
clean = (sales.sort_values("date")
         .drop_duplicates(subset=["sale_id"], keep="last"))
```

## 9. Missingness triage (Ch. 60's liturgy)

```
sales.isna().mean()          # share missing, per column
sales[sales["amount"].isna()][ "weekday"].value_counts() # does missingness correlate?
```

## 10. Impute with a flag

```
sales["was_imputed"] = sales["amount"].isna()
sales["amount"] = sales["amount"].fillna(
    sales.groupby("category")["amount"].transform("median"))
```

# Grouping and Reshaping

## 11. The pivot (totals of X by Y)

```
by_payment = sales.groupby("payment_type")["amount"].agg(["count", "sum", "mean"])
```

## 12. Named aggregations (the AS aliases)

```
by_cat = sales.groupby("category").agg(
    revenue=("amount", "sum"),
    n_sales=("amount", "count"),
    avg_basket=("amount", "mean"))
```

## 13. Two-way pivot (pivot_table)

```
mix = sales.pivot_table(index="month", columns="payment_type",
                          values="amount", aggfunc="sum", fill_value=0)
```

## 14. Share of total (with the decimal point)

```
by_cat["pct_share"] = 100 * by_cat["revenue"] / by_cat["revenue"].sum()
```

## 15. Top-N, overall and per group

```
top10 = sales.groupby("item")["amount"].sum().nlargest(10)
top3_per_cat = (sales.groupby(["category", "item"])["amount"].sum())
```



```
.reset_index()
.sort_values(["category", "amount"],
             ascending=[True, False])
.groupby("category").head(3))
```

## 16. Percentiles and the fence

```
q1, q3 = sales["amount"].quantile([0.25, 0.75])
fence = q3 + 1.5 * (q3 - q1)
whales = sales[sales["amount"] > fence]
```

## Joining (with the defence)

### 17. The merge (LEFT JOIN)

```
enriched = sales.merge(customers, on="customer_id", how="left")
```

### 18. The fan-out defence (two lines, every merge)

```
print(len(sales), "->", len(enriched)) # counts must match
assert sales["amount"].sum() == enriched["amount"].sum() # totals reconcile
```

### 19. The anti-join

```
left_only = sales.merge(customers, on="customer_id",
                        how="left", indicator=True)
orphans = left_only[left_only["_merge"] == "left_only"]
```

### 20. Aggregate, then join (fan-out-proof enrichment)

```
member_value = (sales.dropna(subset=["customer_id"])
                .groupby("customer_id")
                .agg(lifetime=("amount", "sum"),
                    n_purchases=("amount", "count")))
customers = customers.merge(member_value, on="customer_id", how="left")
```

## Time Series

### 21. Resample (the calendar groupby)

```
daily = sales.groupby("date")["amount"].sum().sort_index()
monthly = daily.resample("ME").sum()
```

### 22. Rolling and cumulative (the frames of Ch. 19)

```
daily_7day = daily.rolling(7).mean()
running_total = sales.sort_values("date")["amount"].cumsum()
```

### 23. LAG and month-over-month

```
monthly = monthly.to_frame("revenue")
monthly["mom_pct"] = monthly["revenue"].pct_change() * 100
monthly["yoy"] = monthly["revenue"] - monthly["revenue"].shift(12)
```

## Statistics and Charts

### 24. The six instruments (Ch. 36, one block)

```
from scipy import stats
sun = daily[daily.index.day_name() == "Sunday"]
week = daily[daily.index.day_name() != "Sunday"]
t, p = stats.ttest_ind(sun, week, equal_var=False)      # Welch
r, p_r = stats.pearsonr(oil["price"], oil["bottles"])
chi2, p_c, dof, exp = stats.chi2_contingency(pd.crosstab(x, y))
assert exp.min() >= 5                                # the cell-count rule
s_lo, s_hi = np.percentile(
    [daily.sample(frac=1.0, replace=True).mean() for _ in range(2000)],
    [2.5, 97.5])                                     # the bootstrap CI
```

### 25. The chart pair (fast + craft)

```
ax = monthly.plot(figsize=(9, 4), color="#0F766E")    # fast
ax.set_title("Monthly revenue: growth, and the month-9 dip")
ax.set_ylabel("Revenue (USD)"); ax.spines[["top", "right"]].set_visible(False)
fig = ax.get_figure(); fig.savefig("exhibit.png", dpi=150) # craft: the file
```

## Bonus 1: The Monthly Pipeline (full listing)

Chapter 37's `build_report.py`, complete — the report that builds itself:

```
"""Tariro's monthly report -- builds itself. Run: python build_report.py"""
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
RAW, OUT = Path("data/raw"), Path("output")
TEAL, FENCE = "#0F766E", 21.50
def loading_checks(sales, customers):
    orphans = sales.loc[sales["customer_id"].notna()
        & ~sales["customer_id"].isin(customers["customer_id"])]
    lines = [f"sales rows: {len(sales):,}",
        f"null amounts: {sales['amount'].isna().sum()}",
        f"orphan sales: {len(orphans)}",
        f"total revenue: {sales['amount'].sum():.2f}"]
    return "\n".join(lines), len(orphans), sales["amount"].isna().sum()
def main():
    sales = pd.read_csv(RAW / "sales.csv", parse_dates=["date"])
    customers = pd.read_csv(RAW / "customers.csv")
    sales["month"] = sales["date"].dt.to_period("M")
    log, n_orphan, n_null = loading_checks(sales, customers)
    if n_null > 0:
        raise SystemExit("ABORT: null amounts -- investigate first")
    if n_orphan > 20:
        raise SystemExit(f"ABORT: {n_orphan} orphans (tolerance 20)")
    monthly = sales.groupby("month")["amount"].sum()
```

```

by_day = sales.groupby(sales["date"].dt.day_name())["amount"] \
    .agg(["count", "sum", "mean"])
member_share = (sales.assign(m=sales["customer_id"].notna())
    .groupby("month")["m"].mean() * 100)
latest, prev = monthly.index[-1], monthly.iloc[-2]
out = OUT / str(latest); (out / "exhibits").mkdir(parents=True, exist_ok=True)
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(monthly.index.astype(str), monthly.values, color=TEAL)
ax.set_title(f"Monthly revenue: {monthly.iloc[-1]:.0f} "
    f"({(monthly.iloc[-1]/prev-1)*100:+.1f}% MoM)")
ax.set_ylabel("Revenue (USD)"); fig.autofmt_xdate()
fig.savefig(out / "exhibits" / "revenue.png", dpi=150,
    bbox_inches="tight"); plt.close(fig)
report = f"""# Tariro's Grocery -- Monthly Report {latest}
Revenue: ${monthly.iloc[-1]:.2f}
({(monthly.iloc[-1]/prev-1)*100:+.1f}% vs {monthly.index[-2]}).
Member share of sales: {member_share.iloc[-1]:.1f}%.
Sunday takings averaged ${by_day.loc['Sunday','mean']:.0f} vs weekday
${by_day.drop('Sunday')['mean'].mean():.0f}.
_Caveats: member share is association, not causation;
one shop's 18 months is a sample, not a population._
## Run log

```

```
{log}
```

```

"""
    (out / "report.md").write_text(report)
    print(f"Built: {out/'report.md'}")
if __name__ == "__main__":
    main()

```

## Bonus 2: Generating Tariro's Data

The recipe behind the running case — noise, then planted effects, then the truth file (Ch. 44's method; adapt freely, label always):

```

"""Generate Tariro's dataset: honest noise + planted effects + truth file."""
import numpy as np, pandas as pd
rng = np.random.default_rng(42)          # seeded: reproducible truth
days = pd.date_range("2024-01-01", periods=548, freq="D")  # 18 months
# ---- noise: weekdays vary, lognormal basket amounts (right-skew, Ch. 23)
base_traffic = {"Monday": 30, "Tuesday": 32, "Wednesday": 34,
    "Thursday": 33, "Friday": 42, "Saturday": 46, "Sunday": 26}
rows = []
for d in days:
    n = rng.poisson(base_traffic[d.day_name()])
    payday = 1.35 if d.day >= 25 else 1.0          # PLANT: payday spike
    fri_sat = 1.33 if d.day_name() in ("Friday", "Saturday") else 1.0
    n = int(n * payday * fri_sat)

```

```

    for _ in range(n):
        amount = round(float(rng.lognormal(1.6, 0.55)), 2) # median ~5
        rows.append({"date": d, "amount": min(amount, 420.0)})
sales = pd.DataFrame(rows)
sales["payment_type"] = rng.choice(
    ["Cash", "EcoCash", "Card"], p=[.45, .40, .15],
    size=len(sales))
# ... payment probs drift by month toward EcoCash; items/categories assigned;
# oil price rises +0.80 in month 9, quantities fall ~35% (PLANT);
# member ids attached for ~25% of sales; founding cohort's frequency
# declines 8%/month after month 12 (PLANT: the churn);
# 3 stock weeks dropped, ~1% amounts zeroed, a duplicate row inserted.
# ---- the truth file: score every method against known reality
truth = """# Truth (planted patterns)
- payday (25th+): traffic x1.35
- Friday/Saturday: x1.33
- Sunday: base 26 vs weekday ~32 (the shortfall is TRAFFIC)
- oil: price +0.80 at month 9; quantity -35% (quantity-led collapse)
- EcoCash share: 31% -> 44% across 18 months
- founding cohort: -8%/month frequency after month 12
- flaws: 3 missing stock weeks; 1 duplicated sale; 1 amount lost a zero
"""
Path("truth.md").write_text(truth)

```

*(The full generator, with every planted pattern implemented, ships with the book's repository — the listing above is the skeleton; the `truth.md` is the point: analyses scored against known reality.)*

*The cookbook's one law, inherited from Part V: the notebook that cannot be Restart-and-Run-All'd is a rumour — every snippet above assumes clean, top-to-bottom reproducibility.*

# Appendix D: Statistics Cheat Sheet

## Appendices — The Cookbook

"Which test when, in plain English — plus the formulas, the tool translations, and the reading rules. One page per spread, in bookmark form."

### The Decision Table — Which Test When

Your question	The instrument	The tool calls
"What's typical?"	mean (totals logic) / median (skewed money) / mode (categories)	AVERAGE MEDIAN · AVG SQL · <code>df.mean()</code> · SPSS Descriptives
"How spread out?"	sd (individuals) / IQR (robust)	STDEV.S QUARTILE · <code>df.std()</code> · Explore
"Is this value unusual?"	z-score (if bell) / percentile rank (always) / IQR fence (skewed)	$(x - \bar{x}) / s$ · PERCENTRANK · PERCENTILE_CONT
"Two groups differ? (independent)"	Welch's two-sample t-test	T.TEST(r1,r2,2,3) · <code>ttest_ind</code> · SPSS Independent T · Stata <code>ttest y, by(g)</code>
"Same units, before/after? (paired)"	paired t-test	T.TEST(r1,r2,2,1) · <code>ttest_rel</code> · Paired-Samples T · <code>ttest a==b</code>
"Counts/proportions across categories?"	chi-square (+ Fisher's exact if cells < 5)	CHISQ.TEST · <code>chi2_contingency</code> · Crosstabs · <code>tabulate x y, chi2</code>
"Two numbers move together?"	Pearson r	CORREL · <code>pearsonr</code> · Correlate · <code>pwcorr</code>
"The line through them?"	regression (slope is the finding)	SLOPE RSQ · <code>linregress/ols</code> · Regression · <code>regress</code>
"How wrong might my estimate be?"	CI = estimate ± 2×SE	CONFIDENCE.T · bootstrap (App. C #24) · every SPSS/Stata table
"What fraction survive to month k?"	survival curve / cohort retention	<code>pandas.groupby-unstack</code> (App. C) · KM in any stat package
"Which group does this belong to? (predict)"	logistic regression / classification	Ch. 57 — beyond this sheet's scope

**Before any test:** histogram first (bell → z/t; skewed → fences, percentiles, or transform);  $\alpha$  chosen *before* looking (0.05 default); effect size and CI reported with every p; the three explanations of any correlation (coincidence, confounding, causation) — only an argument licenses the third.

### The Formulas (the ones worth owning)

mean	$\bar{x} = \Sigma x / n$	
sample sd	$s = \sqrt{(\Sigma(x - \bar{x})^2 / (n - 1))}$	-- n-1 for samples

z-score	$z = (x - \bar{x}) / s$	
IQR fence	$Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$	-- flags ~ the outer 0.3% of a bell
SE of mean	$SE = s / \sqrt{n}$	-- quadruple n, halve the error
95% CI	$\bar{x} \pm 2 \times SE$	-- the "2" is the bell's gift
(1.96 exact)		
Welch's t	$t = (\text{mean}_1 - \text{mean}_2) / \sqrt{s_1^2/n_1 + s_2^2/n_2}$	
paired t	$t = \text{mean}(\text{differences}) / (\text{sd}(\text{diff})/\sqrt{n})$	
chi-square	$\chi^2 = \sum (O - E)^2 / E$ ; $E = \text{row} \times \text{col} / \text{grand}$ ; $df = (r-1)(c-1)$	
Pearson r	$r = \sum (zx \cdot zy) / (n-1)$	
regression	$y = a + b \cdot x$ ; $b = r \cdot (sy/sx)$	-- b is the finding
margin (prop)	$ME = 2 \times \sqrt{p(1-p)/n}$	-- worst case $p=.5$ : $n \approx 96$ for $\pm 10\%$ , 385 for $\pm 5\%$

## Reading Any Test Output (the six lines)

Every statistics package prints the same six lines, in every costume — read them in this order: **1)** the group/effect sizes (the finding's *size* — read first, before any p); **2)** the statistic (t,  $\chi^2$ , F — the evidence's strength); **3)** the df (the sample's contribution); **4)** the p-value (the surprise score *if nothing were happening* — never "the probability the null is true"); **5)** the confidence interval (the honest range of the effect); **6)** n (what all of the above is worth). Report in the order: **effect (with CI), then p, then n** — "Sundays are 840 lower (95% CI: 750-930),  $t(206) = 18.4$ ,  $p < 0.001$ ."

## Percentile Method Defaults (the Ch. 24 footnote, in full)

Tools compute percentiles differently on finite data (interpolation vs nearest-rank): **Excel/Calc** PERCENTILE.INC and QUARTILE.INC (interpolate, inclusive); PERCENTILE.EXC (exclusive — can error on small n); **pandas/NumPy** default linear interpolation ( $\approx$  INC); **SQL** PERCENTILE_CONT (continuous interpolation) vs PERCENTILE_DISC; **SPSS/Stata** offer several (SPSS's EXAMINE defaults are documented in output footnotes — read them). Differences vanish on large n and can move a quartile on  $n < 20$ : **state the method when precision matters; never compare percentiles across methods without checking.**

## The Tool Translation Table (Ch. 29's Rosetta Stone, extended)

Task	Spreadsheet	SQL	SPSS (menus)	Stata (commands)	Python
Descriptives	AVERAGE MEDIAN STDEV.S	AVG, PERCENTILE_CONT	Descriptives / Explore	summarize, summarize, detail	.mean() .median() .std()

Task	Spreadsheet	SQL	SPSS (menus)	Stata (commands)	Python
Frequencies / five-number	QUARTILE.INC	quartile query	Frequencies / Explore	tabstat	.describe()
Cross-tab	PivotTable	GROUP BY two cols	Crosstabs (+Chi-square)	tabulate x y, chi2	pd.crosstab
Independent t	T.TEST(...,3)	CTE arithmetic	Independent-Samples T	ttest y, by(g)	ttest_ind
Paired t	T.TEST(...,1)	CTE arithmetic	Paired-Samples T	ttest a==b	ttest_rel
Chi-square	CHISQ.TEST	hand-built E/O	Crosstabs Statistics	tabulate, chi2	chi2_contingency
Correlation	CORREL	dialect-dependent	Correlate (Pearson)	pwcorr	pearsonr, .corr()
Regression	SLOPE RSQ INTERCEPT	dialect-dependent	Regression	regress	linregress, sm.OLS
z-scores	formula	window arithmetic	Descriptives: "Save standardized"	egen z = std(x)	(x-x̄)/s
CI of mean	CONFIDENCE.T	arithmetic	printed by every procedure	printed / ci	sem() / bootstrap
Reproducible script	workbook+log	.sql files	Paste → Syntax .sps	Do-file .do	notebook/script

**SPSS menu paths (Ch. 27/29's walkthrough, compact):** Frequencies (Analyze → Descriptive Statistics → Frequencies); Explore with plots (Analyze → Descriptive Statistics → Explore); Crosstabs with  $\chi^2$  and row % (Analyze → Descriptive Statistics → Crosstabs → Statistics → Chi-square; Cells → Row); Independent t (Analyze → Compare Means → Independent-Samples T Test — read the Levene line; if "equal variances not assumed", read that row); Paired t (Compare Means → Paired-Samples); Correlation (Correlate → Bivariate); Regression (Regression → Linear — the coefficients table is the six lines again). Paste every dialog to Syntax instead of clicking OK — the query pack, SPSS edition.

**The Stata beginner's kit (Ch. 29, with the verbs):** import delimited, describe, summarize, detail, generate, egen, keep/drop, tabulate x y, chi2, ttest, by(), regress, save; comments * or //; run via do-file; the log is the output trail.

## Worked Examples (the arithmetic, end to end)

### The Sunday test, every step

Data: 78 Sundays, mean takings 2,150, sd 590  
390 weekdays, mean takings 2,990, sd 655

1.  $H_0$ : Sunday mean = weekday mean ( $\alpha = 0.05$ , chosen in advance)
2. Difference:  $2,150 - 2,990 = -840$
3. SE of difference (Welch):  
 $\text{sqrt}(590^2/78 + 655^2/390) = \text{sqrt}(4,468 + 1,099) = \text{sqrt}(5,567) \approx 45.7$
4.  $t = -840 / 45.7 \approx -18.4$
5. df (Welch-Satterthwaite):  $\approx 101 \rightarrow$  look up  $t=-18.4$  at 101 df:  $p < 0.0001$
6. Verdict: reject  $H_0$ . Sundays are lower by 840 (95% CI:  $-840 \pm 2 \times 45.7$   
 $= -931$  to  $-748$ ). Report: "Sundays average \$840 below weekdays  
(95% CI \$748–\$931 lower),  $t(101) = -18.4$ ,  $p < 0.001$ ."

## Chi-square, every step

Cross-tab (tier  $\times$  shopped this month):

	Yes	No	row total
Gold	34	6	40
Silver	90	50	140
None	310	150	460
cols	434	206	640

$E(\text{cell}) = \text{row} \times \text{col} / \text{grand}$ :

Gold/Yes:  $40 \times 434/640 = 27.1$

Gold/No:  $40 \times 206/640 = 12.9$

Silver/Yes:  $140 \times 434/640 = 94.9$     Silver/No:  $140 \times 206/640 = 45.1$

None/Yes:  $460 \times 434/640 = 312.0$     None/No:  $460 \times 206/640 = 148.0$

$$\chi^2 = \sum (O-E)^2/E = (34-27.1)^2/27.1 + (6-12.9)^2/12.9 + (90-94.9)^2/94.9 \\ + (50-45.1)^2/45.1 + (310-312)^2/312 + (150-148)^2/148$$

$$= 1.76 + 3.69 + 0.25 + 0.53 + 0.01 + 0.03 \approx 5.5 + \text{smaller terms} \approx 6.3$$

df =  $(3-1)(2-1) = 2 \rightarrow p \approx 0.04\text{--}0.06$  (check every cell: all  $E \geq 5$  ✓)

Verdict at  $\alpha = 0.05$ : borderline – report the effect (Gold 85% vs None 67%)  
with the p and the small-cell caution, and recommend more data (Ch. 27).

## Reading a regression table (the SPSS/Python shape, decoded)

	coef	std err	t	P> t	[0.025	0.975]
const	6.24	0.81	7.70	0.000	4.66	7.82
price	-1.12	0.24	-4.67	0.000	-1.59	-0.65
R-squared	0.76	n = 18				

Read, in order:

1. slope -1.12: each extra dollar of price  $\approx$  1.1 fewer bottles/month
2. its CI [-1.59, -0.65]: excludes zero – the effect's honest range
3.  $R^2$  0.76: 76% of bottle variation tracks price
4.  $p < 0.001$ : not noise (surprise score)
5.  $n = 18$ : one shop, 18 months – the sample-not-population caveat rides along

## The Honesty Rules (the ones this book enforces, final form)

1. **An average is an answer, not the answer** — mean, median, or weighted: say which, and why; check the shape first; report both when they disagree.



2. **Correlation is not causation** — three explanations always; confounders named; the experiment is the licence.
3. **A sample is not a population** — every number from data is an estimate with an error; the CI is the honest form; "is it narrow enough to decide?" is the analyst's question.
4. **Significance is not importance** — effect size + CI + p, always; the 20th test on nothing is "significant" once; declare the question before running.
5. **Missing data is data about the data** — MCAR you may drop, MAR you must model, MNAR you must bound and declare; imputed values carry flags and disclosure sentences.

*Print this appendix if you print one thing: the decision table for the question, the formulas for the arithmetic, the six lines for the output, the five rules for the conscience.*

# Appendix E: Interview Question Bank

---

## Appendices — The Cookbook

"One hundred questions across the stack — the shapes actually asked, each with a line of guidance pointing at the chapter that owns it. Work them aloud, in batches of ten; the ones that make you hesitate are your revision syllabus."

### Mindset and Honesty (1-8)

**1.** What does a data analyst actually do all day? — *Five moves: question, data, clean, analyse, communicate; a day is these plus stakeholder time (Ch. 1, 45).*  
**2.** How do you handle being given a vague request? — *Clarify the decision behind it: "what will you do with the answer?" Half of all requests dissolve or transform (Ch. 54).* **3.** Your analysis contradicts what the stakeholder believes. What do you do? — *Deliver with unchanged rigour, frame options not verdicts, be ready to be wrong yourself (Ch. 60).* **4.** What's the difference between an average and a median, and when do you use each? — *Totals logic vs typicality; money is skewed; report both when they disagree (Ch. 22).* **5.** What questions do you ask of every analysis? — *The five: compared to what, says who, what's missing, what was counted, could it be luck (Ch. 2).* **6.** Tell me about a time you made a mistake with data. — *Fan-out, detached sort, the missing week — any real error plus the check you now run forever (Ch. 8, 17).*  
**7.** How do you know when you can trust a number? — *Reconciled bridge, method known, caveats attached, source named — trust is earned by checks (Ch. 21, 45).* **8.** What's more important: accuracy or deadline? — *The staged answer: cut scope, never rigor — verified core on time, whole truth dated (Ch. 60).*

## Excel (9-23)

**9.** VLOOKUP vs XLOOKUP vs INDEX-MATCH? — *Exact-match joins; XLOOKUP modern, INDEX-MATCH insertion-proof; know why FALSE matters (Ch. 9).* **10.** What does #N/A mean and what do you do? — *Three causes: genuine non-match, spelling/type mismatch, broken key — count them, don't hide them (Ch. 9).* **11.** Walk me through building a PivotTable. — *Select, Insert, four shelves; values re-aimable; % of total; drill by double-click (Ch. 10).* **12.** What's the difference between SUMIF and SUMIFS? — *One criterion vs pairs of (range, criterion); order of arguments flips (App. B).* **13.** How would you clean a column with inconsistent text entries? — *TRIM/CLEAN/case unification on a helper, paste values back, count before/after (Ch. 8).* **14.** How do you handle duplicates in a spreadsheet? — *COUNTIF flag, then decide: dedupe on which key, keep which copy — documented (Ch. 8).* **15.** Explain absolute vs relative references. — *The \$ anchors; F4 cycles; ranges drift when copied unless anchored (App. B).* **16.** What Excel function would you use for a weighted average? — *SUMPRODUCT/SUM — and ask "weighted by what?" before answering (Ch. 22).* **17.** How do you find outliers in Excel? — *QUARTILE + 1.5×IQR fence; never silently delete; investigate, report separately (Ch. 12).* **18.** What's the difference between STDEV.S and STDEV.P? — *Sample (n-1) vs population (n); your default is .S (Ch. 23).* **19.** Text dates are being read as text. Fix? — *DATEVALUE/Text-to-Columns, ISO format, check the locale, reformat real dates (Ch. 8).* **20.** How would you show month-over-month change? — *Pivot by month, % difference from previous — or LAG in SQL; name the tool by audience (Ch. 10, 19).* **21.** What's your reconciliation habit after a lookup join? — *Totals before = totals after; row counts; the fan-out check (Ch. 9, 17).* **22.** Conditional formatting for data quality — how? — *Duplicate values, blank-cell rules, above-fence: the liturgy made visible (App. B).* **23.** When would you NOT use Excel? — *Beyond ~1M rows, multi-table joins at scale, reproducibility requirements, automation — the honest limits (Ch. 14, 58).*

## SQL (24-43)

**24.** Write: total revenue by payment type. — *GROUP BY with SUM and COUNT side by side; sort by revenue (Ch. 16).* **25.** Write: top 3 items by revenue per category. — *Aggregate → ROW_NUMBER() OVER (PARTITION BY) → filter  $rn \leq 3$  — the interview classic (Ch. 19).* **26.** Find the second-highest sale. — *OFFSET/LIMIT or MAX-below-MAX or DENSE_RANK=2; mention ties (Ch. 52).* **27.** INNER vs LEFT JOIN — when each? — *Matches only vs all-left-rows; the choice is an analysis decision, not syntax (Ch. 17).* **28.** How do you find customers with zero orders? — *LEFT JOIN + WHERE right IS NULL — the anti-join (Ch. 17).* **29.** What is a fan-out and how do you catch it? — *Keys repeating both sides; row counts before/after, totals reconcile, aggregate-then-join (Ch. 17).* **30.** WHERE vs HAVING? — *Rows before grouping vs groups after; execution order FROM→WHERE→GROUP→HAVING→SELECT (Ch. 16).* **31.** What does NULL mean? How do you test it? — *Unknown, not zero; IS NULL; fails every comparison; COUNT() vs COUNT(col) (Ch. 15).** **32.** Explain a CTE and why you use one. — *Named step; readable pipelines; testable layers; reusable — the professional default (Ch. 18).* **33.** Write: each category's % of total revenue. — *Grouped CTE + SUM() OVER() or grand-total CTE; 100.0 decimal division (Ch. 18).* **34.** Running total by month? — *SUM() OVER (ORDER BY ...); tie-break key; the frame if partial (Ch. 19).* **35.** Month-over-month change? — *LAG(revenue) over ordered months; pct with decimal division; first row NULL is honest (Ch. 19).* **36.** Deduplicate a table keeping the latest row per id. — *ROW_NUMBER() PARTITION BY id ORDER BY date DESC, keep rn=1 (App. A).* **37.** What's a window function vs GROUP BY? — *Annotate every row vs collapse to one row per group (Ch. 19).* **38.** A query is slow. What do you do? — *Read the shape (sargability), EXPLAIN the plan, know what an index does (Ch. 20).* **39.** What does this inherited query do? (given a multi-join) — *Diagram FROM first: tables, lines, join types, which side is many (Ch. 17, 20).* **40.** LEFT JOIN with a WHERE on the right table — what happens? — *Silently becomes INNER; move the condition into ON or accept it deliberately (Ch. 20).* **41.** Write: count of sales per customer, including customers with none. — *LEFT JOIN from customers + COUNT(s.sale_id) — the column count, not COUNT() (Ch. 16-17).** **42.** NULL in NOT IN — the trap? — *NOT IN against a set containing NULL returns nothing; use NOT EXISTS (App. A).* **43.** How do you document SQL? — *Filename = question; header with purpose/author/expected totals; comment the why; version (Ch. 20).*

## Statistics (44-63)

**44.** What's a p-value, in one sentence? — *Probability of evidence this strong if nothing were happening; not the probability the null is true (Ch. 26).* **45.** 0.05 — magic? — *Convention, chosen in advance; 0.06 is weaker-labelled evidence, not failure (Ch. 26).* **46.** Statistical vs practical significance? — *Not-noise vs worth-money; big n makes trivial significant; report effect sizes (Ch. 26).* **47.** Type I vs Type II error? — *False alarm vs missed effect;  $\alpha$  guards the first; n drives the second (Ch. 26).* **48.** When do you use a paired test? — *Same units measured twice; test the differences; pairing cancels noise (Ch. 26).* **49.** Chi-square — what's it for, and its assumption? — *Association between categories; expected counts  $\geq 5$  per cell, else Fisher's exact (Ch. 27).* **50.** Correlation is 0.9. What can you conclude? — *Strong straight-line association; three explanations; not causation without an argument (Ch. 28).* **51.** What's  $R^2$ ? — *Share of y's variance the line explains;  $0.76 = 76\%$  tracked by x (Ch. 28).* **52.** What's a confidence interval, and how do you read it? —  $\bar{x} \pm 2 \times SE$ ; the procedure* captures truth in 95% of uses — and "is it narrow enough to decide?" (Ch. 25).* **53.** Standard deviation vs standard error? — *Individuals vs statistics; "sales vary by 12" vs "the mean is wrong by  $\sim 1.7$ " (Ch. 25).* **54.** Why does the CLT matter? — *Means of 30+ are near-normal whatever the raw shape; skew in, bell out (Ch. 25).* **55.** What's the Central Limit Theorem's practical use? — *Intervals and tests on averages from skewed data — daily takings from skewed baskets (Ch. 25).* **56.** Your data is right-skewed. What do you report? — *Median headline, fence-based outliers, percentiles; or transform; name the shape (Ch. 22-23).* **57.** Multiple comparisons — why care? — *20 tests on nothing yields one "significant"; declare questions first (Ch. 26).* **58.** Sampling bias example from your own work? — *The in-shop survey missing Sunday-stayers; refusal rates; the frame declared (Ch. 27, 47).* **59.** What's selection bias in a metric? — *Loyalty members self-selected; gym-club salad-eaters; measured instrument vs world (Ch. 28, 59).* **60.** Regression slope interpretation? — *Effect of x per unit, holding others constant — "a dollar costs a bottle" (Ch. 28).* **61.** Why not extrapolate the line? — *No data there; the line doesn't know;  $-3.7$  bottles is a confidence lie (Ch. 28).* **62.** What's a bootstrap? — *Resample with replacement, recompute, take the spread — CIs for anything (Ch. 36).* **63.** Which test: compare churn rate between two plans? — *Two proportions — chi-square (or z-test of proportions); state expected counts (App. D).*

## Python (64-78)

**64.** Why pandas for analysis? — *Reproducible pipeline, scale beyond sheets, joins/stats/charts in one place; automation compounding (Ch. 30).* **65.** DataFrame vs Series? — *The table vs one labelled column; df is a dict of Series (Ch. 33).* **66.** How do you handle missing values in pandas? — *Classify the mechanism first; drop (MCAR), impute-with-flag (MAR), bound+declare (MNAR) (Ch. 60).* **67.** merge — what's how= doing? — *Join direction: left keeps all left rows; indicator=True gives the anti-join (Ch. 34).* **68.** groupby-agg pattern, write it. — *df.groupby(k)[v].agg([...]); named aggregations are the aliases (Ch. 34).* **69.** What's vectorisation? — *Column-at-once operations at C speed; loops explain, one-liners work (Ch. 32).* **70.** When would you use .apply? — *Last resort: row-wise logic with no column idiom; slow and less searchable (Ch. 33).* **71.** How do you make a month column from dates? — *.dt.to_period("M"); parse with to_datetime at load; .dt accessor is the cleaning kit (Ch. 33).* **72.** Resample vs groupby for time? — *Calendar-aware grouping: ME/W/Q; unequal months handled (Ch. 34).* **73.** Rolling average and MoM in pandas? — *.rolling(7).mean(); .pct_change() — LAG pre-assembled (Ch. 34).* **74.** What's a notebook's biggest danger? — *State: cells run out of order; Restart & Run All before trusting (Ch. 30).* **75.** How do you test your pandas analysis? — *Reconcile against a second route (SQL, sheet); assert on known totals; small cases by hand (Ch. 33-34).* **76.** t-test in Python? — *stats.ttest_ind(a, b, equal_var=False) — Welch; read t, p; report effect+CI (Ch. 36).* **77.** Your script must run monthly unattended. What changes? — *Checks with consequences, config constants, dated outputs, logs, fail loudly (Ch. 37).* **78.** What is assert doing in your pipeline? — *The abort guard: nulls, orphans beyond tolerance — never report from untrusted data (Ch. 37).*



## Power BI and Dashboards (79-88)

**79.** Calculated column vs measure? — *Row label vs in-context number; "how much" = measure (Ch. 40).* **80.** What is filter context? — *The rows visible to the measure; clicks are WHERE clauses through the model (Ch. 40).* **81.** What does CALCULATE do? — *Evaluates a measure in a context you steer — the hand-written WHERE (Ch. 40).* **82.** Explain a star schema. — *Facts in the middle, dimensions around; why BI models are fast and sane (Ch. 39).* **83.** Many-to-many warning — what's it telling you? — *Fan-out risk; the design wants a bridge table (Ch. 39).* **84.** Why a date table? — *Time as a dimension; time intelligence needs a marked calendar (Ch. 39).* **85.** Your dashboard is slow. Diagnose. — *Too much shown, repeated heavy measures, too many visuals — Performance analyzer (Ch. 41).* **86.** How do you keep a dashboard honest? — *Data-age stamp, reconciliation card, checks page, caveat sentences (Ch. 41, 43).* **87.** Dashboard vs report — how do you choose? — *Recurring slices vs one argued finding; the brief decides (Ch. 43).* **88.** How would you present findings to non-analysts? — *Headline first, movement, stories with caveats, one interaction, the ask (Ch. 43).*

## Projects and Behaviour (89-100)

**89.** Walk me through a project you're proud of. — *STAR, 90 seconds, action-majority, the number in the result (Ch. 53).* **90.** Tell me about a time you automated something. — *An afternoon → twenty minutes; checks that abort; the before/after is the punchline (Ch. 37).* **91.** A time an analysis changed a decision. — *The save-campaign design; measurable, budgeted, honest about cause (Ch. 46).* **92.** A time you found something others missed. — *The say/do gap; 41% reframed to 15-20% by joining to behaviour (Ch. 47).* **93.** How do you scope a take-home? — *Question first, cut to the deciding finding, ship the honest limits, deliverables foldered (Ch. 45, 53).* **94.** "Isn't this just seasonality?" — *hostile Q&A. — Receive as a gift; show the split; the three explanations in action (Ch. 53).* **95.** What data-quality checks do you run habitually? — *The liturgy: counts, dup keys, orphans, NULLs, the bridge — every load (Ch. 21).* **96.** How do you handle changing requirements mid-project? — *Scope arithmetic aloud: adds X days or replaces Y — costs visible (Ch. 60).* **97.** What's your experience with SPSS/Stata? — *The translation table answer: statistics transferable, syntax a week not a year, artefacts as proof (Ch. 29).* **98.** Why this company / this role? — *Their domain + your portfolio's nearest project; specific duties, specific evidence (Ch. 50-51).* **99.** Where do you want to be in five years? — *The fork answered honestly: craft depth or team — with the compounding habits (Ch. 56).* **100.** What would you do in your first 90 days here? — *The three maps, the first deliverable, the quick win, the self-review — offer it as a plan (Ch. 54).*

## Using the Bank

Work it in batches of ten, *aloud* — the fluency being trained is the spoken sentence, not the known answer. The hesitation list is your revision syllabus: each hesitated question names a chapter; reread that chapter's Key Takeaways, then re-attempt the question cold a week later. Before interviews, rerun the sections matching the round (SQL screens → 24–43; a stats-heavy role → 44–63; the behavioural hour → 89–100 with your own STAR stories beside each). The bank's promise is the book's promise inverted: 62 chapters taught the method; 100 questions prove you can *say* it — which is the interview's whole game.



# Appendix F: Solutions and Walkthroughs

---

## Appendices — The Cookbook

"Attempt the lab first — the struggle is the lesson. Then come here: full walkthroughs for the projects' key labs, the answers with their numbers, the common wrong turns, and the reconciliations that prove each answer. Selected chapter labs included where beginners most often stall."

## How to Use This Appendix

Do not read ahead of your attempt. The labs are where the book actually happens; a solution read instead of attempted is a chapter unlearned. Use this appendix three ways: to *unstick* (read only the first step, then return to your work); to *check* (finish, then compare numbers and reasoning); and to *repair* — when your answer differs, find where the paths diverge, because the divergence is a chapter you half-learned. Numbers below are from the book's generated datasets (your copies will match if you use the repo's files; the *methods* match always).

## Part II — Excel (Chapters 6-13)

### Ch. 8, Lab 3 — the three EcoCash spellings

**The trap:** cleaning with `=UPPER(D2)` fixes case but not the trailing space — "ECOCASH "  $\neq$  "ECOCASH" and the pivot still shows two rows. **The full move:** a helper column `=TRIM(CLEAN(UPPER(D2)))`, filled down; then *paste-as-values* over the original column; then delete the helper. **The check:** `COUNTIF(D:D,"ECOCASH")` before and after — before: three values totalling 6,420; after: one value, same total. If the total moved, you deleted rows instead of cleaning them: undo.

## Ch. 13, Lab 1 — the one-page report

The walkthrough is Chapter 13.3's template filled with your numbers. **The five most common wrong turns, in order of frequency:** (1) the headline is a table of contents ("Monthly Report — June") instead of a finding — fix by writing the oil sentence there instead; (2) a recommendation without a measurement ("improve Sundays" — say *what* number, by *when*); (3) charts with furniture titles — every title should state the finding; (4) the caveat detached (a "limitations" paragraph at the bottom nobody reads — attach caveats to their claims, sentence by sentence); (5) two pages "because there was a lot" — the appendix sheet exists precisely so the page does not have to grow. **The check:** the readability test (Lab 2) — a friend recalls the three findings after ninety seconds or the page has failed, whatever it contains.

## Part III — SQL (Chapters 14-21)

### Ch. 17, Lab 4 — fan-out on purpose

**Build:** create `customer_discounts(customer_id, discount_note)`; insert two rows for member C0102. **Observe:** `SELECT COUNT(*) FROM sales` (their true count) vs the count after `JOIN customer_discounts` — every one of that member's ~180 sales now matches *both* discount rows: ~180 extra rows, and `SUM(amount)` inflated by exactly that member's revenue once over. **Fix:** aggregate first —

```
WITH one_row_per_customer AS (
    SELECT customer_id, COUNT(*) AS n_offers
    FROM   customer_discounts GROUP BY customer_id)
SELECT ... FROM sales s
LEFT JOIN one_row_per_customer o ON o.customer_id = s.customer_id;
```

**The numbers to log:** revenue before (54,013.50), revenue doubled at the join (+~1,940, the member's lifetime spend), revenue restored after the fix. Three numbers; one permanent instinct.

## Ch. 21, Q4 — the price-rise answer

The question: what did the month-9 rise actually do? **The shape:** monthly oil quantity, before vs after, as a paired comparison. **The walkthrough:** CTE 1 — oil sales only, monthly, count and sum; CTE 2 — the price from `suppliers.last_price_change` joined by month; then compare averages: before  $\approx 3.1$  bottles/customer-month, after  $\approx 1.8$  — a  $-1.3$  change; the paired  $t$  on per-item differences (Ch. 26) gives  $t \approx -4.6$ ,  $p \approx 0.0008$ . **The common wrong turn:** comparing *revenue* before/after and concluding "revenue fell 40%" — true but not the answer; revenue = price  $\times$  quantity, and the price rose: the collapse is quantity-led, which recommends a *different* action (cheaper brand trial) than a price-cut reflex. **The sentence to ship:** "The 0.80 price rise cut per-customer quantity from 3.1 to 1.8 bottles/month ( $t = -4.6$ ,  $p < 0.001$ ); the wholesaler's increase was not passed through volume — see the shelf trial proposal."

## Part IV — Statistics (Chapters 22-29)

### Ch. 25, Lab 1 — the CLT by hand

**The setup:** 200 samples of 50 sales; the  $\bar{x}$  of each. **What you should see:** the raw amount histogram — a right-skewed spike near 5 with a long tail to 400+; the sample-means histogram — a symmetric bell centred on 8.42 with  $sd \approx 1.7$  ( $= s/\sqrt{50} \approx 12.10/7.07$ , the SE formula *confirmed by your own simulation*). **The common wrong turns:** sampling *with replacement from the sample means* (nonsense — resample the data); forgetting to reset the sample each draw (one giant drifting sample); and reading the means' spread as "the spread of sales" (it is the spread of *estimates* — the whole point). **The caption pair to paste in your log:** "Left: baskets — skewed, whales visible. Right: averages of 50 — a bell. The Central Limit Theorem, demonstrated in ten minutes."

### Ch. 26, Lab 1 — the Sunday test, reconciled

By hand: group stats  $\rightarrow$  SE  $\rightarrow$   $t \rightarrow$  verdict, exactly as Appendix D's worked example ( $t \approx -18.4$ ,  $p < 0.0001$ ). **Then by tool:** `=T.TEST(sun_range, week_range, 2, 3)` returns the  $p$  alone — *write the  $t$  yourself*; the tool's job is confirmation, not understanding. **The reconciliation that matters:** your hand-computed  $t$  vs the ToolPak/SPSS output to three decimals. **The common wrong turn:** pooling all days into one giant two-group test *including* the promotion Sundays (a mixture) — state your exclusions, always.

Ch. 28, Lab 1 — the correlation paragraph

**The number:**  $r \approx -0.87$ ,  $p < 0.001$  on 18 months of oil. **The paragraph structure** (five clauses, each one clause — this is the template for every correlation you ever report): the correlation with its test; the fitted line's slope as a sentence; the  $R^2$  as a share; the mechanism making causation plausible (a known wholesaler increase — not a mysterious co-drift); and the trial that would close the argument. **The common wrong turns:** reporting  $r$  and stopping (no size — the slope is the business number); claiming causation outright (confounders unexamined — the harvest year, rivals' prices); and extrapolating the line to price 9 (the impossible  $-3.7$  bottles — Appendix D's regression-table reading covers why the CI widens at the edges).

Part V — Python (Chapters 30-37)

Ch. 34, Lab 1 — the pivot tour, reconciled

**The work:** five Chapter-10 pivots as one chained block each. **The reconciliation table to build** (this is the lab's actual deliverable):

pivot (workbook)	pandas	SQL	match?
revenue by month	monthly.sum()	GROUP BY month	✓ to the cent
category revenue	by_cat["revenue"]	GROUP BY category	✓
weekday counts	by_day["n"]	GROUP BY weekday	✓
payment mix	mix.sum(axis=?)	GROUP BY payment	✓
member share	member_share.mean()	AVG(is_member)	✓

**The common wrong turns:** a `.mean()` where the workbook had a *share* (different questions — reread the pivot's Values shelf); month as text sorting "2025-10" before "2025-2" (use `to_period`, not string slicing, for ordering); and reconciling *after* adding derived columns (reconcile at each step, or you cannot localise the drift).

Ch. 37, Lab 1-2 — the pipeline, built and broken

**The build:** Appendix C's full listing, typed. **The failure drill, expected outputs:** (a) deleting one amount from the CSV → the run aborts with "null amounts -- investigate first" *before* writing any output — this is the guard earning its keep; (b) the corrupted key census → abort on duplicates; (c) next month's rows appended → the report regenerates with the new month's headline, and the data-age stamp moves. **The common wrong turns:** catching the `SystemExit` and continuing anyway (the abort is the feature — never wrap it in `try/except-and-carry-on`); paths that only work from one directory (use the `Path(__file__).parent` pattern or run from the project root, and note it in the README); and no dated output folder (overwritten history — the folder-per-run *is* the log).

## Part VI — Power BI (Chapters 38-43)

### Ch. 39, Lab 3 — the fan-out museum, BI wing

**The exhibits:** (1) delete the customers relationship → the table visual shows every sale beside every customer (a cross-join's nonsense — massive row inflation, totals wildly wrong); (2) set cardinality many-to-many on a dimension with duplicate keys → the badge warns, and slicers start double-counting; (3) the fix — deduplicate the dimension in Power Query (Group By on the key), restore one-to-many, and watch the reconciliation card return to 54,013.50. **The lesson to write under the screenshots:** in BI tools the fan-out lives in *cardinality badges* and *missing lines* — the model view is the debugging view.

### Ch. 41, Lab 2 — the five-second test, scored

**What friends typically miss first** (in order): the MoM % (too small), the data-age stamp (too grey), the Sunday story (buried bottom-left — move it up). **The iteration rule:** whatever two of three viewers missed, change *position or size* — not colour. Position is comprehension; colour is decoration. **The finished page's benchmark:** revenue (big, top-left), direction (the MoM, beside it, coloured), one story visible without scrolling. If your page passes three consecutive five-second tests unchanged, it is done — ship it.

## Part VII — The Projects (Chapters 44-49)

### Ch. 46, Lab 1-2 — the cohort grid and the bend

**The grid walkthrough:** last-seen per customer (`groupby().max()`), months-since-join per active month (`((month - join_cohort).n)`), dedupe customer×age, count, unstack, divide by column 0. **What you should see:** column 0 all 100%; early-cohort rows holding 80%+ through month 10; the founding row thinning from month ~12 (−8%/month planted). **The survival curve's honest annotation:** founding cohort 12-month survival  $\approx 0.71$  vs newer cohorts  $\approx 0.84$  — *the drift is value-concentrated*: the founding cohort's CLV (higher spend × longer life) makes its churn the expensive one. **The common wrong turns:** counting *sales* not *customers* in the grid (deduplicate first — the grid counts people); including non-members (no join date — filter `notna`); and averaging retention across cohorts into one number (the entire point is that cohorts differ — never average the time dimension).

## Ch. 47, Lab 3 — the say/do table

**The build:** respondent segments (tier, suburb) joined to till history; within the "definitely would" group, compare Sunday-shopping frequency of *already-Sunday* shoppers vs not. **The expected finding:**  $\sim 2.3\times$  — stated intent concentrates among existing Sunday shoppers. **The sentence to ship:** "Stated Sunday intent is real but concentrated where behaviour already agrees; the promotion's *incremental* pull is nearer 15-20% than 41% — the four-Sunday till trial is the decisive test." **The common wrong turn:** reporting the 41% headline straight (it is *true* and *misleading* — the gap between those two words is this project's whole lesson).

## Part VIII-IX — The Hunt and Beyond (Chapters 50-62)

### Ch. 52, Lab 1 — the six SQL shapes, with the self-check line

Model answers are Appendix A: top-N-per-group (#20/#25), second-highest (OFFSET or DENSE_RANK), anti-join (#16), running total (#21), share-of-total (#19), dedupe-keep-latest (#23). **What to add to each in a live interview:** the assumption ("revenue, not units"), and *the check* — "I'd confirm the total didn't double after the join." That one sentence, spoken habitually, is the difference between a candidate who has used SQL and one who ships SQL.

### Ch. 54, Lab 1 — the 90-day plan, a model skeleton

Days 1-30: run the first hour on the main datasets; 8 coffees (manager, senior, 3 stakeholders, 2 engineers, the report's readers); collect the existing reports + the questions map.  
 Days 31-60: rebuild the monthly pack with checks (ship 2 days early); quick win: automate the "can you pull" of the week (state the before: "3 hours" / deliver: "10 minutes").  
 Days 61-90: runbook tested by handing over one cycle; schedule the second-time tasks; answer the top recurring question permanently; write the self-review with numbers and book the 90-day meeting.

**The common wrong turn:** a plan with no *numbers* (coffees counted, days early, hours saved) — the plan is a deliverable, and deliverables carry numbers. That habit — this book's oldest habit — is what the whole appendix has been demonstrating: *the answer, the method, the wrong turn, the check*.

## Chapter Labs — the Second Wave

### Ch. 2, Lab — the five questions, on a real chart

Take any published business chart. **The audit you should produce** (five lines, one per question): *compared to what* — does the axis start at zero (or flag that it doesn't); *says who* — is the source named; *what's missing* — absent months, an excluded segment, a legend doing surgery; *what was counted* — revenue or transactions, members or sales; *could it be luck* — is a wobble being narrated as a trend. **The most common finding on published charts:** the missing denominator — "80% of customers prefer X" with no n, no comparison, and no interval. That single audit habit, applied weekly to the charts you meet, is the entire mindset chapter in maintenance mode.

### Ch. 5, Lab — the questions ledger, graded

After drafting Tariro's ten questions, grade each on three axes: *decidable* (does a number exist that would settle it?), *affordable* (can this data be had for a reasonable effort?), and *valuable* (does the answer change an action?). The book's arc was built by exactly this grading: the Sunday, oil, and loyalty questions scored high on all three and got Parts; "which suburb has the nicest customers" scored one-of-three and stayed a curiosity. **Your own career version:** keep the graded ledger for your domain — it is Chapter 45's anatomy, pre-step-one.

### Ch. 7, Lab — the order-of-operations trap, built and escaped

The formula  $=2+3*4$  is 14, not 20;  $=(2+3)*4$  is 20. The spreadsheet formula that bites analysts: averaging a column of per-row products — `=AVERAGE(B2:B100*C2:C100)` typed hopefully into one cell is an error or a silent wrong (depending on the tool and array settings), while `=SUMPRODUCT(B2:B100,C2:C100)/SUM(C2:C100)` is the weighted mean, correct by construction. **The lesson in one line:** multiplication belongs *inside* the aggregation, not outside it — the same law as SQL's "aggregate, then join".

### Ch. 10, Lab — the refresh lesson, experienced

Change one amount in the source, watch the pivot *not* change, feel the instinct to distrust the tool — then Data → Refresh and watch it catch up. The lesson is not "remember to refresh" (you will remember after this happens in front of someone); it is *the pivot is a view, not a copy* — and every tool in this book has its own version of the stale-view moment (the dashboard browser cache, the notebook's old kernel state, Power BI's "apply pending changes" warning). Write down which tool staled on you and how you noticed; that entry is your future debugging instinct.

## Ch. 15, Lab — the NULL census, read as a story

Two queries: `COUNT(*)` (6,420) and `COUNT(customer_id)` (~4,900). The difference — ~1,520 — is the non-member sales, and the number itself is a finding: *a quarter of sales are anonymous*. The professional next move is a sentence in the report ("member analysis covers 76% of sales; the anonymous quarter is analysed separately"), not a silent filter. **The trap this lab prevents:** the query that quietly drops the NULL rows and reports "average member basket" as if it were the average basket.

## Ch. 18, Lab — the refactor drill, timed

Rebuild your longest flat query as named CTEs, then hand both versions to a friend. Typical timing: the flat version takes 3-4× longer to explain back, and the explainer invents errors in the flat version ("so it filters *after* the join? which join?"). The one-sentence conclusion for your log: *the CTE version is documentation that cannot go stale* — the query documents itself, or it isn't done.

## Ch. 22, Lab — the two-typical gap, explained to a shop owner

Mean 8.42, median 6.50 on the same data. The sentence that lands with a non-analyst: "half of all sales are under 6.50, but the *average* is 8.42 — bulk buyers drag it up; when you plan for the average customer, you're planning for someone bigger than three-quarters of your customers." That sentence — a number, its partner, and the business consequence — is the two-typical gap fully exploited.

## Ch. 24, Lab — the Sunday pre-analysis, closed one chapter early

Mean Sunday takings 2,150, weekday 2,990, sd of daily takings ~640: the gap (840) is 1.3 sd — every single week. Informally: too consistent to be weather, too large to be noise. The *z* you computed here becomes *t* = 18.4 in Chapter 26 because 78 *Sundays* of consistency compound: the per-week *z* is modest; the accumulated evidence is overwhelming. **The transferable insight:** one observation's surprise and a year's surprise are different arithmetic — the test exists to compound them honestly.

## Ch. 31, Lab — the error museum, curated

The four regulars, deliberately raised and pasted: `NameError` (the typo — "customer_id" vs "cutomer_id"), `TypeError` ("sales: " + 8.42 — the fix is the f-string), `SyntaxError` (the missing quote), `KeyError` (asking `sales["Amount"]` — capital A — the data question in disguise). The museum's value is speed: your debugging time drops the moment error messages become *recognised faces* instead of red walls. Add a fifth wing when you meet it — every analyst's museum grows a personal collection.



### Ch. 33, Lab — the dtype surprise, hunted

`info()` on customers shows joined as object, not datetime: somewhere in the column a value failed to parse (a blank, a "2025/06/14" slash-date, a note someone typed into a date field). The hunt: `customers[customers["joined"].apply(lambda x: not str(x)[:4].isdigit())]` — find the offenders, fix or exclude them *with a logged decision*, re-parse. **The rule the lab installs:** dtype object on a column that should be numeric or date is not a technicality — it is a data-quality finding wearing a type system's clothes.

### Ch. 36, Lab — the three-tool verification, certified

The same Sunday test in spreadsheet, SPSS-or-Stata, and scipy: t to three decimals, p identical to the displayed precision. When all three agree, you have verified not the *answer* but the *understanding* — three implementations of one courtroom, one verdict. The certification line for your log: "t = -18.43 across tools; differences in default Welch/Student handling checked and stated." That single sentence, in an interview, signals a professional upbringing.

### Ch. 42, Lab — the greyscale test

View your dashboard page through a colour-vision simulator (or screenshot → greyscale): the finding that survives is carried by position, label, and length — the honest carriers. What disappears was carried by colour alone — the lie you didn't intend. Typical casualty: the red/green conditional formatting on the MoM card, which in greyscale reads as two identical grey numbers — fix by adding the sign character (+/−) or an arrow, which carries the meaning redundantly. Accessibility fixes are almost always *clarity* fixes for everyone: the labelled, signed, positioned chart is simply the better chart.

### Ch. 44, Lab — the first hour, on national data

The typical surprises in your first real government dataset: totals that don't match the published summary (revisions, exclusions — a documentation read resolves it); codes as numbers (region "1"-"10" — keep them strings, map them with the codebook); implicit missing years (the survey skipped 2020 — a limitation to declare, not a gap to fill). The first hour's real lesson: *documentation is data too* — the methodology PDF you skim is where the dataset tells you its secrets.

## Ch. 49, Lab — the link-check that found the dead exhibit

The monthly link-check (first Friday) on a portfolio typically finds: the screenshot whose file was renamed, the notebook link that points to a local path, and — the expensive one — the project whose README says "report coming soon" from three months ago. Each is a five-minute fix on discovery and a silent credibility leak until then. The portfolio's real maintenance law: *links rot faster than skills* — schedule the check, or the shelf lies about you while you sleep.

## Ch. 55, Lab — the first pitch, whatever the outcome

You approached one real prospect with one specific offer. Outcomes, all four of them, are wins: *yes* — your first engagement, price it at the slightly-uncomfortable number; *no* — log the reason (wrong product? wrong time? wrong price?) — that reason is your market research; *not now* — a warm follow-up date, which is a pipeline, which is the entire discipline; *silence* — also data (the offer wasn't specific enough; make the next one narrower). The freelancer's ledger converts all four into progress; the amateur only counts the yeses.

## Ch. 61, Lab — the switch cost, measured

You rebuilt Project 1's one-pager in a tool you had never used, and it took — typically — 60–70% of the original time, most of it spent on the two or three idioms the new tool names differently. That measured number is your permanent answer to "do you know tool X?" — *the method transfers in a weekend; here is the rebuild I timed*. Tool-anxiety dies at that moment, and employers can hear that it has.

## The Common Wrong Turns, Consolidated

Across every lab in this appendix, five wrong turns account for most of the struggle: **missing parentheses** (in pandas booleans, in spreadsheet formulas, in SQL's AND/OR chains — the fix is always the same: wrap the units of meaning); **the unexamined denominator** (percentages, averages, shares — ask of what, weighted by what); **the silent filter** (NULLs dropped, rows amputated by INNER, the selection you forgot you made — the count-before/after habit catches them all); **skipping the reconciliation** (every doubled total in this appendix traces to it); and **narrating before checking** (the chart's wobble, the model's brilliance, the "significant" — all stories told before the data was asked). Notice that none of the five is a tool problem. The labs teach tools; the wrong turns teach the profession.

# Appendix G: Glossary

---

## Appendices — The Cookbook

"Every term this book taught, one line each — the definitions as the chapters used them, not as dictionaries do. Bold cross-references point to their home chapters."

**ad-hoc analysis** — one-off questions answered by hand, as opposed to automated reporting; the recurring ad-hoc is a pipeline waiting to be built (Ch. 54, 60).

**aggregate** — a function that collapses many rows into one number: COUNT, SUM, AVG, MIN, MAX (Ch. 16).

**aggregation, conditional** — SUM(CASE WHEN...) — building columns-of-conditions in one pass (App. A).

**alias** — a short name for a table (`sales s`) or a computed column (`AS revenue`) (Ch. 17).

**α (alpha)** — the significance level chosen *before* testing; the standard of proof; 0.05 by convention (Ch. 26).

**anti-join** — LEFT JOIN + WHERE right-key IS NULL; the rows with no match (Ch. 17).

**appendix-driven development** — not a real term; if you looked it up, you are thorough, and that is the trait (Ch. 2).

**API** — a programmatic doorway into a system or data source (Ch. 44, 57).

**ATP/ATS** — the systems that parse and screen CVs; mirror honest keywords, keep layouts plain (Ch. 51).

**average (mean)** — the balance point; the only typical that interacts correctly with totals (Ch. 22).

**average, weighted** —  $\Sigma(\text{value} \times \text{weight}) / \Sigma(\text{weight})$ ; every "average" hides a choice of weights (Ch. 22).

**baseline** — the comparison a number needs to mean anything; "compared to what?" (Ch. 2, 52).

**bias (statistics)** — systematic error in one direction; enters at sample, measurement, or model (Ch. 59).

**BI (business intelligence)** — the dashboarding discipline: models, measures, pages, refresh (Part VI).

**bookmark** — a saved state of a dashboard page (filters, visibility) (Ch. 41).

**bootstrap** — resample with replacement, recompute, take the spread; CIs for anything (Ch. 36).

**bridge total** — a known total reconciled at every step of a pipeline; the trust anchor (Ch. 21).

**calendar/date table** — a marked dimension of dates that makes time intelligence possible (Ch. 39).

**CALCULATE** — DAX's context-steering function; a hand-written WHERE for a measure (Ch. 40).

**cardinality** — the one-to-many shape of a relationship; the model's fan-out guard (Ch. 39).

**case study note** — the 150-word portfolio atom: situation, work, finding, impact, limits (Ch. 45).

**caveat** — the honesty attached to a claim at the moment of claiming; travels with the number (Ch. 13).

**Chebyshev's inequality** — the shape-free promise:  $\geq 75\%$  within 2 sd, whatever the distribution (Ch. 24).

**chi-square ( $\chi^2$ )** — the association test for categories; expected counts  $\geq 5$  per cell (Ch. 27).

**churn** — a customer stopping, by a stated definition (no purchase in N days) (Ch. 46).

**CLT (Central Limit Theorem)** — means of 30+ are near-normal whatever the raw shape (Ch. 25).

**cohort** — a group sharing a start; the unit that keeps time honest (Ch. 46).

**confidence interval (CI)** — estimate  $\pm 2 \times \text{SE}$ ; the procedure captures truth in 95% of uses (Ch. 25).

**confounder** — the third variable driving two others; the reason correlation  $\neq$  causation (Ch. 28).

**confusion matrix** — predicted vs actual: a cross-tab of a classifier's performance (Ch. 57).

**correlation (r)** — straight-line association strength,  $-1$  to  $+1$ ; not causation (Ch. 28).

**COUNT(*) vs COUNT(column)** — all rows vs non-NULL rows; the difference is an analysis (Ch. 15).

**cross-tab** — counts of two categorical variables; the pivot's statistical soulmate (Ch. 27).

**CTE (Common Table Expression)** — a named query step (`WITH monthly AS (...)`); the readable pipeline (Ch. 18).

**dashboard** — an interactive page of live measures answering recurring questions (Part VI).

**data dictionary** — the documentation of columns and their meanings; read before touching (Ch. 44).

**Data age stamp** — the freshness marker every honest dashboard displays (Ch. 41).

**data frame (pandas)** — the programmable table; a dict of Series sharing an index (Ch. 33).

**dbt** — SQL-as-modules: versioned, tested transformations; analytics engineering's tool (Ch. 58).

**DAX** — Power BI's formula language; measures evaluated in filter context (Ch. 40).

**degrees of freedom (df)** — the sample's contribution to a test's distribution (Ch. 26, 27).

**descriptive statistics** — numbers that describe what is (typical, spread, shape) without concluding (Ch. 22).

**dimension table** — the who/what/where/when context around a fact table; star schema's points (Ch. 39).

**drillthrough** — right-click into a detail page, pre-filtered; summary to evidence (Ch. 41).

**drill-down** — exploding a summary back into its rows (double-click a pivot number) (Ch. 10).

**DuckDB** — the analytical database engine that queries files directly; the laptop warehouse (Ch. 61).

**effect size** — how big the difference is, with its interval; what business decisions need (Ch. 26).

**empirical rule (68-95-99.7)** — the bell's promise; needs the bell to be true (Ch. 23).

**evaluation context** — the rows a DAX measure can see; clicks become contexts (Ch. 40).

**Excel** — the universal first tool; the whiteboard where everyone starts (Part II).

**expected count (E)** — what a chi-square cell would contain under independence (Ch. 27).

**experiment** — deliberately changing X to watch Y; the licence for causation claims (Ch. 28).

**fact table** — the events in the middle of a star schema; many rows, the numbers (Ch. 39).

**fan-out** — the join trap: keys repeating on both sides multiply rows and totals (Ch. 17).

**feature** — an input column for a model; derived features are the analyst's crafted inputs (Ch. 57).

**field** — a column, in database/BI vocabulary (Ch. 14).

**filter context** — the WHERE a measure evaluates inside (Ch. 40).

**filter propagation** — filters flowing across relationships through the model (Ch. 39).

**five-number summary** — min, Q1, median, Q3, max; the fastest honest portrait (Ch. 12).

**five-second rule** — what a page must communicate in a glance (Ch. 41).

**frame (window)** — the rows a window function can see: 6 PRECEDING, CURRENT ROW... (Ch. 19).

**free stack** — the \$0 toolkit: Calc, SQLite, jamovi, Python, Power BI Desktop, GitHub (Ch. 61).

**GDPR / POPIA / data-protection acts** — the laws built on consent, purpose, minimisation (Ch. 59).

**gateway** — the on-premises agent that lets a cloud service refresh from local sources (Ch. 43).

**GROUP BY** — "for each what"; the aggregate habit (Ch. 16).

**HAVING** — filtering groups after they are built; WHERE cannot (Ch. 16).

**histogram** — the shape of one column; read before any average (Ch. 23).

**honesty rules (the three)** — average  $\neq$  the answer; correlation  $\neq$  causation; sample  $\neq$  population (Ch. 2).

**hypothesis test** — the courtroom for a number: H0 "nothing happened", evidence, surprise score (Ch. 26).

**imputation** — filling missing values, with flags and disclosure; model the mechanism first (Ch. 60).

**INDEX-MATCH** — the insertion-proof lookup pair (Ch. 9).

**index (database)** — the lookup structure that makes key-seeks fast (Ch. 20).

**inner join** — matches only; amputates the unmatched (Ch. 17).

**IQR** —  $Q3 - Q1$ ; the robust spread; powers the outlier fence (Ch. 12, 23).

**join** — match the keys, bring the facts across, check the totals (Ch. 9, 17).

**JSON** — the web's data format; appears when APIs feed your pipelines (Ch. 44).

**Kaplan-Meier / survival curve** — the fraction surviving to each time  $k$  (Ch. 46).

**key (primary/foreign)** — the identifier that makes relationships possible (Ch. 14).

**lag/lead (SQL)** — the previous/next row's value; month-over-month's engine (Ch. 19).

**leakage** — features knowing the future; the silent killer of ML results (Ch. 57).

**LEFT JOIN** — keep all left rows, fill the unmatched with NULLs (Ch. 17).

**lineage** — the clickable path from visual to measure to table to source (Ch. 38).

**loading liturgy** — row counts, key census, orphans, totals, NULLs — every load, forever (Ch. 21).

**log (the)** — the record of cleaning decisions and method notes; the senior-looking part (Ch. 13).

**many-to-many** — keys repeating both sides; fan-out waiting to fire; bridge tables resolve it (Ch. 17).

**margin of error** — the  $\pm$  of a survey estimate;  $\pm 2 \times \text{SE}$  of a proportion (Ch. 25, 27).

**MAR / MCAR / MNAR** — the three missingness mechanisms and their matching responses (Ch. 60).

**marketplace of questions** — not a term either; but your questions ledger is an asset (Ch. 2, 44).

**matplotlib** — Python's chart engine; the second pen this book taught (Ch. 35).

**mean** — see average; the totals-logic typical (Ch. 22).

**measure (DAX)** — a named formula evaluated in context; "how much" lives here (Ch. 40).

**median** — the middle value; the robust typical for money (Ch. 12, 22).

**merge (pandas)** — the join, with `how=` and `indicator=` (Ch. 34).

**metric** — a defined number worth tracking; always ask what it really measures (Ch. 59).

**mode** — the most common value; the only typical for categories (Ch. 22).

**model (data model)** — tables + relationships; where BI correctness is decided (Ch. 39).

**NULL** — unknown, not zero; fails every comparison; test with IS NULL (Ch. 15).

**n** — the count; the denominator; what every percentage owes its reader (Ch. 22, 27).

**notebook (Jupyter)** — code, output, prose in one reproducible document (Ch. 30).

**one-pager** — the report format: headline, findings with recommendations, appendix (Ch. 13).

**orchestrator** — the scheduler of pipelines at scale (Airflow and kin) (Ch. 37, 58).

**outlier** — beyond the fence; investigate, report separately, never silently delete (Ch. 12).

**p-value** — probability of evidence this strong if nothing were happening; a surprise score (Ch. 26).

**paired test** — same units measured twice; test the differences (Ch. 26).

**parameter vs statistic** — the population's fixed truth vs the sample's varying estimate (Ch. 25).

**pandas** — Python's analysis library; the engine of Part V (Ch. 33–34).

**partition (window)** — the group-within-which a window function restarts (Ch. 19).

**Pearson r** — the correlation coefficient (Ch. 28).

**percentile** — the value below which p% of data lies; the shape-free ruler (Ch. 24).

**pivot table** — totals of something for each something else, without formulas (Ch. 10).

**p-hacking** — running many tests until one is significant; prevented by declaring questions first (Ch. 26).

**pipeline** — inputs → checks → analysis → outputs; runs identically, unattended (Ch. 37).

**population** — every unit you want to speak about; you never see it (Ch. 25).



**Power Query** — Power BI's recorded, replayable cleaning editor (Ch. 39).

**practice lab** — the end-of-chapter exercise set; where the book actually happens (every chapter).

**prediction interval** — the honest range around a line's forecast (Ch. 28).

**principal vs manager** — the two senior ladders: hardest problems vs people and priorities (Ch. 56).

**purpose limitation** — data collected for X is used for X (Ch. 59).

**q (quartile)** — the 25th/50th/75th percentiles (Ch. 12).

**query pack** — the folder of saved, headered queries that regenerates a report (Ch. 21).

**R²** — the share of y's variance the line explains (Ch. 28).

**rank trio** — ROW_NUMBER (exact N), RANK (ties, gaps), DENSE_RANK (ties, no gaps) (Ch. 19).

**reconciliation** — checking a number against a second route to it; the trust habit (Ch. 8, 21).

**refresh (scheduled)** — the data updating itself on a timer, with a gateway where needed (Ch. 43).

**regression** — the line of best fit; the slope is the finding (Ch. 28).

**resample (pandas)** — calendar-aware grouping of time series: ME, W, Q (Ch. 34).

**residual** — the vertical miss between point and line; where the next question lives (Ch. 28).

**rolling window** — the moving neighbourhood: 7 PRECEDING and the rest (Ch. 19, 34).

**row context** — the single row a calculated column sees (Ch. 40).

**running total** — SUM OVER (ORDER BY ...); the cumulative line (Ch. 19).

**R² vs r** — the line's share vs the association strength;  $r^2 = R^2$  for one predictor (Ch. 28).

**sample** — the part you can see; the argument from seen to unseen is statistics (Ch. 25).

**sampling distribution** — what a statistic would say across re-runs (Ch. 25).

**sargable** — a filter the engine can seek (bare columns), not scan (functions on columns) (Ch. 20).

**scatter plot** — two numeric variables, one point each; where relationships live (Ch. 11, 28).

**schema** — the structure: tables, columns, types, relationships (Ch. 14).

**SE (standard error)** —  $s/\sqrt{n}$ ; the typical distance between estimate and truth (Ch. 25).

**seasonality** — the repeating calendar pattern; the baseline for honest comparisons (Ch. 34, 53).

**self-selection** — when the members chose themselves (loyalty schemes, gyms, surveys) (Ch. 28).

**sensitivity analysis** — computing the answer under best/worst assumptions; MNAR's honest response (Ch. 60).

**SERIES (pandas)** — one labelled column; the DataFrame's building block (Ch. 33).

**share of total** — each group's slice with its denominator declared (Ch. 18).

**significance vs importance** — not-noise vs worth-money; report effect sizes (Ch. 26).

**skew** — the lopsidedness; right-skew is money's shape (Ch. 23).

**slicer** — the reader-facing filter; a click that becomes a WHERE (Ch. 38, 41).

**small multiples** — the same chart per group, shared axes; comparison by construction (Ch. 42).

**snowflake vs star** — the warehouse shapes; star is the analyst's default (Ch. 39).

**SQL** — the language of data; fifty years the lingua franca (Part III).

**SPSS / Stata** — the classic statistics packages; menus vs do-files (Ch. 29).

**squared error** — deviation²; severity plus positivity (Ch. 23).

**standard deviation (s /  $\sigma$ )** — the typical distance from the mean; individuals' spread (Ch. 23).

**standardisation (z)** —  $(x - \bar{x})/s$ ; units dissolved (Ch. 24).

**STAR interview method** — Situation, Task, Action, Result; 90 seconds (Ch. 53).

**star schema** — facts in the middle, dimensions around (Ch. 39).

**substring/slice** — text by position: substr, [start:end], LEFT/MID/RIGHT (Ch. 15, 31).

**supervised learning** — models fitted to labelled examples: regression, classification (Ch. 57).

**t-test** — the mean-comparison courtroom; Welch's version is the default (Ch. 26).

**take-home** — the interview analysis task; graded on craft and honesty, not sophistication (Ch. 52).

**three explanations** — coincidence, confounding, causation; the correlation triage (Ch. 28).

**time intelligence** — DAX's date-aware calculations: MoM, YTD, same-period-LY (Ch. 40).

**top-N per group** — aggregate, rank, filter; the interview classic (Ch. 19).

**tool-agnosticism** — the method transfers; the accents are cheap (Ch. 29, 61).

**TRIM/clean** — killing the whitespace and case ghosts before they become joins' #N/As (Ch. 8).

**t-test, paired** — see paired test.

**Type I / Type II error** — false alarm vs missed effect (Ch. 26).

**unsupervised learning** — structure without labels: clustering, reduction (Ch. 57).

**validation (cross-)** — rotating the train/test split so the grade is not luck (Ch. 57).

**VAR / RETURN** — DAX's named intermediate steps; CTE habits in formula form (Ch. 40).

**variable (Python)** — a labelled box; names mean things; snake_case (Ch. 31).

**vectorisation** — column-at-once operations; loops explain, one-liners work (Ch. 32).

**VLOOKUP / XLOOKUP** — the spreadsheet join; exact match always (Ch. 9).

**warehouse (data)** — the rented, columnar, cloud-scale database: BigQuery, Snowflake (Ch. 58).

**weighted mean** — see average, weighted.

**window function** — annotate every row without collapsing any (Ch. 19).

**XLOOKUP** — see VLOOKUP; the modern form (Ch. 9).

**z-score** — the standard ruler; +2.0 is a once-in-twenty value if the bell holds (Ch. 24).

# About this Edition

---

## *Production notes*

**The Practical Data Analyst's Handbook — From Zero to Your First Data Job — A Complete Beginner's Course in Excel, SQL, Statistics, Python and Power BI** was typeset directly from its Markdown manuscript with `build_pdf.py` (ReportLab): 7 x 10 inch textbook trim, running headers, a linked table of contents, and PDF outline bookmarks.

- Edition: First Edition, 2026 (compiled 05 October 2026)
- Contents: 62 chapters in 9 parts, plus 7 appendices
- Volume: approximately 99,487 words of prose and 1409 lines of code (estimated 384 print pages)
- Toolchain: Markdown manuscript, `build_book.py` (HTML edition), `build_pdf.py` (this PDF)
- Typeface: DejaVu (serif body, sans display, mono code)

To rebuild either edition after editing the manuscript, run `python3 build_book.py` then `python3 build_pdf.py`. The companion HTML edition is screen-optimised and regenerates in seconds.

Thank you for reading this book — and for teaching from it. The data is already pouring in somewhere near you.